

CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Requirements Specifications

for

GIT it!

Team Code:

Team 2627-Sem1-IT411-34

Change History

Version	Date	Author	Status	Description of Changes
0.1	April 23, 2026	Team 2526-sem2-it332-34	Draft	Initial draft created. Sections 1.1 Purpose, 1.2 Scope, and 1.3 Definitions skeleton established.
0.2	April 25, 2026	Team 2526-sem2-it332-34	Draft	Completed 1.3 Definitions, Acronyms and Abbreviations. Added 1.4 References with full citation list and source URLs.
0.3	April 28, 2026	Team 2526-sem2-it332-34	Draft	Completed Section 2 Overall Description: 2.1 Product Perspective including system architecture, content library scope, KPI framework, and pedagogical design model. Added 2.2 Product Functions table.
0.4	April 30, 2026	Team 2526-sem2-it332-34	Draft	Completed 2.3 User Characteristics, 2.4 Constraints, and 2.5 Assumptions and Dependencies.
0.5	May 3, 2026	Team 2526-sem2-it332-34	Draft	Completed Section 3.1 External Interface Requirements: 3.1.1 Hardware Interfaces, 3.1.2 Software Interfaces, and 3.1.3 Communications Interfaces.
0.6	May 7, 2026	Team 2526-sem2-it332-34	Draft	Began Section 3.2 Functional Requirements. Completed System Access use cases SA.1 Student Registration, SA.2 Student Login, and SA.3 Administrator Login with use case descriptions, diagrams, and activity diagrams.
0.7	May 10, 2026	Team 2526-sem2-it332-34	Draft	Completed Module 0 Orientation (0.1–0.7) and Module 1 Local Repository Foundations (1.1–1.8) use cases and FR tables.
0.8	May 13, 2026	Team 2526-sem2-it332-34	Draft	Completed Module 2 Branching and Collaboration (2.1–2.11), Module 3 Conflict Resolution (3.1–3.5), Module 4 Advanced Recovery and History (4.1–4.5), Administrative Management (AM.1–AM.6), and AI-Assisted Learning Support (AI.1) use cases.
0.9	May 17, 2026	Team 2526-sem2-it332-34	Revised	Completed Section 3.3 Non-Functional Requirements: Performance, Security, and Reliability. Revised 1.2 Scope to include Full System Scope and MVP Core Implementation Scope split.
1.0	May 22, 2026	Team 2526-sem2-it332-34	Approved	Completed Appendix A: Needs Assessment Survey Summary, Appendix B: SUS Questionnaire Instrument, and Appendix C: TAM Questionnaire Instrument. Final review and

				consolidation. Resolved internal inconsistencies across sections. Applied team and adviser feedback. Final approved version submitted for Capstone 2 evaluation.
1.1	17 October 2026	Team 2627-sem1-it411-34	Draft for review	Updated scope to current stories, chapters, drills, Shop, and staff functions; prior MVP exclusions conflicted with the website.
1.1	17 October 2026	Team 2627-sem1-it411-34	Draft for review	Aligned registration, shared sign-in, onboarding, and Home with current routes; separate administrator login was obsolete.
1.1	17 October 2026	Team 2627-sem1-it411-34	Draft for review	Revised tier access, multi-clear, retry, replay, and file actions; old flows omitted these behaviors and orientation topic 0.8.
1.1	17 October 2026	Team 2627-sem1-it411-34	Draft for review	Proposed CAR processability and eligible RTA definitions; implementation differs from earlier study measures and targets.
1.1	17 October 2026	Team 2627-sem1-it411-34	Draft for review	Clarified stars, mastery rank, GitCoins, and companions; prior text conflated scoring, progression, and purchases.
1.1	17 October 2026	Team 2627-sem1-it411-34	Draft for review	Separated browser simulation from backend verification and updated token fallback; old interface claims assigned responsibility incorrectly.
1.1	17 October 2026	Team 2627-sem1-it411-34	Draft for review	Updated authoring, learner management, and analytics status; audit-log viewing remains unconfirmed in the current release.
1.1	17 October 2026	Team 2627-sem1-it411-34	Final	Proposed guided-teaching amendment for syntax and examples; strict No-Answer wording conflicts with current guidance.
1.2	28 October 2026	Team 2627-sem1-it411-34	Draft	Replaced the deferred AI.1 Query AI Support use case with the implemented companion-based Git tutor: authenticated general and command-guide help, scenario recognition with protected conceptual teaching, provider failure recovery, and requirements FR-AI-01 to FR-AI-03, with an updated use case diagram, use case description, and activity diagram. Updated scope, definitions, product functions, user characteristics, constraints, assumptions, software and communications interfaces, and the survey-to-design mapping that described AI as deferred or unavailable. Reason: the AI chatbot is now implemented. The earlier wireframes for AI.1 are replaced by placeholders for implemented interface screenshots, which are pending.
1.3	30 October 2026	Team 2627-sem1-it411-34	Draft for review	Rewrote 3.3 Non-functional requirements against the deployed system: measurable performance targets with workload, throughput and error rate;

				actual rate limits and data processors; an uptime monitor and weekly manual backups for the free hosting tier; new usability, maintainability and research-target groups.
1.3	30 October 2026	Team 2627-sem1-it411-34	Draft for review	Updated interfaces, constraints and assumptions: Render Key Value replaces Upstash Redis; current framework versions; phone-width support; Vercel, Brevo and OAuth services; current API route groups.
1.3	30 October 2026	Team 2627-sem1-it411-34	Draft for review	Updated sign-up and sign-in: email verification by one-time code, GitHub and Google sign-in, sign-in lockout, password reset and account security (FR-AUTH-01 to FR-AUTH-12).
1.3	30 October 2026	Team 2627-sem1-it411-34	Draft for review	Updated gameplay: current star rules; retired the daily streak and the pre-practice modal requirements; level unlock order; added Challenge Gates (FR-CHAL), Squire's Drill (FR-DRILL), chests and achievements.
1.3	30 October 2026	Team 2627-sem1-it411-34	Draft for review	Aligned KPI definitions with KPI_METHODOLOGY.md and defined the admin requirements FR-ADMIN-17 to FR-ADMIN-24; content authoring was removed from the product.
1.3	30 October 2026	Team 2627-sem1-it411-34	Draft for review	Replaced the AI.1 screenshot placeholders (Figures AI-SRS-3 to AI-SRS-6) with screenshots of the implemented tutor, captured from a local build of the current main branch.
1.4	30 October 2026	Team 2627-sem1-it411-34	Draft for review	Revised NFR-PERF-01, NFR-PERF-04, NFR-PERF-06 and NFR-PERF-07 to the capacity measured on the free hosting tier (STD version 3.2, INC-10), with the reason for each change in a new table of target revisions: command confirmation targets by number of learners, learners joining over about 10 minutes, a separate level start target for when the battle art must be downloaded, and requests rejected by the hosting platform counted as errors.

Table of Contents

1. Introduction 16

1.1. Purpose 16

1.2. Scope 16

1.3. Definitions, Acronyms and Abbreviations 18

KPI Terms 18

System Terms 19

Content Terms 22

Technical Acronyms 23

1.4. References 24

2. Overall Description 27

2.1. Product perspective 27

2.2. Product Functions 32

2.3. User characteristics 34

2.4. Constraints 36

2.5. Assumptions and dependencies 39

3. Specific Requirements 41

3.1. External interface requirements 41

3.1.1. Hardware interfaces 41

3.1.2. Software interfaces 42

3.1.3. Communications interfaces 48

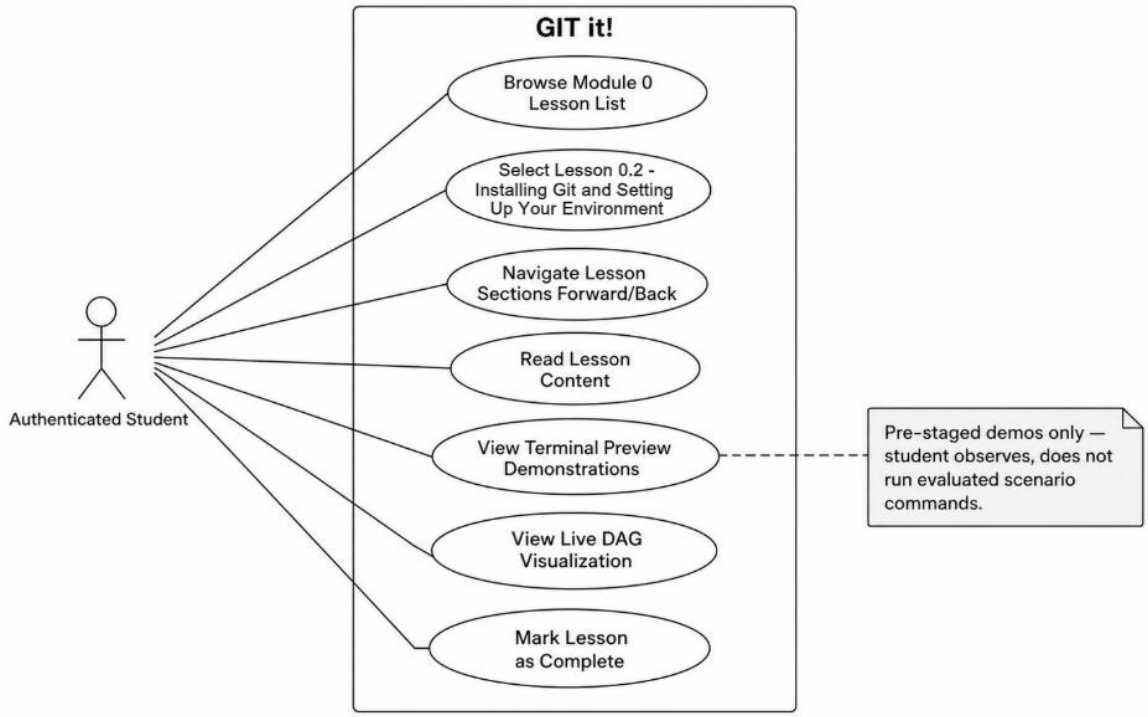
3.2. Functional requirements 49

System Access 50

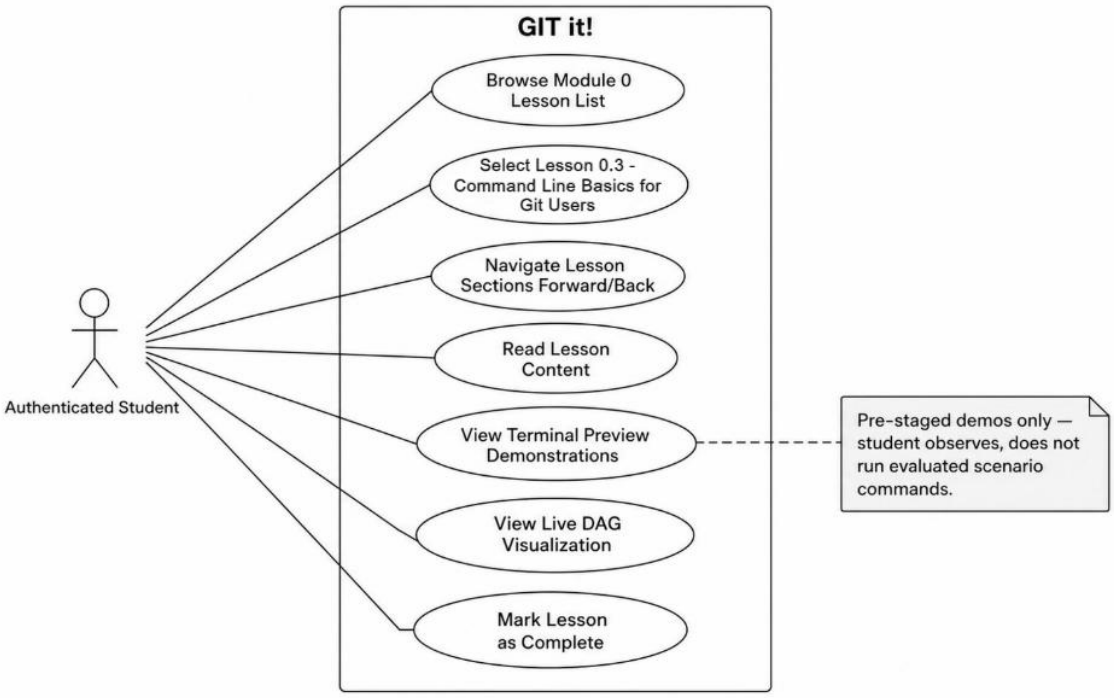
Functional Requirements Table 50

Module 0: Orientation 65

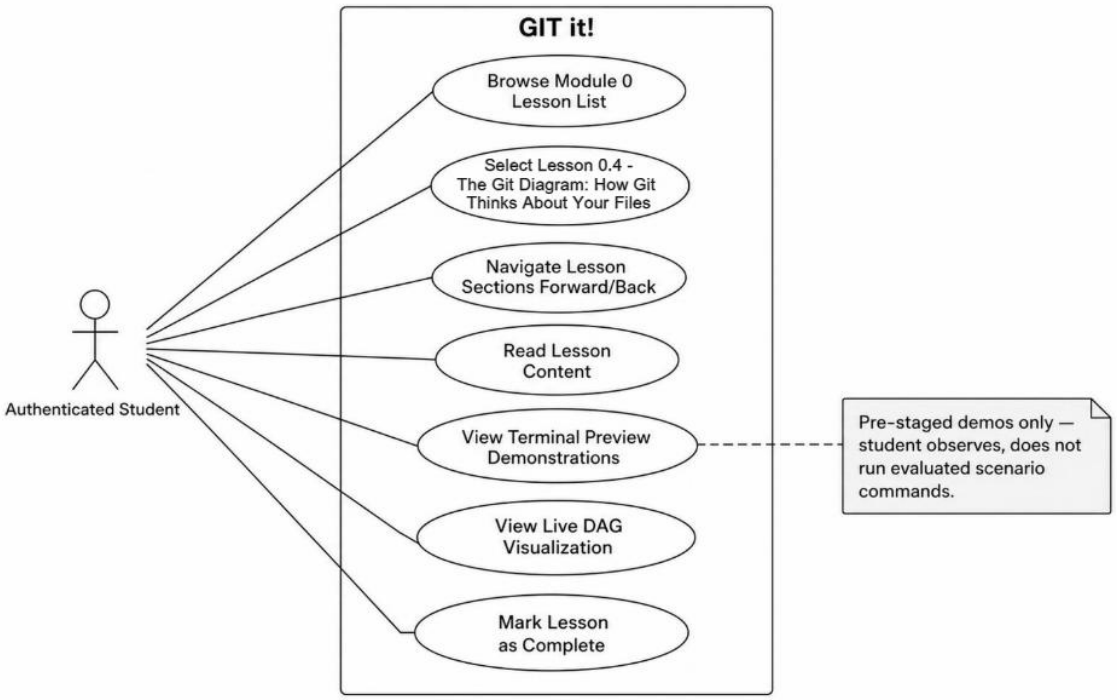
Functional Requirements Table 65


GIT it!
Browse Module 0 Lesson List
Select Lesson 0.2 - Installing Git and Setting Up Your Environment
Navigate Lesson Sections Forward/Back
Read Lesson Content
View Terminal Preview Demonstrations
View Live DAG Visualization
Mark Lesson as Complete
Authenticated Student
Pre-staged demos only — student observes, does not run evaluated scenario commands.

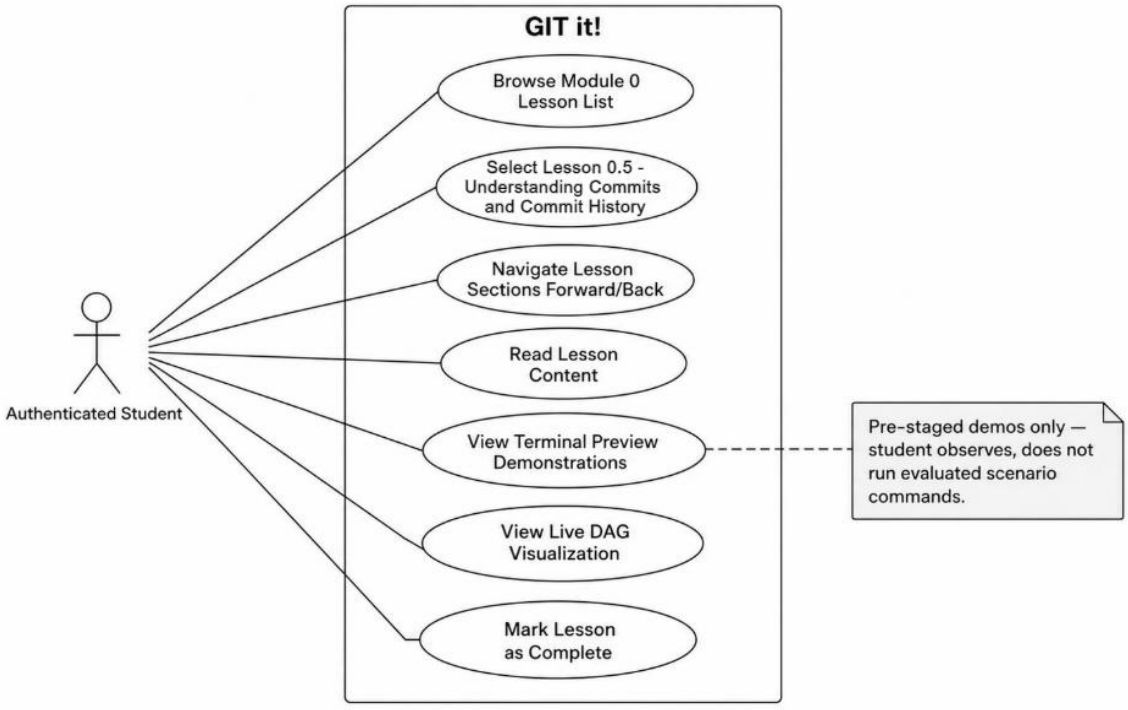
.69


GIT it!
Browse Module 0 Lesson List
Select Lesson 0.3 - Command Line Basics for Git Users
Navigate Lesson Sections Forward/Back
Read Lesson Content
View Terminal Preview Demonstrations
View Live DAG Visualization
Mark Lesson as Complete
Authenticated Student
Pre-staged demos only — student observes, does not run evaluated scenario commands.

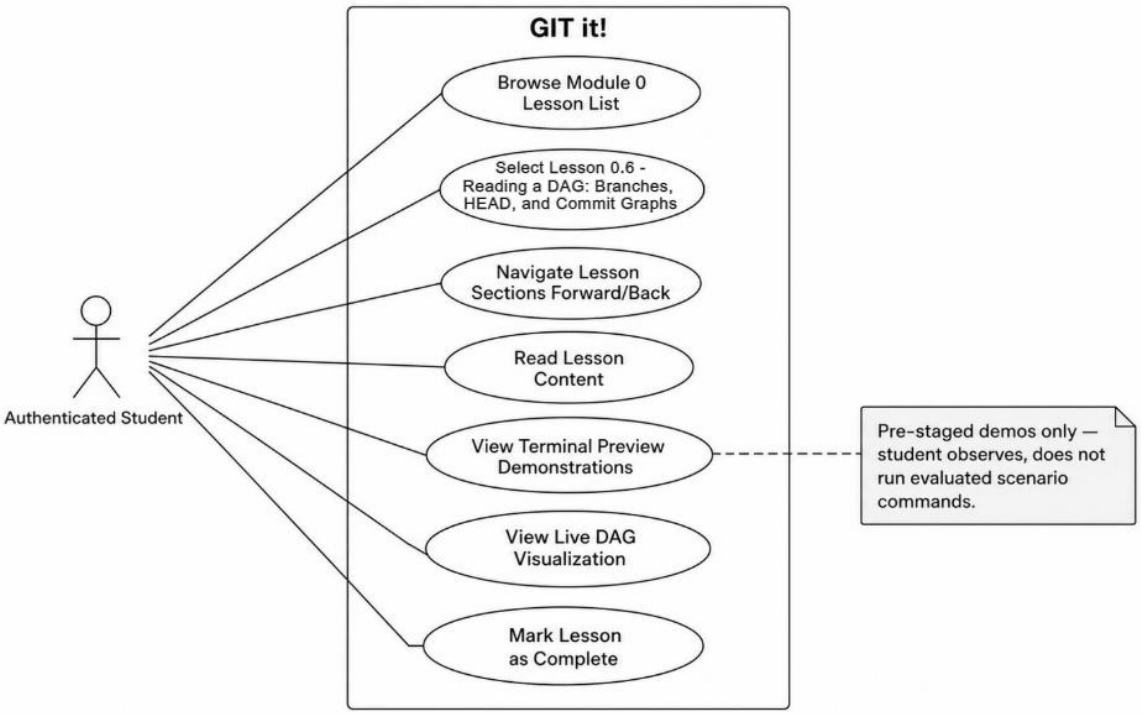
.....73


GIT it!
Browse Module 0 Lesson List
Select Lesson 0.4 - The Git Diagram: How Git Thinks About Your Files
Navigate Lesson Sections Forward/Back
Read Lesson Content
View Terminal Preview Demonstrations
View Live DAG Visualization
Mark Lesson as Complete
Authenticated Student
Pre-staged demos only — student observes, does not run evaluated scenario commands.

.....77

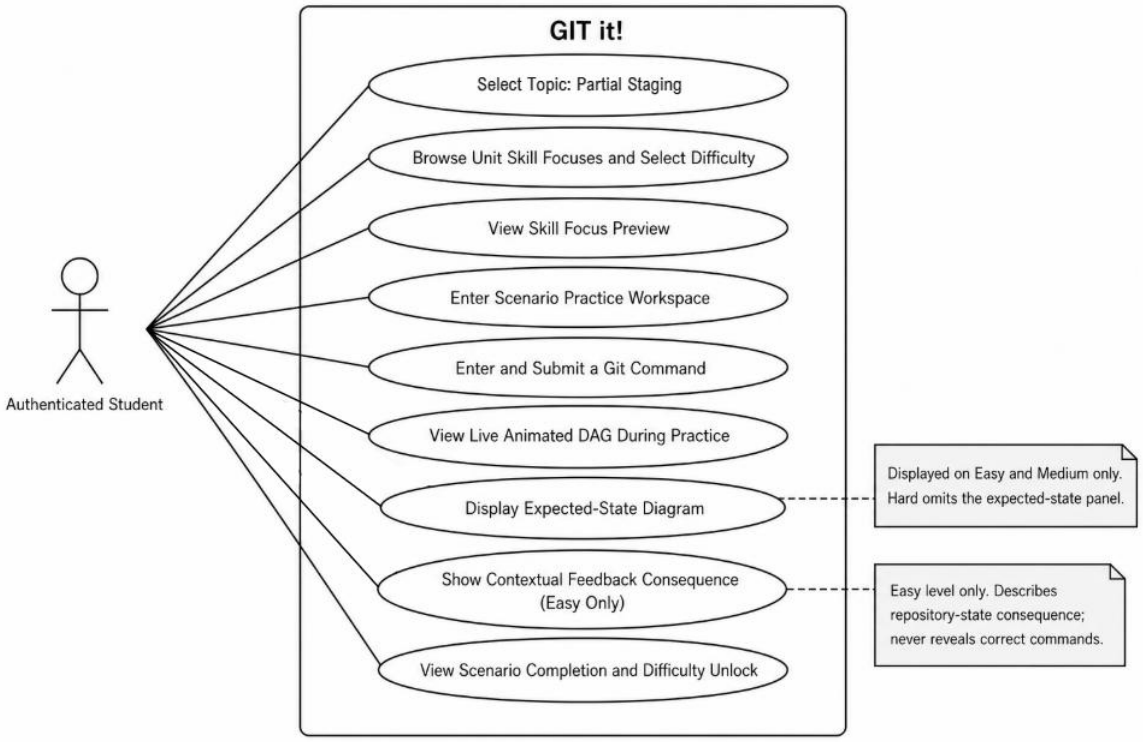

GIT it!
Browse Module 0 Lesson List
Select Lesson 0.5 - Understanding Commits and Commit History
Navigate Lesson Sections Forward/Back
Read Lesson Content
View Terminal Preview Demonstrations
View Live DAG Visualization
Mark Lesson as Complete
Authenticated Student
Pre-staged demos only — student observes, does not run evaluated scenario commands.

....81

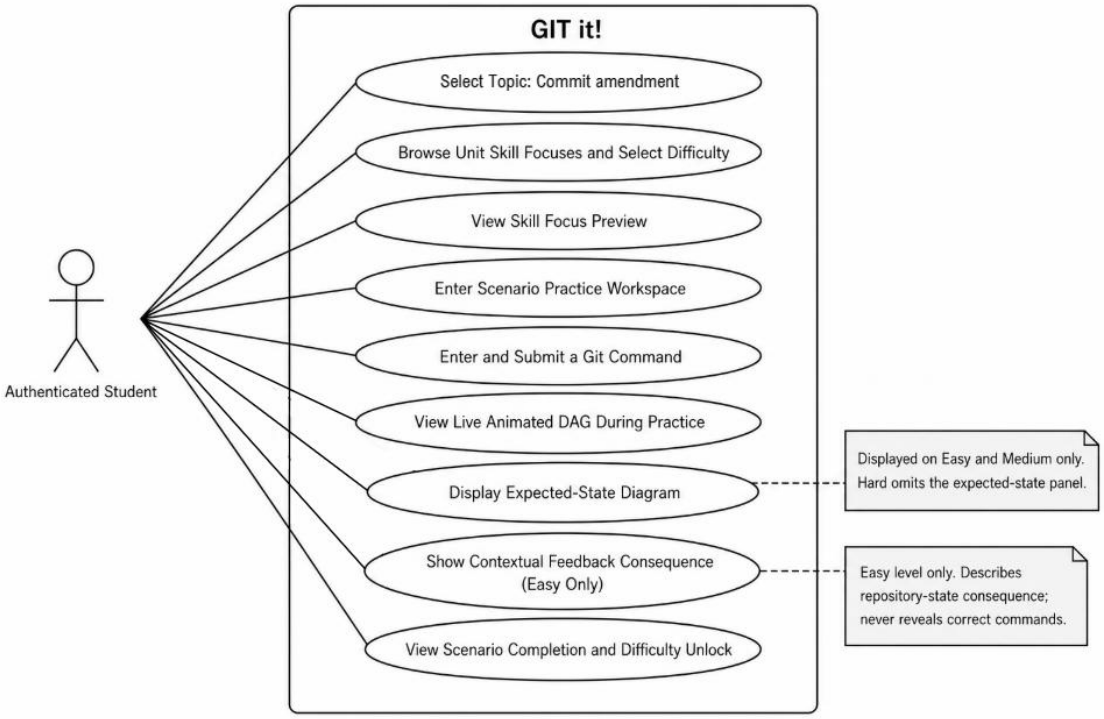


..85

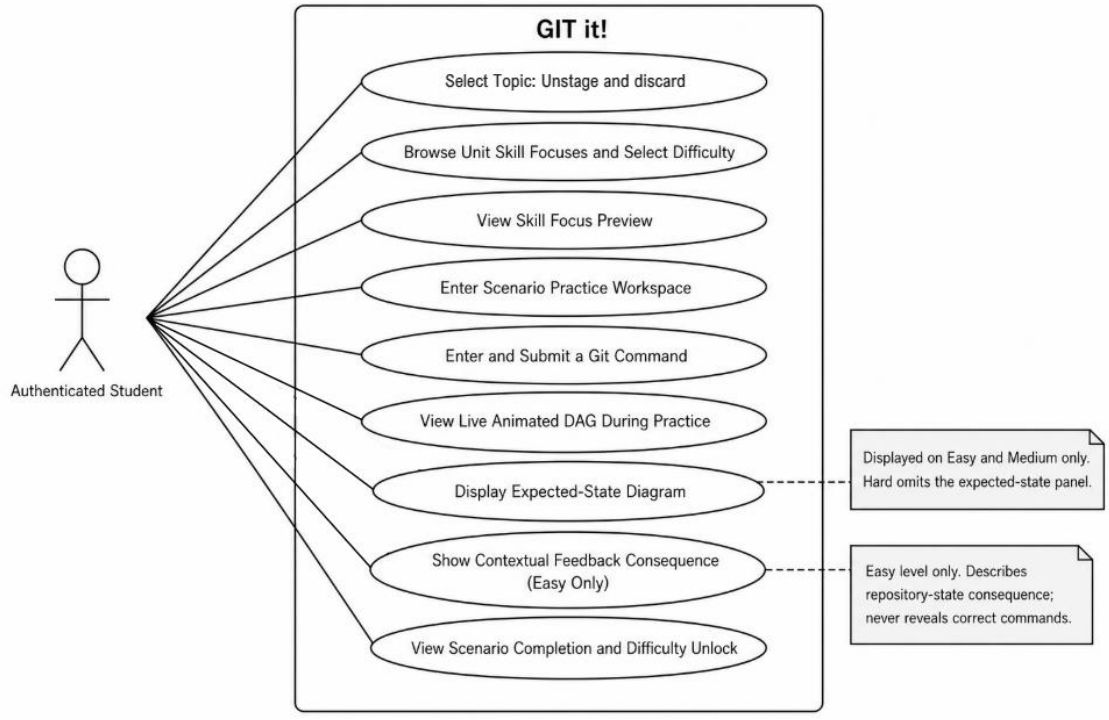
Module 1 — Local Repository Foundations 98



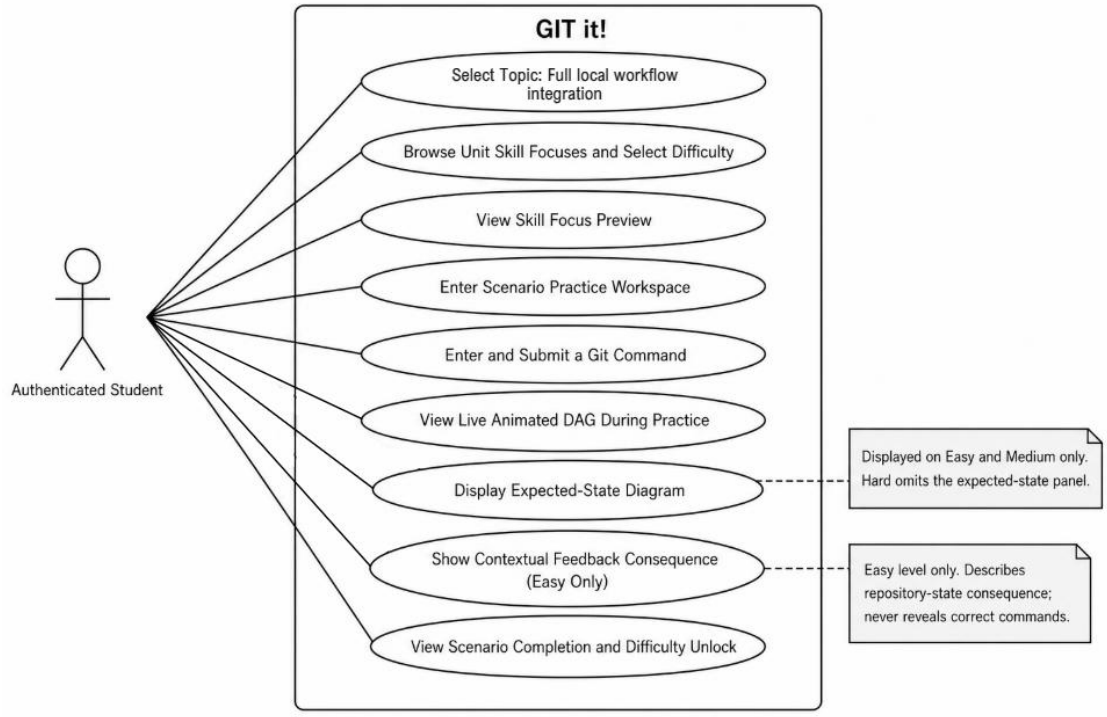
..116


GIT it!
Select Topic: Commit amendment
Browse Unit Skill Focuses and Select Difficulty
View Skill Focus Preview
Enter Scenario Practice Workspace
Enter and Submit a Git Command
View Live Animated DAG During Practice
Display Expected-State Diagram
Show Contextual Feedback Consequence (Easy Only)
View Scenario Completion and Difficulty Unlock
Authenticated Student
Displayed on Easy and Medium only. Hard omits the expected-state panel.
Easy level only. Describes repository-state consequence; never reveals correct commands.

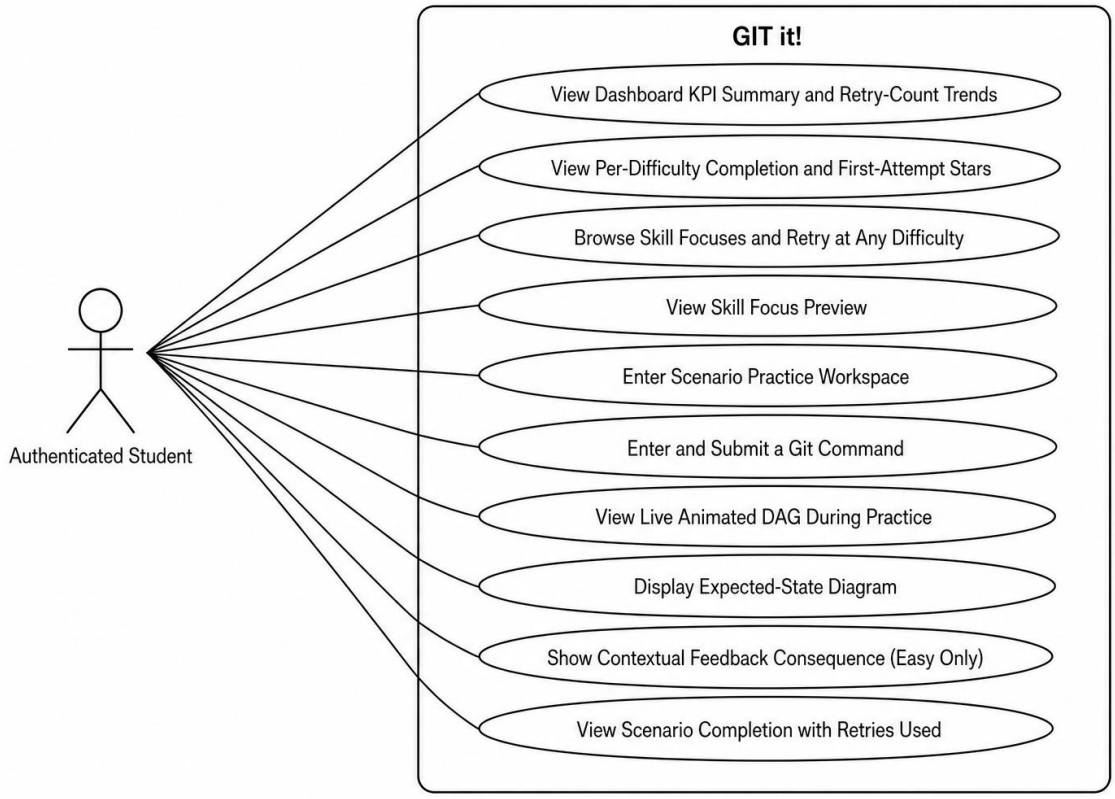
..... 122


GIT it!
Select Topic: Unstage and discard
Browse Unit Skill Focuses and Select Difficulty
View Skill Focus Preview
Enter Scenario Practice Workspace
Enter and Submit a Git Command
View Live Animated DAG During Practice
Display Expected-State Diagram
Show Contextual Feedback Consequence (Easy Only)
View Scenario Completion and Difficulty Unlock
Authenticated Student
Displayed on Easy and Medium only. Hard omits the expected-state panel.
Easy level only. Describes repository-state consequence; never reveals correct commands.

..... 128

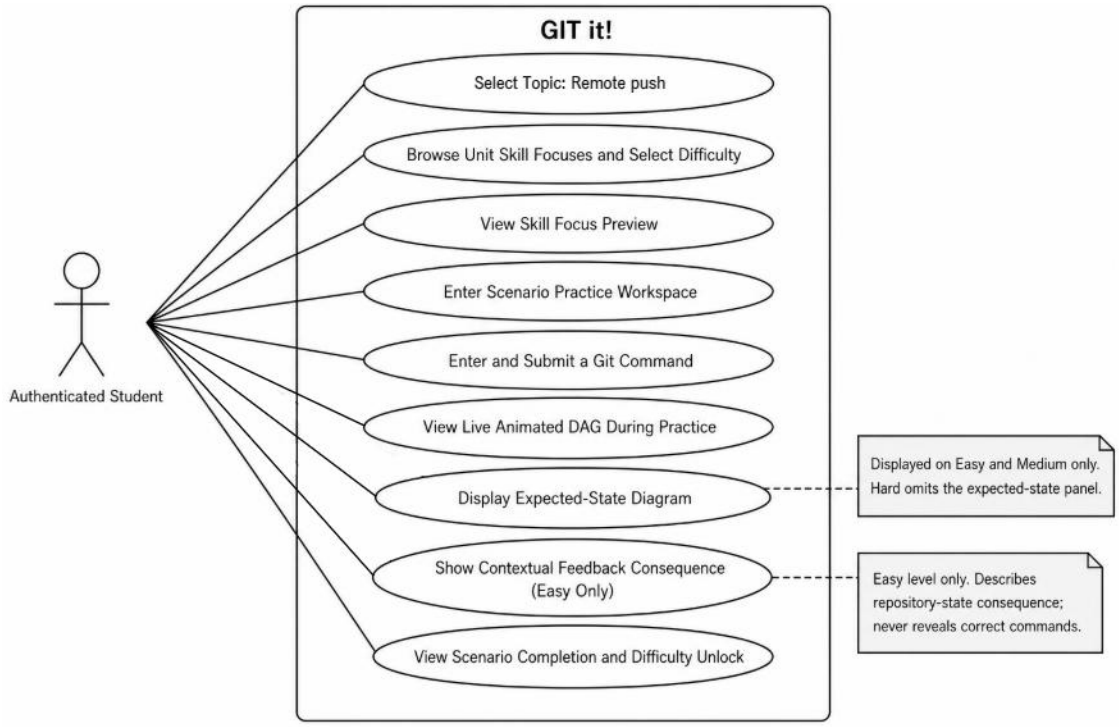


..... 134

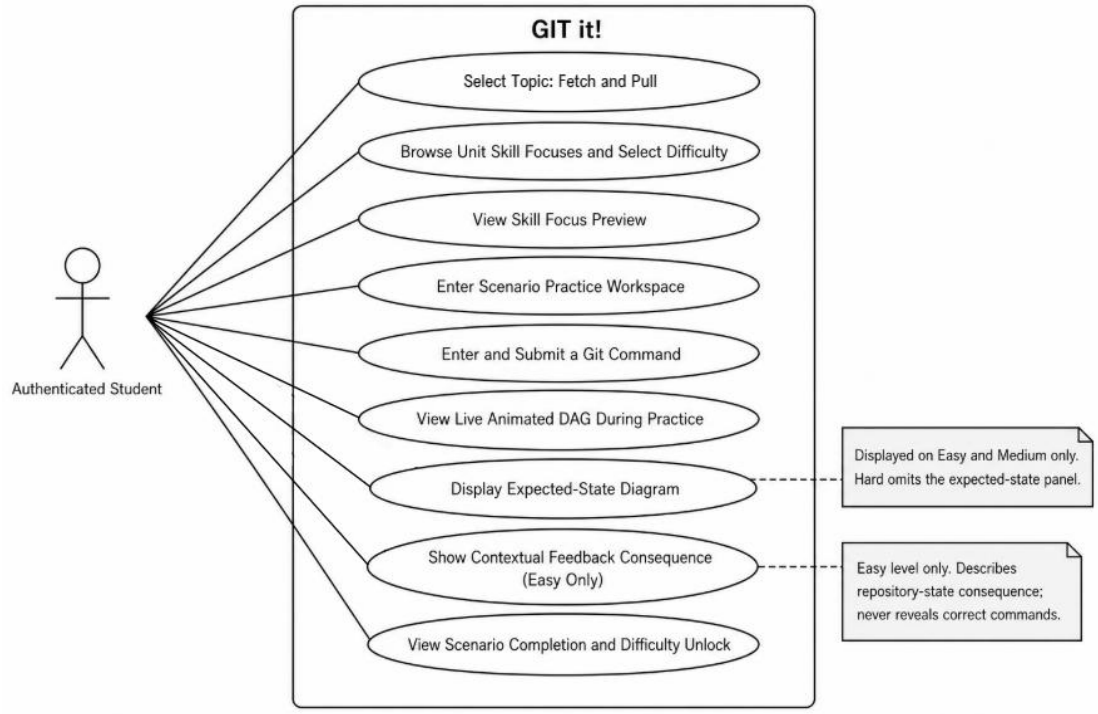


..... 137

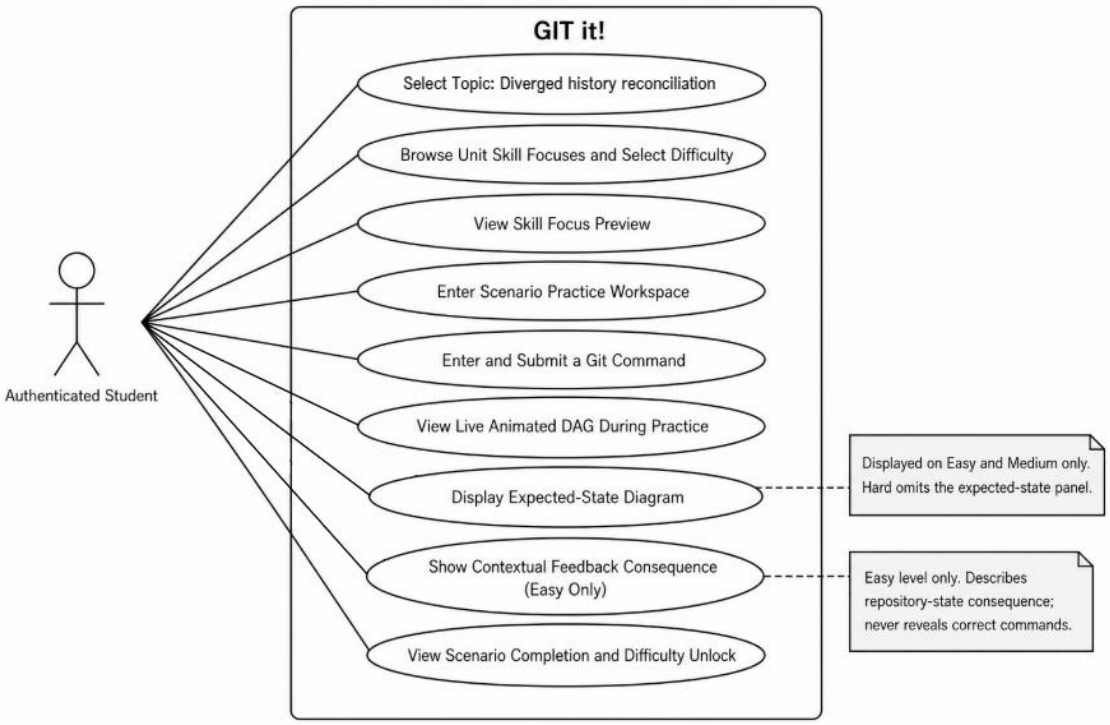
Module 2 — Branching and Collaboration 141


GIT it!
Select Topic: Remote push
Browse Unit Skill Focuses and Select Difficulty
View Skill Focus Preview
Enter Scenario Practice Workspace
Enter and Submit a Git Command
View Live Animated DAG During Practice
Display Expected-State Diagram
Show Contextual Feedback Consequence (Easy Only)
View Scenario Completion and Difficulty Unlock
Authenticated Student
Displayed on Easy and Medium only. Hard omits the expected-state panel.
Easy level only. Describes repository-state consequence; never reveals correct commands.

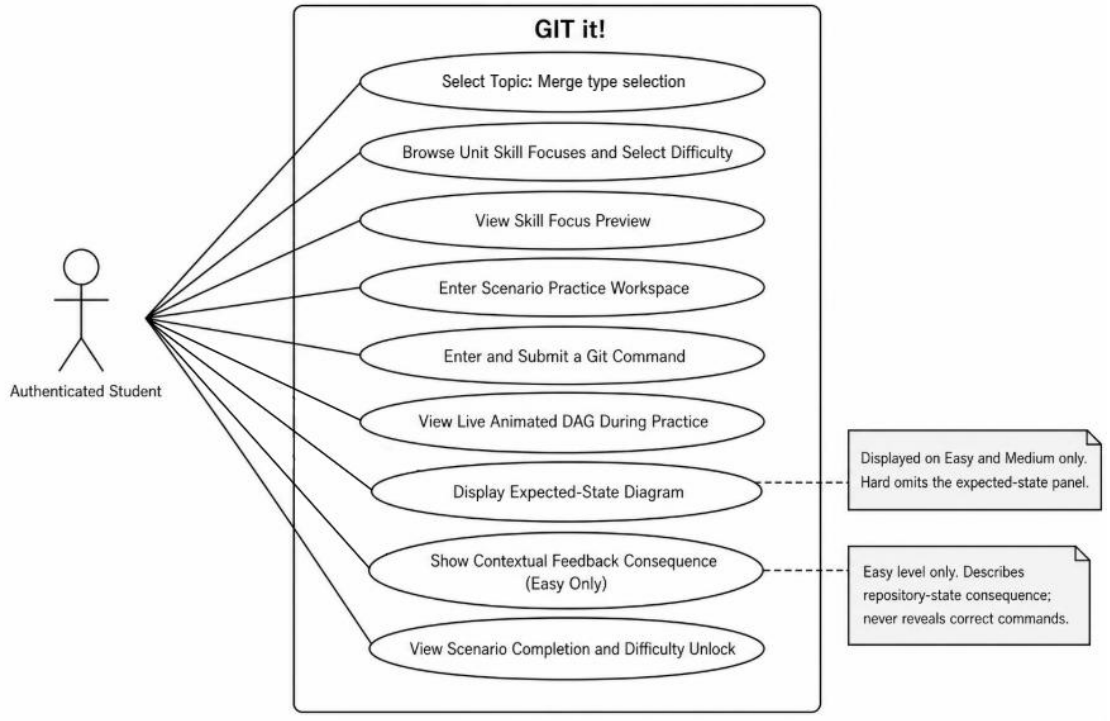
..... 160


GIT it!
Select Topic: Fetch and Pull
Browse Unit Skill Focuses and Select Difficulty
View Skill Focus Preview
Enter Scenario Practice Workspace
Enter and Submit a Git Command
View Live Animated DAG During Practice
Display Expected-State Diagram
Show Contextual Feedback Consequence (Easy Only)
View Scenario Completion and Difficulty Unlock
Authenticated Student
Displayed on Easy and Medium only. Hard omits the expected-state panel.
Easy level only. Describes repository-state consequence; never reveals correct commands.

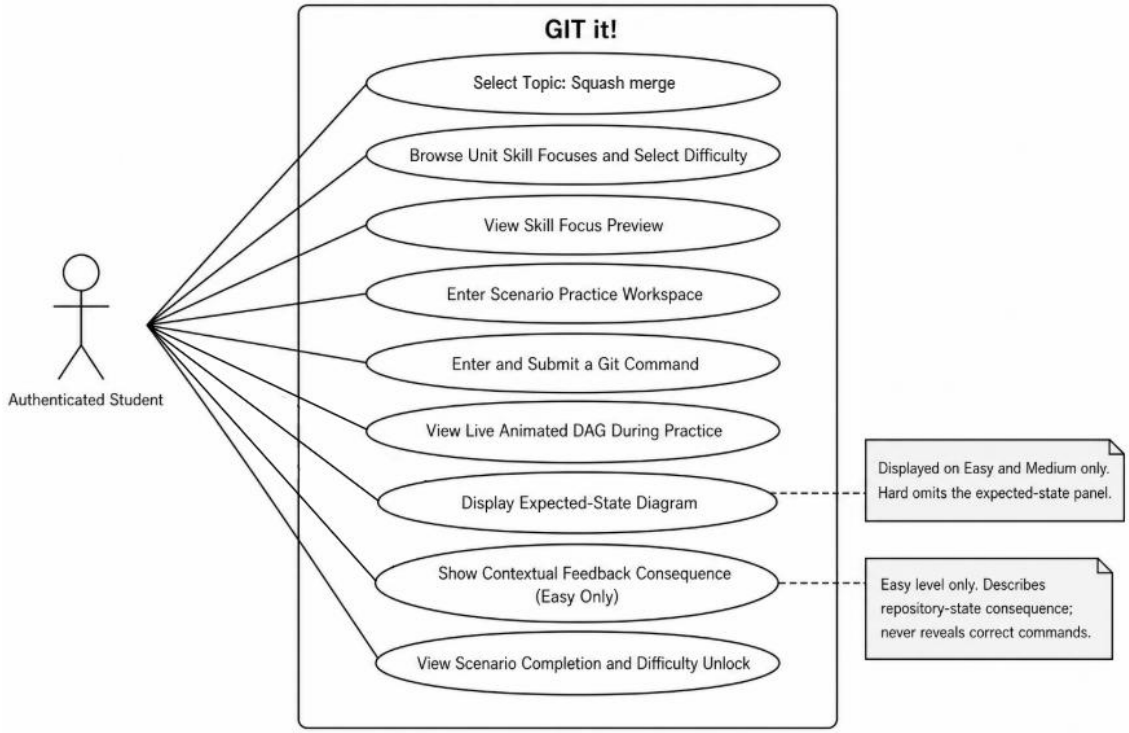
..... 167


GIT it!
Select Topic: Diverged history reconciliation
Browse Unit Skill Focuses and Select Difficulty
View Skill Focus Preview
Enter Scenario Practice Workspace
Enter and Submit a Git Command
View Live Animated DAG During Practice
Display Expected-State Diagram
Show Contextual Feedback Consequence (Easy Only)
View Scenario Completion and Difficulty Unlock
Authenticated Student
Displayed on Easy and Medium only. Hard omits the expected-state panel.
Easy level only. Describes repository-state consequence; never reveals correct commands.

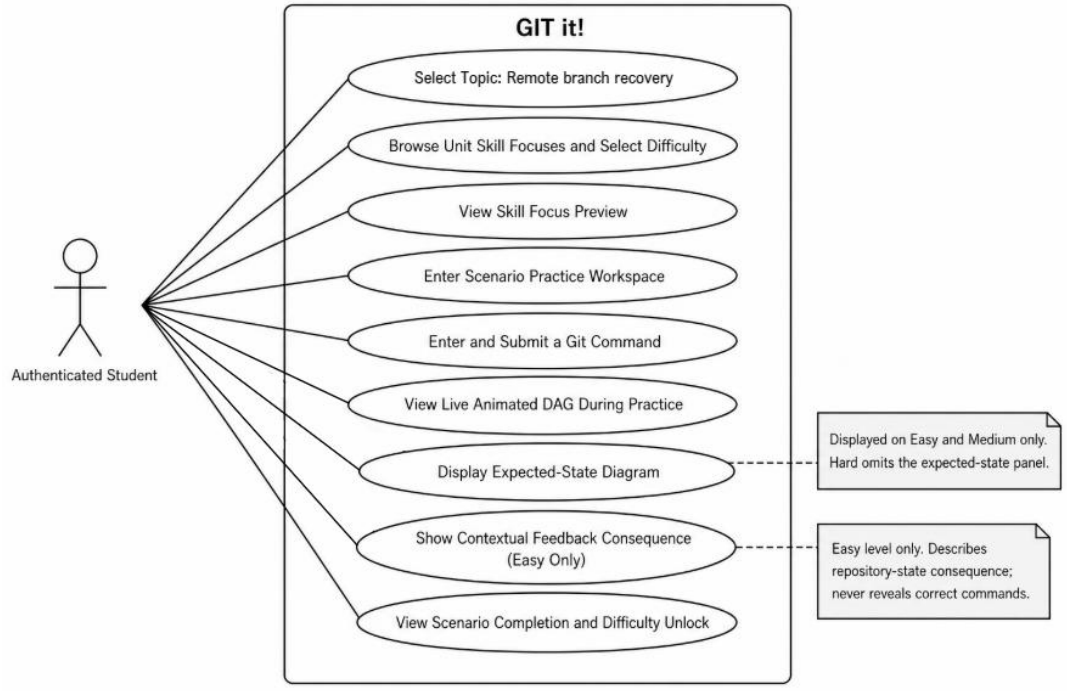
..... 173


GIT it!
Select Topic: Merge type selection
Browse Unit Skill Focuses and Select Difficulty
View Skill Focus Preview
Enter Scenario Practice Workspace
Enter and Submit a Git Command
View Live Animated DAG During Practice
Display Expected-State Diagram
Show Contextual Feedback Consequence (Easy Only)
View Scenario Completion and Difficulty Unlock
Authenticated Student
Displayed on Easy and Medium only. Hard omits the expected-state panel.
Easy level only. Describes repository-state consequence; never reveals correct commands.

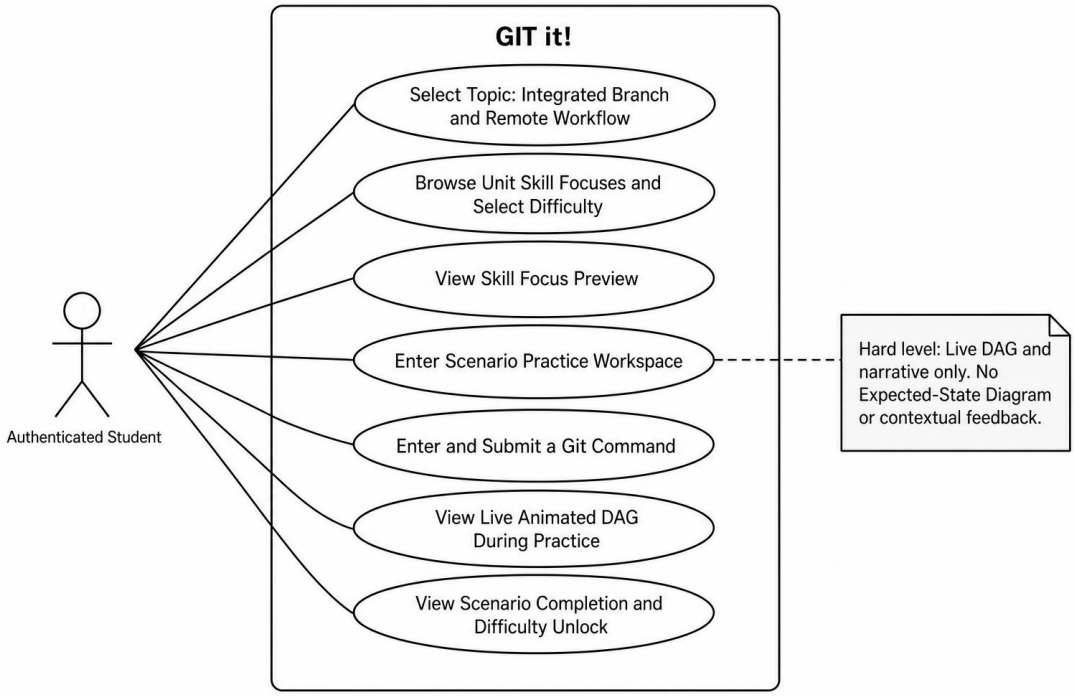
..... 179


GIT it!
Select Topic: Squash merge
Browse Unit Skill Focuses and Select Difficulty
View Skill Focus Preview
Enter Scenario Practice Workspace
Enter and Submit a Git Command
View Live Animated DAG During Practice
Display Expected-State Diagram
Show Contextual Feedback Consequence (Easy Only)
View Scenario Completion and Difficulty Unlock
Authenticated Student
Displayed on Easy and Medium only. Hard omits the expected-state panel.
Easy level only. Describes repository-state consequence; never reveals correct commands.

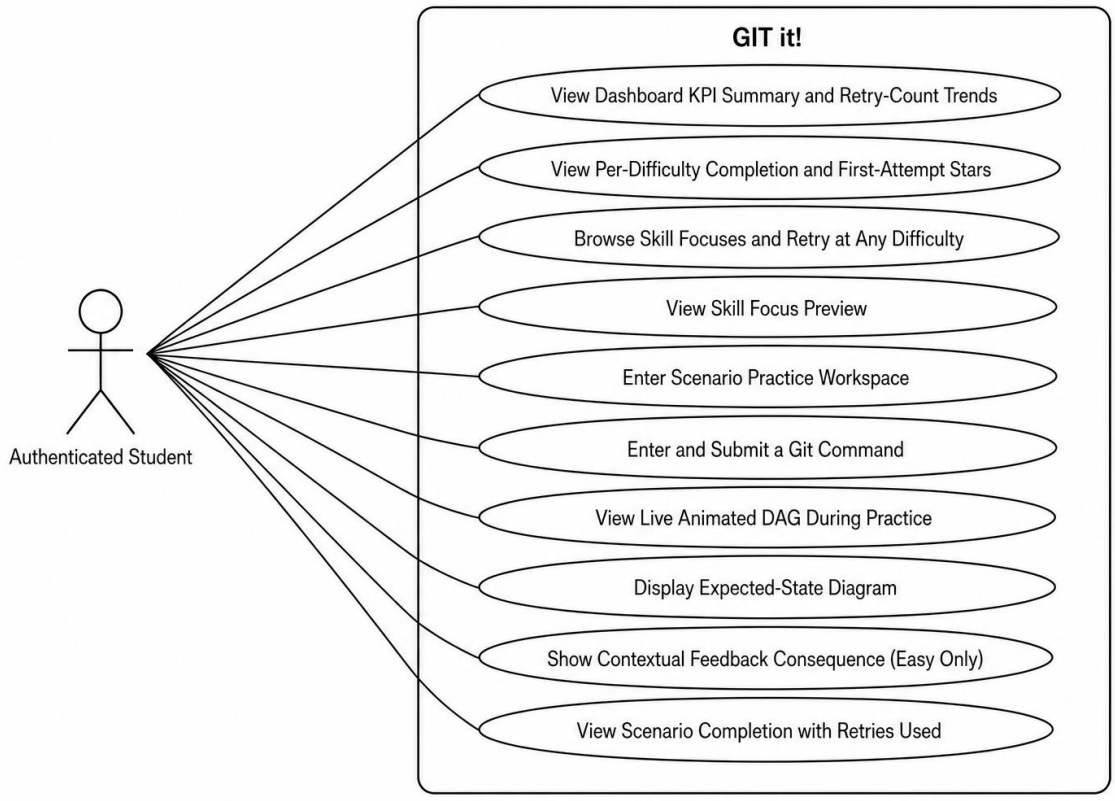
..... 185


GIT it!
Select Topic: Remote branch recovery
Browse Unit Skill Focuses and Select Difficulty
View Skill Focus Preview
Enter Scenario Practice Workspace
Enter and Submit a Git Command
View Live Animated DAG During Practice
Display Expected-State Diagram
Show Contextual Feedback Consequence (Easy Only)
View Scenario Completion and Difficulty Unlock
Authenticated Student
Displayed on Easy and Medium only. Hard omits the expected-state panel.
Easy level only. Describes repository-state consequence; never reveals correct commands.

..... 191

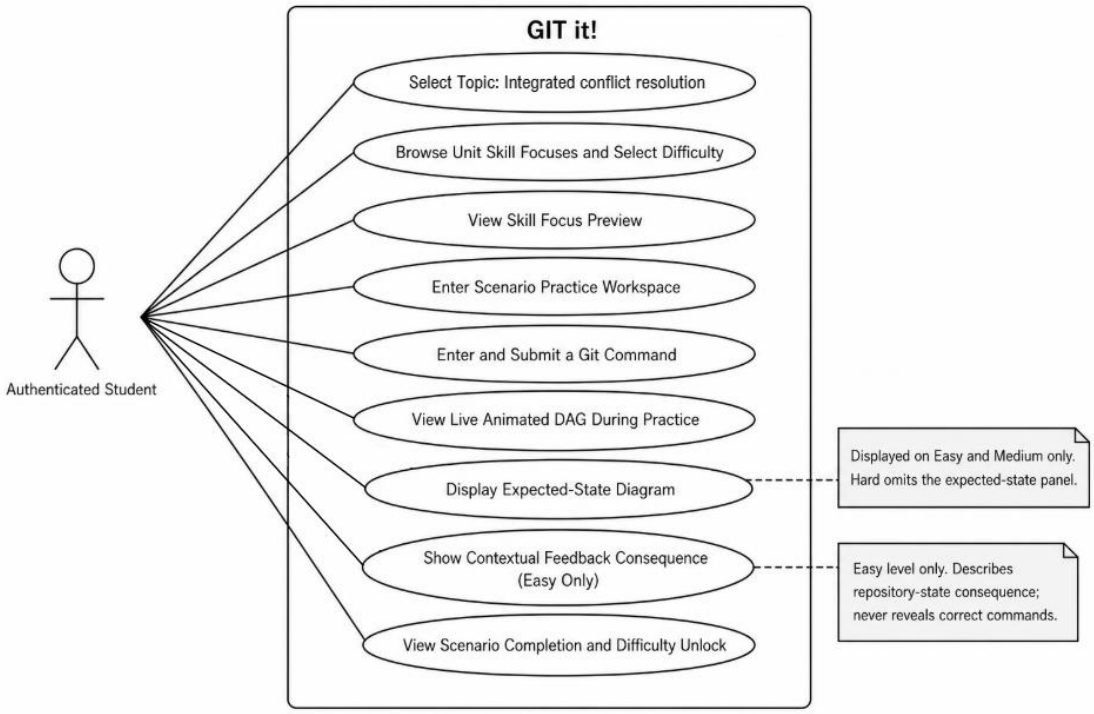


..... 197

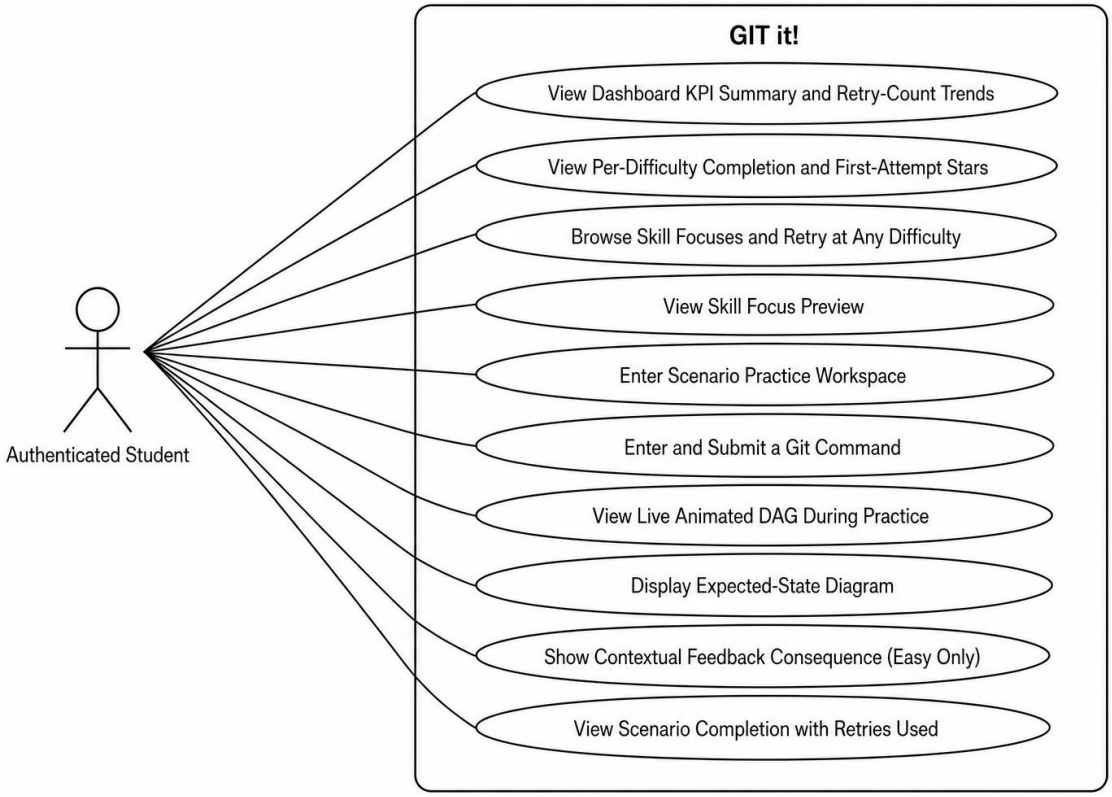


..... 201

Module 3 — Conflict Resolution 205

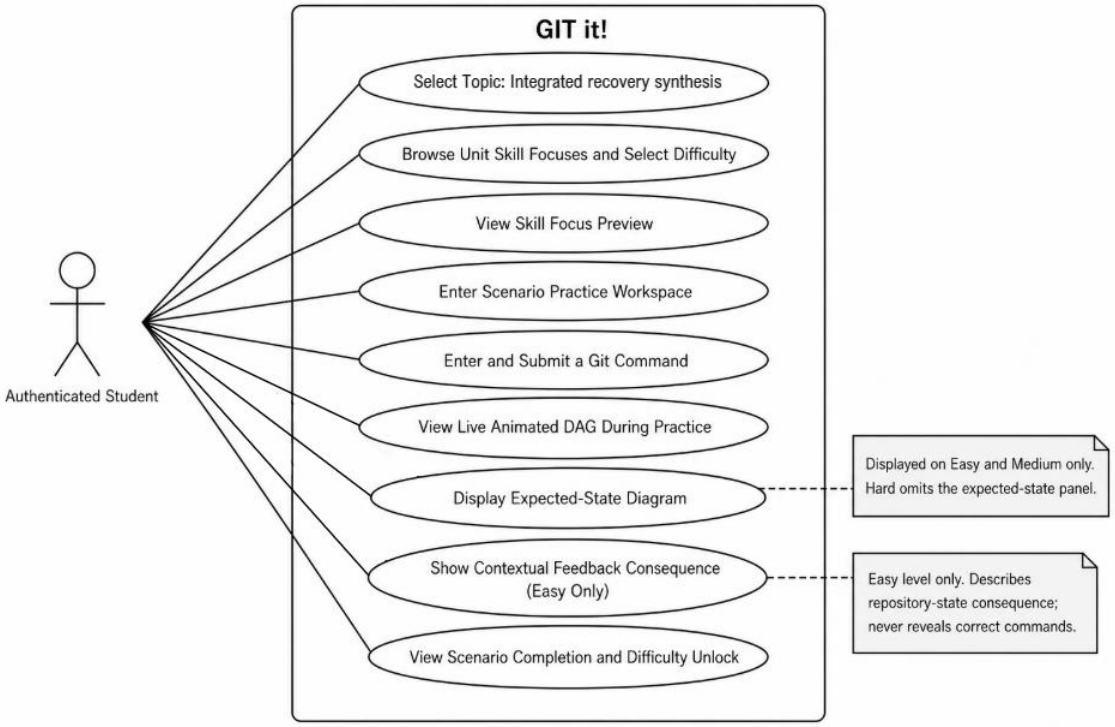


..... 223

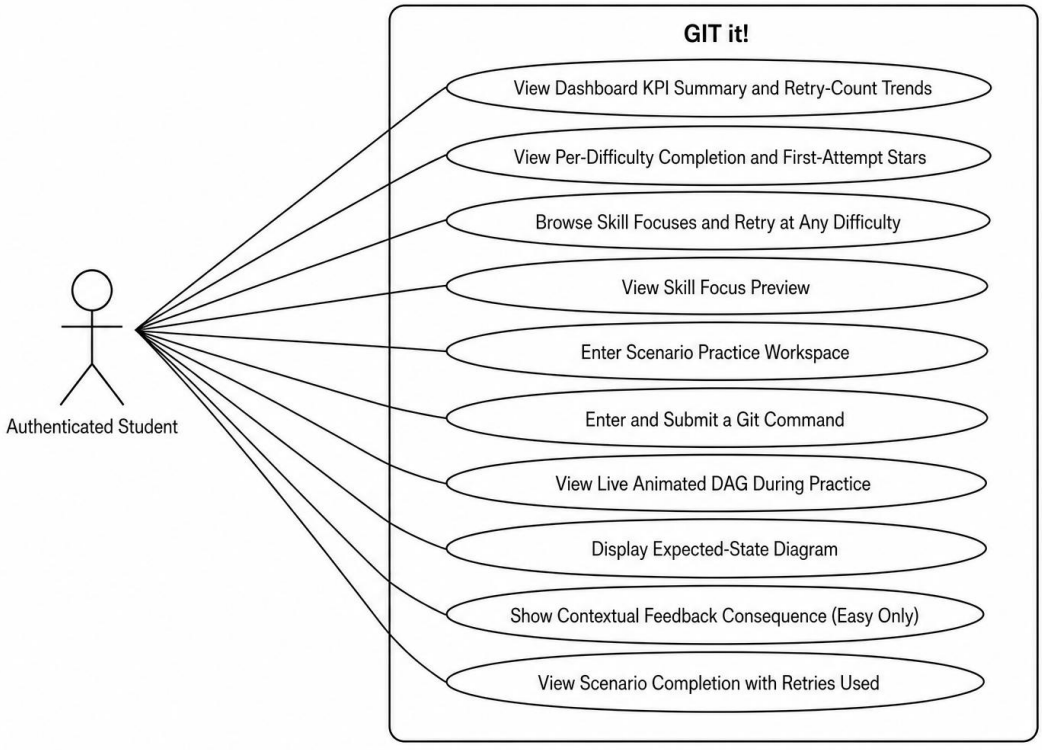


..... 226

Module 4 — Advanced Recovery and History 230



..... 248



251

Cross-Cutting Functional Requirements 253

Administrative Management 254

AI-Assisted Learning Support 258

3.3. Non-functional requirements 266
Performance 266
Performance target revisions (version 1.4) 267
Security 268
Reliability and Availability 270
Usability and Accessibility 271
Maintainability 272
Research Validation Targets 272
Appendix A: Needs Assessment Survey Summary 274
A.1. Overview 274
A.2. Respondent Profile 274
A.3. Key Findings 274
A.3.1 Git Usage Context 274
A.3.2 Error Response Behavior 274
A.3.3 Common Mistakes in Team Projects 275
A.3.4 Conflict and Panic Situations 275
A.3.5 How Problems Were "Resolved" 276
A.4. Student-Identified Learning Needs 276
Student-Identified Learning Needs 276
A.5. Connection to System Requirements 277
Appendix B: SUS Questionnaire Instrument 279
B.1. Overview 279
B.2. Instrument 279
Scoring Method 280
Appendix C: TAM Questionnaire Instrument 281
C.1. Overview 281
Scoring Targets 281
C.2. Instrument 281
Perceived Usefulness (PU) 281
Perceived Ease of Use (PEOU) 282
Behavioral Intention to Use (BI) 282
Scoring Method 283

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for GIT it!, a gamified, web-based Git learning **platform** developed for IT411 at Cebu Institute of Technology – University. It describes stories and chapters, orientation and lessons, adventure encounters and challenge trials, simulated repository practice, player progression, rewards, administration, and measurable learning outcomes for the capstone evaluation.

This document is the requirements baseline for the development team, project adviser, Capstone 2 evaluators, and project stakeholders. Revision 1.1 records proposed requirement changes needed to align the baseline with the implemented system. The project adviser and evaluation team should review these proposed changes before treating them as approved research criteria. Revision 1.3 aligns the non-functional requirements, external interfaces and functional requirements with the system deployed on 30 October 2026.

1.2. Scope

GIT it! is a self-directed, gamified Git learning platform. Learners navigate stories and chapters, complete orientation and lessons, rehearse with drills, and solve repository scenarios in guided adventures and independent challenges. A browser-side Git simulator gives immediate feedback; the backend validates supported state transitions and controls persisted progress, completion, and rewards. A live repository graph makes Git state visible. The platform does not invoke a real Git executable or pass student input to an operating-system shell. Guided command introductions may display command syntax and examples; this implemented behavior is recorded as a proposed amendment to the earlier absolute Guided Teaching Policy.

Full System Scope

The complete system supports the following core functionalities:

- Student registration with username, email, password confirmation, email verification by one-time code, and player setup; sign-in with a password or with GitHub or Google; students and staff use the shared sign-in interface, with staff access controlled by role
- Module 0 Orientation (optional introductory content covering Git concepts, DAG literacy, and platform conventions; students are encouraged but not required to complete it before proceeding to scenario practice)
- Home with player profile, progress, results, activity, mastery, and performance summaries
- Stories and chapter maps with available learning paths, chapter progress, adventure levels, challenge trials, orientation, and drill entry points
- Lesson Pages containing pre-authored instructional content, command references, examples, callouts, and optional DAG illustrations for supported learning content
- Command-form previews and teaching introductions that present Git syntax, examples, and contextual command guidance
- Adventure and challenge workspaces with simulated Git terminal, repository graph, project files, task context, counted-command allowance, and difficulty-appropriate guidance. File workspace actions are simulated and are verified by the backend before they affect accepted state.

- Per-level Easy, Medium, and Hard tiers made of ordered scenarios; a tier completes when every scenario is cleared and unlocks the next tier. Within a chapter, levels open in order, and each module closes with a Challenge Gate
- State-based evaluation accepts supported transitions that reach the authored completion conditions. Failed runs may be retried with an alternate eligible variant; replay is recorded separately from primary performance measures.
- Replay practice after a tier has a completion record; replay activity is excluded from primary performance metrics and reward eligibility.
- AI-assisted learning support, in which the equipped companion acts as a Git tutor on Home, the story map, the shop, and learner-owned command guides, with protected conceptual teaching and explicit recovery states (UC-AI-01)
- Staff administration includes the KPI overview, learner management and moderation, the GitCoin economy ledger, story and chapter publishing, analytics, and feature settings. Admin actions are recorded in an admin action log, and the KPI overview lists the most recent entries.
- Continuous session, step, command, completion, and progress logging for all KPI computation

GIT it! does not connect to external Git hosting services or execute student input through a real Git binary or operating-system shell. The browser runs the supported Git simulation for immediate interaction. The backend normalizes submitted state, validates command metadata and transitions, evaluates authored completion rules, and authoritatively persists accepted progress and rewards.

The following are explicitly **out of scope** for the full system:

- Institutional LMS integration or multi-institution deployment
- Offline mode or progressive web app functionality
- Social or collaborative features between students (shared sessions, peer review)
- Native mobile application (iOS or Android)
- Pre-test or post-test assessment instruments

MVP Core Implementation Scope

The current release provides the implemented learner experience across the Runebound Turret's Modules 0–4, including orientation, story and chapter navigation, adventure and challenge practice, drills, profile and progress views, rewards, and the companion shop.

AI-assisted learning support is implemented as a companion-based Git tutor (AI.1); its runtime availability depends on provider credentials, the scenario index, and network access. Implemented staff functions include the KPI overview, learner management, the economy ledger, story and chapter publishing, analytics, and feature settings; admin actions are recorded in an admin action log.

- Module 0 Orientation (optional introductory content covering Git concepts, DAG literacy, and platform conventions; students are encouraged but not required to complete it before proceeding to scenario practice)
- AI-Assisted Learning Support

The starter curriculum is seeded before deployment. Staff can publish or unpublish stories and chapters; scenario content changes go through the curriculum source and its validation guards.

Implementation status note — Stories, chapters, orientation, adventures, challenges, drills, the companion shop, learner progress, staff KPI overview, learner management, story and chapter publishing, and the the AI tutor tutor are represented in the current system.

Organizational Objective

GIT it! is designed to reduce the incidence of destructive Git workaround behaviors among BSIT students in team-based development projects where version control proficiency is required, and to provide log-derived KPI data that quantifies Git competency development without requiring manual assessment instruments.

1.3. Definitions, Acronyms and Abbreviations

KPI Terms

Term	Definition
ARC — Average Retry Count	Primary KPI. Failed or meaningfully abandoned attempts before first mastery, summed over mastered tiers and divided by the number of mastered tiers. Successful repetitions are not retries. Target: ≤ 2 (Modules 1–3); ≤ 3 (Module 4).
CAR — Command Accuracy Rate	Primary KPI. Benchmark-relative command efficiency: the mean of 100 × S × min(1, T / max(1, A)) over assessed runs, where S is 1 for a successful run, T is the validated reference count of counted commands and A is the run's counted commands, including incorrect input. The assessed run is the first run that achieves mastery of a tier. Target: ≥ 70%.
HLCR — Hard-Level Completion Rate	Primary KPI. Mastered Hard tiers ÷ attempted Hard tiers × 100. Measures students' ability to resolve scenarios with minimal scaffolding. Target: ≥ 70% (Modules 1–3); ≥ 65% (Module 4).
KPI	Key Performance Indicator. A metric computed exclusively from system-generated run, command and checkpoint records, used to evaluate learning outcome achievement.

RTA — Retry Transfer Accuracy	Primary KPI for Modules 3–4. Transfer checkpoints cleared on their first run ÷ eligible novel transfer checkpoints × 100. A transfer checkpoint serves a reserved, structurally different variant the learner has not seen. Measures skill transfer over memorization. Target: ≥ 65%.
SCR — Scenario Completion Rate	Validity gate KPI. Required tiers mastered ÷ unique required tiers attempted × 100. Target: ≥ 80% per module.

System Terms

Term	Definition
AI-Assisted Learning Support	The authenticated, companion-based Git tutor (AI Tutor) available on Home, the story map, the shop, and within command guides for learner-owned runs (UC-AI-01).
AI Tutor	The equipped companion's chat help that answers Git, version-control, and verified platform questions and provides protected conceptual guidance within command guides.
FAQS	Saved Git concept explanations in the tutor window that are available without a provider request.
Guide Token	A signed token that binds a player, run, and teaching key to command-guide help; it cannot execute commands, complete runs, or award progress.
Protected Conceptual Teaching	Tutor guidance that explains concepts and placeholder syntax without supplying scenario-specific values, commands, ordered solutions, or answer confirmation.

Command Adapter	Backend command-processing logic that maps supported student-submitted Git command text to internal simulator behavior. Unrecognized or unsupported commands are returned as invalid or unprocessable without being executed through an operating-system shell.
Completion-Gated Difficulty Progression	The system-enforced difficulty access model within each learning objective. Easy is initially available to the student. Medium unlocks automatically after the student completes the Easy difficulty instance. Hard unlocks automatically after the student completes the Medium difficulty instance. Difficulty instances cannot be accessed out of sequence.
Contextual Feedback Panel	An Easy-difficulty-only interface component that displays a plain-language description of the repository-state consequence of the student's most recent processed command, without labeling the command as correct or incorrect, suggesting the next command, or revealing the correct sequence.
Counted Action Command	A Git command that changes or is intended to change the repository state. Counted toward the counted-command allowance and toward CAR computation. Distinguished from Diagnostic (Non-Counted) Commands.
Diagnostic (Non-Counted) Command	A read-only, state-inspecting Git command (e.g., `git status`, `git log`, `git diff`, `git reflog`, `git branch -v`). Never counted toward the counted-command allowance or CAR. Freely available at all difficulty levels in all scenario sessions.
target-state guidance	A static diagram showing the target repository state the student must reach to complete the current scenario step. Present on Easy and Medium difficulty levels only. Absent at Hard. Does not reveal the command path, only the goal state.
Fading Scaffolding	The pedagogical design principle of progressively removing instructional support as students advance from Easy to Medium to Hard difficulty. Easy provides full scaffolding: repository graph, target-state guidance, contextual feedback panel, and counted-command allowance. Medium removes the contextual feedback panel. Hard removes the target-state guidance, leaving only the repository graph, narrative context, and counted-command allowance.
Hard-Level Scenario	A scenario difficulty instance with minimal scaffolding: repository graph, narrative context, and counted-command allowance only. No target-state guidance, no contextual feedback panel.

repository graph	The real-time repository-state graph rendered in the practice workspace. It displays commit nodes, branch pointers, HEAD position, parent-child edges, and related state information. It updates as soon as the browser simulation processes a command; the backend then verifies the transition.
Guided Teaching Policy	Proposed policy amendment for review: guided introductions may present syntax and worked examples, while scenario-specific solutions remain outside the teaching policy. The AI tutor (AI.1) applies protected conceptual teaching and server-side answer checks, which reduce but do not guarantee prevention of scenario-answer disclosure; the earlier absolute prohibition is not represented as implemented behavior.
Project Structure Panel	A read-only panel within the practice workspace that displays the simulated repository's current file and directory structure. Updates to reflect the current repository state after each processed command. Provides students with file-level context relevant to staging, conflict resolution, and other file-operation scenarios.
counted-command allowance	A visible counter in the practice workspace showing how many counted action commands the student has remaining before the maximum counted-command limit is reached. Present at all difficulty levels.
Repository State Simulator	The browser-side simulation provides immediate repository feedback; the backend validates submitted transitions and controls persisted run state. Does not execute external Git binaries or OS shell commands from student input.
Replay	A playable re-attempt mode available after a completion record exists for the difficulty tier — a difficulty tier with a completion record. Provides the live terminal, live DAG, and difficulty-appropriate scaffolding. Logged separately from primary attempts. Does not reveal correct commands or command sequences.
practice workspace	The level workspace used for adventure tiers, Challenge trials and replays. Used when a requirement or constraint applies equally to both contexts. Command-guide AI help (AI.1) may be used alongside an active practice session; it provides protected conceptual guidance only and does not execute commands.
Session Log	A system-generated record created for each scenario practice attempt, capturing session identity, student identity, scenario and difficulty metadata, session outcome (Completed, Failed, Abandoned, or Review), and timestamps. The primary data source for SCR, HLCR, ARC, and RTA computation.

Command guide	An in-workspace card that teaches the command form for the learner's next move — its usage, a part-by-part reading of its syntax and the change it makes — in the scenario that is that form's home. It replaced the earlier pre-practice level details modal.
State-Based Evaluator	The backend component that compares the normalized post-command repository state snapshot against the expected target state for the current scenario step. Returns TargetMatched or TargetNotYetMatched. Does not perform command-string matching. Accepts any valid command path to the correct target state.
Step Log	A system-generated record created for each command submitted within a scenario session, capturing the submitted command, command classification (Action or Diagnostic), evaluator result, and resulting repository state snapshot. The primary data source for CAR computation.
TargetMatched	The evaluator result indicating that the student's post-command repository state satisfies the expected target state for the current scenario step. Advances the scenario.
TargetNotYetMatched	The evaluator result indicating that the student's post-command repository state does not yet satisfy the expected target state. Does not mark the session as Failed; the student continues from the resulting state.

Content Terms

Term	Definition
Difficulty Instance	One specific difficulty-level version (Easy, Medium, or Hard) of a learning objective. Each carries its own command-count policy, scaffolding configuration, and variant pool.
Lesson Page	A single scrollable instructional page rendered for each chapter. Contains concept explanations, examples, command snippets, callouts, and DAG illustrations. Accessed from the Stories and chapter map. Separate from the mission briefing and the command guide.
Retry Variant	A structurally distinct, pre-authored variation of a scenario difficulty instance served on retry after a Failed or Abandoned session. It differs from the prior attempt in branch names, commit topology, file-state markers, or starting repository state.

Scenario	A self-contained Git problem-solving exercise presented with a professional narrative context, a starting repository state, and one or more required resolution steps.
learning objective	The reusable Git problem pattern targeted by a set of scenario difficulty instances (e.g., Manual conflict resolution, Remote push, Hard reset recovery). Each skill focus provides Easy, Medium, and Hard difficulty instances.
Scenario Variant	A structurally distinct, pre-authored template-based version of a learning objective, differing in branch names, commit structures, file-state markers, or starting repository topology.

Technical Acronyms

Acronym	Expansion
API	Application Programming Interface
BSIT	Bachelor of Science in Information Technology
CIT-U	Cebu Institute of Technology – University
CORS	Cross-Origin Resource Sharing
DAG	Directed Acyclic Graph — the data structure Git uses to represent commit history, where nodes are commits and directed edges point to parent commits
DRF	Django REST Framework
HEAD	In Git: a special pointer that refers to the currently checked-out commit or branch tip
HTTP(S)	HyperText Transfer Protocol (Secure)
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token — a compact, stateless token format used for authentication (RFC 7519)
ORM	Object-Relational Mapper
RA 10173	Republic Act No. 10173 — Philippine Data Privacy Act of 2012
REST	Representational State Transfer
SPA	Single-Page Application

SRS	Software Requirements Specification
SUS	System Usability Scale — a standardized ten-item usability questionnaire; target score ≥ 70
TAM	Technology Acceptance Model
TLS	Transport Layer Security
TTL	Time-To-Live

1.4. References

Ref	Document / Standard	Report No.	Year	Publisher	Source
[1]	IEEE Recommended Practice for Software Requirements Specifications	IEEE Std 830-1998	1998	IEEE Computer Society	https://ieeexplore.ieee.org/document/720574
[2]	Data Privacy Act of 2012	RA 10173	2012	Republic of the Philippines	https://www.privacy.gov.ph/data-privacy-act/
[3]	JSON Web Token (JWT)	RFC 7519	2015	IETF	https://datatracker.ietf.org/doc/html/rfc7519
[4]	Transport Layer Security (TLS) Protocol Version 1.2	RFC 5246	2008	IETF	https://datatracker.ietf.org/doc/html/rfc5246
[5]	React Documentation	—	Online	Meta Open Source	https://react.dev
[6]	Vite Documentation	—	Online	Vite Contributors	https://vitejs.dev
[7]	Django / Django REST Framework Documentation	—	Online	Django Software Foundation / Encode OSS	https://docs.djangoproject.com / https://www.django-rest-framework.org
[8]	PostgreSQL Documentation	—	Online	PostgreSQL Global	https://www.postgresql.org/docs/

				Development Group	
[9]	Supabase Documentation	—	Online	Supabase Inc.	https://supabase.com/docs
[10]	Redis Documentation	—	Online	Redis Ltd.	https://redis.io/docs
[11]	Render Key Value Documentation	—	Online	Render	https://render.com/docs/key-value
[12]	Aligning Software Engineering Education with Industrial Needs: A Meta-Analysis	—	2019	IEEE Access (Garousi et al.)	https://doi.org/10.1109/ACCESS.2019.2936660
[13]	Closing the Gap Between Software Engineering Education and Industrial Needs	—	2020	IEEE Software (Garousi et al.)	https://doi.org/10.1109/MS.2018.2880823
[14]	Analysis of Software Engineering Skills Gap in the Industry	—	2022	ACM TOCE (Akdur)	https://doi.org/10.1145/3567837
[15]	SUS: A Quick and Dirty Usability Scale	—	1996	Taylor & Francis (Brooke)	https://www.taylorfrancis.com/chapters/edit/10.1201/9781498710411-35/sus-john-brooke
[16]	Perceived Usefulness, Perceived Ease of Use, and User Acceptance of IT	—	1989	MIS Quarterly (Davis)	https://doi.org/10.2307/249008
[17]	Challenges and Confusions in Learning Version Control with Git	—	2014	Springer (Isomöttönen & Cochez)	https://doi.org/10.1145/2674683.2674694

[18]	A Behavioral Approach to Understanding the Git Experience	—	2021	HICSS (Milliken et al.)	https://hdl.handle.net/10125/71425
[19]	Cognitive Load Theory	—	2011	Springer (Sweller, Ayres & Kalyuga)	https://doi.org/10.1007/978-1-4419-8126-4
[20]	Effects of Feedback in a Computer-Based Learning Environment	—	2015	Review of Educational Research (Van der Kleij et al.)	https://doi.org/10.3102/0034654314564881
[21]	A Systematic Literature Review of Automated Feedback Generation	—	2019	ACM TOCE (Keuning et al.)	https://doi.org/10.1145/3231711
[22]	Teaching Tip: Rethinking How We Teach Git	—	2025	Journal of Information Systems Education (Wagner & Thurner)	https://aisel.aisnet.org/jise/vol36/iss1/1/
[23]	Gemini API Documentation	---	2026	Google LLC	https://ai.google.dev/gemini-api/docs
[24]	Groq API Documentation	---	2026	Groq, Inc.	https://console.groq.com/docs

2. Overall Description

2.1. Product perspective

Background and Problem Context

Version control is the most consistently cited unmet competency among computing graduates internationally [11][12][13]. Despite Git being the industry standard for collaborative software development, undergraduate programs across multiple geographic and institutional contexts produce graduates who lack reliable proficiency under realistic collaborative conditions. Employers report elevated onboarding costs and technical screening failures directly attributable to version control deficiencies [12][13]. Akdur [13] confirmed that version control ranks among the top deficiencies flagged by industry managers evaluating new hires — a pattern that has remained stable across multiple geographic and institutional contexts.

The root cause is not exposure but practice. Students encounter Git in coursework but rarely practice it under realistic collaborative conditions. Isomöttönen and Cochez [17] documented that novice developers consistently fail to build an accurate mental model of repository state transitions, leading to avoidance behaviors when Git operations produce unexpected results. Milliken et al. [18] further characterized this as a behavioral pattern: students default to destructive workarounds not because they lack awareness of correct procedures, but because they lack the practiced confidence to execute them under pressure.

A preliminary survey of 44 third-year BSIT students confirmed that this gap is acute and behaviorally entrenched within the target population:

- 91% reported pasting Git errors directly into AI tools and executing returned commands without understanding them
- 75% described resolving merge conflicts by discarding a teammate's code entirely rather than resolving the conflict
- 45% deleted the local repository folder and re-cloned from remote to escape an unresolvable repository state
- 16% abandoned the repository entirely and created a new one to resolve a conflict

These behaviors share a common characteristic: students are bypassing the problem rather than resolving it. Each workaround avoids engaging with the repository state directly, which means the underlying skill gap is never addressed. Left unresolved, these patterns carry forward into graded capstone projects and team-based development work where destructive workarounds have real consequences for collaborative codebases.

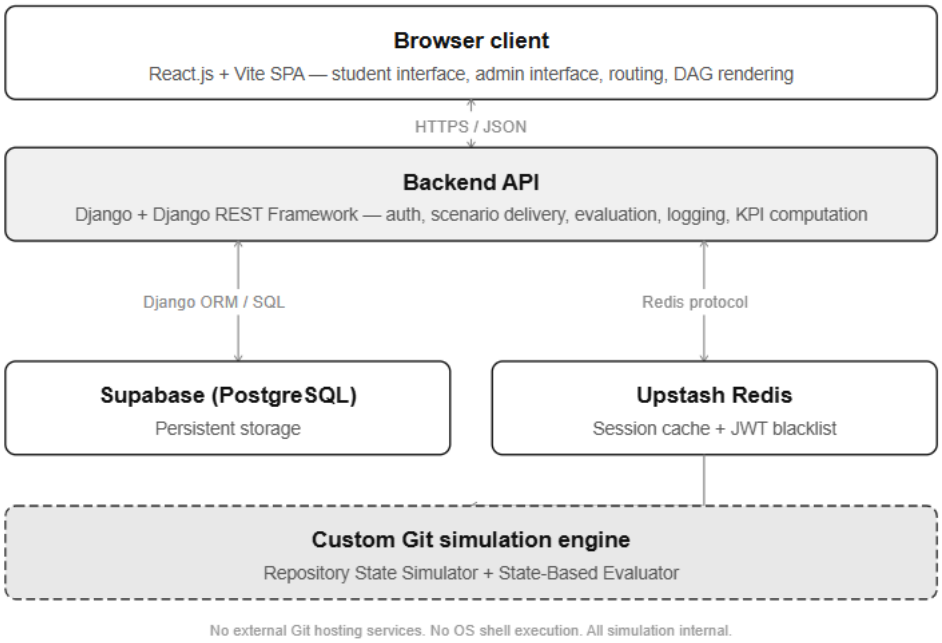
No locally available instructional tool provided consequence-safe, scenario-driven practice with state-based evaluation, visual repository feedback, or difficulty-differentiated instructional support tailored to this population.

GIT it! is developed in direct response to these documented behavioral patterns. It is a self-contained web platform that provides students with a consequence-safe environment to practice resolving realistic Git repository failure scenarios. Through progressive scaffolding removal, real-time repository state visualization, and state-based evaluation that accepts any valid command path to the correct target state, the platform is designed to replace avoidance behaviors with practiced, confident Git reasoning.

System Architecture

GIT it! follows a strict client-server architecture. All application components are self-contained, except that the AI tutor's backend service calls configured external language-model providers (Google Gemini and Groq). The system does not connect to any external Git hosting service and does not execute student input through an operating system shell at any point.

GIT it! System Architecture Diagram



Component	Role
Browser (React.js + Vite SPA)	Student-facing and admin-facing interface. Communicates with the backend exclusively through RESTful HTTPS endpoints using JSON payloads.
Django + Django REST Framework	Backend API responsible for authentication, scenario delivery, command processing, state-based evaluation coordination, session logging, KPI computation, and dashboard data retrieval.
Supabase (PostgreSQL)	Primary persistent data store for user accounts, chapters, scenario definitions, variant pools, session logs, step logs, completion records, and progress records. Hard dependency — system is non-operational if unavailable.
Redis (Render Key Value)	PostgreSQL (Supabase) is the authoritative store for accounts, runs, commands, progress and rewards. Redis caches rate-limit counters, the sign-in lockout, refresh-token revocation lookups, GitHub and Google sign-in tickets and tutor replies; if it is unavailable the system keeps serving from PostgreSQL (NFR-REL-02).

Custom Git Simulation Engine	Browser simulation and backend verification components for authored scenario rules. Manages isolated simulated repository sessions and processes student-entered Git commands through an application-controlled Command Adapter. Not an external service. Does not execute student input through an OS shell or external Git binary.
External AI Providers (Google Gemini, Groq)	Generate AI tutor replies and support semantic scenario recognition when called by the backend tutoring service. The browser does not contact them, credentials remain server-side, and they own no application data.

Content Library Scope

The platform's starter content library is seeded before deployment. It is organised into three levels:

- **Lesson.** An instructional page. A *scenario-bearing lesson* has one Scenario Skill Focus attached, which the student reaches through the lesson's View Scenarios action. Module 0 lessons are content-only and have no scenarios.
- **Difficulty instance.** One playable level of a Scenario Skill Focus. Each skill focus has three: Easy, Medium and Hard.
- **Scenario variant.** One authored version of a difficulty instance. A student's first attempt uses a primary variant. After a Failed or Abandoned session, the system serves a structurally different variant when one is available.

The library contains the following content. Counts of *scenario variants* describe authored content. They are not student attempts, which the system logs separately as *sessions*.

Module	Lessons	Scenario-bearing lessons	Difficulty instances	Easy variants	Medium variants	Hard variants	Total scenario variants
0	8	0	0	0	0	0	0
1	6	6	18	28	24	25	77
2	9	9	27	45	36	36	117

Module	Lessons	Scenario-bearing lessons	Difficulty instances	Easy variants	Medium variants	Hard variants	Total scenario variants
3	3	3	9	15	11	12	38
4	3	3	9	11	11	11	33
Total	**29**	**21**	**63**	**99**	**82**	**84**	**265**

The 21 scenario-bearing lessons correspond one-to-one with Specific Objectives SO 1.1–1.6, SO 2.1–2.9, SO 3.1–3.3 and SO 4.1–4.3. The remaining Specific Objectives in each module (SO 1.7–1.8, 2.10–2.11, 3.4–3.5 and 4.4–4.5) are module-level KPI targets and have no lesson of their own.

Scenario variants per lesson are not uniform. Most lessons have 13 variants (5 Easy, 4 Medium, 4 Hard). Lessons with a different count:

Lesson	Easy / Medium / Hard	Total
1.1 Initializing Repositories	3 / 4 / 4	11
1.3 Staging and Committing	5 / 4 / 5	14
3.1 Resolving Merge Conflicts Manually	5 / 3 / 4	12
4.1 Recovering from Hard Resets	5 / 5 / 5	15
4.2 Reversing Pushed Commits Safely	3 / 3 / 3	9
4.3 Completing Rebase Recovery Sequences	3 / 3 / 3	9

KPI Framework

All learning outcomes are measured exclusively through system-generated session and step logs using the following Key Performance Indicators:

KPI	Full Name	Formula	Target	Scope
SCR	Scenario Completion Rate	100 × required tiers mastered ÷ unique required tiers attempted	≥ 80% per module (validity gate)	Modules 1–4 · Validity gate
CAR	Command Accuracy Rate	Mean of 100 × S × min(1, T / max(1, A)) over assessed runs (S: run succeeded; T: validated reference command count; A: counted commands)	≥ 70% per Specific Objective	Modules 1–4 · Primary
HLCR	Hard-Level Completion Rate	100 × mastered Hard tiers ÷ attempted Hard tiers	≥ 70% (Modules 1–3); ≥ 65% (Module 4)	Modules 1–4 · Primary
ARC	Average Retry Count	Failed or abandoned attempts before first mastery, summed over mastered tiers ÷ mastered tiers; successful repetitions are not retries	≤ 2 (Modules 1–3); ≤ 3 (Module 4)	Modules 1–4 · Primary
RTA	Retry Transfer Accuracy	100 × transfer checkpoints cleared on their first run ÷ eligible novel transfer checkpoints (Modules 3–4); no eligible checkpoint yields no rate	≥ 65% (Modules 3–4)	Modules 3–4 · Primary

KPI definitions follow KPI_METHODOLOGY.md: they are computed from tier mastery and assessment checkpoints rather than raw session counts. CAR counts the commands that use the action budget, including incorrect input; diagnostic and shell commands are excluded. Replays and transfer runs never add practice mastery credit.

Platform usability and technology acceptance are assessed post-deployment using the System Usability Scale (SUS, target ≥ 70) and a TAM-based survey (Perceived Usefulness ≥ 4.0, Perceived Ease of Use ≥ 3.5, Behavioral Intention to Use ≥ 4.0 on a 5-point Likert scale). These are external validation instruments and do not contribute to primary KPI measurement.

Pedagogical Design Model

GIT it! implements three interconnected pedagogical design principles:

Fading Scaffolding — Instructional support is progressively removed as students advance through difficulty levels within each learning objective. Easy provides full scaffolding: repository graph, target-state guidance, contextual feedback panel, and counted-command allowance. Medium removes the contextual feedback panel, retaining the repository graph, target-state guidance, and counted-command allowance. Hard removes the target-state guidance, leaving only the repository graph, narrative context, and counted-command allowance. The repository graph is the single persistent visual anchor present at all difficulty levels [19].

State-Based Evaluation — The system evaluates supported repository transitions against authored completion rules rather than matching a single command string. The frontend simulation supplies immediate feedback; the backend verifies submitted metadata and transitions before accepting state or updating completion, mastery, and rewards. [20][21]

Guided Teaching Policy — The implemented command introduction may display syntax and a worked command example. This behavior conflicts with the previous absolute Guided Teaching Policy. Revision 1.1 proposes replacing that prohibition with a distinction between concept-level teaching examples and scenario-specific solutions. The application behavior and revised policy must be evaluated together; this SRS does not claim compliance with the superseded absolute prohibition.

Retry and Transfer — A retry rotates to another variant of the same scenario. Retry Transfer Accuracy (RTA) is measured on reserved transfer checkpoints in Modules 3–4: structurally different variants the learner has not seen, played without answer scaffolding and graded on their first run. [21]

Data Privacy Scope

Student personal data is limited to a username, email address, password hash, an optional GitHub or Google account link, and the IP address and browser of each sign-in session. It is encrypted at rest (AES-256) and in transit (TLS) in compliance with the Philippine Data Privacy Act of 2012 (RA 10173) [2]. Personal data is shared only with the processors listed in NFR-SEC-10, and learning records are not shared with third parties. Session and step logs are used exclusively for KPI computation and platform analytics within the system.

2.2. Product Functions

The following table summarizes the system's primary functions by user role. Detailed functional requirements are specified in Section 3.2.

Function	Description	Primary Actor
Student Registration and Authentication	Account creation, login, JWT-based session management, token refresh, and logout	Student
Module 0 Orientation	Eight optional, no-terminal orientation lessons covering Git concepts, DAG literacy, command anatomy, and platform conventions	Student
Home	Profile, Progress (activity and the KPI instrument) and Skills (spellbook and achievements) views	Student

Stories and chapter map	Story and chapter maps with orientation lessons, available adventures and challenges, drills, progress indicators, and tier access states	Student
Challenge Gates	Each module's closing challenge: Easy, Medium and Hard trials that open once every level of the module has all three tiers complete	Student
Squire's Drill	Optional recall practice on the commands an adventure level uses; resumes after a refresh and pays no GitCoins	Student
Lesson Page	Scrollable page that renders pre-authored instructional content, including concept explanations, command examples, callouts, and optional diagrams. Lesson Pages are separate from the mission briefing, the command guide and the level workspace. Opening a Lesson Page does not initiate a scenario session and does not generate scenario KPI data. Student	Student
Mission briefing and command guide	The briefing shows the scenario's story and task when a level is first played and can be reopened during the run; the command guide teaches the next command form in its home scenario	Student
practice workspace	Terminal-style command input, terminal output, repository graph, target-state guidance when permitted by difficulty, contextual feedback when permitted by difficulty, project structure panel, scenario brief, and remaining counted-command tracking	Student
State-Based Scenario Evaluation	Browser simulation provides immediate feedback; backend verification accepts supported state transitions and records completion and rewards.	System
Completion-Gated Difficulty Progression	Automatically unlocks Medium after Easy completion and Hard after Medium completion within each learning objective	System
Adaptive Retry with Changed Variants	Serves a structurally different, pre-authored scenario variant on retry after Failed or Abandoned primary sessions when another valid variant is available	System
Replay	Playable re-attempt of a mastered scenario difficulty instance; logged separately from primary attempts; no answer reveal	Student

Session and Step Logging	Continuous session, step, command, completion, and progress log-writing for all KPI computation	System
Progress and KPI Indicators	Log-derived KPI and progress indicators visible to the student in real time on the dashboard	Student
AI-Assisted Learning Support	Authenticated companion-based AI tutor providing Git and platform explanations, suggested topics, and protected conceptual guidance in command guides; it does not execute commands or change assessed progress	Student
Learning Analytics Dashboard	Log-derived KPI and progress analytics across all students, accessible to the Administrator	Administrator
Learner Management	Search learners, view each learner's KPIs, grant GitCoins, grant or remove staff rights, and activate or deactivate accounts	Administrator
Story and Chapter Publishing	Edit story and chapter details and publish or unpublish them; scenario content is seeded from the curriculum source	Administrator
Admin Action Log	GitCoin grants and adjustments, staff and activation changes, and settings changes are recorded; the KPI overview lists the most recent	Administrator

Note 1: Functions listed with System as the primary actor are triggered automatically by platform events and do not require direct user initiation. These functions do not appear as use cases in Section 3.2 but are specified as functional requirements where applicable.

Note 2: Guest access is limited to the public landing page, login screen, and registration screen. No system functions are available to unauthenticated users beyond account creation.

2.3. User characteristics

Student (Primary User)

Students are third-year BSIT students enrolled in team-based development courses such as Information Management 2 and Applications Development. They have been exposed to Git in prior coursework but lack reliable proficiency under realistic collaborative conditions. This population exhibits a consistent

pattern of avoidance and workaround behaviors when confronted with unresolvable repository states — defaulting to destructive workarounds such as discarding teammate contributions, re-cloning repositories, or relying on AI tools to execute commands they do not understand [11][17][18].

Students are non-technical end users with respect to the platform itself — they should be able to navigate the Home, select scenarios, and enter Git commands without requiring external documentation. Their technical engagement is with Git concepts, not with platform configuration. Frequency of use is driven by course requirements during the evaluation period. Primary goals are completing learning objectivees at all three difficulty levels and improving Git command proficiency as reflected in the dashboard KPI indicators. Key concerns include understanding why their commands did not produce the expected repository state, avoiding loss of progress records, and not having correct answers revealed prematurely.

These characteristics are the basis for several requirements specified in Section 3. The student's non-technical relationship with the platform and sensitivity to cognitive overload motivate the fading scaffolding model — specifically, the provision of a repository graph, contextual feedback panel, and target-state guidance at Easy difficulty, with progressive removal at Medium and Hard. The documented avoidance and workaround behaviors motivate the consequence-safe practice environment and state-based evaluation. The earlier absolute answer prohibition is recorded as a proposed teaching-policy amendment because current guided introductions include command examples. The concern over loss of progress records motivates the session and step logging requirements. The original absolute No-Answer requirement conflicts with command examples in the implemented teaching flow. Revision 1.1 proposes a concept-guidance boundary; the AI tutor (AI.1) applies protected conceptual teaching to scenario content.

Administrator

The Administrator is the authorized platform operator responsible for story and chapter publishing, learning analytics review, learner management, and reviewing the admin action log. The Administrator uses the system during active deployment periods to monitor student progress through log-derived KPI dashboards and to manage student accounts. Primary concerns are KPI data integrity and platform availability. Technical proficiency is expected to be moderate to high — sufficient to navigate the administrative interface and interpret log-derived analytics.

The Administrator's moderate-to-high technical proficiency and primary concern with data integrity and platform availability motivate the requirements for a log-derived Learning Analytics Dashboard, learner management functions, and the admin action log specified in Section 3. The expectation that the Administrator operates without real-time developer support motivates the requirement that the administrative interface be navigable without external documentation.

Guest (Unregistered Visitor)

An unregistered visitor who reaches the platform's public-facing pages — the landing page, login screen, or registration screen. Access is limited to public-facing content and account creation. No learning content is accessible without registration. This user class requires no technical proficiency beyond operating a standard web browser.

The Guest's limited access scope and minimal technical proficiency motivate the requirement that no learning content, scenario data, or KPI information be accessible to unauthenticated users, and that the registration flow require no technical knowledge beyond standard form completion.

2.4. Constraints

Regulatory and Legal

Constraint	Description
RA 10173 Compliance	Student personal data (see Data Minimization) shall be encrypted at rest and in transit, and its collection, storage and processing shall comply with the Philippine Data Privacy Act of 2012 (RA 10173).
Data Minimization	Personal data is limited to what sign-in, account security and KPI computation need: a username, email address, password hash, an optional GitHub or Google account link, and the IP address and browser of each sign-in session. No other personal data is collected.
Safety Classification	GIT it! is not a safety-critical system and presents no risk to physical safety. Security considerations are limited to data privacy compliance under RA 10173, authentication integrity via JWT token policy, and transport security via HTTPS/TLS 1.2+.

Hardware and Environment

Constraint	Description
No Offline Mode	The platform requires an active internet connection of at least 1 Mbps. Offline access is not supported.
Browser Compatibility	The platform requires a modern web browser — Chrome, Firefox, or Edge (latest two major versions) — with JavaScript enabled. No native application installation is required.
Supported Screen Sizes	All primary interface components shall be usable without horizontal scrolling from a 390 × 844 phone screen up to desktop sizes.

Runtime Environment Requirements	The backend runs on Python 3.13 with Django 6.0 and Django REST Framework 3.18. The frontend uses React 19, TypeScript 6 and Vite 8. No runtime environment outside these stacks shall be introduced without explicit project approval.

Interface and Integration

Constraint	Description
No External Git Execution	The system shall not execute learner input through an operating-system shell or connect practice runs to external Git hosting services. Browser simulation supplies immediate feedback; backend services verify submitted transitions and persist accepted outcomes.
JWT Token Policy	Access tokens shall have a 15-minute expiry. Refresh tokens shall have a 7-day expiry and shall be stored exclusively in an httpOnly, Secure, SameSite=Strict cookie inaccessible to JavaScript. On logout, refresh tokens shall be added to the database-backed token revocation records with a TTL equal to the token's remaining lifetime.
External AI Providers	The AI tutor's backend service may call configured external language-model providers over HTTPS. Provider credentials shall remain server-side, browsers shall not contact providers, and answer fields shall not be included in provider prompts.

Operational

Constraint	Description
Concurrent User Support	The system shall support 40 concurrent learners during the evaluation period, as defined in NFR-PERF-04. Supporting 100 concurrent learners requires a larger server than the free hosting tier.
Ordered, Completion-Gated Progression	Chapters are open to every learner. Within a chapter, levels open in order: a level unlocks once the previous level's Easy tier is complete. Within a level, Easy is available first, Medium unlocks after Easy and Hard after Medium. Each module's Challenge Gate opens once every required level has all three tiers complete. State-based evaluation is a practice feedback mechanism, not a quiz assessment.

Replay Eligibility	Replay is available for a difficulty tier with a completion record. Replay runs are logged separately and excluded from primary completion, reward, and performance calculations.
Seeded Content	The curriculum is seeded before deployment from the committed curriculum source: modules, levels and tiers, ordered scenarios and their variants, orientation lessons, command forms, Challenge Gates and drills. Staff can publish or unpublish stories and chapters but do not author scenarios in the app. Content counts are not permanent SRS commitments.
Deployment Criticality	GIT it! is deployed during a defined academic evaluation period. System unavailability during this period directly affects students' ability to complete required scenario practice and compromises the integrity of KPI data collection. The platform is not safety-critical in the sense of risk to life or property; however, it is academically critical — extended downtime during the evaluation window cannot be compensated for by alternative means and would invalidate the research outcomes the platform is designed to measure.

Security and Access Control

Constraint	Description
Guided Teaching Policy	Proposed for adviser review: the system may provide concept-level command syntax and worked examples in guided introductions, but shall not present a scenario's required solution as a teaching answer. The AI tutor (AI.1) is instructed against scenario-specific solutions and checks protected replies against server-side answer data; this filtering reduces disclosure risk without guaranteeing prevention.
Access Control Enforcement	All learning content, scenario data, session records, step logs, KPI indicators, and administrative functions shall be access-controlled by role. Students shall not access administrative functions. Administrators shall not participate in scenario sessions that generate KPI data. Unauthenticated users shall not access any content beyond the public landing page, login screen, and registration screen.
Admin Action Log	The system shall record administrative actions — GitCoin grants and adjustments, staff and activation changes, and settings changes — with the acting staff member. Only staff can see them (the KPI overview lists the most recent), and no platform interface can modify or delete them.

Measurement and Data

Constraint	Description
KPI Measurement Methodology	All learning outcomes shall be measured exclusively through system-generated session and step logs. No pre-test or post-test instruments are used. The platform itself is the measurement instrument for all primary KPIs.
CAR Command Counting	CAR counts the commands that use the action budget, including incorrect input. Diagnostic and shell commands are excluded unless an author enables diagnostic assessment for a diagnostic objective.

2.5. Assumptions and dependencies

- If the deployment environment does not support the required Python/Django runtime or lacks HTTPS capability, the system cannot be deployed and the requirements in Section 3.1 would require revision.
- The system has a hard dependency on a running Supabase (PostgreSQL) instance. If the database is unavailable, persistent storage and core application functions will not be operational.
- Redis provides a shared cache for refresh-token revocation and other cache-dependent controls. PostgreSQL holds the authoritative revocation record. If Redis is unavailable, refresh-token revocation is checked in the database; if both checks fail, validation fails closed. Other database-backed learning records remain available, and rate limits and the sign-in lockout pause until Redis returns.
- The evaluation deployment uses free hosting tiers: Vercel for the frontend, a Render Free web service and Render Free Key Value for the API and cache, and Supabase Free for PostgreSQL. Render puts an idle API to sleep after 15 minutes, which the uptime monitor prevents (NFR-REL-01), and Supabase Free has no automatic backups, which the weekly manual export replaces (NFR-REL-04).
- AI tutor replies depend on configured provider credentials, the generated scenario index, and network access. When they are unavailable, the system shows an explicit unavailable or recovery state, and saved FAQS remain available without a provider request.
- The database is the source of truth: Redis never owns learning or account data, so losing it can pause cache-dependent controls but cannot lose records.
- Students are assumed to have a valid email address for registration and a stable internet connection of at least 1 Mbps. The frontend requires JavaScript to be enabled in the browser.
- Student registration is not permanently restricted to a specific institutional email domain unless an approved deployment policy explicitly requires it. The system supports general valid email registration.
- The curriculum is seeded into the database before deployment from the committed curriculum source, and staff can publish or unpublish stories and chapters. Seeded content includes modules, levels and tiers, ordered scenarios and their variants, orientation lessons, command forms, Challenge Gates and drills. Exact content counts are not treated as permanent SRS commitments as the curriculum may be revised prior to final deployment.
- Platform usability and technology acceptance shall be assessed post-deployment using the System Usability Scale (SUS, target ≥ 70) and a TAM-based survey (Perceived Usefulness ≥ 4.0, Perceived

Ease of Use ≥ 3.5, Behavioral Intention to Use ≥ 4.0 on a 5-point Likert scale). These are external validation instruments and do not contribute to primary KPI measurement.

3. Specific Requirements

3.1. External interface requirements

3.1.1. Hardware interfaces

GIT it! is a browser-based SPA and does not interface with hardware peripherals, dedicated hardware components, or device drivers beyond the standard capabilities of a modern web browser. No native application installation, browser plugin, or hardware driver is required on the client device. No device-level hardware access (camera, microphone, USB, Bluetooth) is required or used by the system.

Client Device Minimum Requirements

Item	Requirement
Web Browser	Chrome, Firefox, or Edge (latest two major versions) with JavaScript enabled
Display	≥ 390 × 844 pixels (phone) up to desktop; color display required for graph rendering
Internet Connection	≥ 1 Mbps stable connection
Input	Keyboard and pointing device, or a touch screen with an on-screen keyboard

Server Configuration

Item	Requirement
Application server (evaluation)	Render Free web service in Singapore running the Django API in Docker with one Gunicorn worker and two threads
Database	Supabase PostgreSQL (Free plan), reached through Supabase's connection pooler over TLS
Cache	Render Key Value (Free), Redis-compatible
Frontend hosting	Vercel (static single-page app; forwards /api to the API server)
Recommended for 100 learners	Application host with 4 vCPU and 8 GB RAM, with managed PostgreSQL and Redis (DEPLOYMENT.md)

Specialized hardware	None required

The evaluation deployment shall sustain the NFR-PERF-04 workload of 40 concurrent learners; 100 concurrent learners require the recommended configuration.

3.1.2. Software interfaces

GIT it! interfaces with the following software products and internal services. All external interfaces use documented, stable APIs over HTTPS. No proprietary binary protocols are used.

Required Software Products

Name	Mnemonic	Version	Source	Purpose
React.js	React	19.x	Meta Open Source — https://react.dev	Frontend SPA framework; renders all student-facing UI components
Vite	Vite	8.x	Vite Contributors — https://vitejs.dev	Frontend build tool and development server
Django	Django	6.0	Django Software Foundation — https://docs.djangoproject.com	Backend web framework; handles routing, middleware, ORM, and application logic
Django REST Framework	DRF	3.18	Encode OSS — https://www.django-rest-framework.org	REST API layer for all client-server communication; serialization, authentication, and endpoint routing
Supabase (PostgreSQL)	Supabase	Current stable	Supabase Inc. — https://supabase.com/docs	Primary persistent data store; hosts all relational data including user accounts, scenario definitions, session logs, step logs, completion records, and progress records. Hard dependency — system is non-operational if unavailable

Render Key Value	Redis	Redis 7 compatible	Render — https://render.com/docs/key-value	Shared cache for rate limits, the sign-in lockout, refresh-token revocation lookups, GitHub and Google sign-in tickets and tutor replies. Soft dependency — the system keeps serving from PostgreSQL if it is unavailable (NFR-REL-02).
Vercel	Vercel	Hobby plan	Vercel Inc. — https://vercel.com	Hosts the built frontend and forwards /api requests to the API server, so the browser sees one origin
Render	Render	Free web service	Render — https://render.com	Runs the Django API container in the Singapore region
Brevo	Brevo	Transactional email API	Brevo — https://www.brevo.com	Sends email-verification codes and password-reset links over HTTPS
GitHub and Google sign-in	OAuth 2.0	Provider APIs	GitHub — https://docs.github.com; Google — https://developers.google.com/identity	Optional sign-in with a provider-verified email address

Custom Git Simulation Engine	GitSim	Internal	Internal — not an external service	Browser Git simulator for instant feedback and backend transition verifier that checks each proposed state change before it is persisted; neither invokes an external Git binary or operating-system shell
Google Gemini API	Gemini	Configured models	Google LLC --- https://ai.google.dev/gemini-api/docs	Language-model and embedding provider for AI tutor replies and semantic scenario recognition; called only by the backend. Soft dependency --- the tutor returns an unavailable state when no provider responds
Groq API	Groq	Configured model	Groq, Inc. --- https://console.groq.com/docs	Language-model provider used after the configured Gemini models for AI tutor replies; called only by the backend

Software Interface Summary

Interface	**Type**	**Purpose**	**Protocol / Format**
React.js + Vite SPA	Frontend client	Student-facing browser interface; all user interactions, rendering, routing, module navigation, lesson content rendering, mission briefings and command guides, DAG rendering, and scenario workspace behavior	HTTPS / JSON
Django + Django REST Framework	Backend API	Authentication, content delivery, scenario delivery, command processing, state-based evaluation coordination, session logging, KPI computation, dashboard data retrieval, retry handling, and review handling	HTTPS REST / JSON

Supabase (PostgreSQL)	Relational database	Persistent storage for users, profiles, session records, chapters, lessons, orientation progress, scenario definitions, difficulty instances, command policies, variants, generation blueprints, scenario sessions, step logs, command logs, completion records, KPI checkpoints, wallet transactions, and progress records. All writes involving completion records use database transactions and unique constraints to prevent duplicate entries	Django ORM / SQL
Redis (Render Key Value)	Cache	Rate-limit counters, the sign-in lockout, refresh-token revocation lookups, sign-in tickets and tutor replies; PostgreSQL remains authoritative	Redis protocol (private network)
Git simulator and transition verifier	Browser module and internal backend service	Instant browser simulation; backend transition verification, state evaluation and snapshot persistence for each run	Browser module; Python internal calls
Django tutoring service + External AI providers	External AI services	Reply generation and semantic scenario recognition for the AI tutor, with provider rotation, cooldowns, and bounded deadlines; credentials are held server-side	HTTPS / JSON

Interface Definitions

React.js ↔ Django REST Framework (Primary Client-Server Interface)

The frontend SPA communicates with the backend exclusively through RESTful HTTPS endpoints. All request and response payloads are encoded in JSON (Content-Type: application/json). Authentication state is carried via a JWT access token transmitted in the HTTP Authorization header using the Bearer scheme. Refresh tokens are stored exclusively in an httpOnly, Secure, SameSite=Strict cookie inaccessible to JavaScript.

Django ORM ↔ Supabase (PostgreSQL) Interface

The backend communicates with the Supabase-hosted PostgreSQL database through the Django ORM over a standard PostgreSQL wire protocol connection (port 5432, TLS-encrypted). No direct SQL outside the ORM layer is used for application logic. The database schema is managed through Django migrations.

Django ↔ Redis (Render Key Value) Interface

The backend uses the configured Redis connection as a shared cache, including refresh-token revocation lookups. Revocation is also recorded in PostgreSQL, which is authoritative when the cache is unavailable. The database remains the source of truth for learning, account, and reward records. If Redis is unreachable, cache reads return nothing and writes are skipped, so requests continue (NFR-REL-02).

Browser Git Simulator ↔ Backend Transition Verifier Interface

The Git engine has two parts that follow the same rules: a browser simulator that processes each command instantly, and a backend verifier inside the Django process. The browser sends the command with its proposed next repository state; the backend validates the JSON shape, verifies the transition with its own engine, evaluates the scenario's state requirements (TargetMatched or TargetNotYetMatched), and persists the accepted state. No subprocess calls, OS shell invocations, or external Git binary calls are made for learner input.

Django ↔ External AI Providers Interface

The backend tutoring chat service calls configured Google Gemini models and then the configured Groq model over HTTPS/JSON, with credentials sent in headers and never in URLs. The service tries models in order, skips models without credentials or in cooldown, and may start the next model while a slow attempt continues; the first usable response is used. Provider calls are bounded by configured deadlines, and the browser never contacts a provider. When no provider responds, the tutor returns an unavailable state rather than a fabricated answer.

Implemented API Route Groups

Route Group	**Example Endpoints**	**Purpose**
Authentication	/api/auth/register/, /api/auth/email-verification/confirm/, /api/auth/login/, /api/auth/oauth/exchange/, /api/auth/refresh/, /api/auth/logout/, /api/auth/me/, /api/auth/password-reset/request/	Registration, email verification, sign-in with a password or GitHub and Google, token refresh, logout, profile, password reset and change, session sign-out
Stories and chapters	/api/stories/, /api/chapters/, /api/chapters/{id}/overview/, /api/chapters/{id}/book/, /api/chapters/{id}/chests/	Story map, chapter overview, Chapter Book and chests

Orientation	/api/chapters/{id}/orientation/, /api/orientation-lessons/{id}/, /api/orientation-lessons/{id}/complete/	Module 0 lessons and their completion
Adventure runs	/api/adventure-level-tiers/{id}/runs/, /api/adventure-tier-runs/{id}/, …/submit-command/, …/files/, …/retry/, …/introduction/complete/	Start or resume a tier, submit commands, edit project files, retry, and finish command guides
Challenge runs	/api/challenge-trials/{id}/runs/, /api/challenge-runs/{id}/, …/submit-command/, …/files/, …/retry/	Challenge Gate trials
Squire's Drill	/api/adventure-levels/{id}/drill/, …/drill/run/, …/drill/results/	Drill content, checkpoint and results
the AI tutor	/api/tutoring/chat/	Tutor conversation
Command forms and skills	/api/command-forms/{id}/preview/, /api/skills/learned/	Command previews and the learned spellbook
Progress	/api/progress/dashboard/, /api/progress/performance/, /api/progress/stats/, /api/progress/stats/activity/, /api/progress/wallet/	Home data, KPIs, activity and GitCoin wallet
Player and shop	/api/player/preferences/, /api/player/loadout/companion/, /api/shop/catalog/, /api/shop/catalog/purchase/	Preferences, equipped companion and companion shop
Administration	/api/admin/overview/, /api/admin/users/, /api/admin/analytics/, /api/admin/economy/…, /api/admin/stories/, /api/admin/chapters/, /api/admin/settings/	Staff-only console
Health	/api/health/live/, /api/health/ready/	Liveness and readiness probes

Critical Boundary

Student-submitted Git command text shall be parsed and translated into supported internal simulator operations by the Command Adapter before any repository state is modified. The browser simulator runs inside the learner's browser and the transition verifier inside the backend server process. At no point shall student input be passed directly to an operating-system shell or executed as a real external Git command.

3.1.3. Communications interfaces

GIT it! communicates exclusively over standard internet protocols. No proprietary communications protocols or local network protocols are used between the client and server. All client-server communication shall conform to the following specifications.

Protocol / Mechanism	Specification
HTTPS (TLS 1.2+)	All client-server communication shall be encrypted using HTTPS with TLS 1.2 or higher (RFC 5246 [4]). Plain HTTP connections shall not be accepted in production; all HTTP requests shall be redirected to HTTPS. Authentication tokens, user credentials, and session data shall not be transmitted over unencrypted channels under any circumstances.
REST / JSON	The backend shall expose RESTful JSON endpoints. All request and response bodies shall use JSON encoding (Content-Type: application/json). Standard HTTP status codes shall be used, including 200, 201, 204, 400, 401, 403, 404, 409, 429, 500, and 503. No stack traces or internal error details shall be exposed in production API error responses.
JWT Access Token	Short-lived access token (15-minute expiry) transmitted by the frontend in the Authorization: Bearer <token> header on all authenticated requests.
JWT Refresh Token	Long-lived refresh token (7-day expiry) stored exclusively in an httpOnly, Secure, SameSite=Strict cookie inaccessible to client-side JavaScript. On logout, the refresh token shall be added to the database-backed token revocation records with a TTL equal to the token's remaining lifetime.
CORS	Cross-Origin Resource Sharing shall be restricted to approved frontend application origins in production. Requests from unauthorized origins shall be rejected at the CORS preflight stage.
Database Connection	The backend shall connect to Supabase PostgreSQL through Supabase's connection pooler over TLS (sslmode=require). Connections are reused for up to 60 seconds, with a 10-second lock timeout and a 30-second statement timeout.

WebSockets / Long Polling	The system does not use WebSocket connections or long-polling mechanisms. All scenario state updates — including DAG updates following command processing — are delivered as synchronous HTTP responses to client-initiated requests. There are no server-initiated push notifications in the current release.
External Network Boundaries	The system shall not execute learner commands through an operating-system shell or connect practice sessions to external Git hosting services. Browser simulation provides immediate repository feedback; the backend verifies supported transitions and persists accepted state. Backend data and cache connections use configured secure transports. The backend tutoring service may call configured external AI providers over HTTPS; browsers do not contact providers and provider credentials remain server-side.
Third-Party HTTPS APIs	The backend calls Brevo (email) and GitHub and Google (optional sign-in) over HTTPS. The browser never calls them directly, and no API key is sent to the frontend.

3.2. Functional requirements

System Access

Functional Requirements Table

FR ID	Summary	Requirement Statement
FR-AUTH-01	Registration validation	The system shall validate the username (3–30 characters: letters, numbers, dots, underscores or hyphens), the email format, the password (at least 8 characters, not entirely numeric, not a commonly used password, and not too similar to the username or email), and that the password and its confirmation match, before creating an account.
FR-AUTH-02	Duplicate account rejection	The system shall reject registration when the username or email address already belongs to an account and shall return HTTP 409.
FR-AUTH-03	Password hashing	The system shall hash all passwords using a secure one-way password hashing algorithm before storage. Plaintext passwords shall never be persisted or logged.
FR-AUTH-04	Player profile on registration	The system shall create the learner's player profile, onboarding state and a one-time sign-up grant of 150 GitCoins when an account is created.
FR-AUTH-05	Email verification before first sign-in	The system shall email a 6-digit verification code, valid for 15 minutes, when an account is created. Entering the code verifies the address and signs the learner in. Until then, sign-in returns HTTP 403 with the code email_unverified and mails a new code when the last one can no longer be used; a resend is available after 60 seconds.
FR-AUTH-06	Email and password authentication	The system shall authenticate a learner by username or email and verify the submitted password against the stored password hash.
FR-AUTH-07	Token issuance on login	The system shall issue a short-lived access token (15-minute expiry) and a long-lived refresh token (7-day expiry) upon successful login.
FR-AUTH-08	Secure refresh token storage	The system shall store refresh tokens exclusively in httpOnly, Secure, SameSite=Strict cookies inaccessible to JavaScript running in the browser.
FR-AUTH-09	Generic sign-in error and lockout	The system shall return the same message — "Incorrect username/email or password" (HTTP 401) — for every failed sign-in, whether the account does not exist or the password is wrong, to prevent account enumeration. After 5 failed attempts for the same username or email from the same IP address, sign-in is locked for 5 minutes (HTTP 429).

FR-AUTH-10	GitHub and Google sign-in	The system shall let a learner sign in with GitHub or Google. The provider must supply a verified email address. A new provider identity links to the existing account with that email, which also verifies it, or creates a new account. Staff accounts and deactivated accounts cannot sign in this way.
FR-AUTH-11	Password reset by email	The system shall email a password-reset link that is valid for one hour. The request always shows the same neutral confirmation, whether or not the email belongs to an account. The new password must pass the rules in FR-AUTH-01 and differ from the current password.
FR-AUTH-12	Account security settings	The system shall let a signed-in learner change the password (current password required; the new one must differ), sign out every other session while keeping the current one, and sign out everywhere.
FR-ADMIN-01	Staff authorization through shared sign-in	The system shall provide a shared sign-in page and enforce staff-only access through account authorization on protected administrative operations.
FR-ADMIN-02	Administrator role authentication only	The system shall authenticate staff through the shared sign-in flow and reject access to staff-only routes or APIs from accounts without staff authorization with HTTP 403.
FR-ADMIN-03	External administrator account provisioning	The system shall not permit staff privileges to be assigned through student self-registration. Authorized staff status is provisioned outside the learner registration form.
FR-ADMIN-04	Administrator-scoped JWT token issuance	The system shall issue the standard authenticated session and enforce staff authorization on protected administrative operations.
FR-ADMIN-05	Student token rejection on admin endpoints	The system shall reject requests to staff-only endpoints from authenticated accounts without staff authorization and shall return HTTP 403.
FR-ADMIN-17	KPI overview	The system shall show staff a KPI overview with the five KPIs (SCR, CAR, HLCR, ARC, RTA) averaged over learners, excluding staff accounts, together with the economy settings and the most recent admin actions.
FR-ADMIN-18	Learner KPI panel	The system shall let staff search learners and open a learner's KPI panel, computed with the same formulas as the learner's own Progress view.

FR-ADMIN-19	Analytics filters	The system shall let staff limit learning analytics to a date range in Manila time and open a module's detail. Analytics shall never show solution commands, answer keys or evaluator rules.
FR-ADMIN-20	Read-only analytics	The system shall compute admin analytics from recorded runs, commands and checkpoints, and viewing analytics shall not change any learner data.
FR-ADMIN-21	Learner moderation and grants	The system shall let staff grant GitCoins to a learner, grant or remove staff rights, and activate or deactivate an account. Each action is recorded in the admin action log with the acting staff member.
FR-ADMIN-22	Economy ledger	The system shall show staff the GitCoin transaction ledger and let them post an adjustment, which is recorded in the admin action log.
FR-ADMIN-23	Story and chapter publishing	The system shall let staff edit a story's or chapter's title, summary, world, difficulty and order, and publish or unpublish it. Scenario content is seeded from the curriculum source and is not edited in the app.
FR-ADMIN-24	Feature settings	The system shall let staff switch shop purchases on or off; each change is recorded in the admin action log.

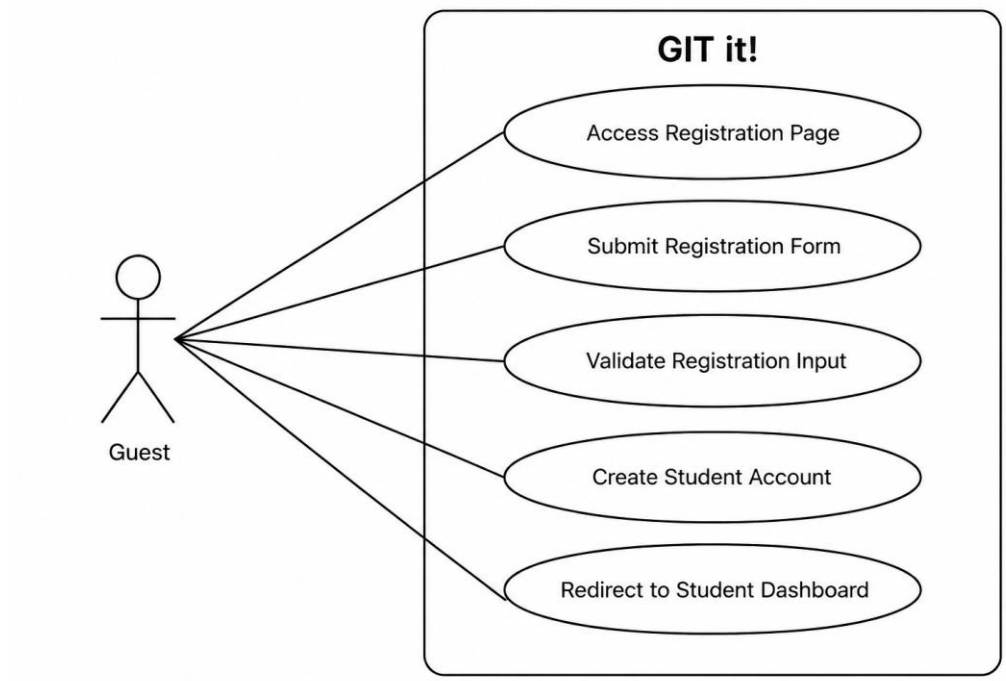
FR-SEC-01	General input validation	The system shall validate and sanitize all user-supplied input fields on both the client side and the server side before processing. This requirement applies to all input surfaces, including registration and login forms, command input in the practice workspace, and any other field through which a student may submit data to the system. Server-side validation shall be the authoritative gate; client-side validation serves as a usability aid only.
FR-GAM-01	Encounter star scoring	The system shall award 1 star for clearing a scenario, a second star when its counted commands are within the scenario's command target, and a third star when it is also cleared with at most one retry. A tier's stars are the lowest of its scenarios' best stars. Opening the command guide never costs a star, and transfer checkpoints are not graded.
FR-GAM-02	Mastery-derived rank	The system shall derive the displayed rank from the backend mastery score and show progress to the next rank. Rank is a progression indicator and shall not be presented as GitCoins or encounter stars.
FR-GAM-03	GitCoin reward ledger	The system shall maintain a player GitCoin balance and transaction ledger. It shall grant the 150-GitCoin sign-up grant, first-clear rewards once per tier (Easy 25, Medium 35, Hard 50) and per Challenge Gate trial (Easy 50, Medium 75, Hard 100), and chapter chest rewards, and shall record shop purchases as debits; a repeated request shall not duplicate a transaction.
FR-GAM-04	Companion ownership and equipment	The system shall display the companion catalog and prices, permit a purchase only when the player has sufficient GitCoins, record ownership, and allow equipment selection only from owned companions. The first successful purchase shall equip the companion automatically when no companion is equipped.

FR-GAM-05	Chapter chests	The system shall show chests on the chapter map that pay GitCoins at chapter-completion milestones; each chest pays once.
FR-GAM-06	Achievements count clears	The system shall award achievements for what the learner has cleared, never for daily visits or streaks.
FR-MET-01	Command Accuracy Rate (CAR)	CAR shall be the mean, over assessed runs, of 100 × S × min(1, T / max(1, A)), where S is 1 when the assessed run succeeded, T is the validated reference count of counted commands and A is the run's counted commands, including incorrect input. The assessed run is the first run that achieves mastery of a tier. Diagnostic and shell commands are not counted. Target ≥ 70%.
FR-MET-02	Retry Transfer Accuracy (RTA)	RTA shall be 100 × P / C, where C is the number of eligible novel transfer checkpoints in Modules 3–4 and P is those cleared on their first run. A transfer checkpoint serves a reserved, structurally different variant the learner has not seen, hides answer scaffolding, and earns no stars or rewards. No eligible checkpoint yields no rate. Target ≥ 65%.
FR-MET-03	Mastery-based completion KPIs	SCR shall be 100 × required tiers mastered ÷ unique required tiers attempted (target ≥ 80%). HLCR shall be 100 × mastered Hard tiers ÷ attempted Hard tiers (target ≥ 70%; Module 4 ≥ 65%). ARC shall be the failed or abandoned attempts before first mastery, summed over mastered tiers, divided by the mastered tiers (target ≤ 2; Module 4 ≤ 3). Replays and transfer runs never add mastery credit.

FR-TEACH-01	Guided teaching policy (proposed)	Proposed for review: guided introductions may show command syntax and worked examples for concepts, but shall not reveal the required solution to a specific active scenario. This proposed policy replaces the earlier absolute No-Answer wording only if approved; implementation alignment and adviser approval remain open.
FR-INT-01	Simulation and backend verification	The browser-side simulator shall provide immediate terminal and repository-graph feedback. The backend shall validate supported submitted transitions and remain authoritative for persisted state, accepted completion, mastery updates, and rewards.

SA.1 Student Registration

Use Case Diagram

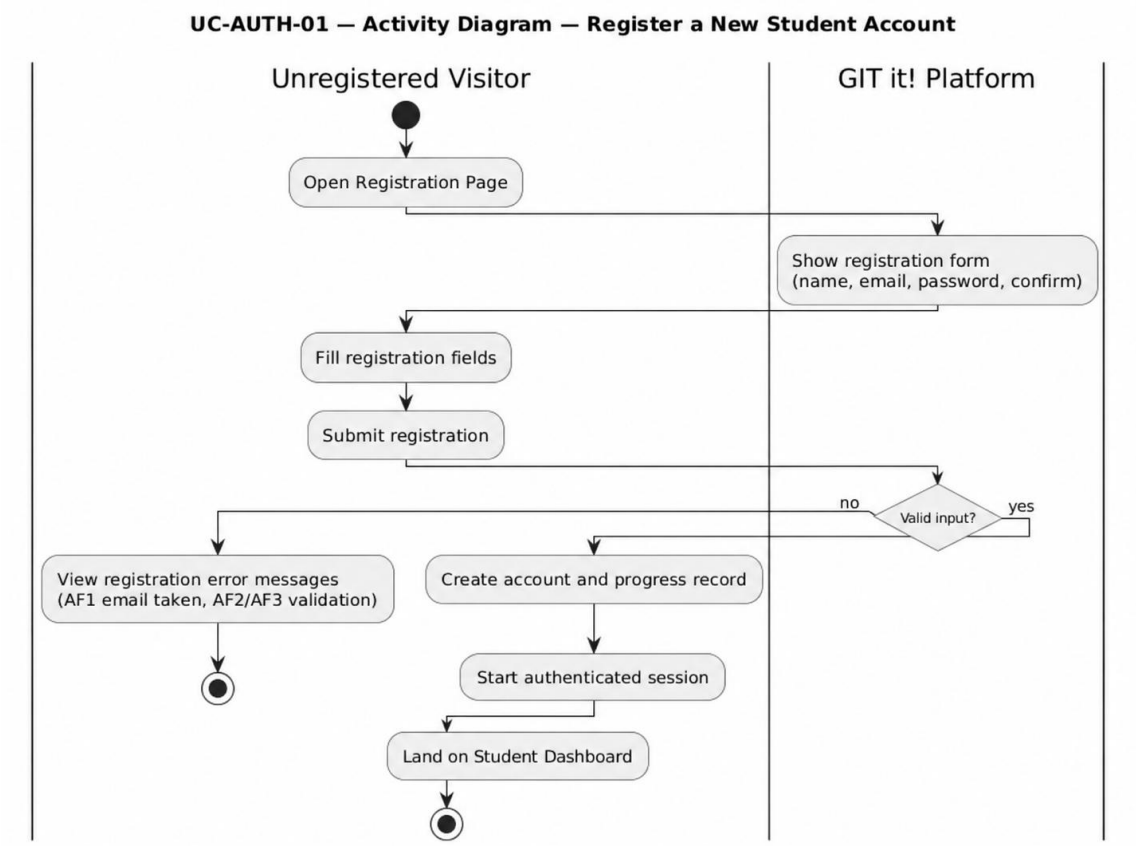


Use Case Description

Field	Description
Use Case ID	UC-AUTH-01

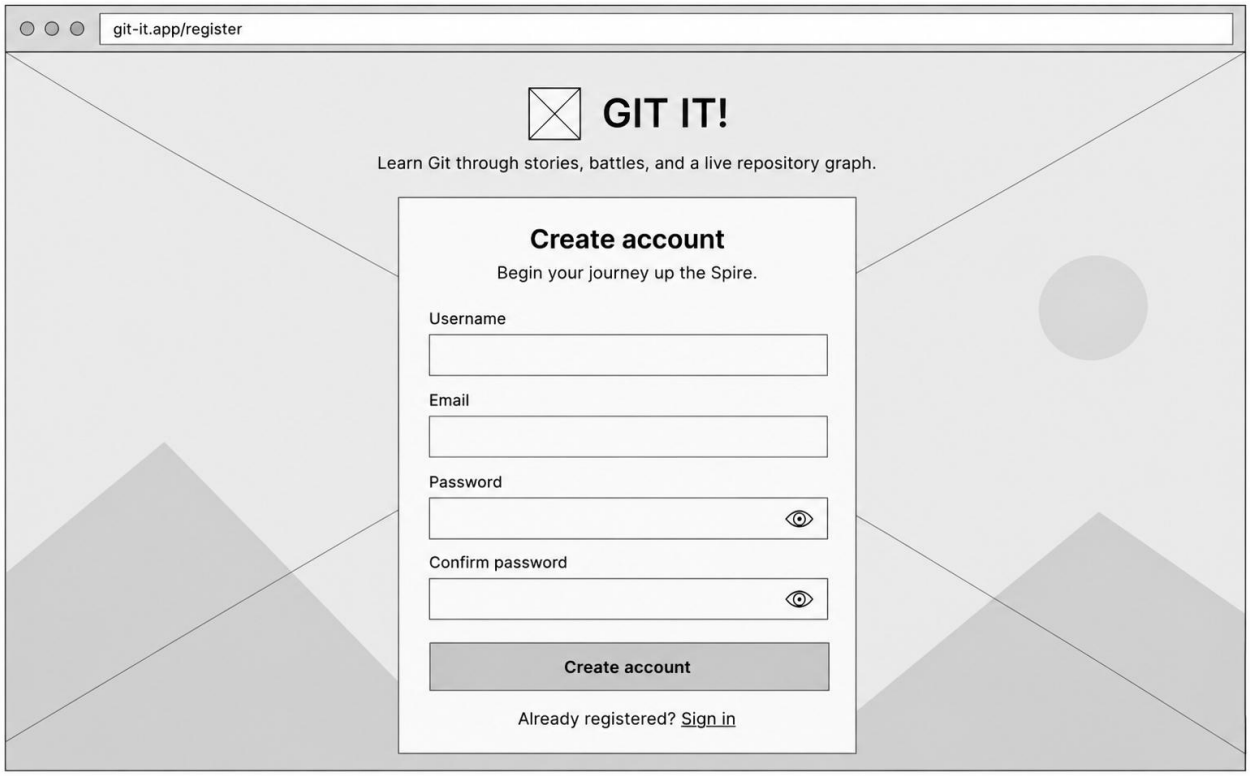
Use Case Name	User Registration
Actor(s)	Guest (Unregistered Visitor)
Preconditions	The visitor has no account. The registration page is reachable from the landing page and the sign-in page.
Main Flow	1. Visitor opens the registration page. 2. System displays the form: username, email address, password and confirm password, with GitHub and Google sign-in as alternatives. 3. Visitor fills in the form and submits. 4. System validates the username, email, password rules and confirmation (FR-AUTH-01). 5. System checks that the username and email are not already registered. 6. System hashes the password and creates the account, player profile, onboarding state and 150-GitCoin sign-up grant. 7. System emails a 6-digit verification code, valid for 15 minutes. 8. Visitor enters the code. 9. System verifies the email address, issues an access token (15 minutes) and a refresh token (7 days, stored in an httpOnly, Secure, SameSite=Strict cookie), and opens the game, starting onboarding.
Postconditions	A verified learner account and player profile exist, and the learner is signed in.
Alternative Flows	AF1 — Username or email already registered: HTTP 409 with "An account already exists for this username" (or email). AF2 — Client-side validation failure: inline field errors appear before submission. AF3 — Server-side validation failure: HTTP 400 with field-level errors. AF4 — Wrong or expired code: HTTP 400; the visitor can request a new code after 60 seconds.
Functional Requirements	FR-AUTH-01, FR-AUTH-02, FR-AUTH-03, FR-AUTH-04, FR-AUTH-05, FR-AUTH-10, FR-SEC-01

Activity Diagram



]

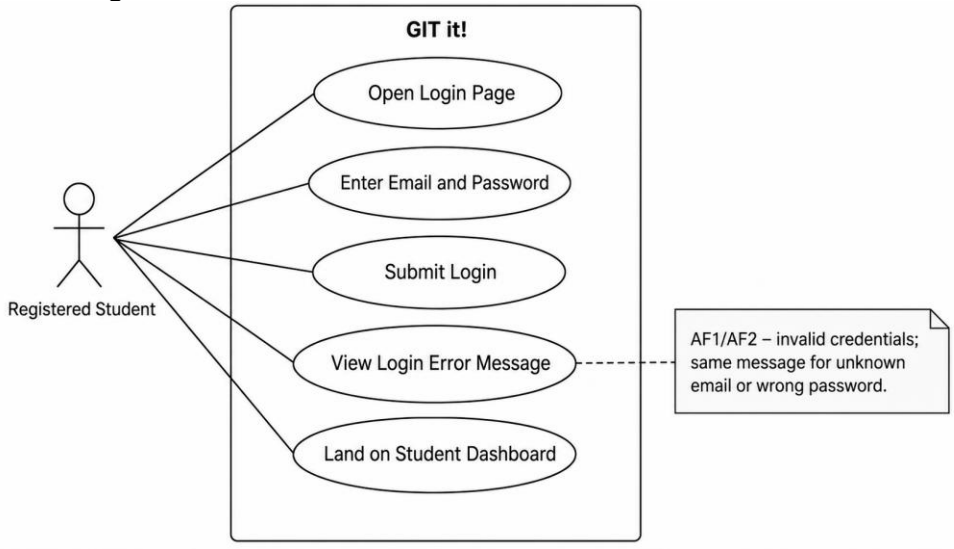
Wireframe



s

SA.2 Student Login

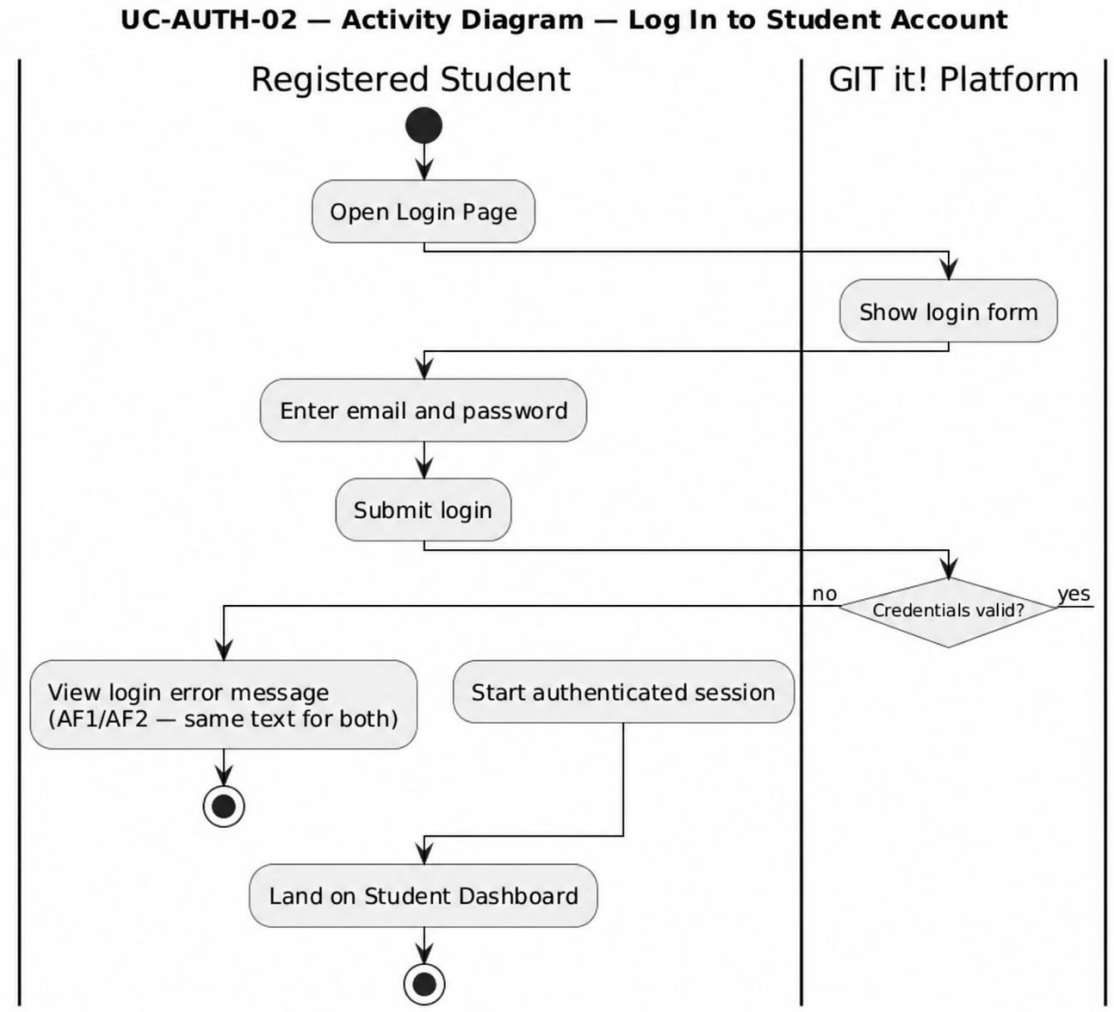
Use Case Diagram



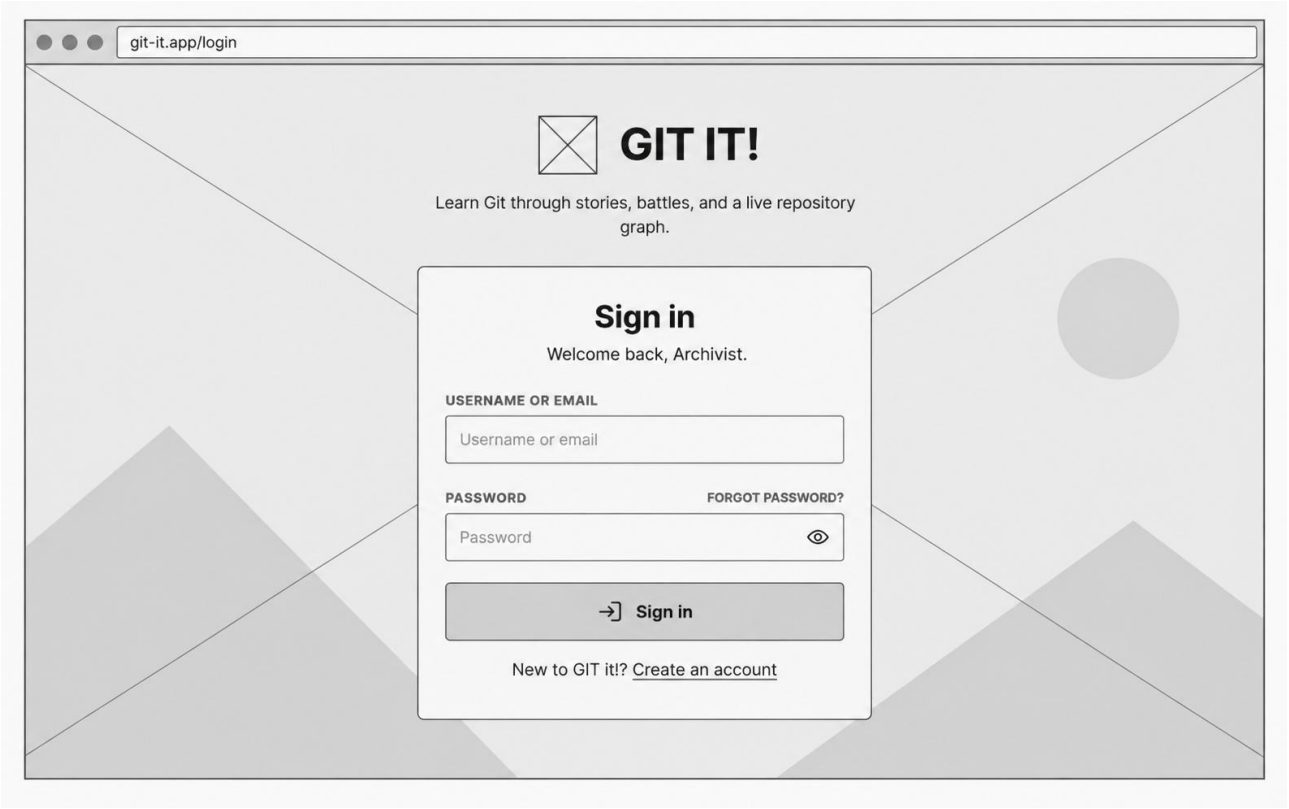
Use Case Description

Field	Description
Use Case ID	UC-AUTH-02
Use Case Name	Shared Sign-In
Actor(s)	Registered Student
Preconditions	The student has a registered account. The /login page is accessible.
Main Flow	1. Student opens /login. 2. System displays the sign-in form (username or email, and password) and GitHub and Google sign-in buttons. 3. Student submits credentials. 4. System finds the account by username or email and verifies the password against the stored hash. 5. System issues an access token (15 minutes) and a refresh token (7 days) stored in an httpOnly, Secure, SameSite=Strict cookie. 6. Frontend keeps the access token in memory and opens Home, continuing onboarding if it is not finished.
Postconditions	The student is authenticated. Access and refresh tokens are issued.
Alternative Flows	AF1 — Wrong password or unknown account: HTTP 401 with the same message, "Incorrect username/email or password". AF2 — Five failed attempts for the same account from the same IP address: sign-in is locked for 5 minutes (HTTP 429). AF3 — Email not verified: HTTP 403 (email_unverified); a new code is mailed when needed and the verification step opens. AF4 — GitHub or Google: the provider confirms the identity and the student is signed in (FR-AUTH-10).
Functional Requirements	FR-AUTH-06, FR-AUTH-07, FR-AUTH-08, FR-AUTH-09, FR-AUTH-10, FR-SEC-01

Activity Diagram

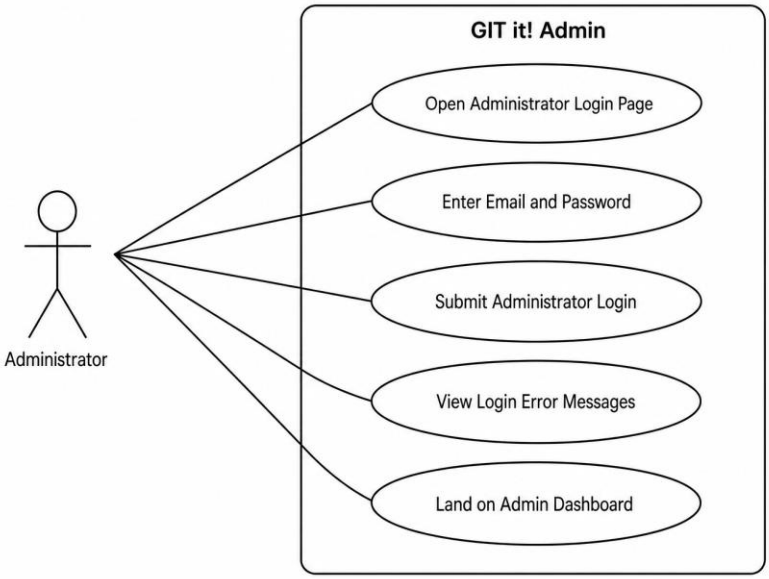


Wireframe



SA.3 Staff Access Through Shared Sign-In

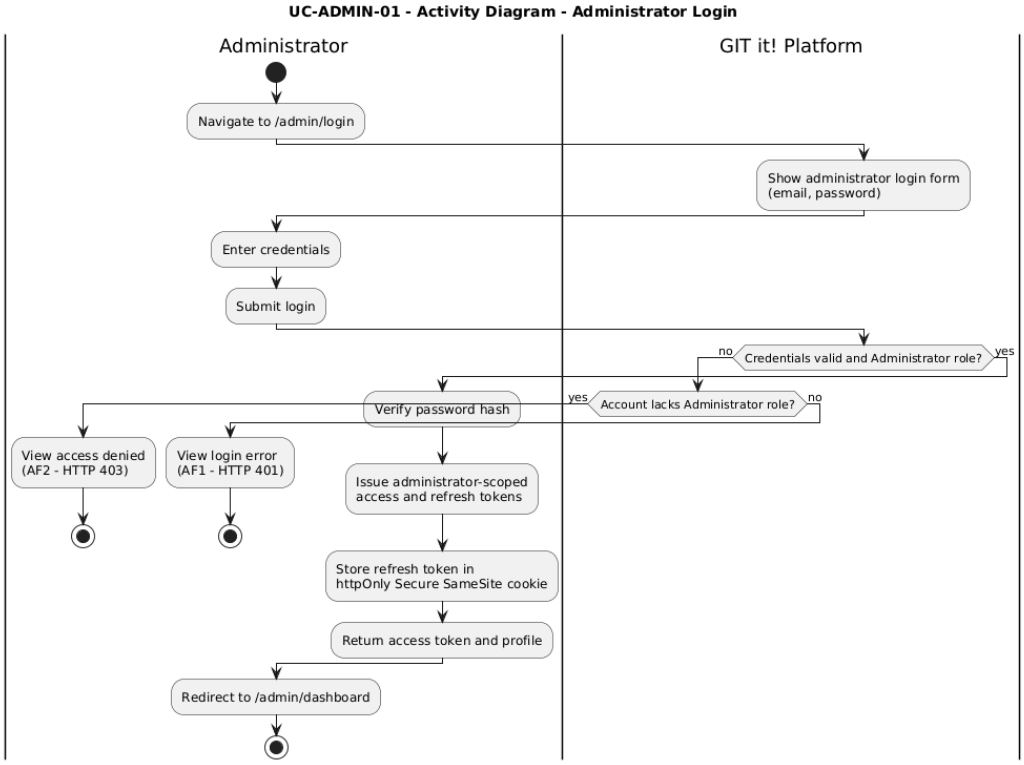
Use Case Diagram



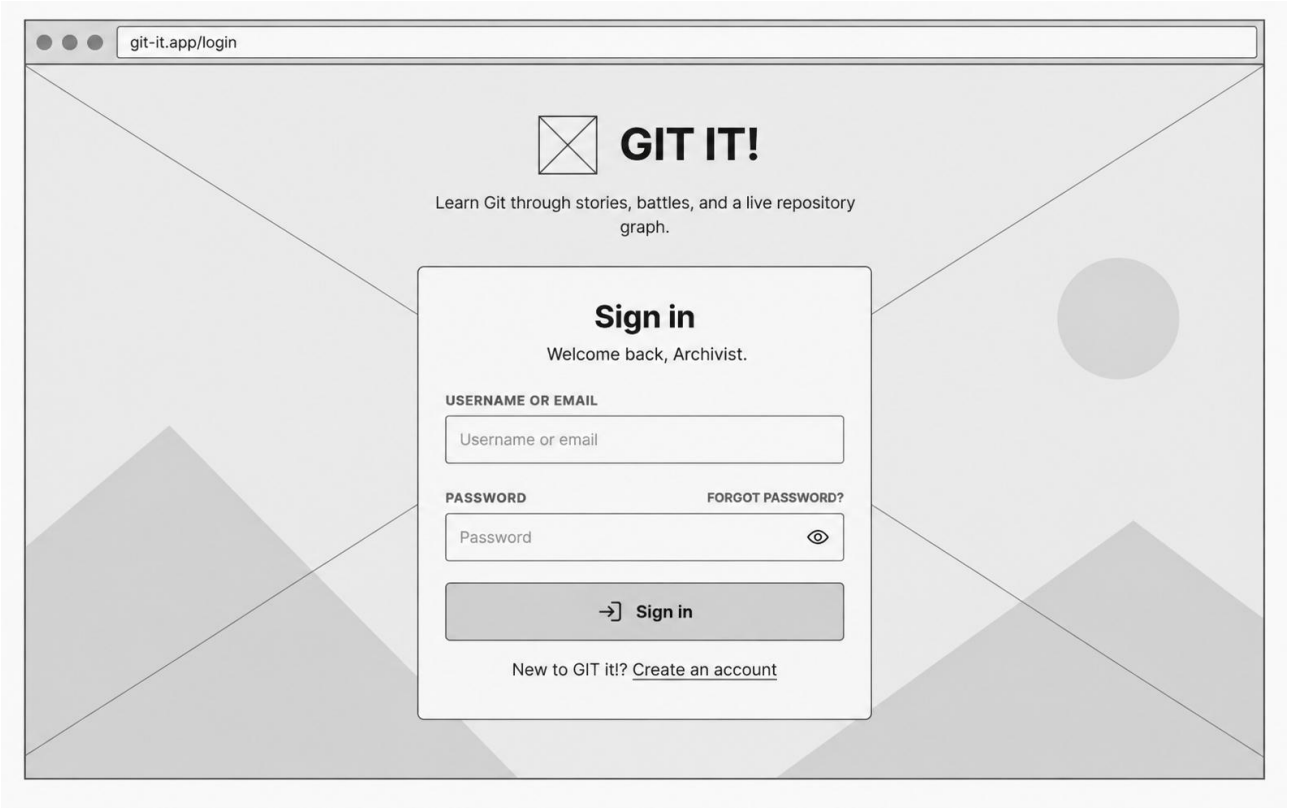
Use Case Description

Field	Description
Use Case ID	UC-ADMIN-01
Use Case Name	Staff Access Through Shared Sign-In
Actor(s)	Administrator
Preconditions	The staff account is provisioned with staff authorization. The shared /login page is available; staff functions require authorization.
Main Flow	1. Staff member opens the shared /login page. 2. System displays the shared sign-in form for username or email and password. 3. Staff member submits credentials. 4. System verifies the password against the stored hash. 5. System issues the standard access token and a refresh token stored in an httpOnly, Secure, SameSite=Strict cookie. 6. System returns the user profile, including staff status. 7. Frontend shows the admin console entry for staff accounts.
Postconditions	The staff member is signed in and can open the admin console.
Alternative Flows	AF1 — Invalid credentials: HTTP 401, "Incorrect username/email or password". AF2 — Account without staff status: it signs in as a learner; opening /admin returns to Home and staff APIs return HTTP 403. AF3 — Staff accounts cannot be created through self-registration and cannot use GitHub or Google sign-in.
Functional Requirements	FR-ADMIN-01, FR-ADMIN-02, FR-ADMIN-03, FR-ADMIN-04, FR-ADMIN-05

Activity Diagram



Wireframe



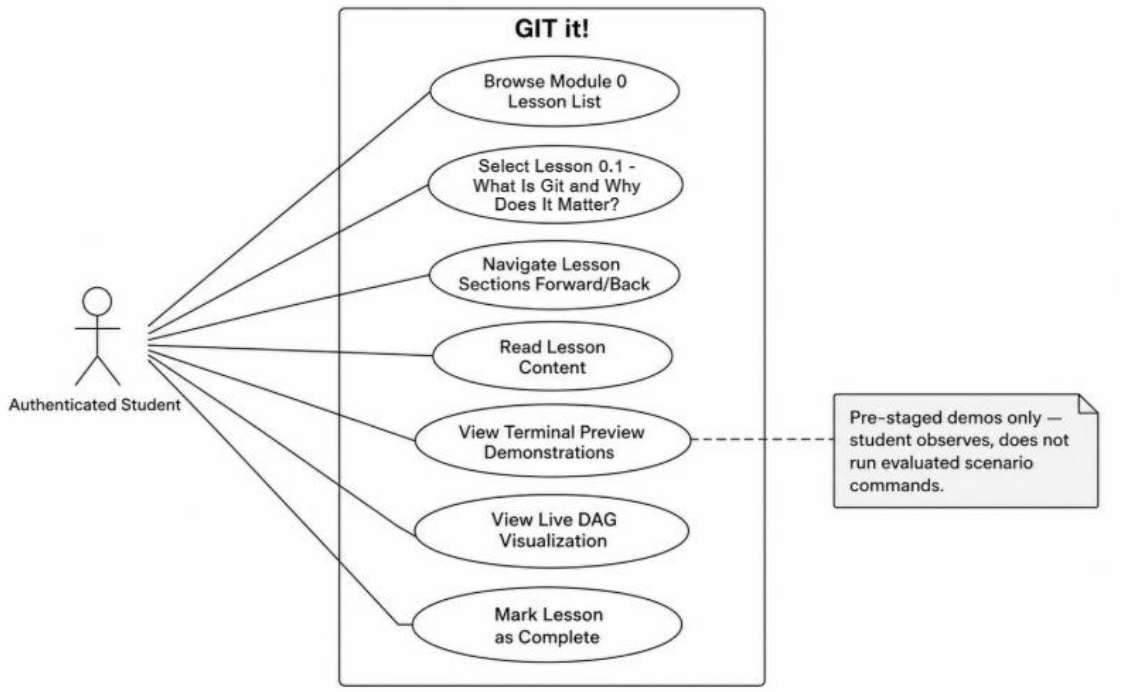
Module 0: Orientation

Functional Requirements Table

FR ID	Summary	Requirement Statement
FR-ORIENT-01	Guided walkthrough interface	The system shall present each Module 0 lesson as a guided walkthrough with a content panel, terminal preview panel, and live DAG panel.
FR-ORIENT-02	Section navigation	The system shall allow students to navigate forward and backward through lesson sections within a lesson without losing section-level progress.
FR-ORIENT-03	Mark as Complete action	The system shall require students to explicitly click Mark as Complete at the end of a lesson to record lesson completion.
FR-ORIENT-04	Progress persistence	The system shall save a student's last completed section so that re-entering an incomplete lesson resumes from the last saved section.
FR-LES-02	No scenario data generation	The system shall not generate scenario session logs, CAR, HLCR, ARC, or RTA data for any Module 0 lesson.
FR-LES-04	Lesson completion visibility	The system shall display each lesson's completion status (Not Started, In Progress, Complete) on the Module 0 lesson list.
FR-TERM-PREVIEW-01	Pre-staged demonstrations	The system shall present pre-staged Git command demonstrations in the terminal preview panel without creating evaluated sessions or consuming command budgets.
FR-TERM-PREVIEW-02	Simulated terminal output	The system shall display simulator-style terminal output for each demonstrated command in the terminal preview panel.
FR-TERM-PREVIEW-03	Fallback to static screenshots	The system shall display static annotated screenshots of command outputs if the interactive terminal preview panel is unavailable.
FR-DIAG-01	Non-counted command registration	The system shall designate diagnostic commands as non-counted system-wide; all designated diagnostic commands are excluded from the counted-command budget in every scenario session, regardless of whether the student has completed Module 0.
FR-DIAG-02	Reflog and graph command registration	The system shall designate git reflog and git log --graph as non-counted diagnostic commands system-wide for all scenario sessions.

0.1 Complete the Git and Version Control Conceptual Walkthrough

Use Case Diagram



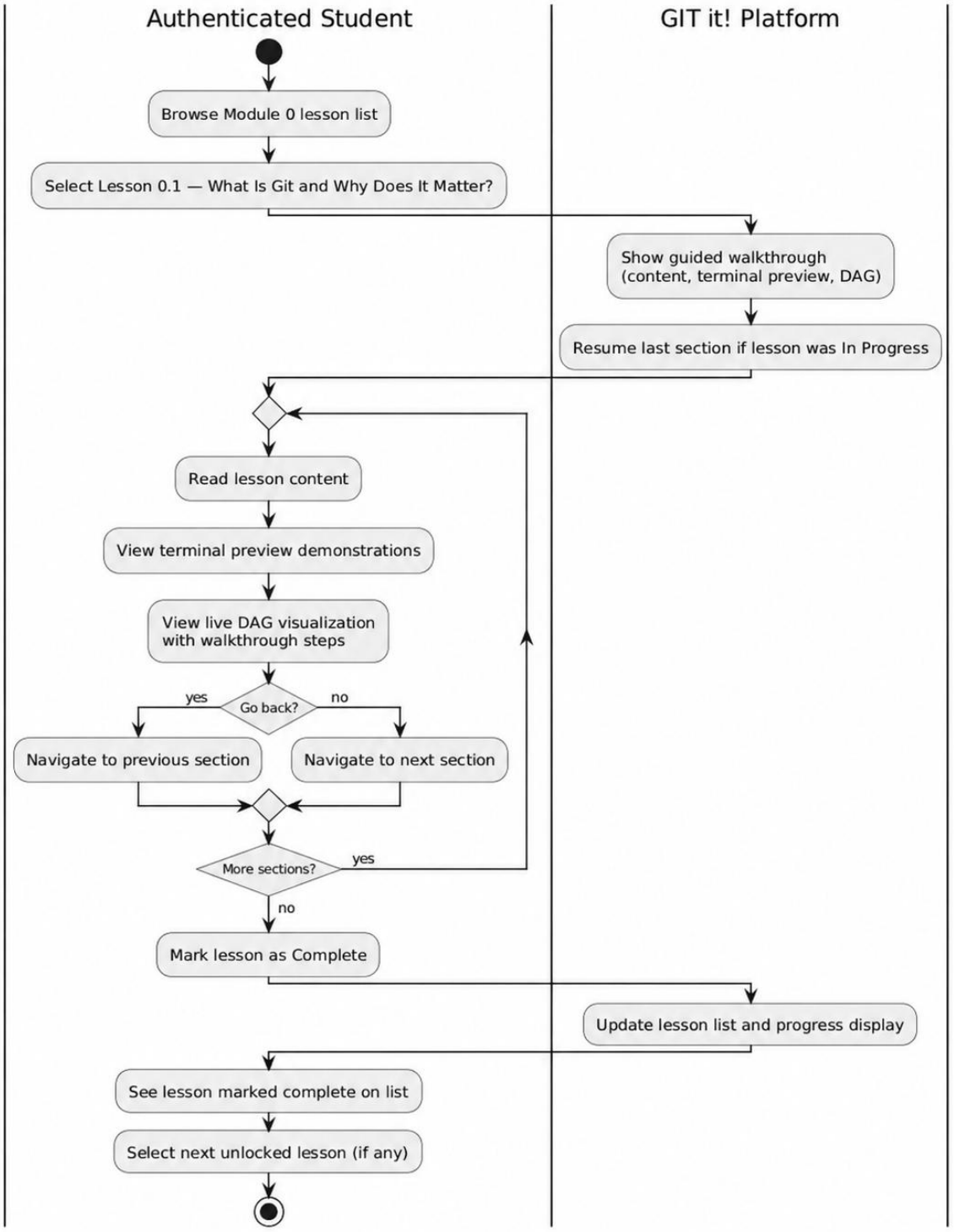
Use Case Description

Field	Description
Use Case ID	UC-ORI-01
Use Case Name	Complete the Git and Version Control Conceptual Walkthrough
Curriculum SO	SO 0.1
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Module 0 Orientation is available and Lesson 0.1 is the student's next incomplete lesson. No prior Module 0 lesson completion is required for Lesson 0.1.
Main Flow	1. Student navigates to the Home and opens the Module 0 Orientation unit. 2. Student selects Lesson 0.1 — What Is Git and Why Does It Matter? 3. Platform loads the guided walkthrough interface showing the lesson content panel, an embedded terminal preview panel, and the live DAG panel. 4. Student reads the conceptual content about version control, centralized vs. distributed VCS, and the three problems Git solves. 5. Student views annotated DAG illustrations embedded in the walkthrough that depict how Git history is structured across distributed clones. 6. Student interacts with the terminal preview panel to observe pre-loaded Git command demonstrations (e.g., example git log outputs) without submitting counted commands.

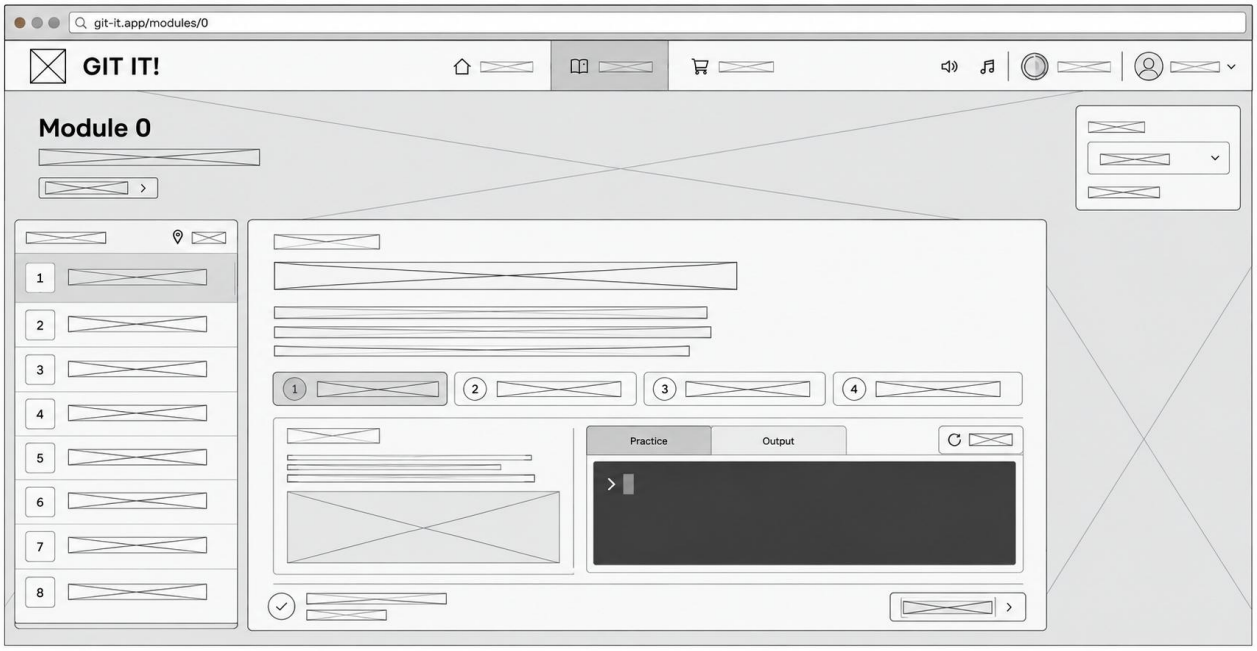
	7. Student sees the DAG panel update to reflect any pre-staged demonstrations as the walkthrough advances to each content section. 8. Student clicks Next Section to progress through all content segments of Lesson 0.1. 9. Student reaches the end of the lesson and clicks Mark as Complete. 10. Platform records Lesson 0.1 as completed and updates the student's progress tracker.
Postconditions	• Lesson 0.1 is recorded as complete in the student's progress tracker. • No scenario session log, CAR, HLCR, ARC, or RTA data is generated.
Alternative Flows	AF1 — Student navigates back mid-lesson: Allowed; completion recorded only on Mark as Complete. AF2 — Student exits before completing: Lesson remains In Progress; progress saved at last section. AF3 — Terminal preview unavailable: Static annotated command output screenshots replace the interactive preview.
Functional Requirements	FR-ORIENT-01, FR-ORIENT-02, FR-ORIENT-03, FR-ORIENT-04, , FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-LES-02, FR-LES-04, FR-TERM-PREVIEW-01, FR-TERM-PREVIEW-02, FR-ACC-01, FR-ACC-02

Activity Diagram

UC-ORI-01 — Activity Diagram — Complete the Git and Version Control Conceptual Walkthrough

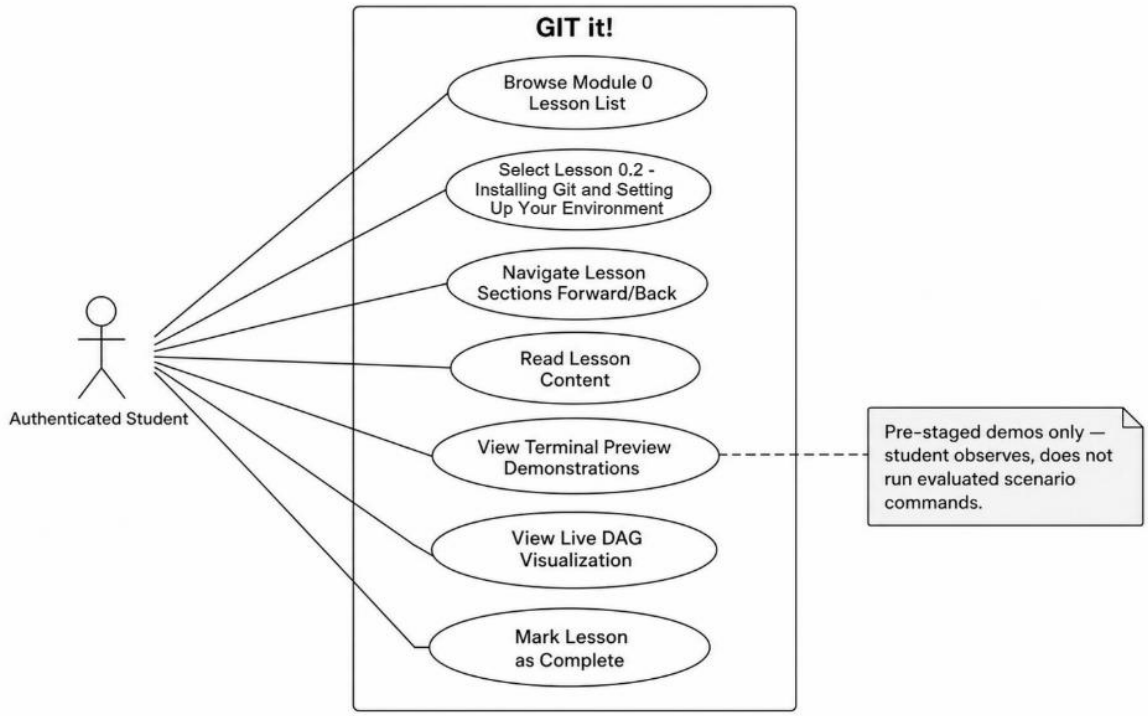


Wireframe



0.2 Complete the Git Installation and Environment Setup Walkthrough

Use Case Diagram

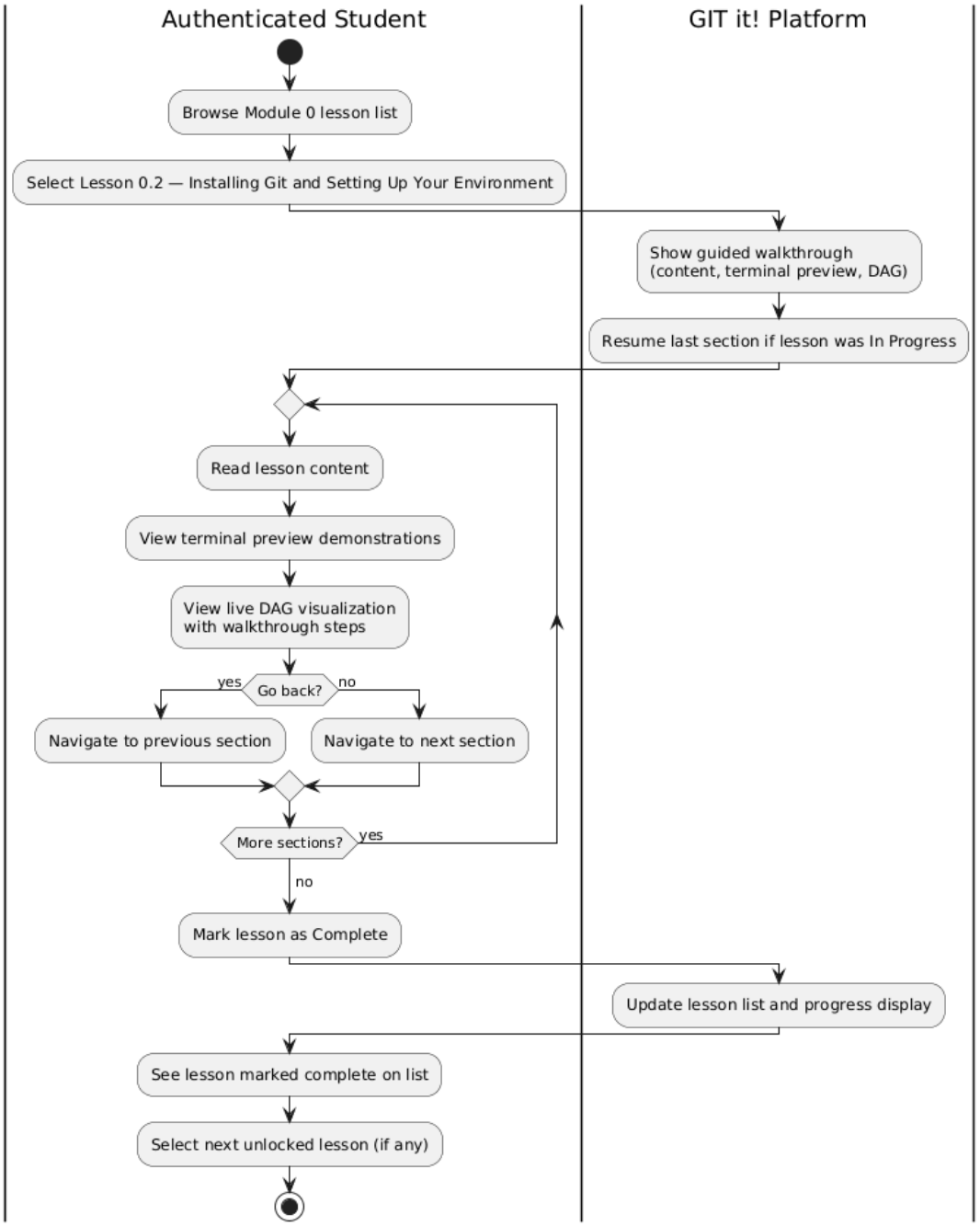


Use Case Description

Field	Description
Use Case ID	UC-ORI-02
Use Case Name	Complete the Git Installation and Environment Setup Walkthrough
Curriculum SO	SO 0.2
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Lesson 0.1 has been completed. Lesson 0.2 is the next incomplete orientation lesson.
Main Flow	1. Student selects Lesson 0.2 — Installing Git and Setting Up Your Environment from the Module 0 lesson list. 2. Platform loads the guided walkthrough with a content panel, a terminal preview panel pre-configured with a simulated shell environment, and the live DAG panel. 3. Student reads the installation instructions for Windows, macOS, and Linux. 4. Student views the terminal preview panel displaying pre-staged command demonstrations: git --version, git config --global user.name, git config --global user.email, and git config --list. 5. Student observes each command's simulated output in the terminal preview panel as the walkthrough steps through each demonstration. 6. Student reads the explanation of why user.name and user.email are embedded in commits as authorship metadata. 7. Student views the DAG panel showing a sample annotated commit node with author and email metadata fields highlighted. 8. Student reads the explanation of git configuration scope levels (system, global, local) and how local overrides global. 9. Student clicks through all lesson sections and reaches the end. 10. Student clicks Mark as Complete. 11. Platform records Lesson 0.2 as completed and updates the progress tracker.
Postconditions	• Lesson 0.2 is recorded as complete in the student's progress tracker. • No scenario session log, CAR, HLCR, ARC, or RTA data is generated.
Alternative Flows	AF1 — Student navigates back mid-lesson: Allowed; completion recorded only on Mark as Complete. AF2 — Student exits before completing: Lesson remains In Progress; progress saved at last section. AF3 — Terminal preview unavailable: Static annotated command output screenshots replace the interactive preview.
Functional Requirements	FR-ORIENT-01, FR-ORIENT-02, FR-ORIENT-03, FR-ORIENT-04, , FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-LES-02, FR-LES-04, FR-TERM-PREVIEW-01, FR-TERM-PREVIEW-02, FR-TERM-PREVIEW-03, FR-ACC-01, FR-ACC-02

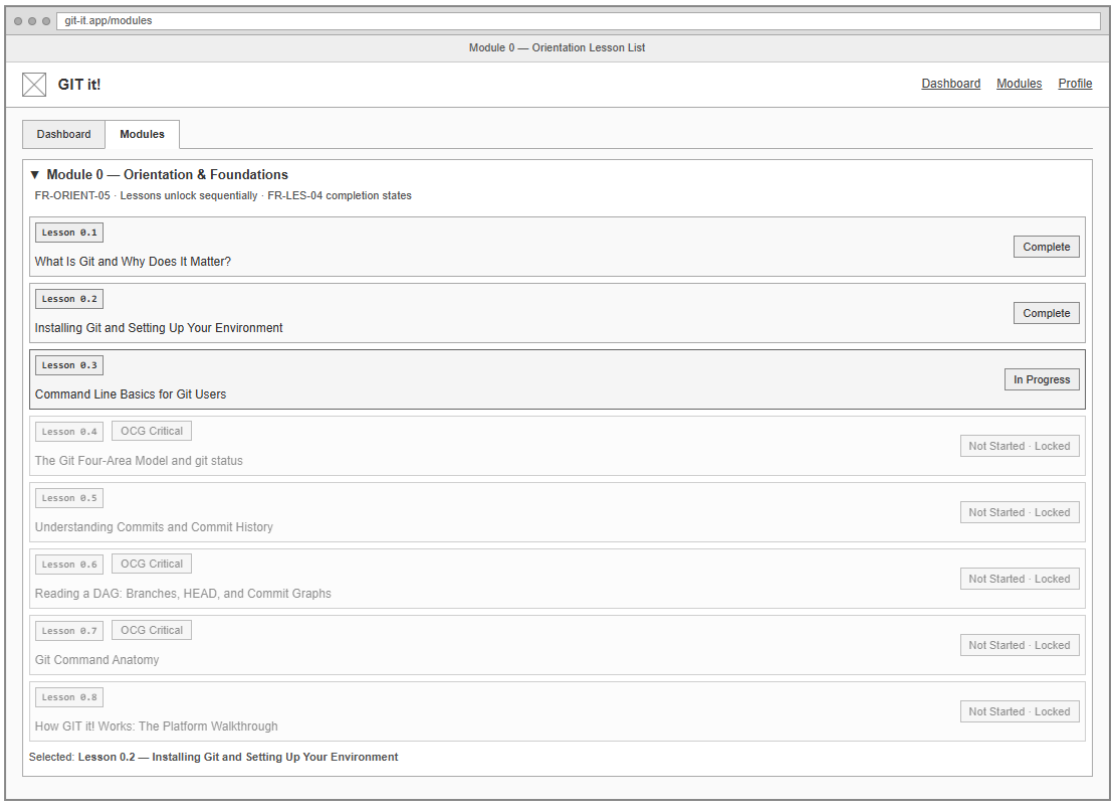
Activity Diagram

UC-ORI-02 — Activity Diagram — Complete the Git Installation and Environment Setup Walkthrough

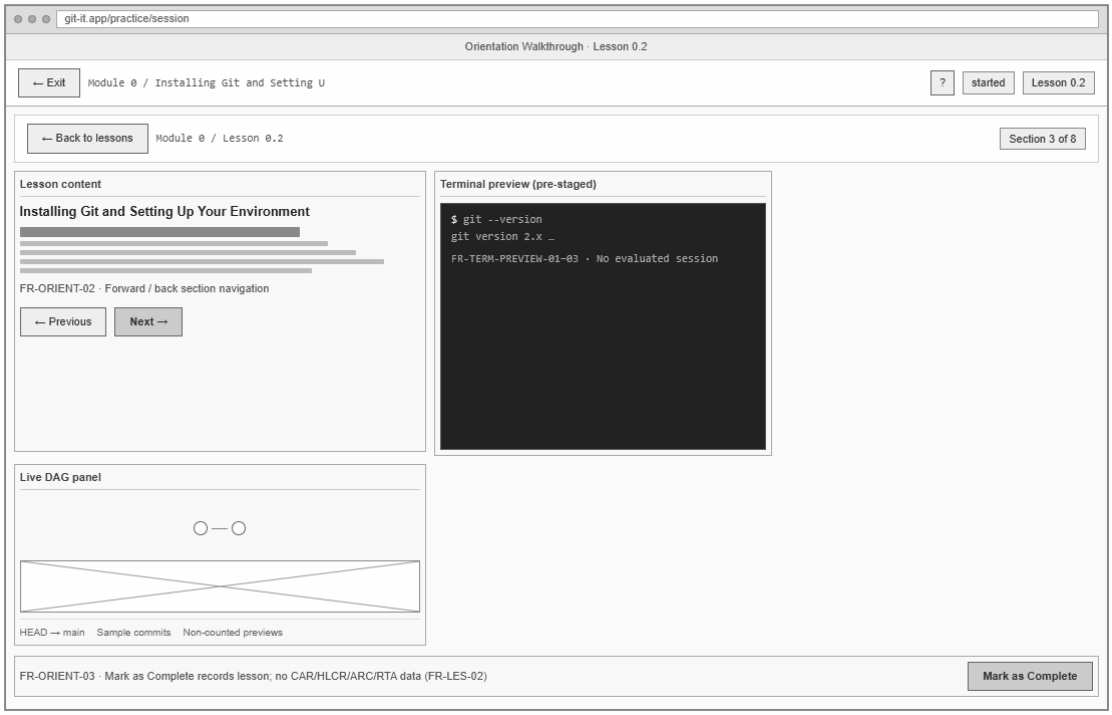


Wireframe

1 · LESSON LIST · UC-ORI-02

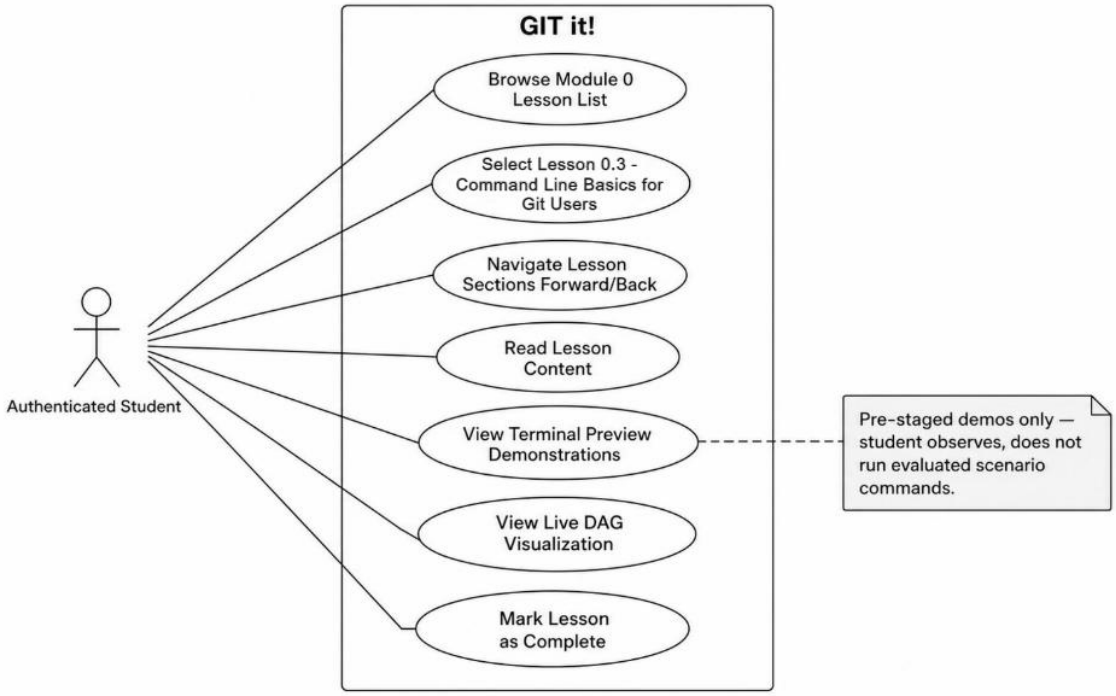


2 · GUIDED WALKTHROUGH · UC-ORI-02



0.3 Complete the Command Line Basics for Git Users Walkthrough

Use Case Diagram



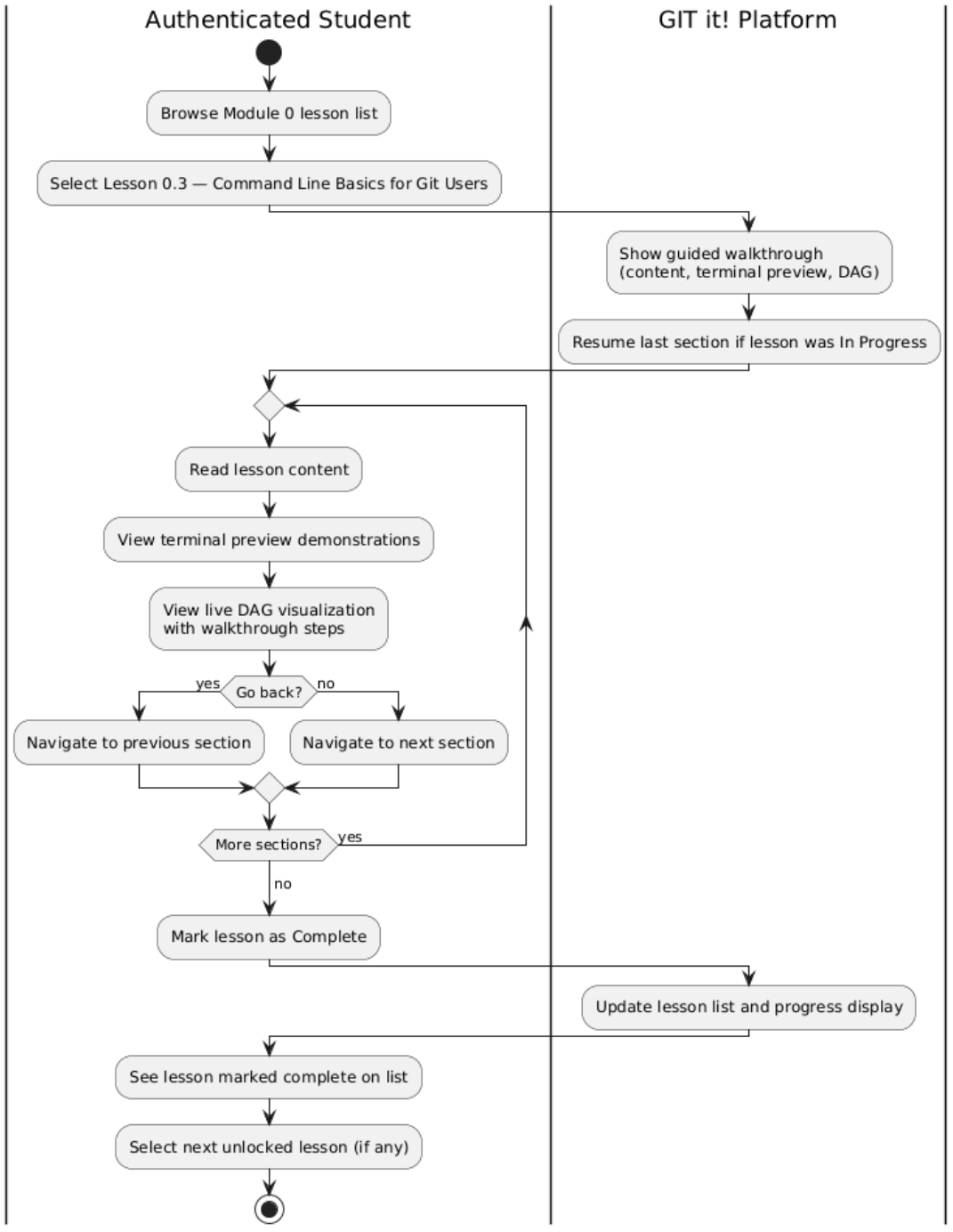
Use Case Description

Field	Description
Use Case ID	UC-ORI-03
Use Case Name	Complete the Command Line Basics for Git Users Walkthrough
Curriculum SO	SO 0.2
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Lessons 0.1 and 0.2 have been completed. Lesson 0.3 is the next incomplete orientation lesson.
Main Flow	1. Student selects Lesson 0.3 — Command Line Basics for Git Users from the Module 0 lesson list. 2. Platform loads the guided walkthrough with a content panel, an interactive terminal preview panel, and the live DAG panel. 3. Student reads the explanation of the working directory, relative and absolute paths, and how filesystem location errors cause Git errors. 4. Student observes terminal preview demonstrations of pwd, ls, ls -la, and cd navigating a sample directory tree. 5. Student reads the explanation of hidden files and the .git directory; the DAG panel highlights the .git directory in a sample repository structure diagram.

	6. Student observes terminal preview demonstrations of touch, cat, echo > file, and echo >> file creating and reading files in the simulated environment. 7. Student reads how file creation commands are used in Git scenario walkthroughs and why GUI editors are not required. 8. Student clicks through all lesson sections. 9. Student clicks Mark as Complete. 10. Platform records Lesson 0.3 as completed and updates the progress tracker.
Postconditions	• Lesson 0.3 is recorded as complete in the student's progress tracker. • No scenario session log, CAR, HLCR, ARC, or RTA data is generated.
Alternative Flows	AF1 — Student navigates back mid-lesson: Allowed; completion recorded only on Mark as Complete. AF2 — Student exits before completing: Lesson remains In Progress; progress saved at last section. AF3 — Terminal preview unavailable: Static annotated command output screenshots replace the interactive preview.
Functional Requirements	FR-ORIENT-01, FR-ORIENT-02, FR-ORIENT-03, FR-ORIENT-04, , FR-DAG-01, FR-DAG-03, FR-LES-02, FR-LES-04, FR-TERM-PREVIEW-01, FR-TERM-PREVIEW-02, FR-TERM-PREVIEW-03, FR-ACC-01, FR-ACC-02

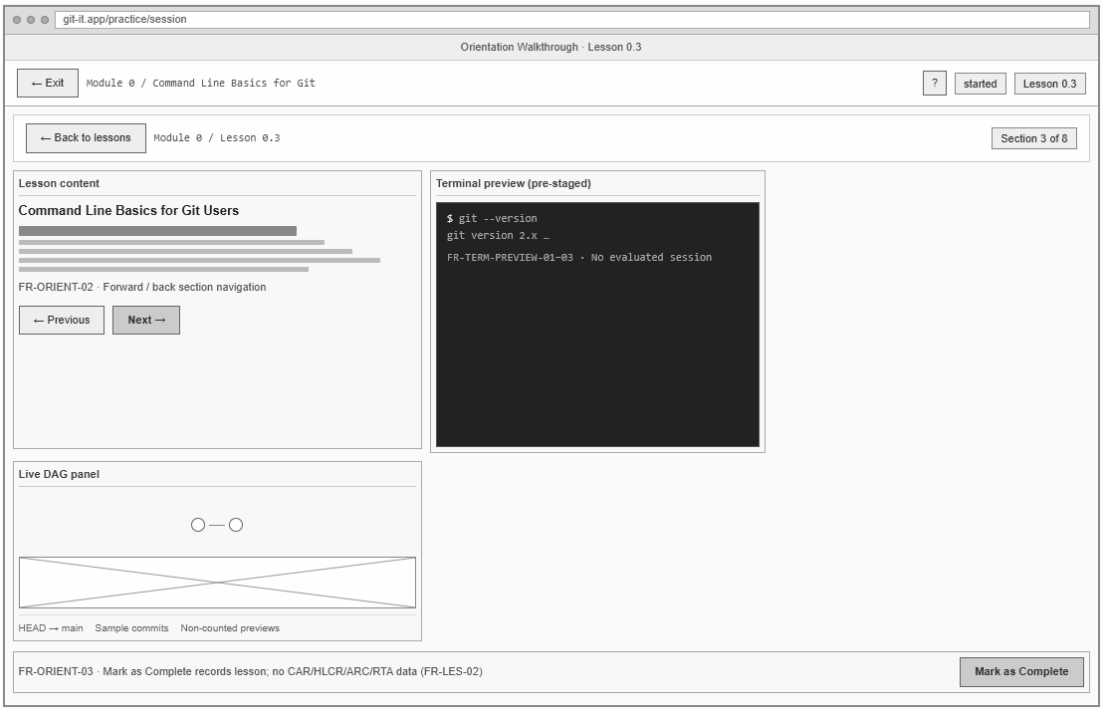
Activity Diagram

UC-ORI-03 — Activity Diagram — Complete the Command Line Basics for Git Users Walkthrough



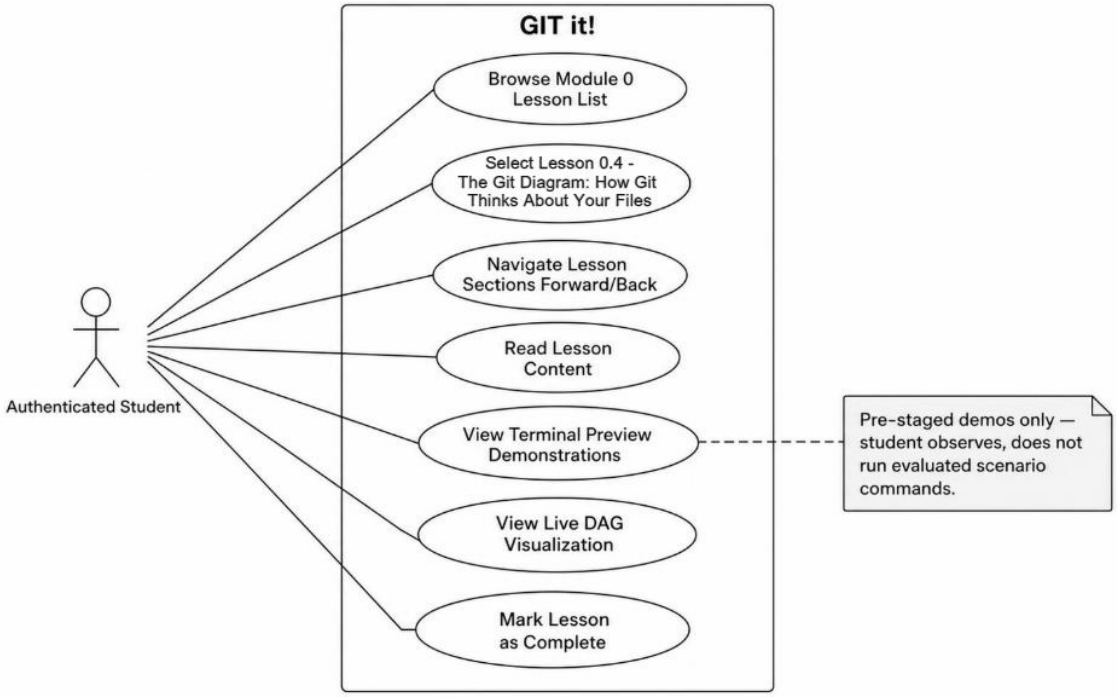
Wireframe

2 · GUIDED WALKTHROUGH · UC-ORI-03



0.4 Complete the Git Four-Area Model and git status Walkthrough

Use Case Diagram

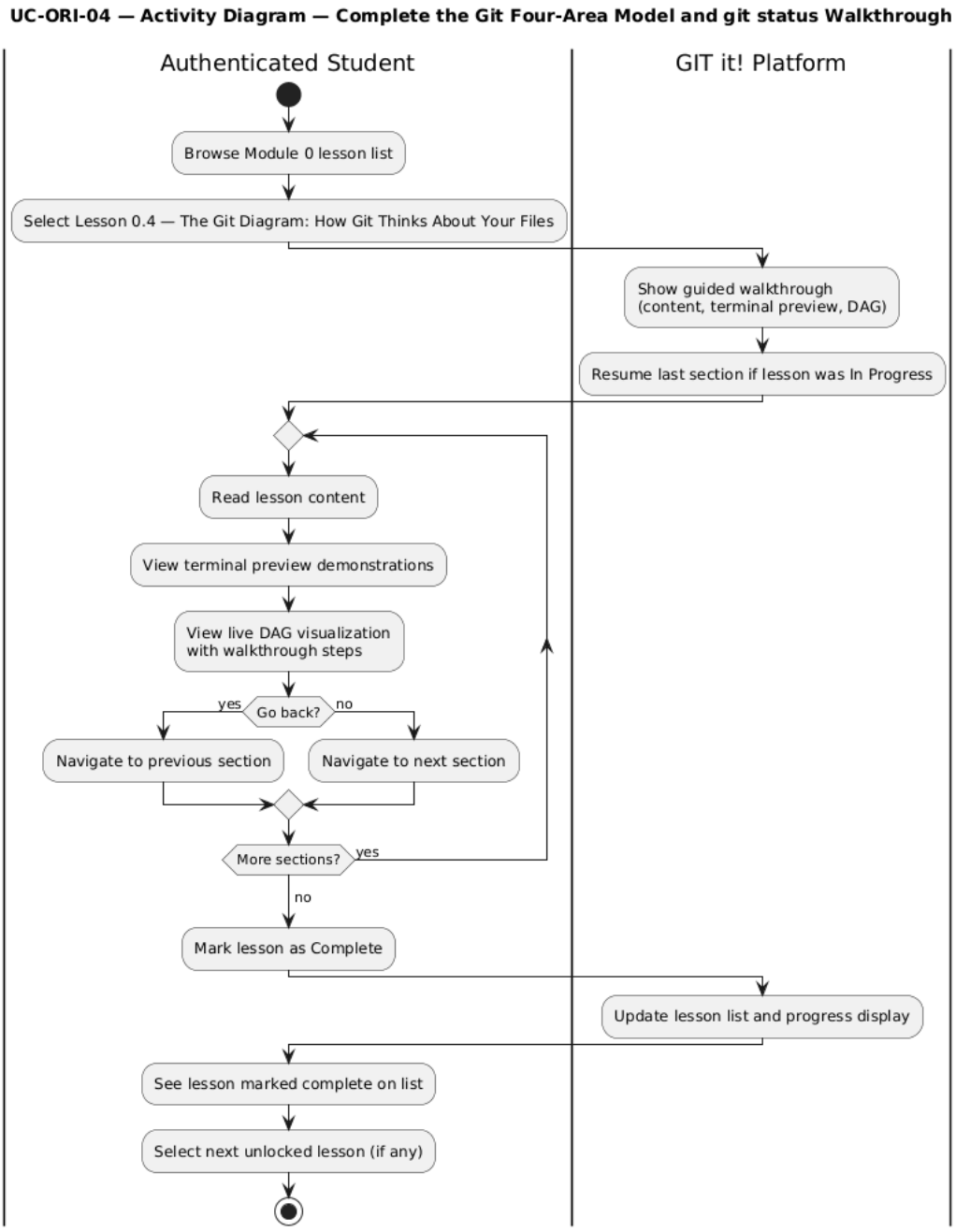


Use Case Description

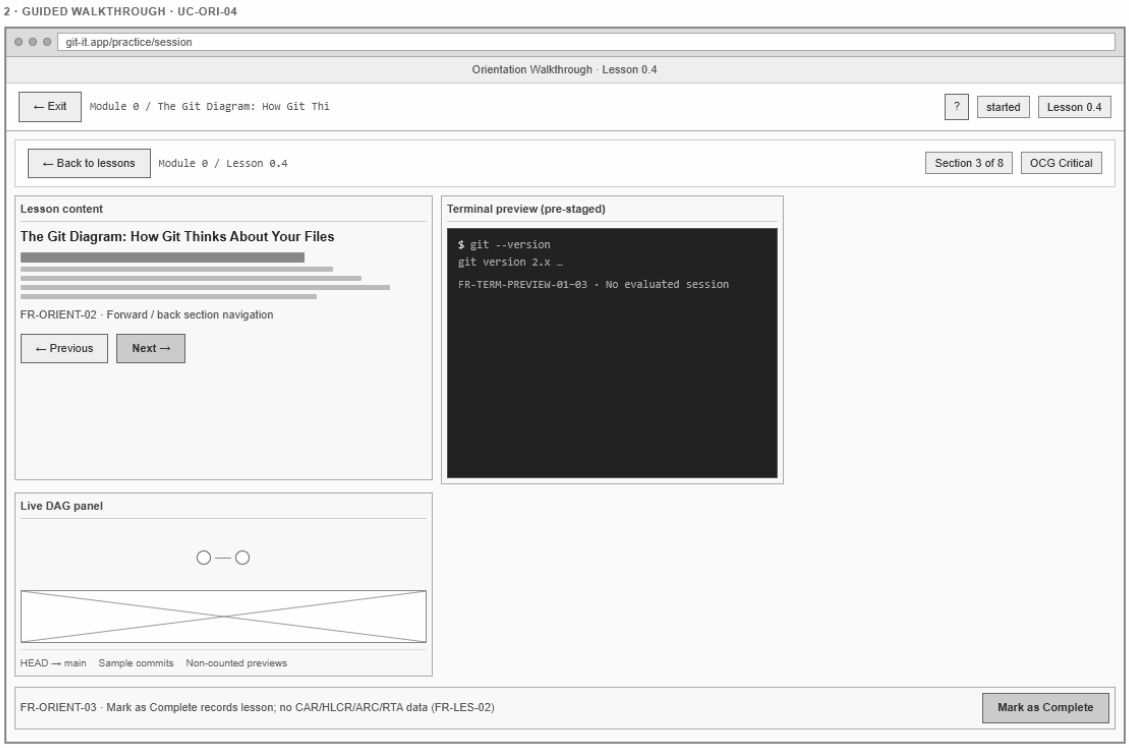
Field	Description
Use Case ID	UC-ORI-04
Use Case Name	Complete the Git Four-Area Model and git status Walkthrough
Curriculum SO	SO 0.1 / SO 0.3
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Lessons 0.1–0.3 have been completed. Lesson 0.4 is the next incomplete orientation lesson.
Main Flow	1. Student selects Lesson 0.4 — The Git Diagram: How Git Thinks About Your Files from the Module 0 lesson list. 2. Platform loads the guided walkthrough with a content panel, an interactive terminal preview panel, and the live DAG panel. 3. Student reads the description of Git's four areas: working directory, staging area, local repository, and remote repository. 4. Student views an animated four-area diagram in the DAG panel showing a file moving from the working directory to the staging area and then to the local repository as the walkthrough advances.

	5. Student reads the explanation of why the staging area exists and how it enables intentional, focused commits. 6. Student observes a pre-staged terminal preview demonstration of git status run after creating a new file, after staging it with git add, and after committing — seeing the output change at each step. 7. Student reads the annotation of each git status output section mapped to the four-area model (untracked, modified unstaged, staged). 8. Student reads the introduction of the live DAG panel as a visual representation of the local repository area. 9. Student observes the DAG panel update in sync with each terminal preview demonstration step. 10. Student clicks through all lesson sections. 11. Student clicks Mark as Complete. 12. Platform records Lesson 0.4 as completed and updates the progress tracker.
Postconditions	• Lesson 0.4 is recorded as complete in the student's progress tracker. • No scenario session log, CAR, HLCR, ARC, or RTA data is generated. • git status is registered as a non-counted diagnostic command for all future scenario sessions this student accesses.
Alternative Flows	AF1 — Student navigates back mid-lesson: Allowed; completion recorded only on Mark as Complete. AF2 — Student exits before completing: Lesson remains In Progress; progress saved at last section. AF3 — DAG animation unavailable: A static sequence of annotated four-area diagrams is displayed in place of the animated panel. AF4 — Terminal preview unavailable: Static annotated command output screenshots replace the interactive preview.
Functional Requirements	FR-ORIENT-01, FR-ORIENT-02, FR-ORIENT-03, FR-ORIENT-04, , FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-LES-02, FR-LES-04, FR-TERM-PREVIEW-01, FR-TERM-PREVIEW-02, FR-TERM-PREVIEW-03, FR-DIAG-01, FR-ACC-01, FR-ACC-02

Activity Diagram

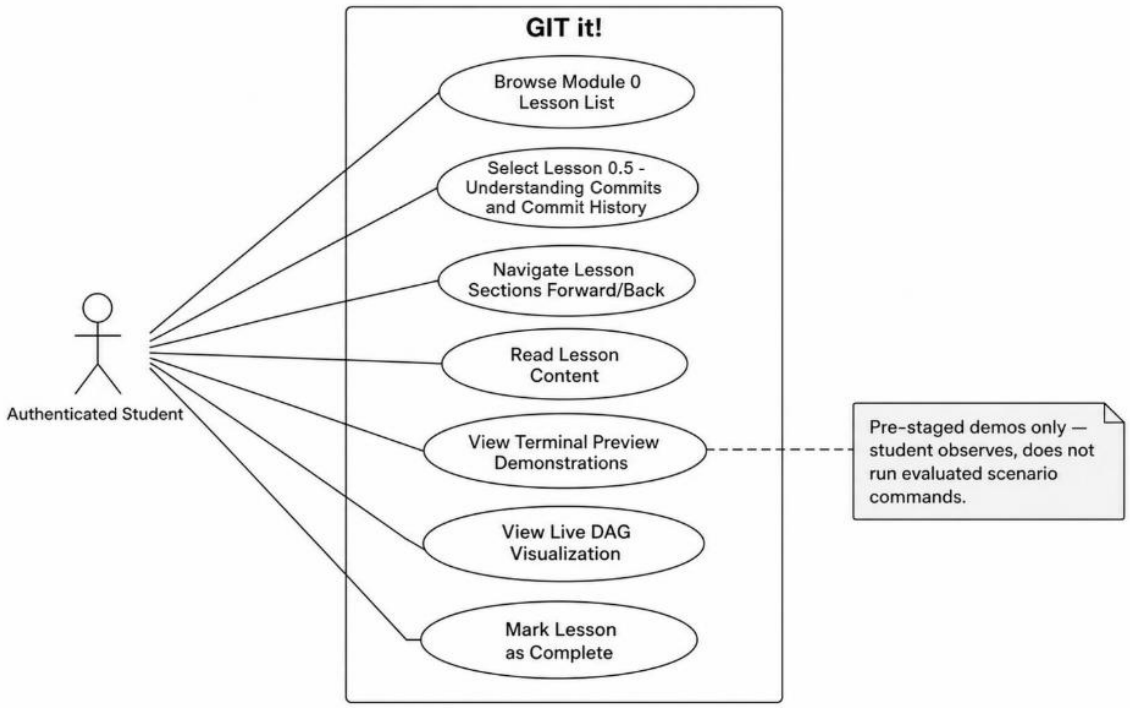


Wireframe


2 · GUIDED WALKTHROUGH · UC-ORI-04
git-it.app/practice/session
Orientation Walkthrough · Lesson 0.4
← Exit
Module 0 / The Git Diagram: How Git Thi
?
started
Lesson 0.4
← Back to lessons
Module 0 / Lesson 0.4
Section 3 of 8
OCG Critical
Lesson content
The Git Diagram: How Git Thinks About Your Files
FR-ORIENT-02 · Forward / back section navigation
← Previous
Next →
Terminal preview (pre-staged)
$ git --version
git version 2.x …
FR-TERM-PREVIEW-01–03 · No evaluated session
Live DAG panel
HEAD → main · Sample commits · Non-counted previews
FR-ORIENT-03 · Mark as Complete records lesson; no CAR/HLCR/ARC/RTA data (FR-LES-02)
Mark as Complete

0.5 Complete the Commits and Commit History Conceptual Walkthrough

Use Case Diagram



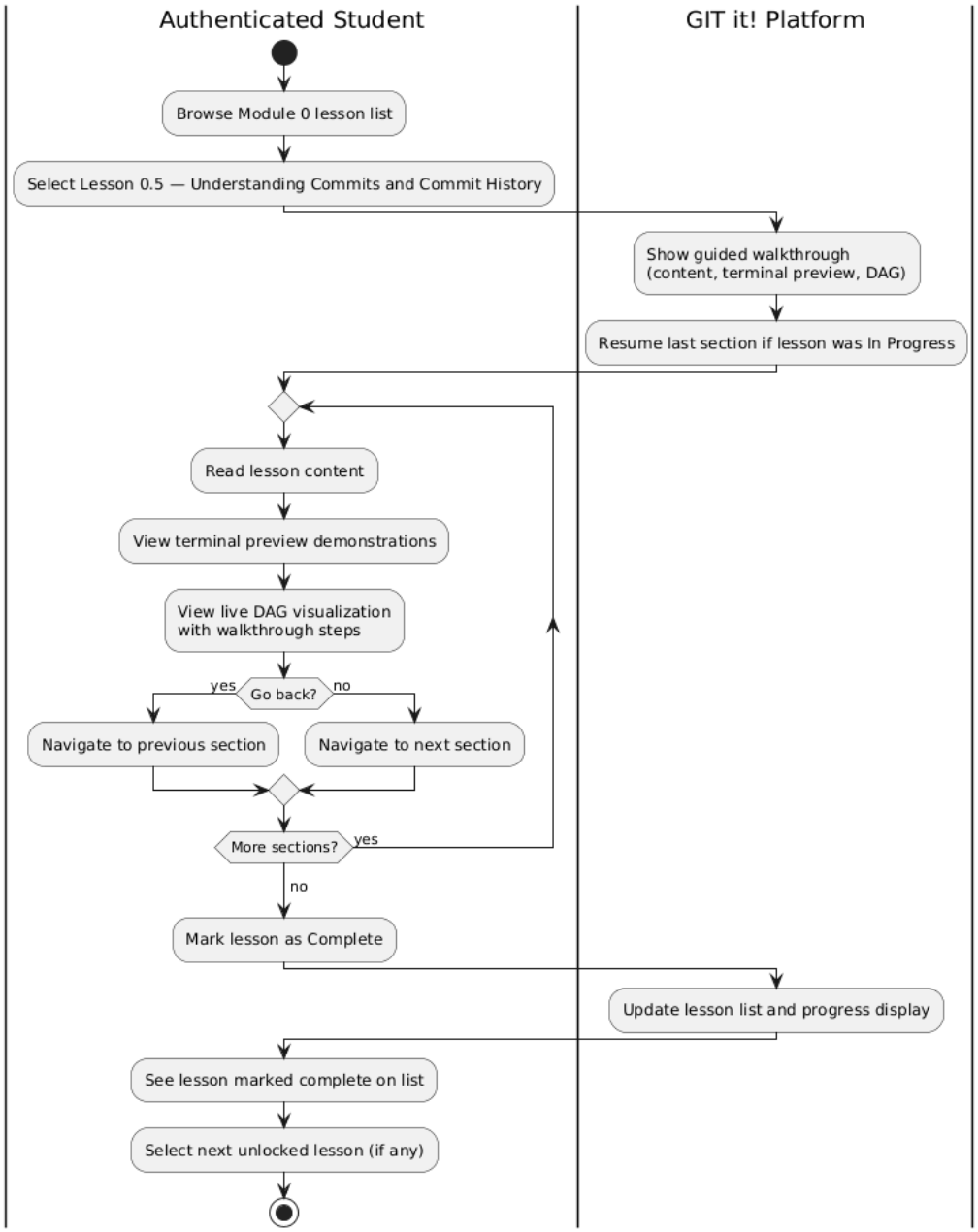
Use Case Description

Field	Description
Use Case ID	UC-ORI-05
Use Case Name	Complete the Commits and Commit History Conceptual Walkthrough
Curriculum SO	SO 0.1
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Lessons 0.1–0.4 have been completed. Lesson 0.5 is the next incomplete orientation lesson.
Main Flow	1. Student selects Lesson 0.5 — Understanding Commits and Commit History from the Module 0 lesson list. 2. Platform loads the guided walkthrough with a content panel, a terminal preview panel, and the live DAG panel. 3. Student reads the anatomy of a Git commit: SHA-1 hash, author, timestamp, message, tree pointer, and parent pointer(s). 4. Student views an annotated commit node in the DAG panel with each anatomy field labeled and explained.

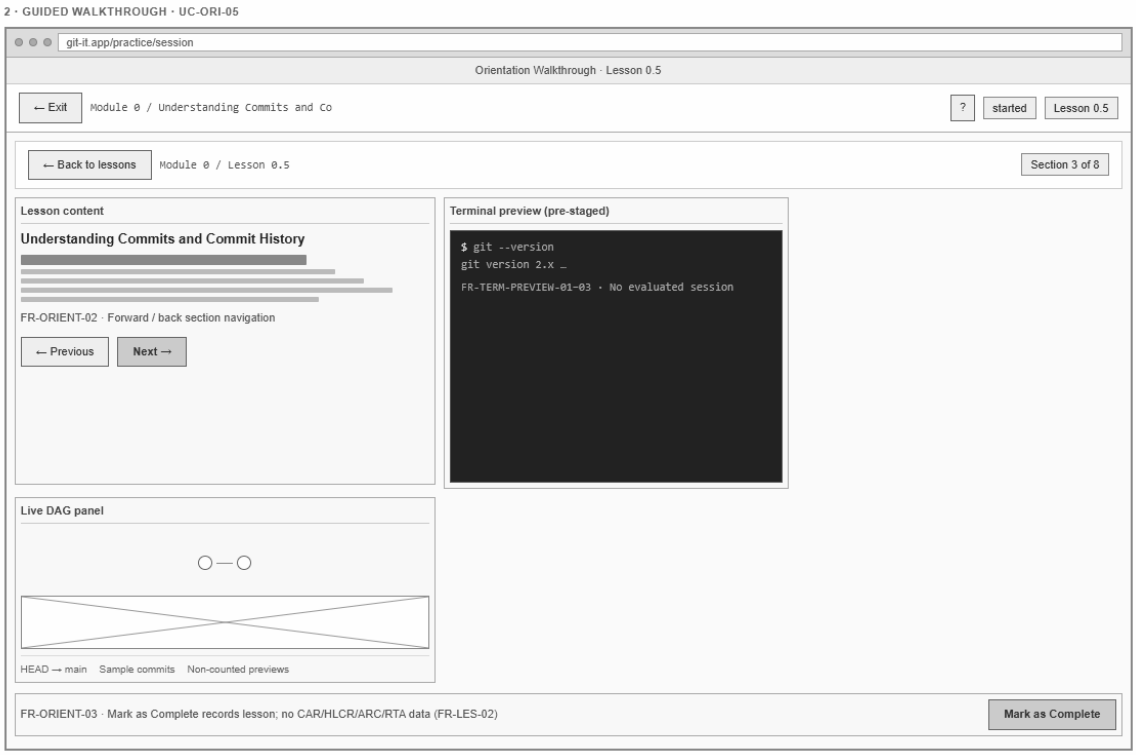
	5. Student reads the explanation of how commits form a chain through parent pointers and how the chain becomes a Directed Acyclic Graph when branches are added. 6. Student observes the DAG panel render a sample linear commit chain, then a branched DAG, as the walkthrough advances. 7. Student reads the explanation of commit immutability: any change to a commit's content or parent creates a new commit with a different hash. 8. Student views a terminal preview demonstration showing git log --oneline output for the sample DAG shown in the panel. 9. Student reads the explanation of good commit message conventions (imperative mood, concise subject line). 10. Student clicks through all lesson sections. 11. Student clicks Mark as Complete. 12. Platform records Lesson 0.5 as completed and updates the progress tracker.
Postconditions	• Lesson 0.5 is recorded as complete in the student's progress tracker. • No scenario session log, CAR, HLCR, ARC, or RTA data is generated.
Alternative Flows	AF1 — Student navigates back mid-lesson: Allowed; completion recorded only on Mark as Complete. AF2 — Student exits before completing: Lesson remains In Progress; progress saved at last section. AF3 — DAG animation unavailable: Static annotated commit chain and branched DAG diagrams replace animated renders. AF4 — Terminal preview unavailable: Static annotated git log --oneline output screenshots replace the interactive preview.
Functional Requirements	FR-ORIENT-01, FR-ORIENT-02, FR-ORIENT-03, FR-ORIENT-04, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-LES-02, FR-LES-04, FR-TERM-PREVIEW-01, FR-TERM-PREVIEW-02, FR-ACC-01, FR-ACC-02

Activity Diagram

UC-ORI-05 — Activity Diagram — Complete the Commits and Commit History Conceptual Walkthrough

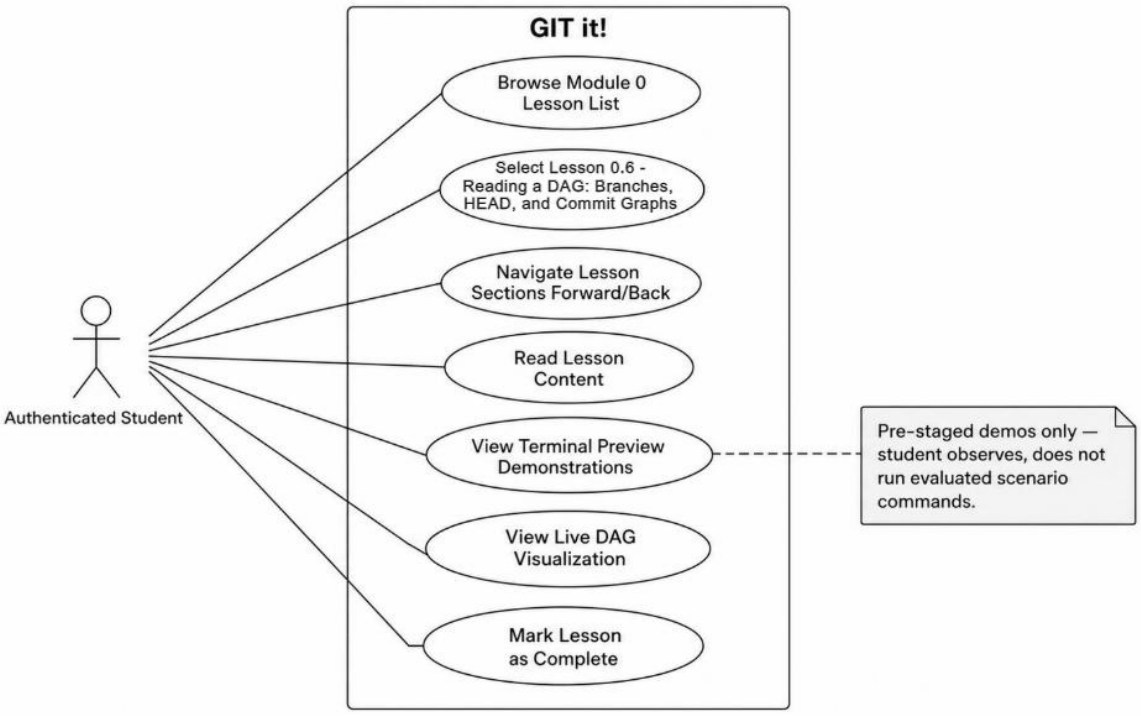


Wireframe


2 · GUIDED WALKTHROUGH · UC-ORI-05
git-it.app/practice/session
Orientation Walkthrough · Lesson 0.5
← Exit
Module 0 / Understanding Commits and Co
?
started
Lesson 0.5
← Back to lessons
Module 0 / Lesson 0.5
Section 3 of 8
Lesson content
Understanding Commits and Commit History
FR-ORIENT-02 · Forward / back section navigation
← Previous
Next →
Terminal preview (pre-staged)
$ git --version
git version 2.x …
FR-TERM-PREVIEW-01–03 · No evaluated session
Live DAG panel
HEAD → main
Sample commits
Non-counted previews
FR-ORIENT-03 · Mark as Complete records lesson; no CAR/HLCR/ARC/RTA data (FR-LES-02)
Mark as Complete

0.6 Complete the DAG Reading, HEAD, and Branch Divergence Walkthrough

Use Case Diagram



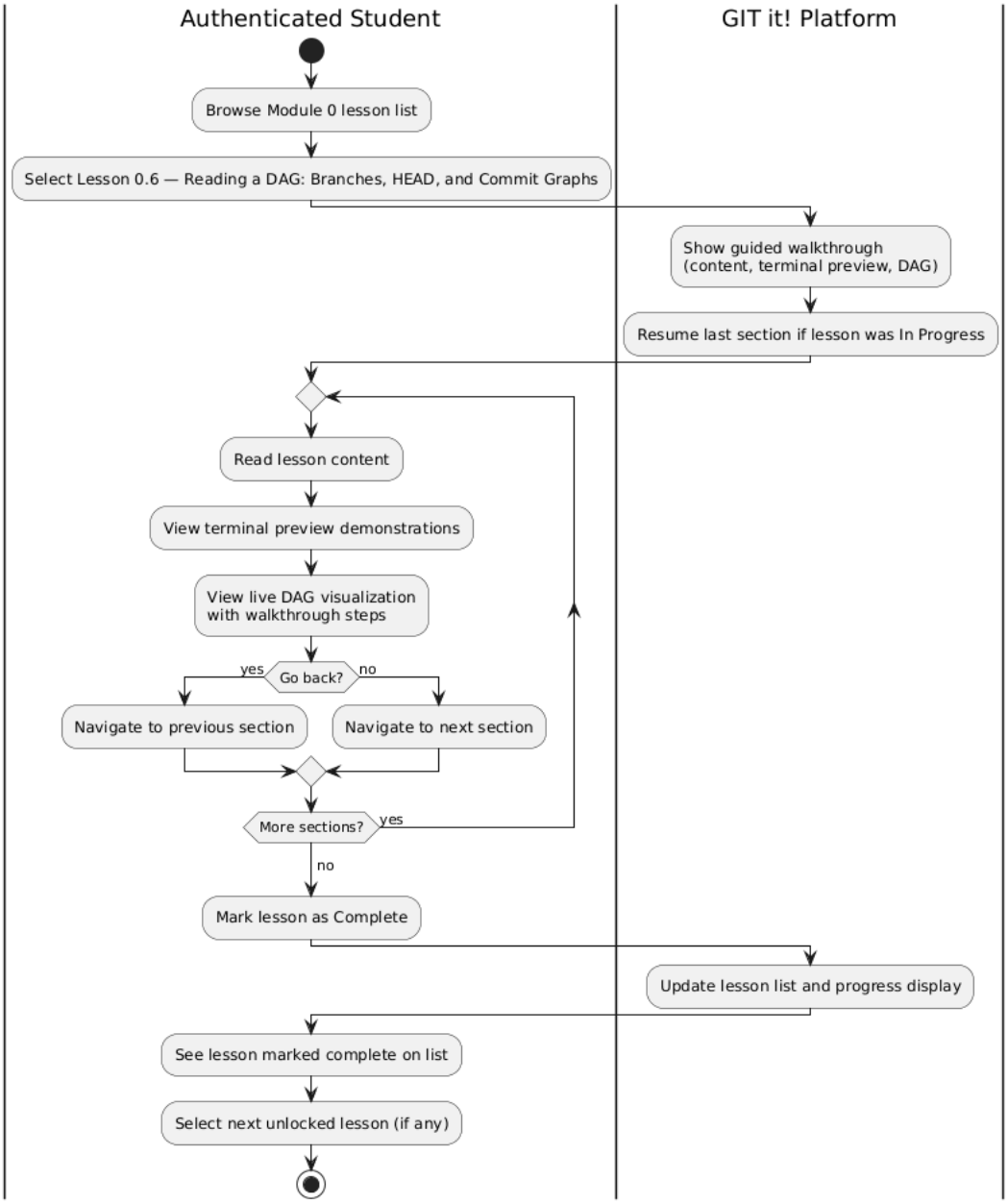
Use Case Description

Field	Description
Use Case ID	UC-ORI-06
Use Case Name	Complete the DAG Reading, HEAD, and Branch Divergence Walkthrough
Curriculum SO	SO 0.3 / SO 0.4
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Lessons 0.1–0.5 have been completed. Lesson 0.6 is the next incomplete orientation lesson.
Main Flow	1. Student selects Lesson 0.6 — Reading a DAG: Branches, HEAD, and Commit Graphs from the Module 0 lesson list. 2. Platform loads the guided walkthrough with a content panel, a terminal preview panel, and the live DAG panel. 3. Student reads the description of DAG components: commit nodes, branch labels, HEAD pointer, edges, merge commits, and divergence points. 4. Student views the live DAG panel rendering a sample repository with multiple branches, HEAD attached to a branch, and visible merge commits.

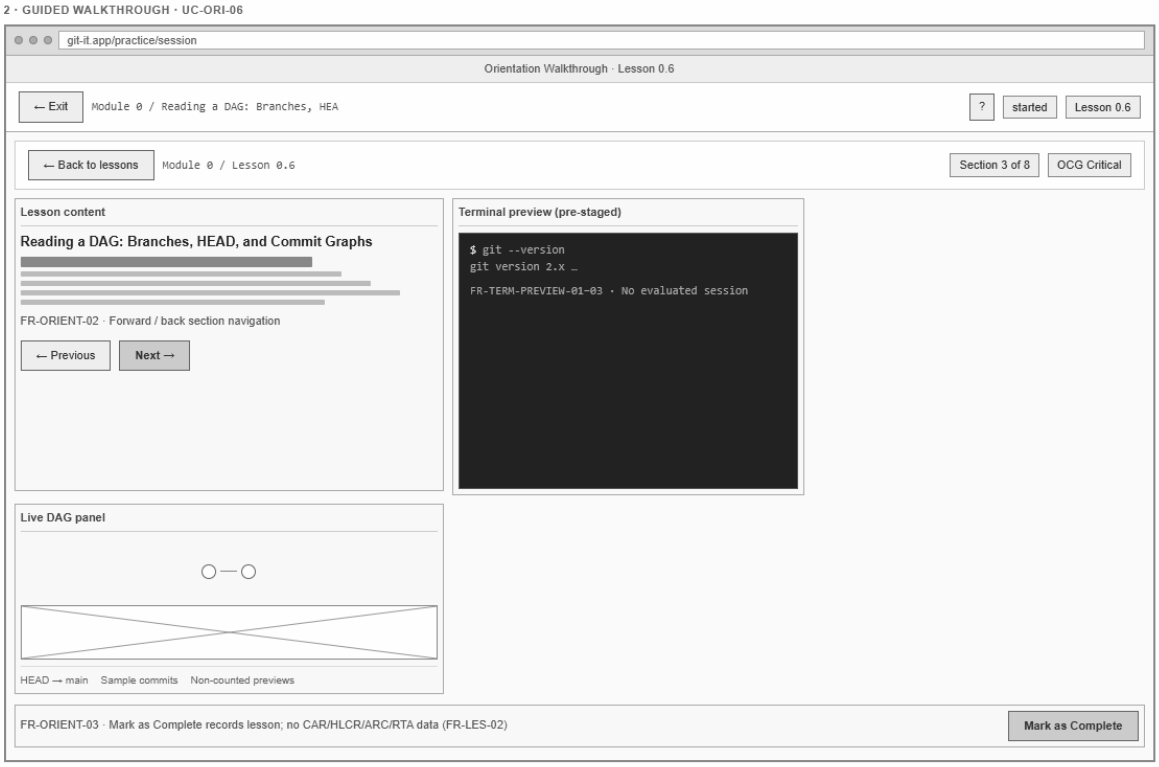
	5. Student reads the detailed explanation of HEAD in attached state (pointing to a branch, which points to a commit) and in detached state (pointing directly to a commit hash). 6. Student observes the DAG panel update to show detached HEAD state as the walkthrough steps through a git checkout <hash> demonstration. 7. Student views a terminal preview demonstration of git log --oneline --graph --all and reads the walkthrough's annotation of the ASCII art output, mapping each element to the live DAG. 8. Student views a terminal preview demonstration of git log --oneline --decorate and reads how branch and tag labels appear in linear history. 9. Student reads the explanation of common ancestor (merge base) and divergence between two branches. 10. Student observes the DAG panel highlight the common ancestor commit as the walkthrough identifies the divergence point in a sample two-branch graph. 11. Student clicks through all lesson sections. 12. Student clicks Mark as Complete. 13. Platform records Lesson 0.6 as completed and updates the progress tracker.
Postconditions	• Lesson 0.6 is recorded as complete in the student's progress tracker. • No scenario session log, CAR, HLCR, ARC, or RTA data is generated. • All git log variants (git log, git log --oneline, git log --oneline --graph --all, git log --oneline --decorate, git log --oneline --graph) are registered as non-counted diagnostic commands for all future scenario sessions.
Alternative Flows	AF1 — Student navigates back mid-lesson: Allowed; completion recorded only on Mark as Complete. AF2 — Student exits before completing: Lesson remains In Progress; progress saved at last section. AF3 — DAG animation unavailable: Static annotated DAG diagrams showing attached HEAD, detached HEAD, and divergence structure replace animated renders. AF4 — Terminal preview unavailable: Static annotated ASCII graph and log output screenshots replace the interactive preview.
Functional Requirements	FR-ORIENT-01, FR-ORIENT-02, FR-ORIENT-03, FR-ORIENT-04, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-LES-02, FR-LES-04, FR-TERM-PREVIEW-01, FR-TERM-PREVIEW-02, FR-TERM-PREVIEW-03, FR-DIAG-01, FR-DIAG-02, FR-ACC-01, FR-ACC-02

Activity Diagram

UC-ORI-06 — Activity Diagram — Complete the DAG Reading, HEAD, and Branch Divergence Walkthrough

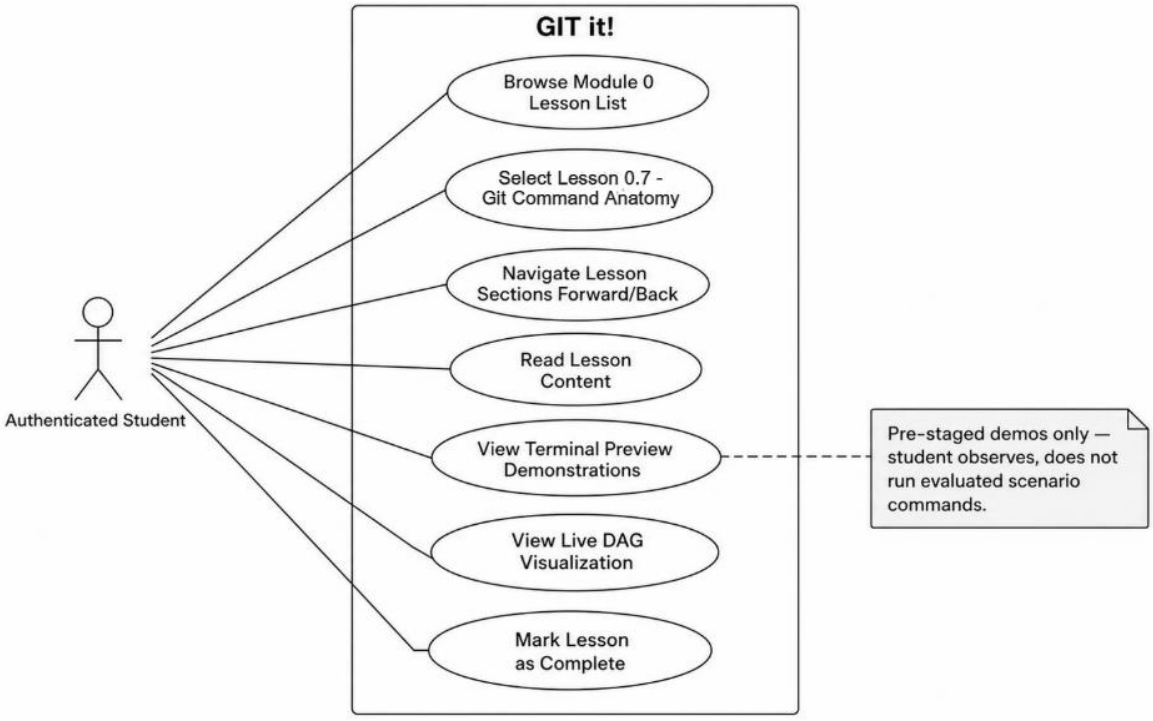


Wireframe


2 · GUIDED WALKTHROUGH · UC-ORI-06
git-it.app/practice/session
Orientation Walkthrough · Lesson 0.6
← Exit
Module 0 / Reading a DAG: Branches, HEA
?
started
Lesson 0.6
← Back to lessons
Module 0 / Lesson 0.6
Section 3 of 8
OCG Critical
Lesson content
Reading a DAG: Branches, HEAD, and Commit Graphs
FR-ORIENT-02 · Forward / back section navigation
← Previous
Next →
Terminal preview (pre-staged)
$ git --version
git version 2.x …
FR-TERM-PREVIEW-01-03 · No evaluated session
Live DAG panel
HEAD → main
Sample commits
Non-counted previews
FR-ORIENT-03 · Mark as Complete records lesson; no CAR/HLCR/ARC/RTA data (FR-LES-02)
Mark as Complete

0.7 Complete the Git Command Anatomy Walkthrough

Use Case Diagram

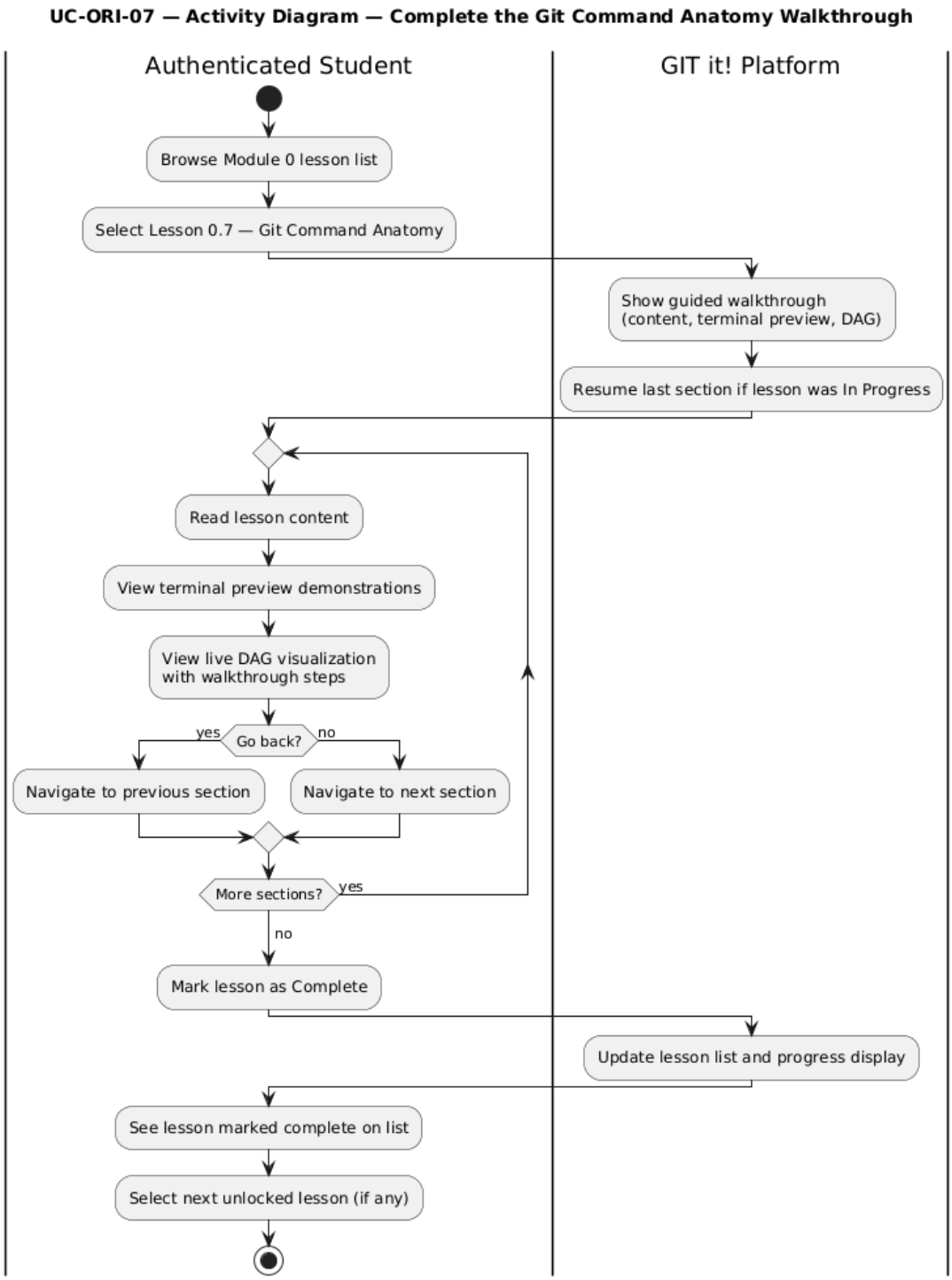


Use Case Description

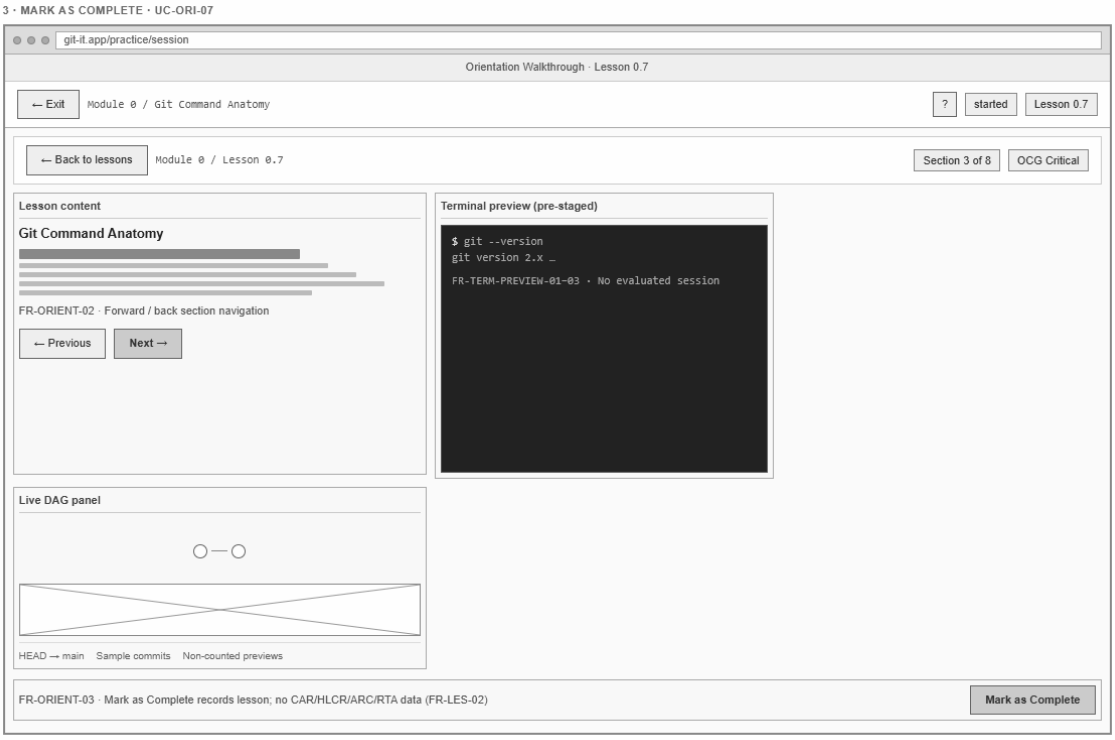
Field	Description
Use Case ID	UC-ORI-07
Use Case Name	Complete the Git Command Anatomy Walkthrough
Curriculum SO	SO 0.2
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Lessons 0.1–0.6 have been completed. Lesson 0.7 is the next incomplete orientation lesson.
Main Flow	1. Student selects Lesson 0.7 — Git Command Anatomy from the Module 0 lesson list. 2. Platform loads the guided walkthrough with a content panel, a terminal preview panel, and the live DAG panel. 3. Student reads the anatomy of a Git command: base command (git), subcommand, options/flags, and arguments, with a labeled structural diagram. 4. Student views annotated decompositions of example commands in the terminal preview panel: git commit -m, git push --force-with-lease, and git log --oneline --graph --all.

	5. Student reads the distinction between short flags (-m) and long flags (--message) and their equivalence. 6. Student reads the explanation of positional arguments and views terminal preview examples: git merge <branch> and git diff <commit1> <commit2>. 7. Student views a terminal preview demonstration of git commit --help output with the synopsis line annotated to explain square brackets (optional) and angle brackets (required). 8. Student reads examples of Git error messages and their plain-English meanings: fatal: not a git repository, error: failed to push some refs, and CONFLICT (content): Merge conflict in <file>. 9. Student observes the terminal preview panel display each error message with an annotation explaining which command component caused it. 10. Student clicks through all lesson sections. 11. Student clicks Mark as Complete. 12. Platform records Lesson 0.7 as completed and updates the progress tracker.
Postconditions	• Lesson 0.7 is recorded as complete in the student's progress tracker. • No scenario session log, CAR, HLCR, ARC, or RTA data is generated. • git help, git <subcommand> --help, and git <subcommand> -h are registered as non-counted diagnostic commands for all future scenario sessions.
Alternative Flows	AF1 — Student navigates back mid-lesson: Allowed; completion recorded only on Mark as Complete. AF2 — Student exits before completing: Lesson remains In Progress; progress saved at last section. AF3 — Terminal preview unavailable: Static annotated command decomposition diagrams and error message screenshots replace the interactive preview.
Functional Requirements	FR-ORIENT-01, FR-ORIENT-02, FR-ORIENT-03, FR-ORIENT-04, , FR-DAG-01, FR-LES-02, FR-LES-04, FR-TERM-PREVIEW-01, FR-TERM-PREVIEW-02, FR-TERM-PREVIEW-03, FR-DIAG-01, FR-ACC-01, FR-ACC-02

Activity Diagram



Wireframe


3 · MARK AS COMPLETE · UC-ORI-07
git-it.app/practice/session
Orientation Walkthrough · Lesson 0.7
← Exit
Module 0 / Git Command Anatomy
?
started
Lesson 0.7
← Back to lessons
Module 0 / Lesson 0.7
Section 3 of 8
OCG Critical
Lesson content
Git Command Anatomy
FR-ORIENT-02 · Forward / back section navigation
← Previous
Next →
Terminal preview (pre-staged)
$ git --version
git version 2.x …
FR-TERM-PREVIEW-01-03 · No evaluated session
Live DAG panel
HEAD → main
Sample commits
Non-counted previews
FR-ORIENT-03 · Mark as Complete records lesson; no CAR/HLCR/ARC/RTA data (FR-LES-02)
Mark as Complete

practice workspace — Shared Functional Requirements

The following functional requirements apply across all scenario practice use cases in Modules 1–4 and are referenced collectively rather than repeated per use case.

FR ID	Summary	Requirement Statement
FR-DAG-01	Live DAG visible at all difficulties	The system shall present and keep visible the repository graph at Easy, Medium, and Hard during scenario practice.
FR-DAG-02	DAG reflects each processed command	The system shall update the repository graph after every simulator-processed command to reflect the current internal repository state.
FR-DAG-03	Observable commit graph semantics	The system shall render commit history, branch pointers, and HEAD position so students can observe reachability and attached or detached HEAD during practice.
FR-DAG-04	Animated state transitions	The system shall animate repository state transitions in the live DAG when the internal state changes.
FR-DAG-05	Graph available at every screen size	The system shall keep the repository graph available during a run, without horizontal scrolling, at screen widths from 390 px up.
FR-DAG-06	Expected-state diagram on Easy and Medium	The system shall display the target-state guidance on Easy and Medium only and shall not display it on Hard.
FR-DASH-02	Per-difficulty completion on skill focus cards	The system shall display per-scenario completion status for Easy, Medium, and Hard on each learning objective card within an expanded chapter.
FR-DASH-04	Encounter star indicators	The system shall display the 1–3 stars earned for a cleared scenario and each tier's stars (the lowest of its scenarios' best stars), based on the clear, the command target and the retry allowance (FR-GAM-01).
FR-DASH-05	Per-scenario retry-count trends	The system shall display retry-count trends per learning objective derived from completed and abandoned session history, and shall display 'No trend available' when insufficient historical data exists.
FR-DASH-07	KPI summary indicators on dashboard	The system shall display KPI summary indicators for the active published scenario library on the Home, including module-relevant metrics such as SCR, CAR, HLCR, RTA, and ARC when sufficient session data is available.
FR-DASH-15	Expand unit to show skill focuses	The system shall allow an authenticated student to expand a chapter entry on the Stories and chapter map and view learning objective entries directly for scenario-bearing units.
FR-DASH-16	Consistent card and session data	The system shall ensure unit-level learning objective cards use the same scenario records, difficulty access rules, and completion states as the scenario practice session entry points.
FR-DASH-17	Three difficulties with access states	The system shall display Easy, Medium, and Hard for every learning objective with the current access state: Not Started, In Progress, Completed, Locked, Failed, or Abandoned.

FR-DASH-18	Easy → Medium → Hard gating	The system shall lock Medium until Easy is completed and shall lock Hard until Medium is completed for the same learning objective.
FR-DASH-19	Difficulty action labels	The system shall display the action label and icon for each difficulty as Start, Continue, Replay, Lock, or Retry according to the student's current session and completion state.
FR-DASH-20	Level and tier access order	The system shall open levels within a chapter in order — a level unlocks once the previous level's Easy tier is complete — and tiers in the order Easy, Medium, Hard. It shall not gate practice on quizzes, manual approval or other chapters' progress.
FR-PREVIEW-01	Retired: pre-practice modal title	Retired in version 1.3. The pre-practice level details and command guide modal was replaced by the mission briefing (FR-PREVIEW-02) and the in-workspace command guide (FR-TEACH-01).
FR-PREVIEW-02	Mission briefing	The system shall open a mission briefing with the scenario's story and task the first time a level is played, and let the learner reopen it from Story during the run.
FR-PREVIEW-03	No solution leakage	The system shall not show a scenario's solution sequence, target-state rules, evaluator data or hidden notes in the mission briefing, the command guide, tips or the AI tutor.
FR-PREVIEW-04	Retired: preview demo commands	Retired in version 1.3. Preview demo commands were removed together with the pre-practice modal.
FR-PREVIEW-05	Retired: preview step navigation	Retired in version 1.3. The pre-practice modal and its steps were removed.
FR-PREVIEW-06	Retired: Start Practice and Skip routing	Retired in version 1.3. Learners start a tier directly from its level on the story map.
FR-PREVIEW-07	Replay remains playable	The system shall allow learners to play a replay after tier completion, log it separately from primary runs, and exclude it from primary rewards and performance measures.
FR-SCENE-01	Load allowed variant on session start	The system shall load a valid variant from the selected difficulty instance's variant pool when a new practice session starts, after validating that the chosen difficulty action is allowed.
FR-SCENE-02	Simulator only in runs	The system shall run the repository simulator only inside adventure and challenge runs.
FR-SCENE-03	Difficulty-appropriate scaffolding	The system shall display Easy with live DAG, target-state guidance, and contextual feedback; Medium with live DAG and target-state guidance; and Hard with live DAG and narrative context only.
FR-SCENE-04	Session log on scenario load	The system shall create and persist a session log when a scenario loads, recording scenario, difficulty instance, difficulty level, variant, timestamp, entry point, and command-count policy.
FR-SCENE-05	Terminal focused on load	The system shall focus the terminal input by default when a scenario practice workspace loads so the student can begin typing immediately.

FR-SCENE-05A	Load from chapter entry actions	The system shall load a replay from an eligible completed difficulty-tier entry point and record the run as a replay, separate from primary attempts.
FR-SCENE-05B	Authored command-count policy per instance	The system shall attach an authored command-count policy to each loaded difficulty instance, including minimum counted-command threshold, maximum counted-command limit, and non-counted diagnostic command patterns.
FR-SCENE-05C	Per-instance narrative and targets	The system shall configure each difficulty instance with its own narrative or task prompt, starting repository state, target-state rule, target-state guidance behavior, and variant pool.
FR-SCENE-06	Simulator processes implemented commands	The system shall pass every submitted command to the repository state simulator and shall update internal repository state only when the command maps to an implemented simulator operation.
FR-SCENE-07	State-based evaluation	The system shall evaluate practice progress by comparing the resulting repository state to the expected target state, not by matching a single authored command sequence.
FR-SCENE-08	Accept valid alternate paths	The system shall accept any simulator-processable command or sequence that reaches the correct target state, even when it differs from the authored reference sequence.
FR-SCENE-09	Live DAG updates after commands	The system shall update the repository graph after every simulator-processed command at all difficulty levels.
FR-SCENE-10	Command submission logging	The system shall log every command submission with timestamp, attempt number, result category, command classification, counted-command increment, and running counted-command total.
FR-SCENE-11	Easy contextual feedback	The system shall display a state-change description in the contextual feedback panel on Easy after each simulator-processed command, and shall show terminal-style output only for unprocessable commands.
FR-SCENE-12	Proposed guided teaching boundary in workspace	Proposed teaching boundary: the practice workspace may provide general command syntax and concept examples, but shall not present a scenario-specific required solution sequence as the answer. Adviser review is required before this amendment becomes the approved baseline.
FR-SCENE-12A	State equivalence dimensions	The system shall evaluate state equivalence using commit graph relationships, branch pointers, HEAD position, staging and working-tree status, conflict status, and required file markers when applicable.
FR-SCENE-12B	Classify before counter update	The system shall classify simulator-processed commands against the session command-count policy before updating the counted-command allowance or CAR-related totals.
FR-SCENE-12C	Fail on counted-command limit	The system shall mark a scenario session as Failed when the maximum counted-command limit is reached before all required target states are achieved.

FR-SCENE-13	Persist completion immediately	The system shall persist scenario completion records immediately when the final step receives a target-matched result.
FR-SCENE-14	Encounter star scoring rules	The system shall score each cleared scenario with 1–3 stars as defined in FR-GAM-01. Opening the command guide never costs a star.
FR-SCENE-15	Unlock next difficulty	The system shall unlock Medium after Easy completion and unlock Hard after Medium completion for the same learning objective.
FR-SCENE-16	Retired: daily streak update	Retired in version 1.3. The daily streak was removed on 24 October 2026; achievements count what the learner cleared (FR-GAM-06).
FR-SCENE-17	Different variant on retry	The system shall serve a different variant from the difficulty instance pool on retry after a failed or abandoned session when a structurally different variant is available.
FR-SCENE-18	Session outcome logging	The system shall log retries used, session duration, difficulty instance, variant, command-count policy, counted and non-counted command totals, completion status, and first-attempt eligibility.
FR-SCENE-19	Completion screen	The system shall show a completion screen with the scenario, difficulty and stars earned, explaining each star criterion and why one was missed, without revealing correct commands or sequences.
FR-TERM-01	Accept Git command input	The system shall accept free-text Git command input in the scenario practice terminal input field.
FR-TERM-02	Chronological command output	The system shall display entered commands and their results in chronological order in the terminal output area with a prompt prefix.
FR-TERM-03	In-session command history	The system shall support Up and Down arrow keys to navigate command history within the current session.
FR-TERM-04	One command at a time	The system shall process one command per run at a time; a command sent while the previous one is still being verified is rejected with HTTP 409 and can be sent again. The terminal stops accepting input once the run ends.
FR-TERM-05	Keyboard-accessible terminal	The system shall provide a keyboard-accessible terminal interface operable without a mouse.
FR-TERM-06	Simulated Git only	The system shall treat the terminal as a simulated Git interface and shall never execute submitted text through a real shell, operating-system process, or external Git binary.
FR-TERM-07	Neutral message for bad commands	The system shall display a neutral terminal-style message for malformed or unsupported commands, leave repository state unchanged, and log the attempt as unprocessable.
FR-TERM-08	Simulator coverage for curriculum	The system shall implement every Git operation required by the approved curriculum scenario library and active variant pools in the repository state simulator.

FR-TERM-09	Remaining counted-command counter	The system shall display a counted-command allowance derived from the active session's authored command-count policy.
FR-VAR-01	Structurally distinct retry variants	The system shall maintain structurally distinct scenario variants for skill focuses that support retry-after-failure practice.
FR-VAR-02	No identical variant on RTA retry	The system shall not serve the identical variant from the immediately preceding failed or abandoned session when another structurally different valid variant exists.
FR-VAR-03	Record variant lineage	The system shall store variant identifier and prior-session relationship on every scenario session record.
FR-VAR-04	RTA eligibility when variant changes	The system shall mark retry attempts ineligible for RTA measurement when a structurally different variant was required but not served.
FR-CHAL-01	Challenge Gate per module	The system shall provide one Challenge Gate per module with Easy, Medium and Hard trials of two variants each. The gate opens once every required level in the module has all three tiers complete; Medium opens after the Easy trial and Hard after Medium.
FR-CHAL-02	Challenge scaffolding	The system shall show the live graph, the Expected diagram and contextual feedback on Easy trials, the live graph and contextual feedback on Medium, and the live graph only on Hard.
FR-CHAL-03	Challenge runs	The system shall play Challenge trials in the same workspace as adventure tiers, with the same state-based grading, command budget, retry and replay rules.
FR-CHAL-04	Challenge rewards	The system shall grant 50, 75 and 100 GitCoins once for the first clear of the Easy, Medium and Hard trial.
FR-DRILL-01	Optional recall practice	The system shall offer a Squire's Drill for each adventure level with recall questions on the command forms that level uses. The drill is optional, unlocks nothing and pays no GitCoins.
FR-DRILL-02	Drill resume and clear	The system shall save drill progress after every answer so a refresh resumes mid-drill, and shall count a drill as cleared only when every card reaches the top of its ladder.

0.8 Complete the GIT it! Platform Walkthrough

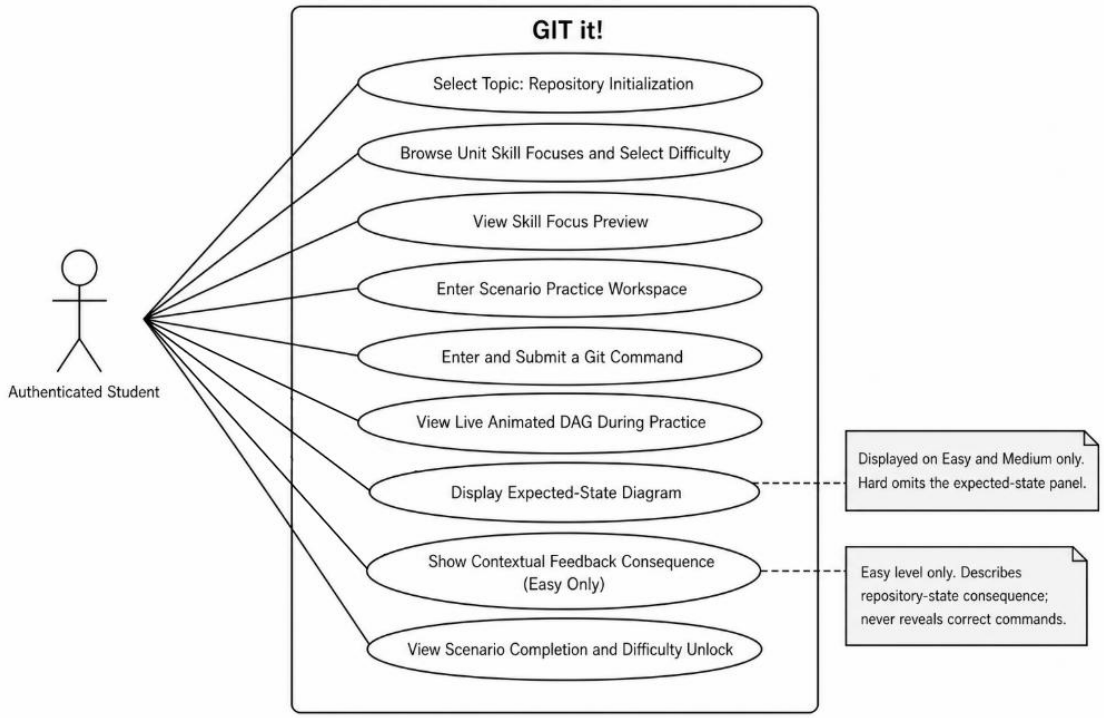
This optional orientation topic explains the current journey through stories, chapters, adventures, challenges, drills, the companion shop, equipment, and Home. Learners can enter available practice without completing orientation. Lesson progress is recorded separately from scenario performance measures.

FR-ORIENT-08 — The system shall present a platform walkthrough and record its lesson progress without treating that progress as a scenario clear, command submission, or reward event.

Module 1 — Local Repository Foundations

1.1 Practice Initializing Local Git Repositories

Use Case Diagram

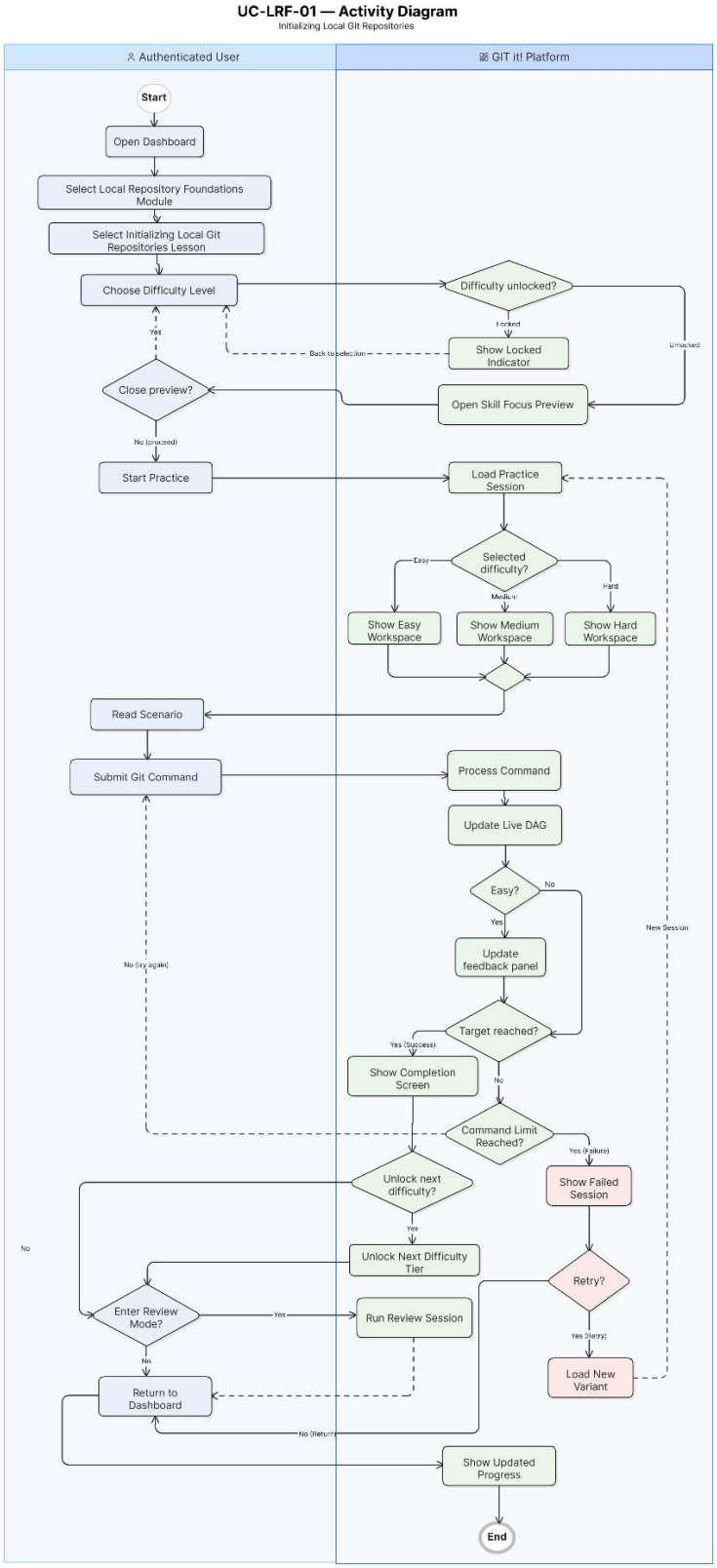


Use Case Description

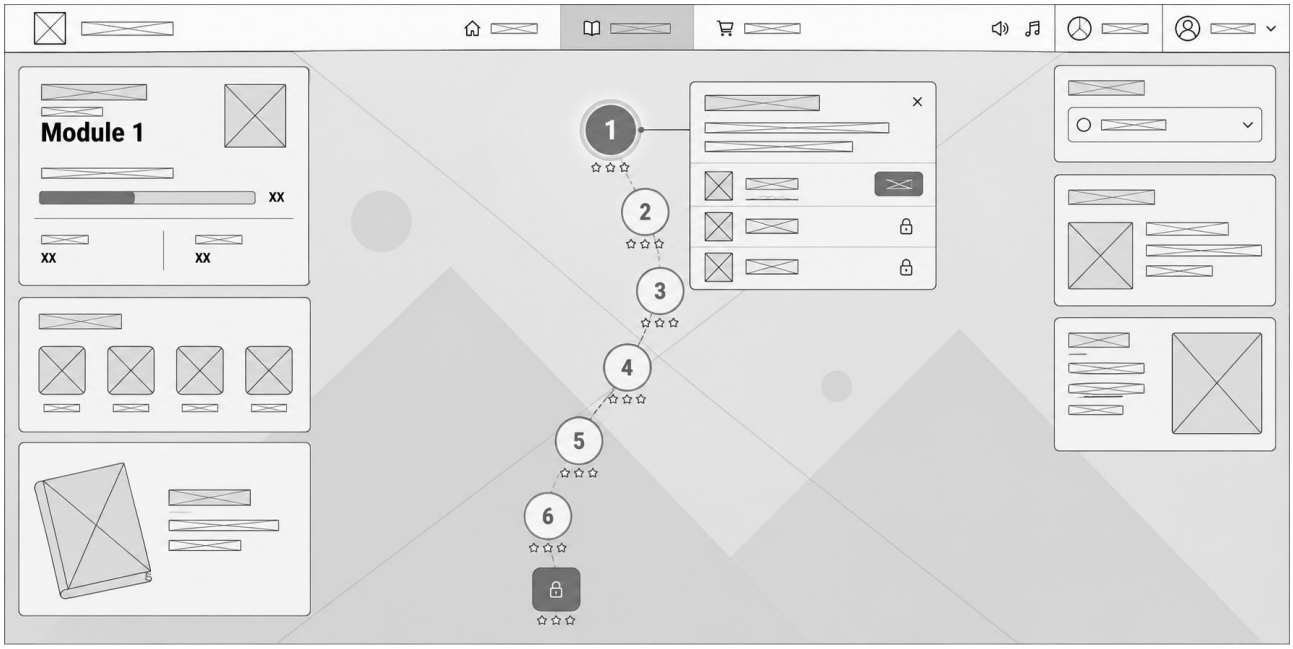
Field	Description
Use Case ID	UC-LRF-01
Use Case Name	Practice Initializing Local Git Repositories
Curriculum SO	SO 1.1
Actor(s)	Authenticated Student

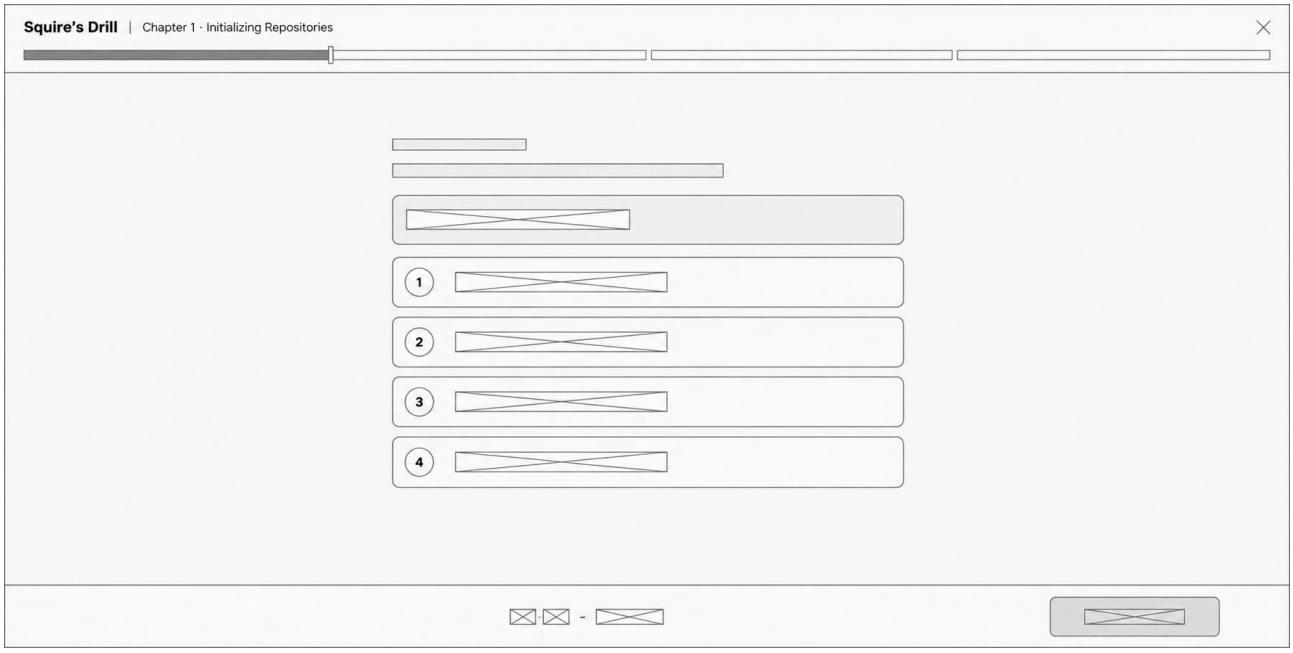
Preconditions	Student is logged in. The chapter and learning objective for Repository initialization are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map. 2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable). 3. Student selects the level for Repository initialization. 4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action. 5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story. 6. System opens the level workspace on the tier's next uncleared scenario. 7. Student reads the scenario narrative and task prompt describing initializing a new local Git repository in varied directory contexts. 8. Student types a Git command in the terminal input and presses Enter. 9. Student sees the command echoed in the terminal output area with simulator-style feedback. 10. Student sees the repository graph update when the command changes repository state. 11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command. 12. On Medium: student views the target-state guidance; contextual feedback panel is not shown. 13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance). 14. Student checks the counted-command allowance after counted action commands. 15. Student repeats steps 8–14 until the practice goal is reached or the session ends. 16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced initializing a new local Git repository in varied directory contexts in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed. AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG. AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry. AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map. AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

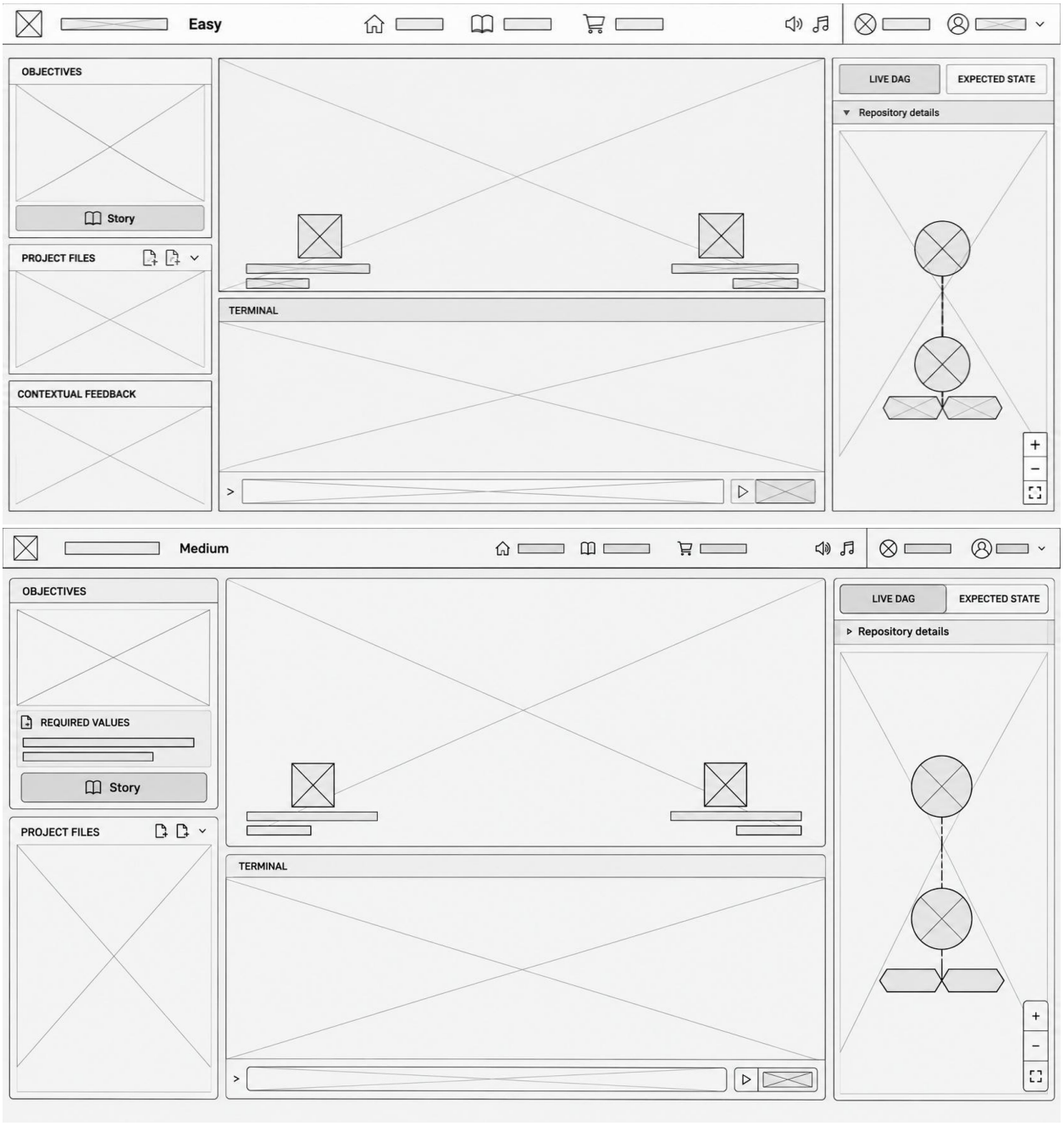
Activity Diagram

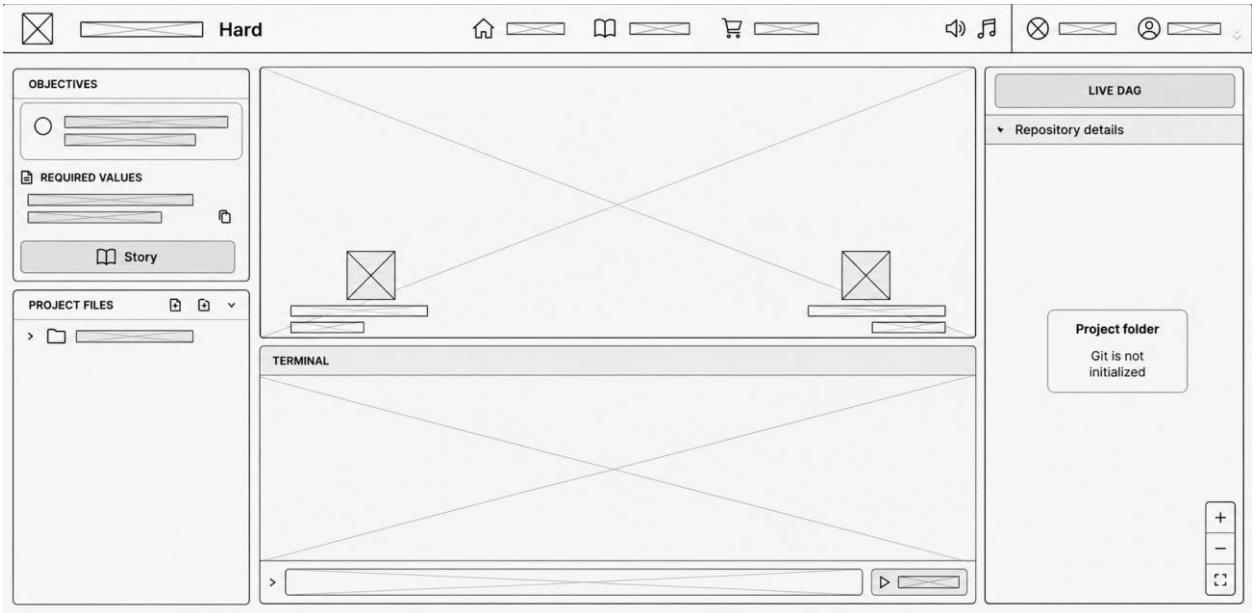


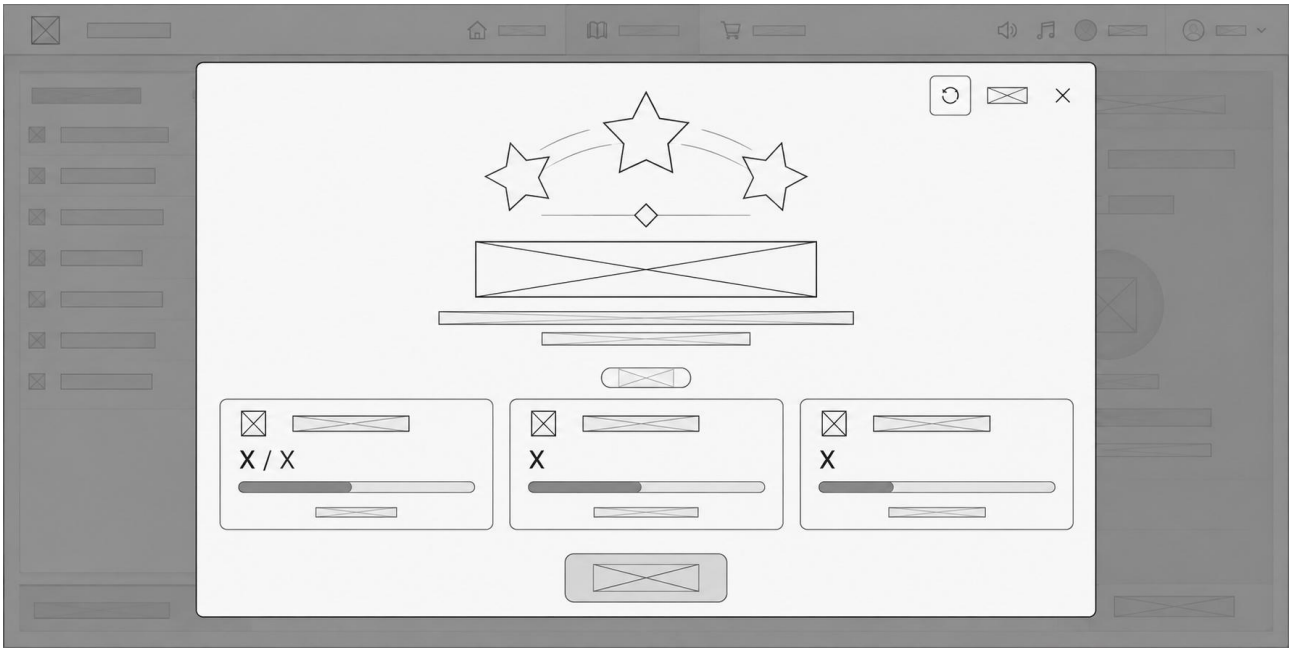
Wireframe


Module 1
XX
XX
XX
1
2
3
4
5
6


Squire's Drill | Chapter 1 · Initializing Repositories
1
2
3
4

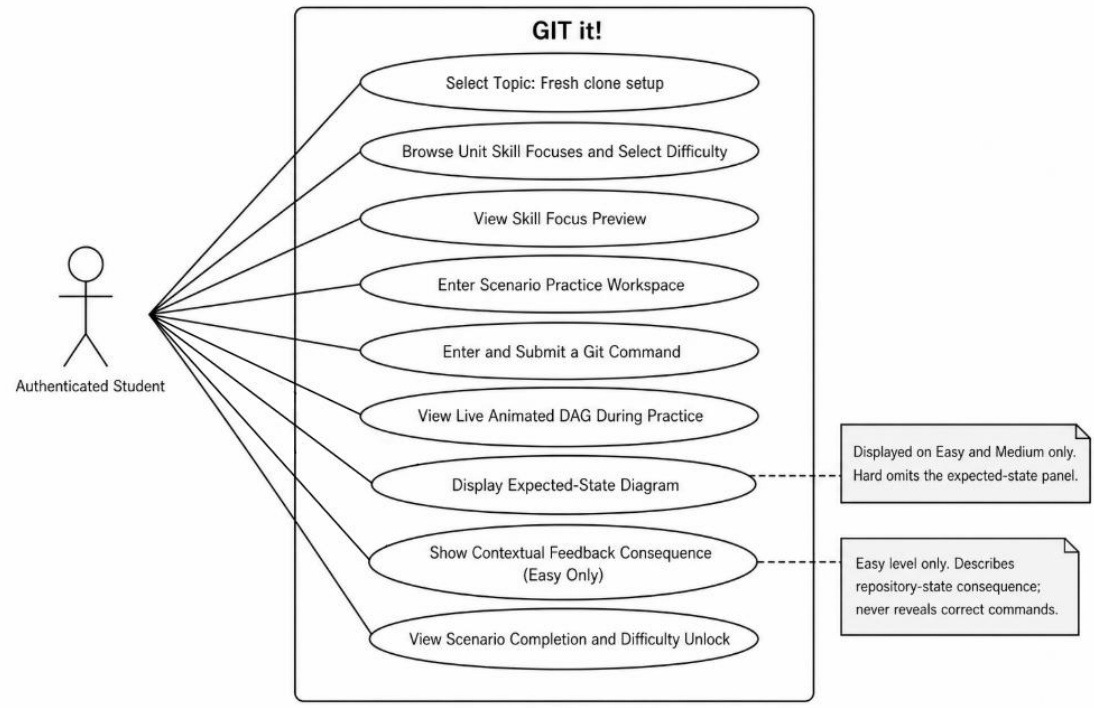

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details





1.2 Practice Cloning Remote Repositories

Use Case Diagram

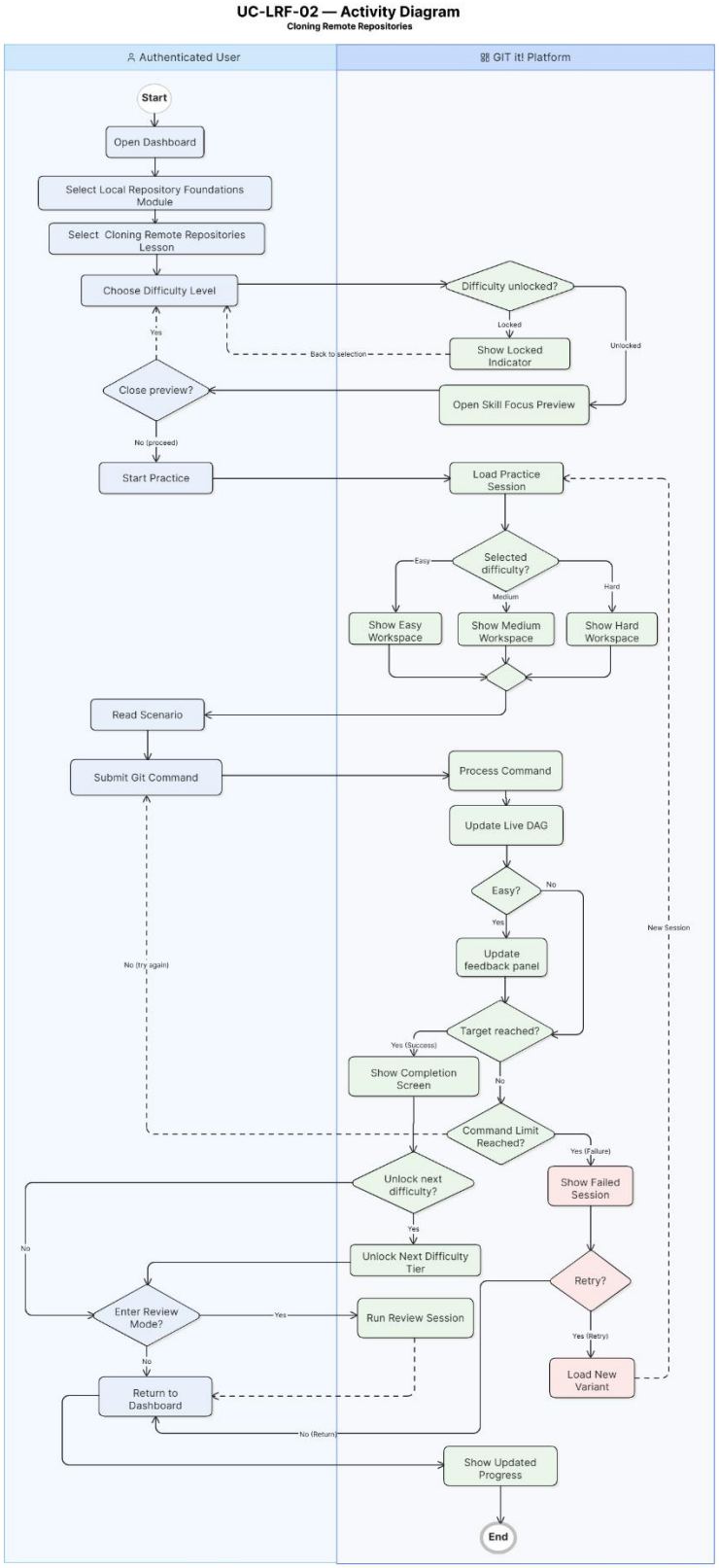


Use Case Description

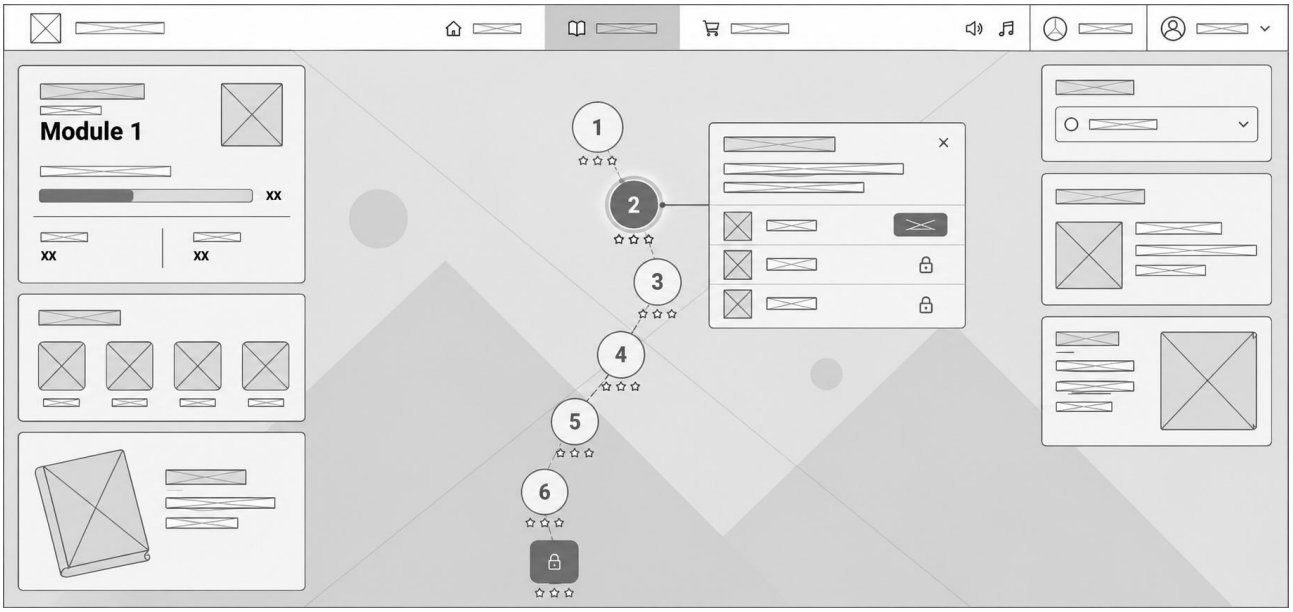
Field	Description
Use Case ID	UC-LRF-02
Use Case Name	Practice Cloning Remote Repositories
Curriculum SO	SO 1.2
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Fresh clone setup are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map. 2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable). 3. Student selects the level for Fresh clone setup. 4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action. 5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story. 6. System opens the level workspace on the tier's next uncleared scenario. 7. Student reads the scenario narrative and task prompt describing cloning a remote repository using HTTPS or SSH URLs and custom destination directories. 8. Student types a Git command in the terminal input and presses Enter. 9. Student sees the command echoed in the terminal output area with simulator-style feedback. 10. Student sees the repository graph update when the command changes repository state.

	11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command. 12. On Medium: student views the target-state guidance; contextual feedback panel is not shown. 13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance). 14. Student checks the counted-command allowance after counted action commands. 15. Student repeats steps 8–14 until the practice goal is reached or the session ends. 16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced cloning a remote repository using HTTPS or SSH URLs and custom destination directories in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed. AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG. AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry. AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map. AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

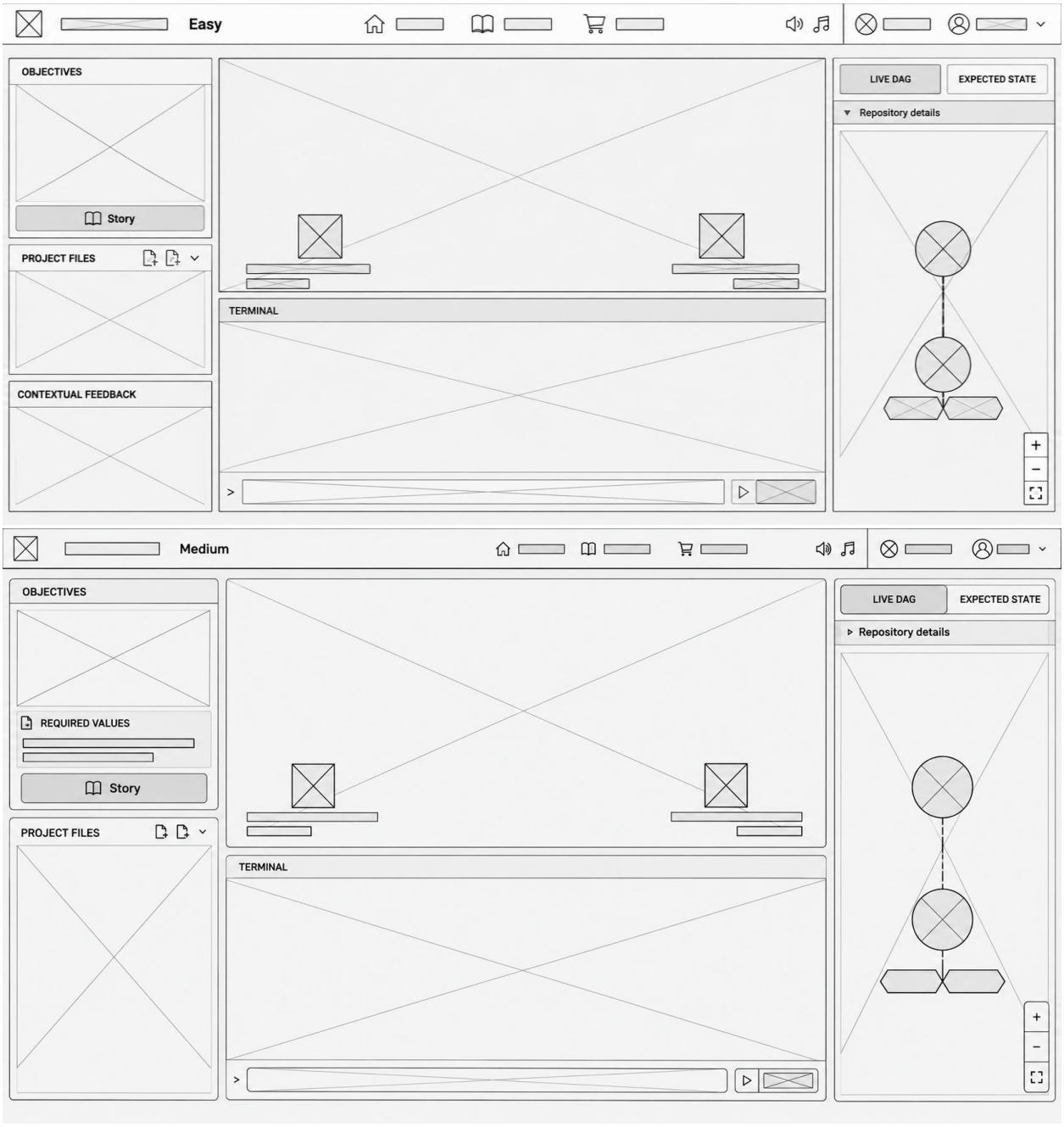
Activity Diagram

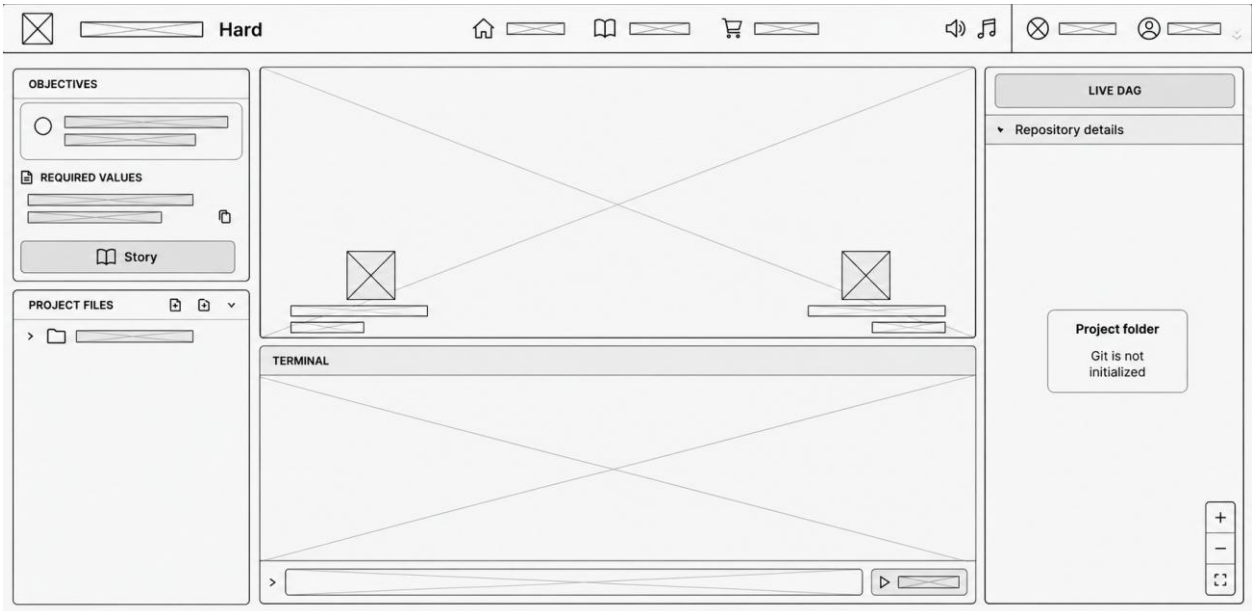


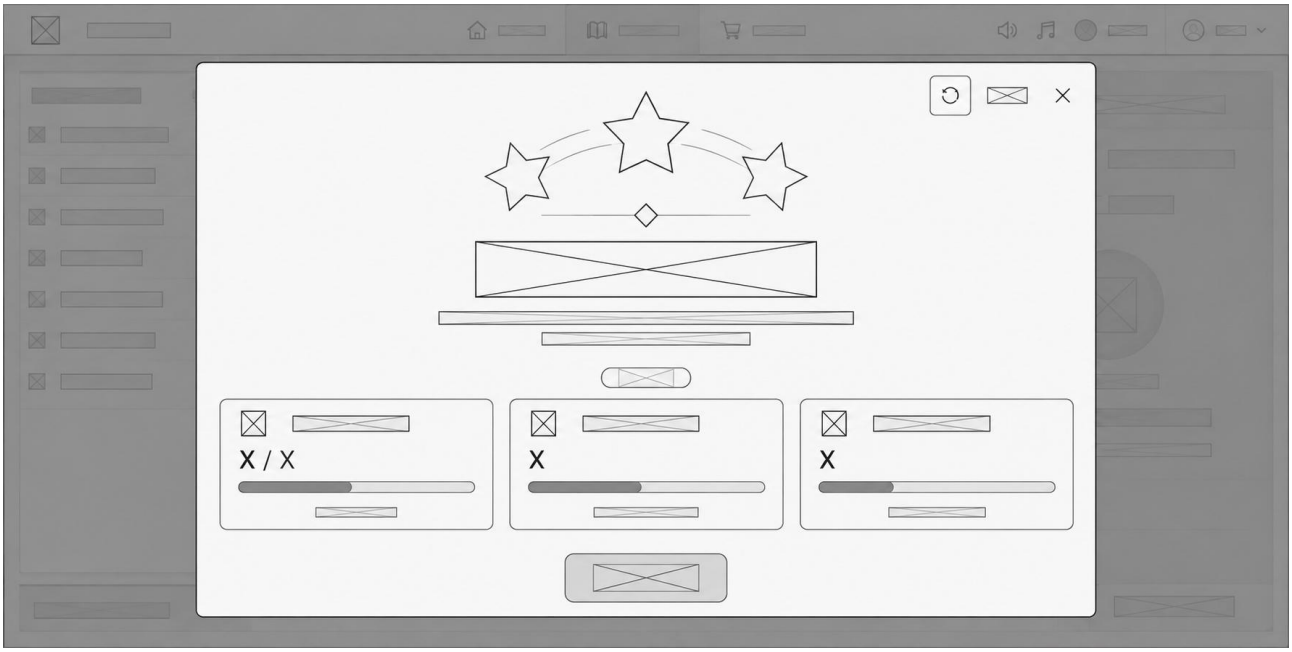
Wireframe





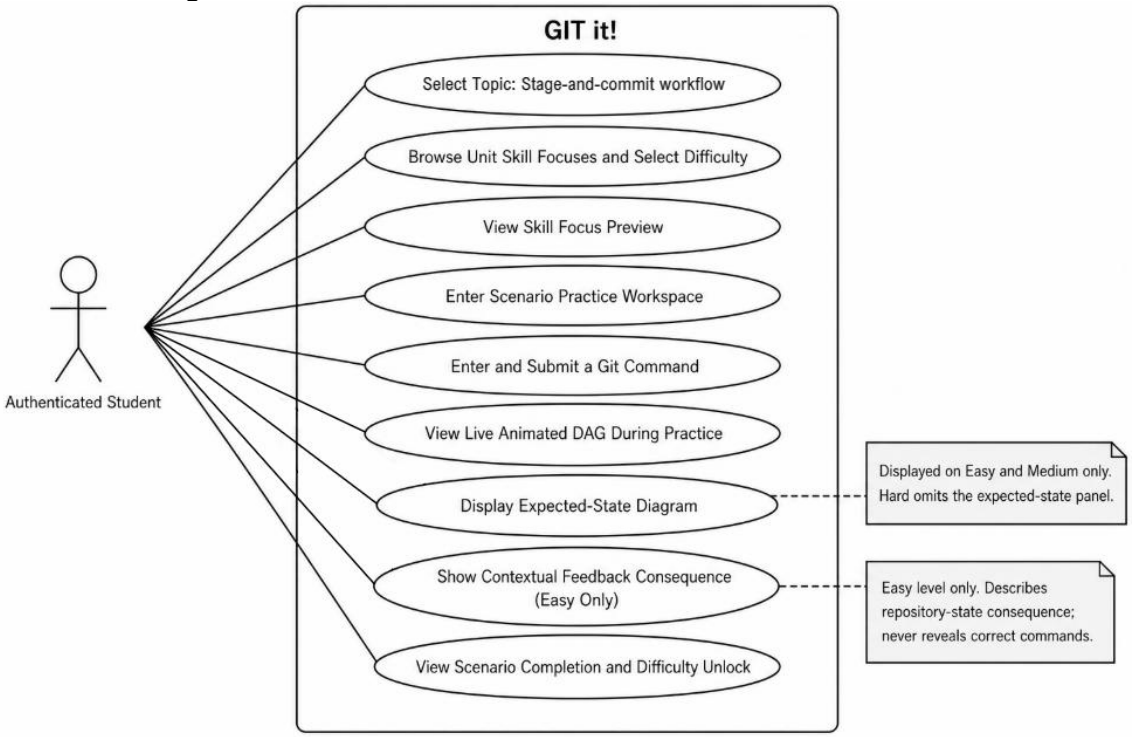

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

1.3 Practice Staging and Committing Changes

Use Case Diagram

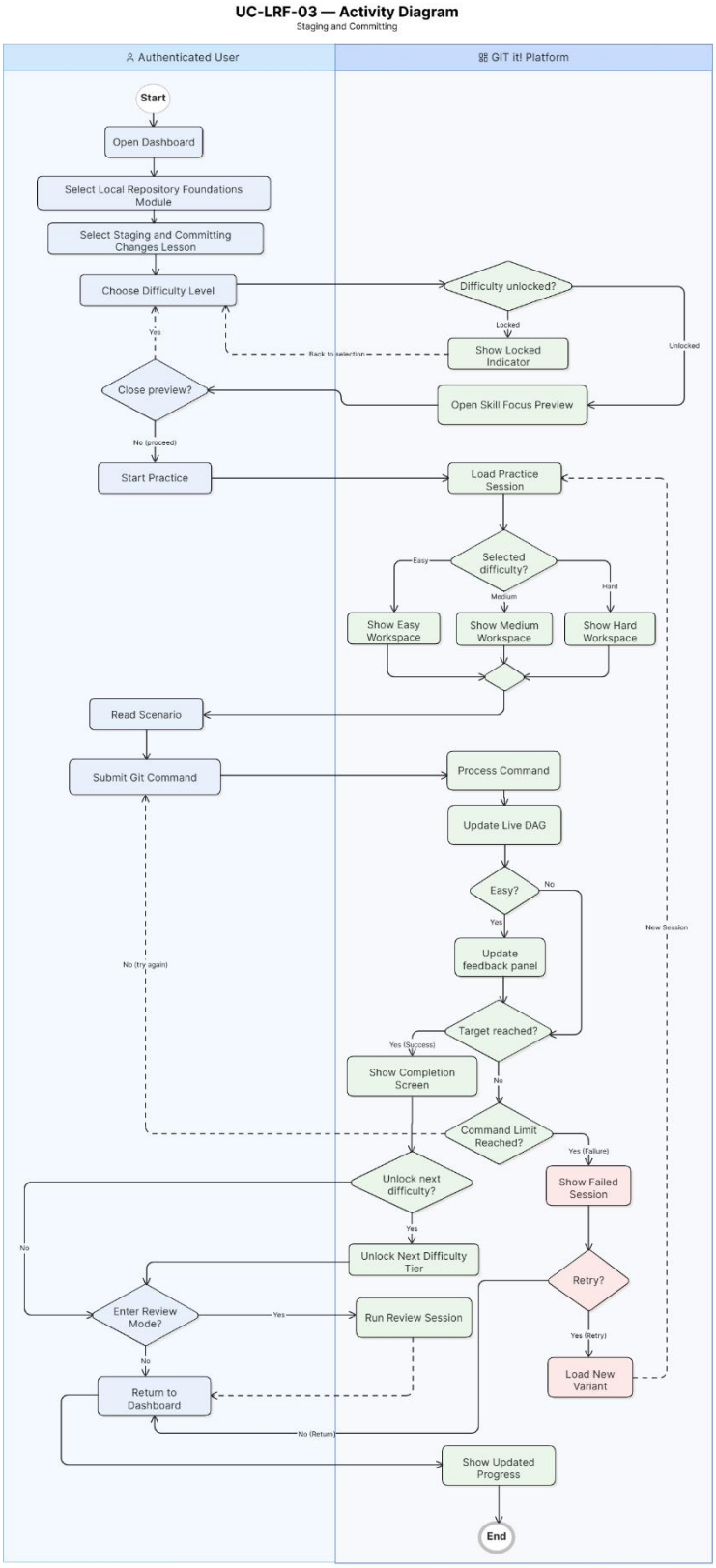


Use Case Description

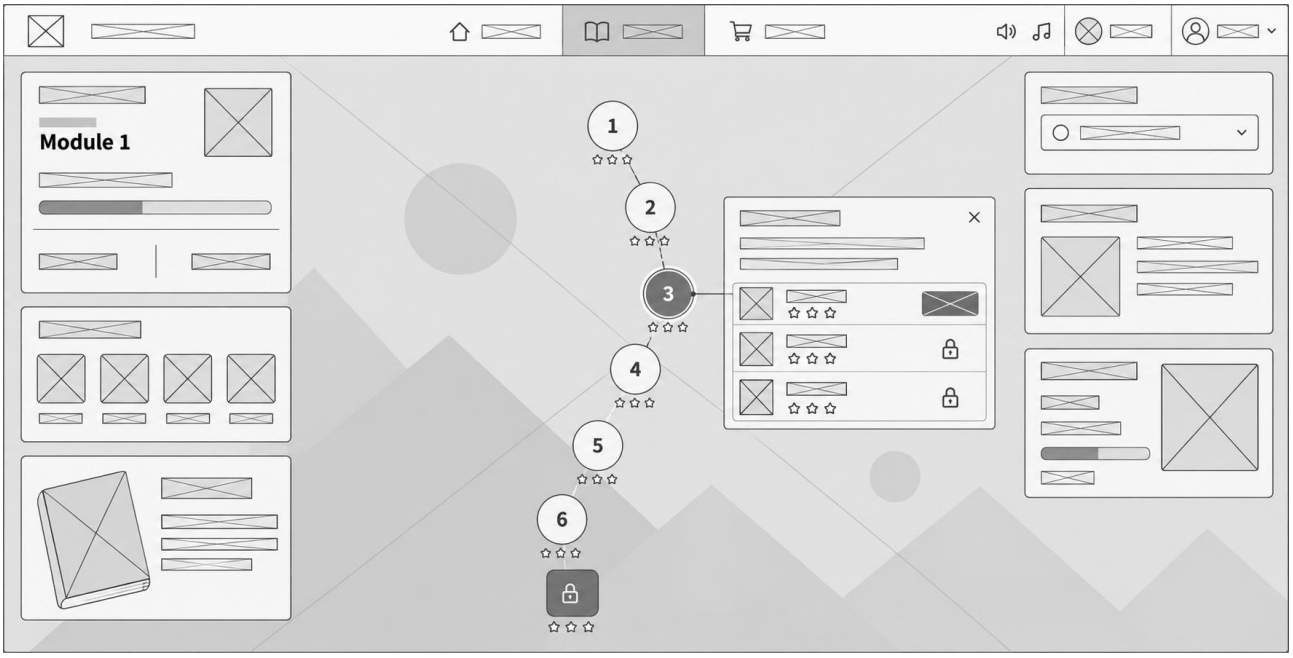
Field	Description
Use Case ID	UC-LRF-03
Use Case Name	Practice Staging and Committing Changes
Curriculum SO	SO 1.3
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Stage-and-commit workflow are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Stage-and-commit workflow.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing staging files or directories and creating valid commits with descriptive imperative-mood messages.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12.

	On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced staging files or directories and creating valid commits with descriptive imperative-mood messages in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

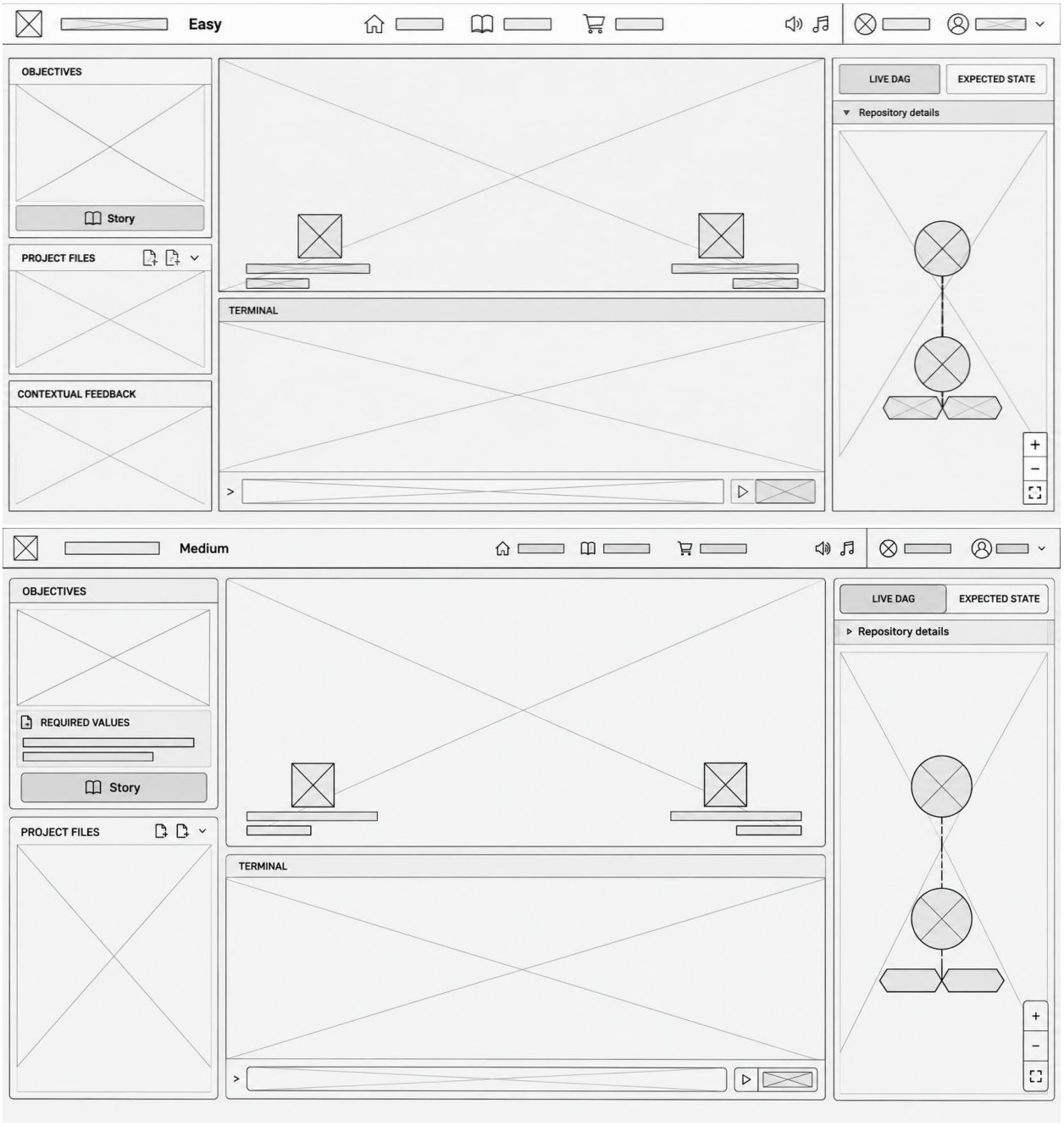
Activity Diagram

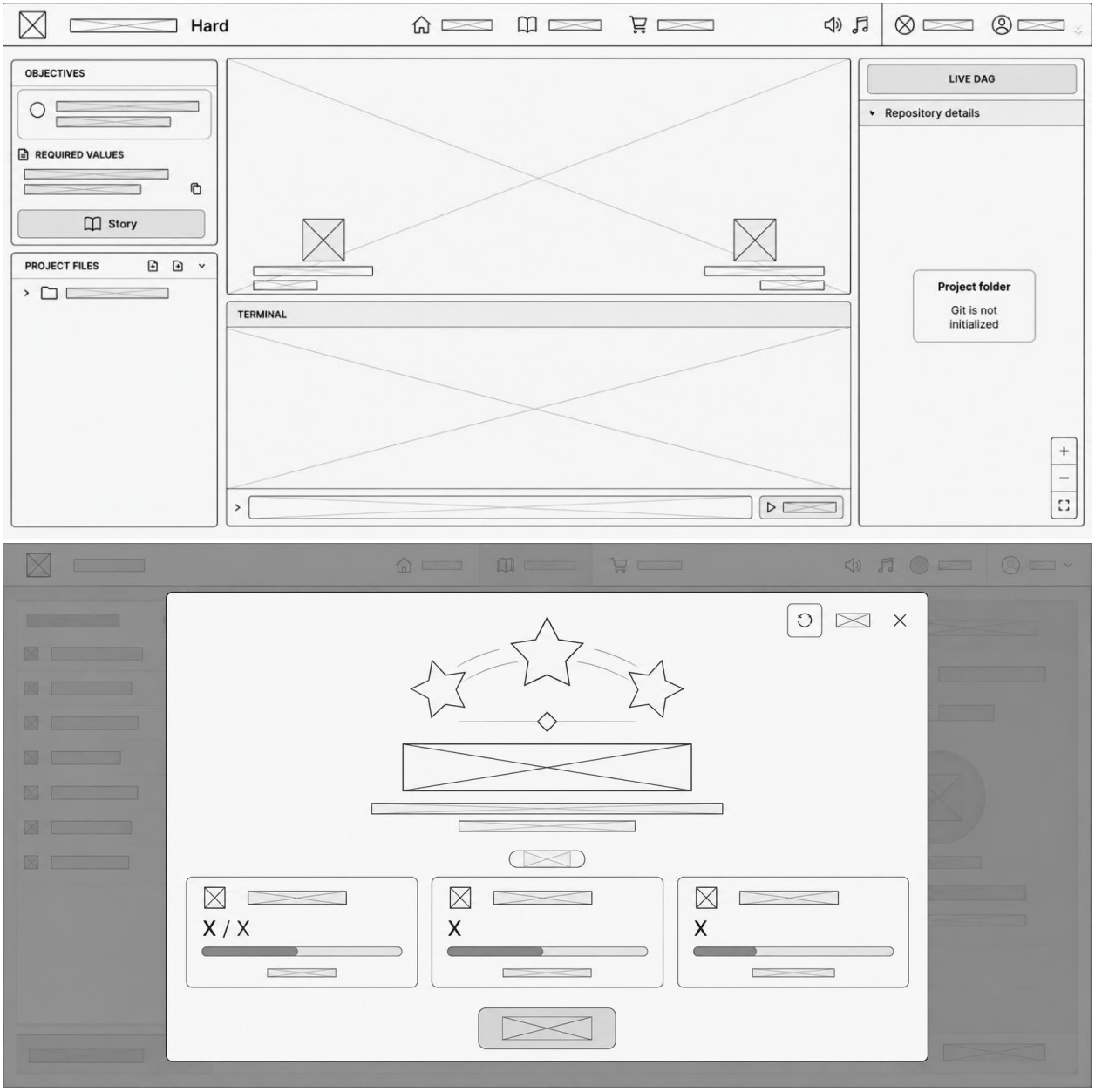


Wireframe



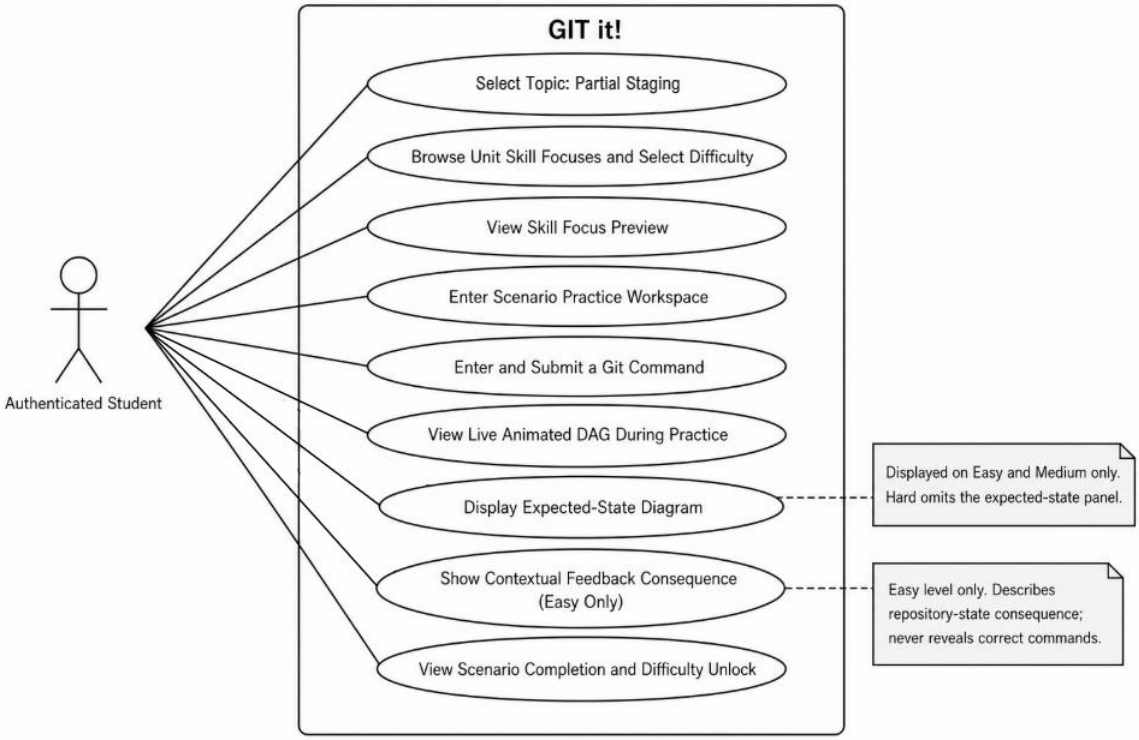



Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized
X / X
X
X

1.4 Practice Partial Staging

Use Case Diagram

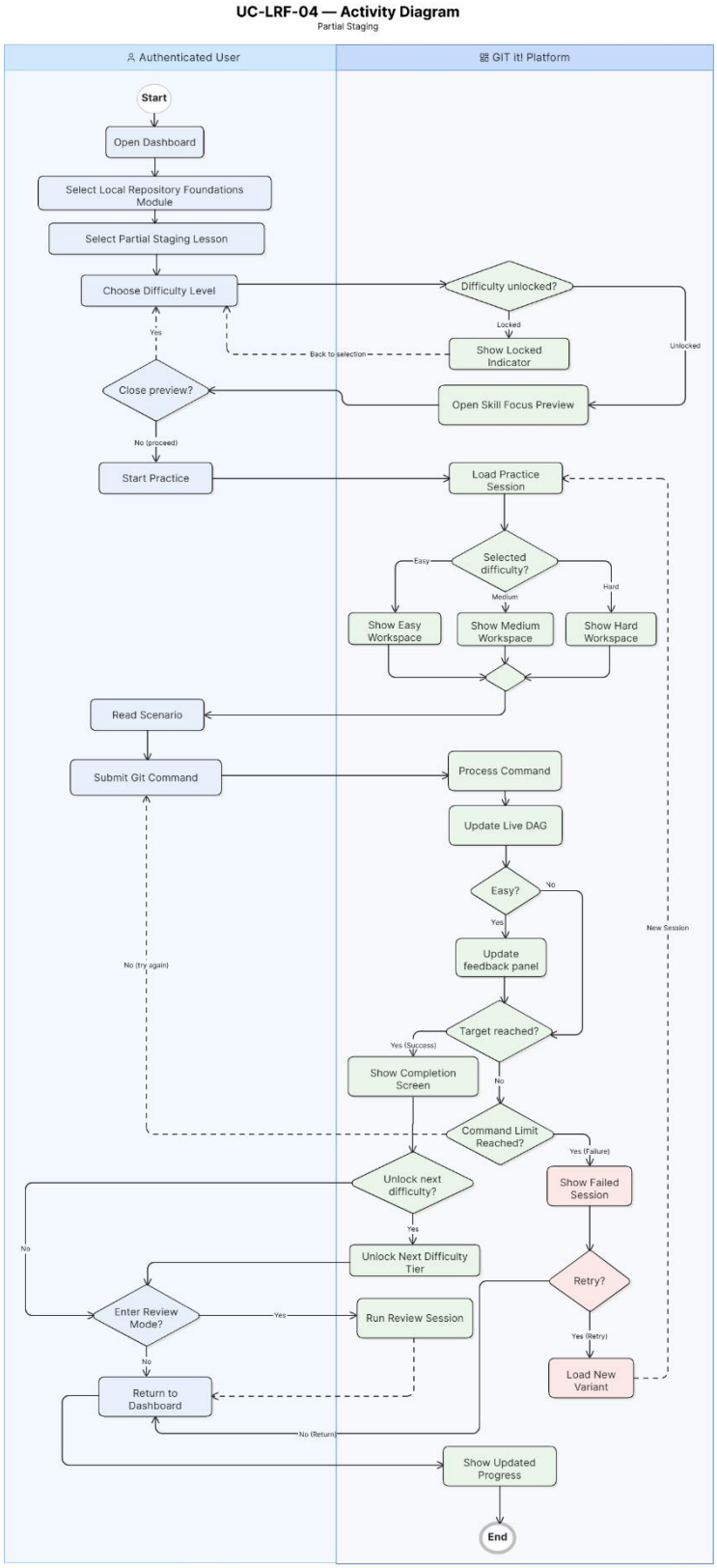


Use Case Description

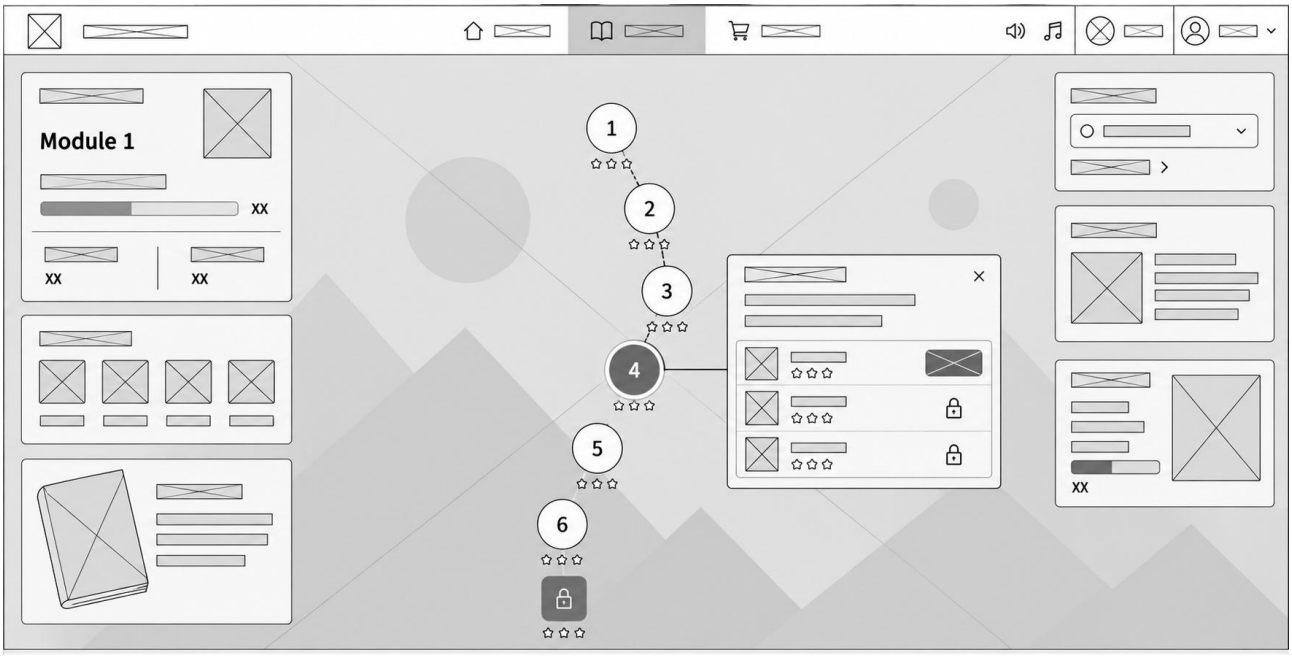
Field	Description
Use Case ID	UC-LRF-04
Use Case Name	Practice Partial Staging
Curriculum SO	SO 1.4
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Partial staging are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Partial staging.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing selecting specific hunks or partial file changes for staging.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On

	Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced selecting specific hunks or partial file changes for staging in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

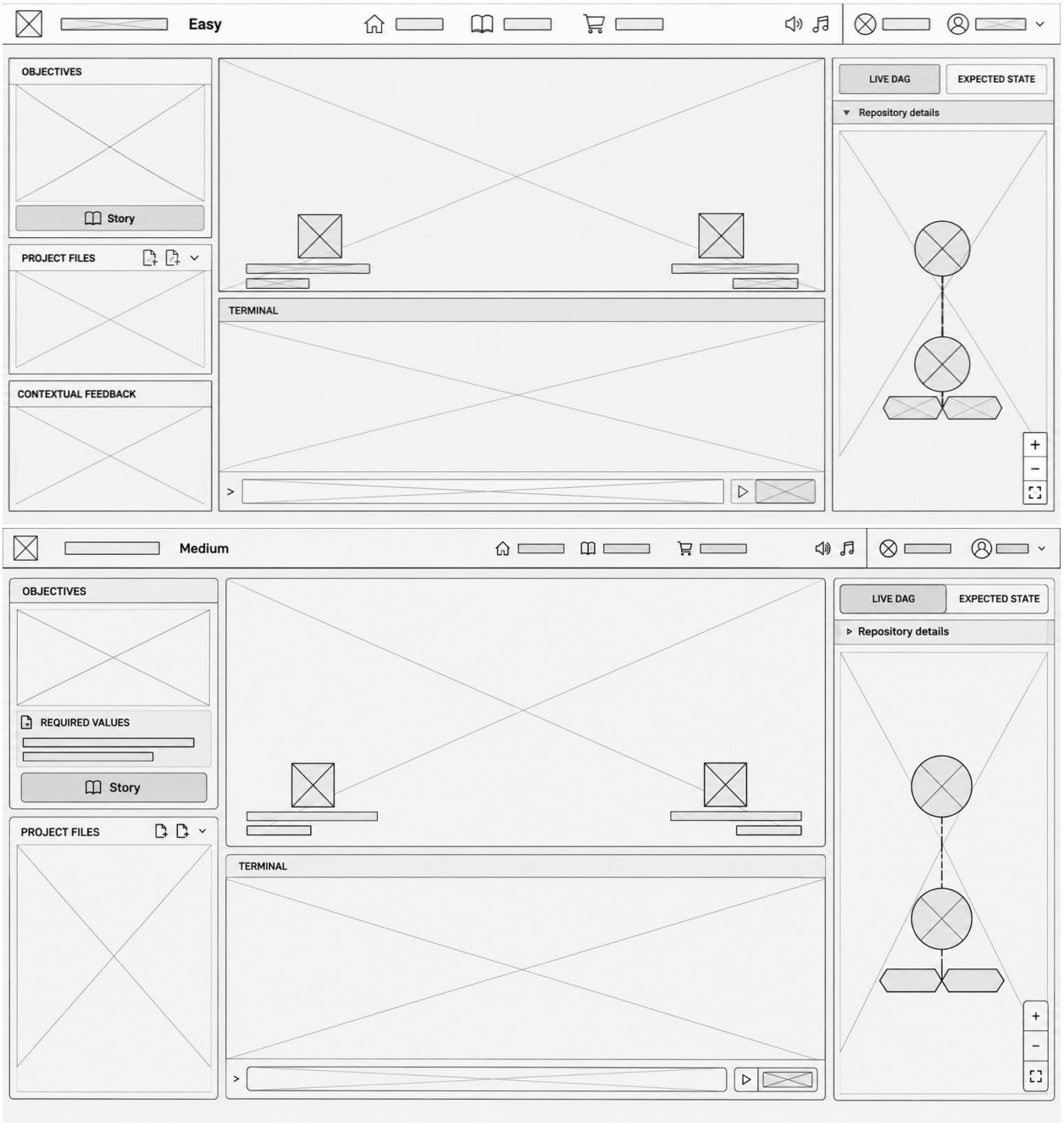
Activity Diagram

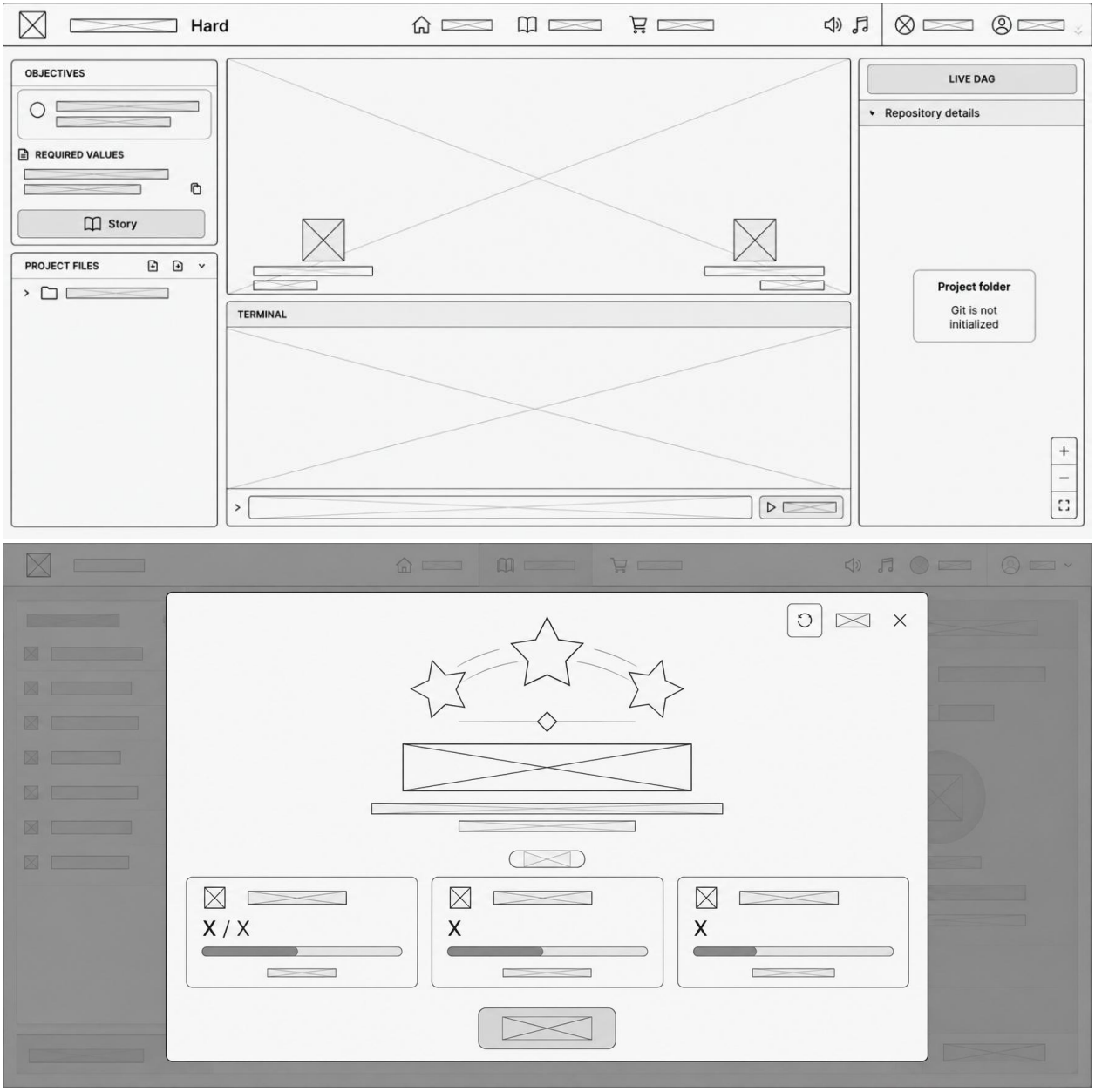


Wireframe



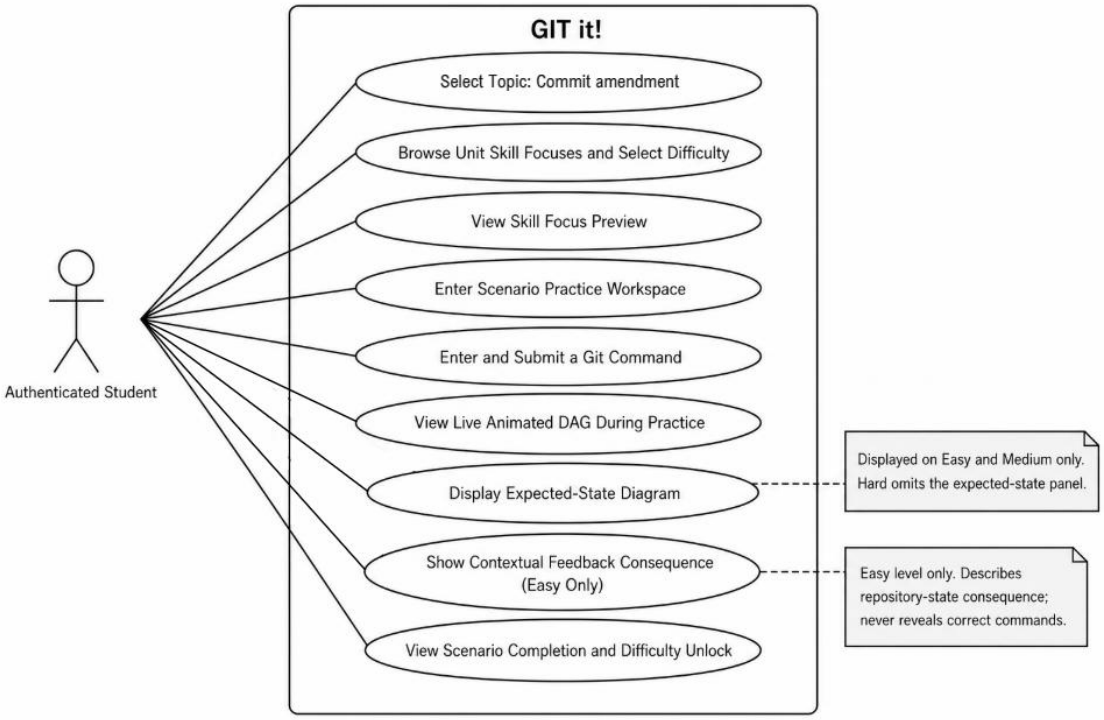



Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized
X / X
X
X

1.5 Practice Amending the Most Recent Commit

Use Case Diagram

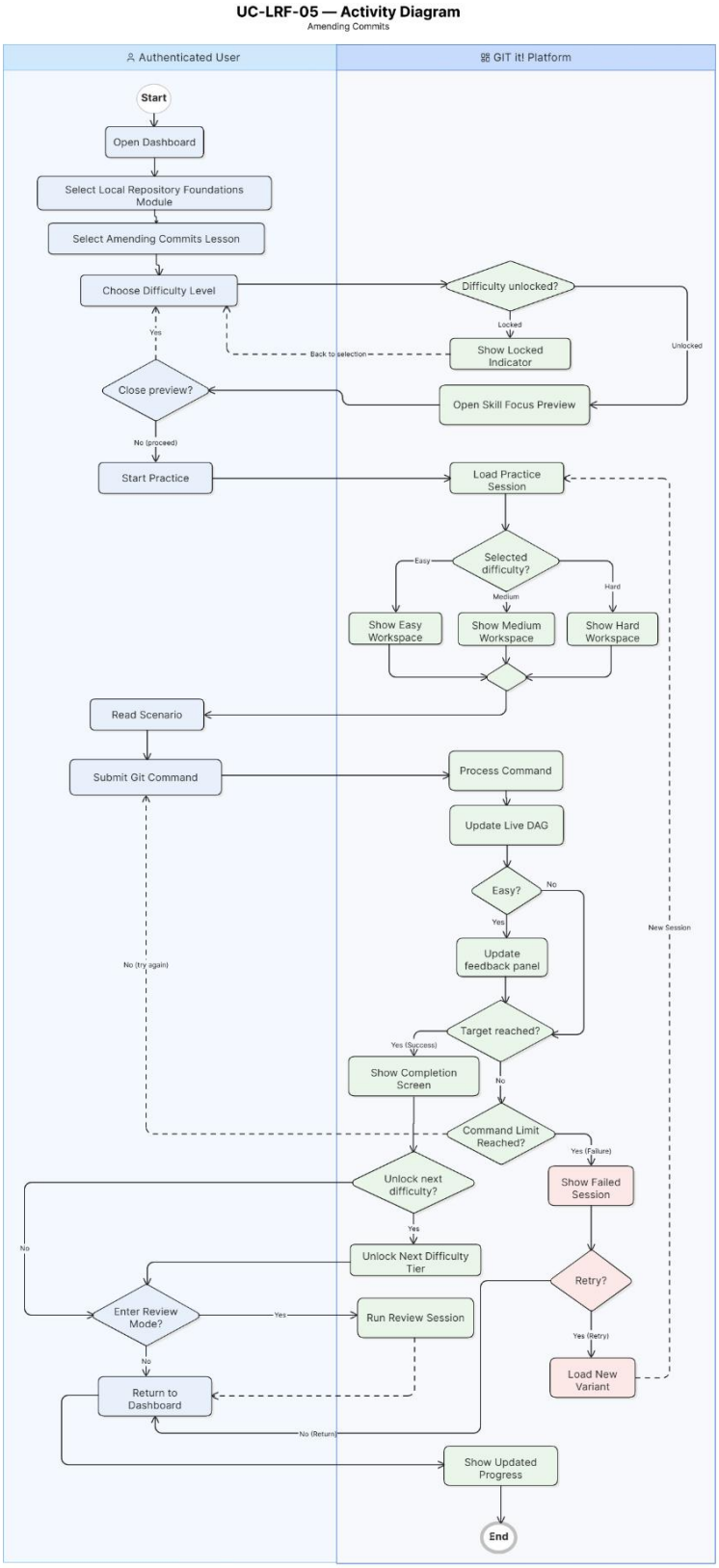


Use Case Description

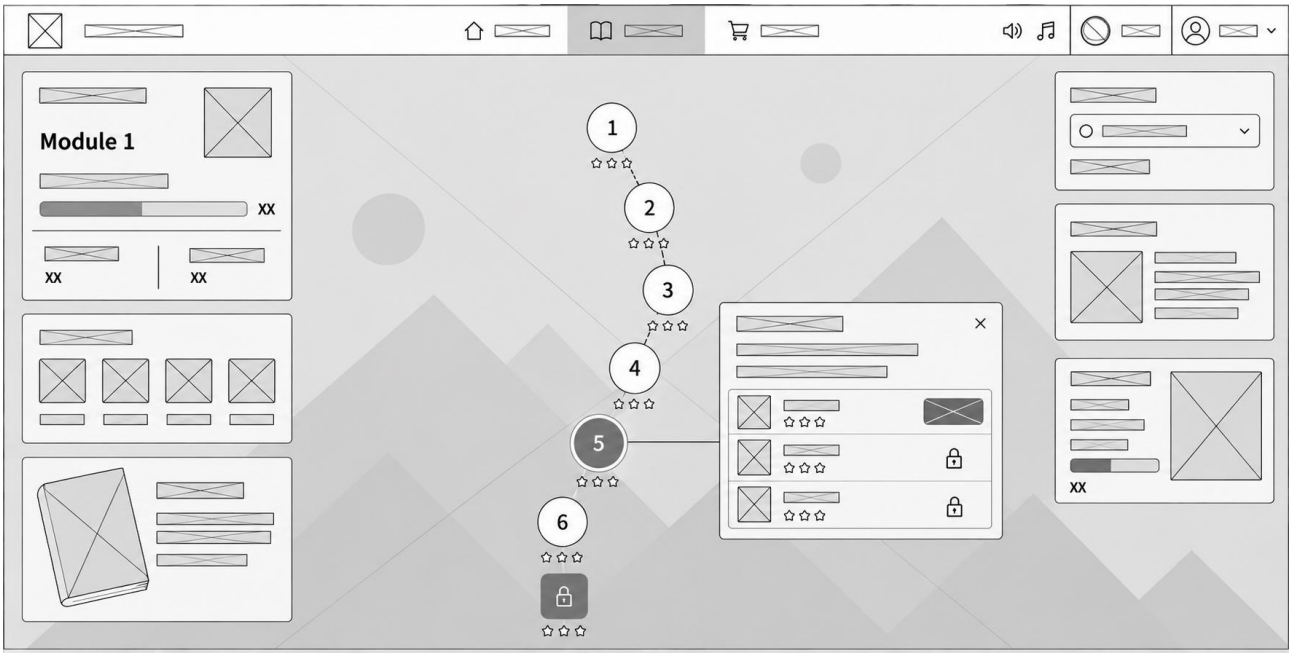
Field	Description
Use Case ID	UC-LRF-05
Use Case Name	Practice Amending the Most Recent Commit
Curriculum SO	SO 1.5
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Commit amendment are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Commit amendment.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing amending the latest commit message or staged content.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On

	Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced amending the latest commit message or staged content in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

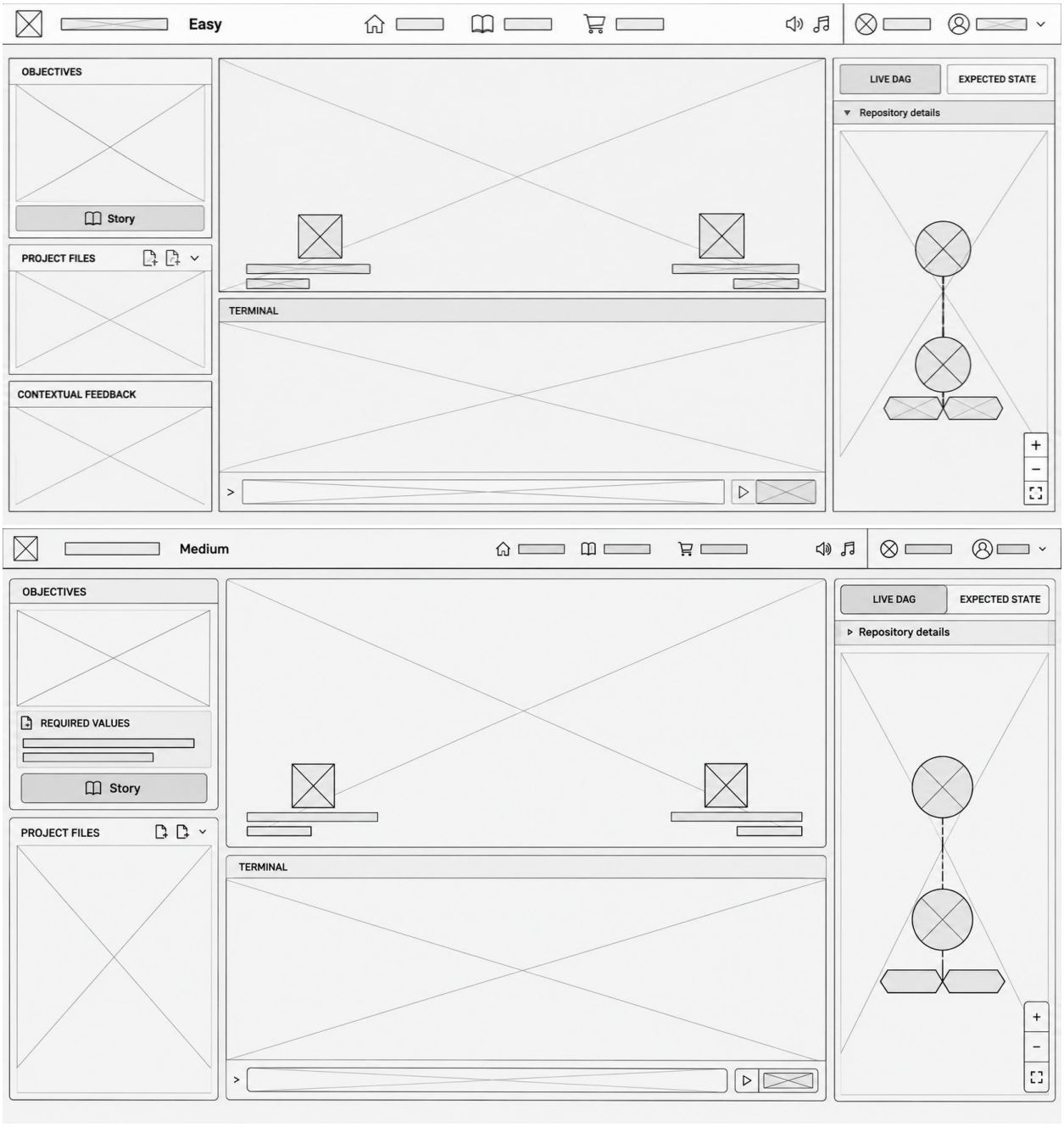
Activity Diagram

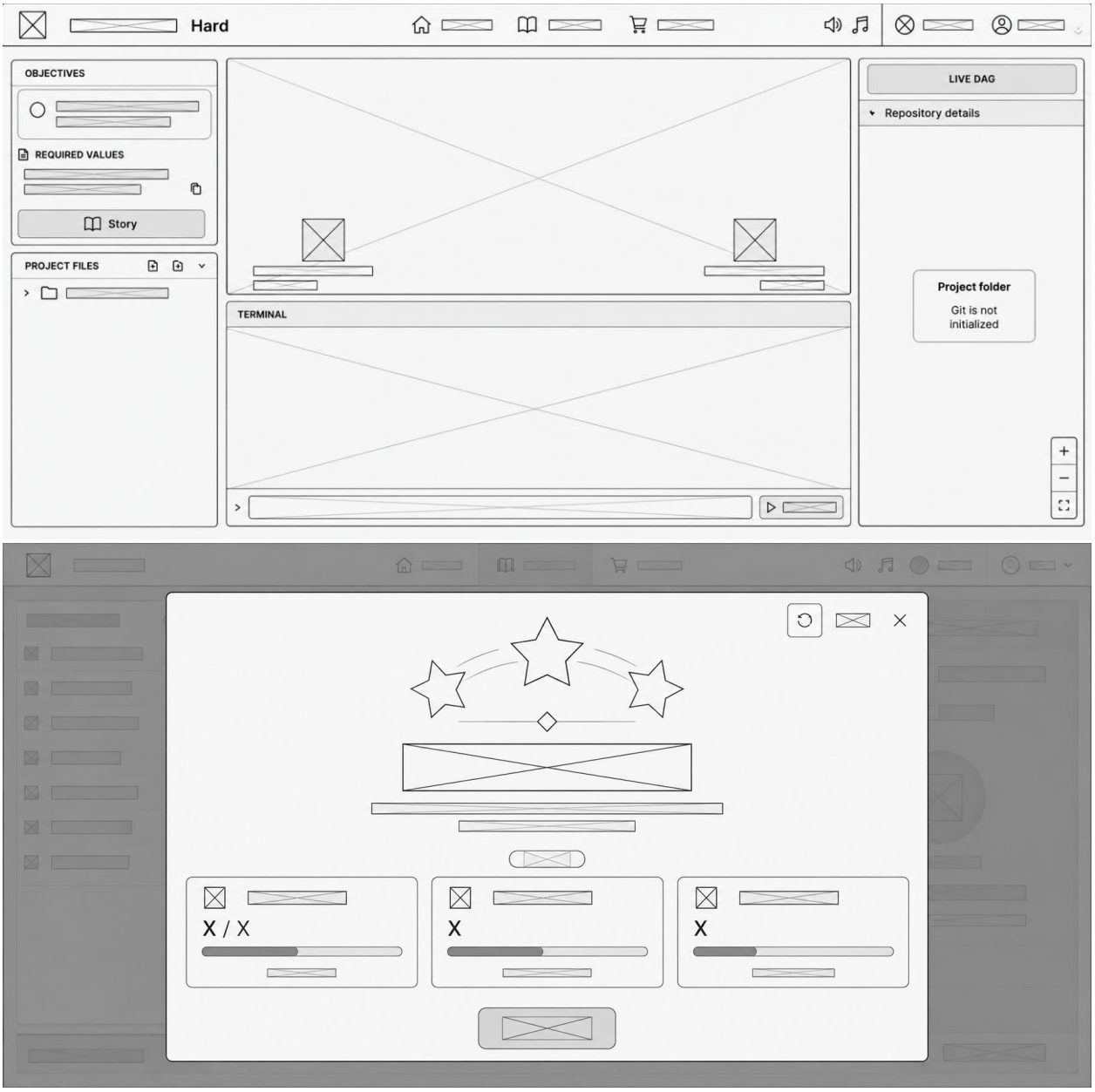


Wireframe



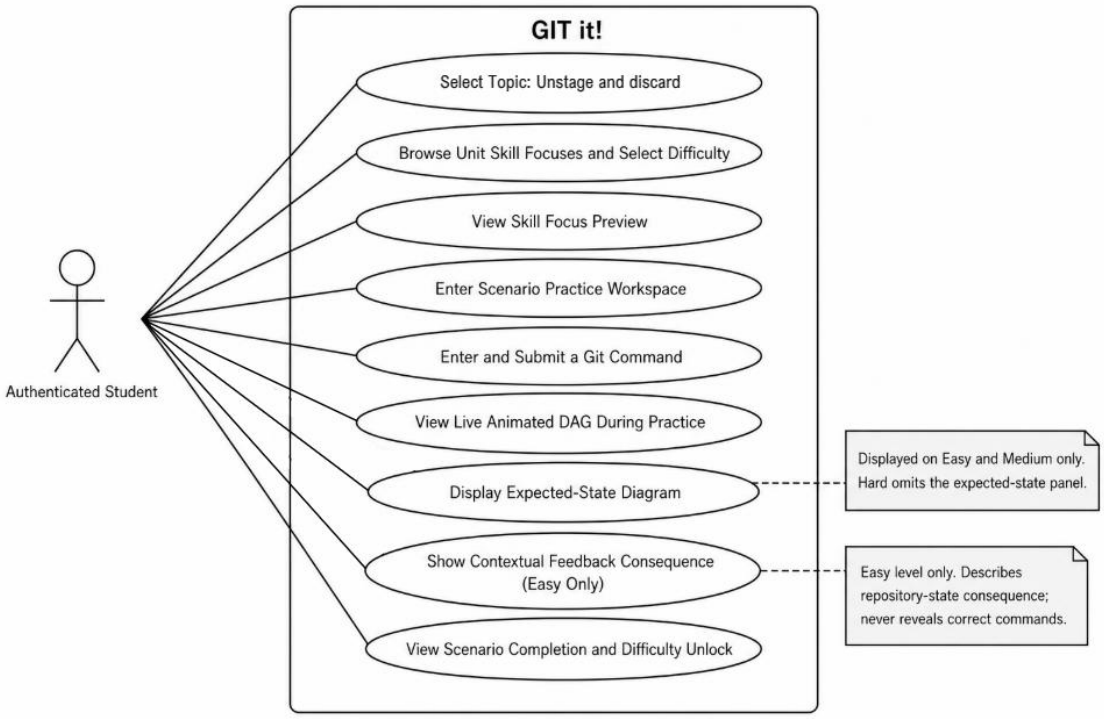



Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized
X / X
X
X

1.6 Practice Unstaging and Discarding Changes

Use Case Diagram

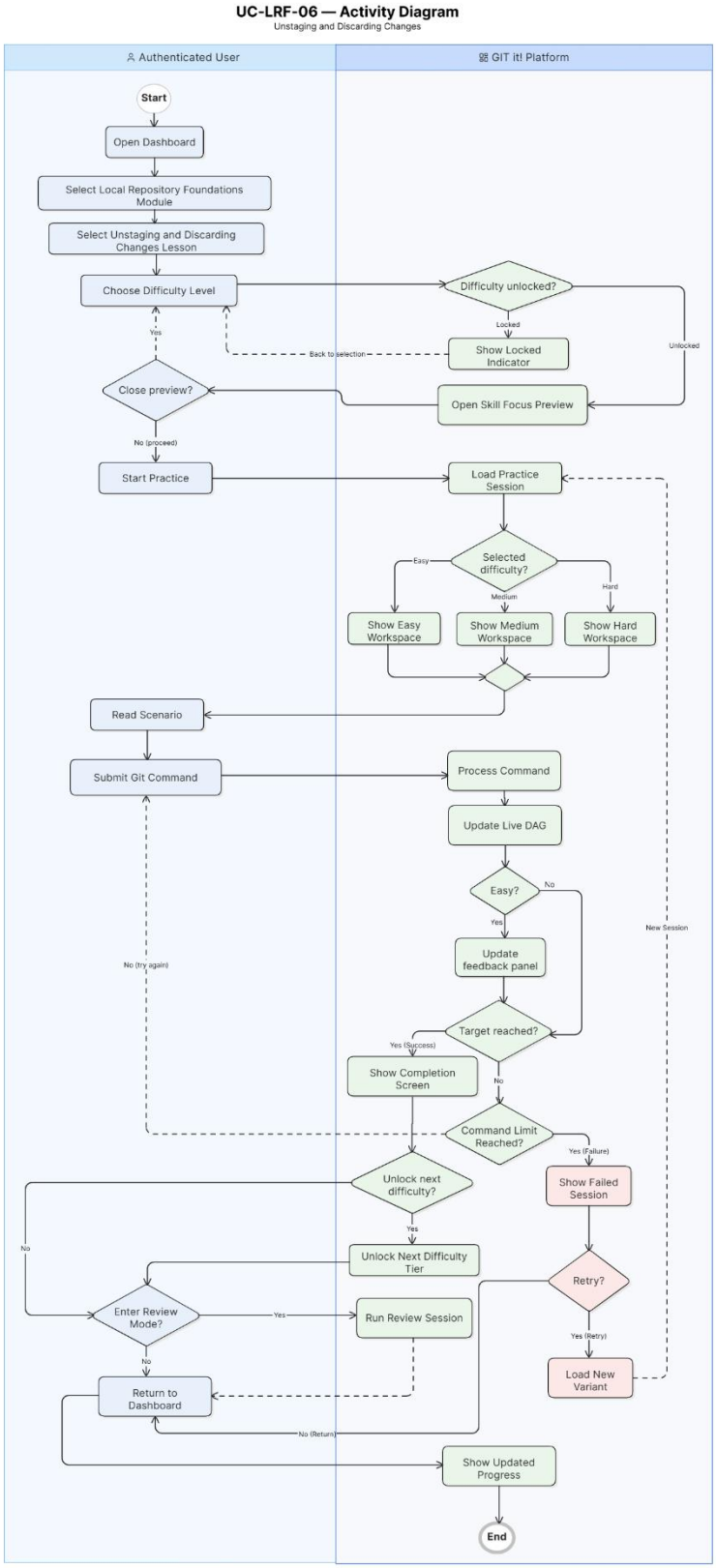


Use Case Description

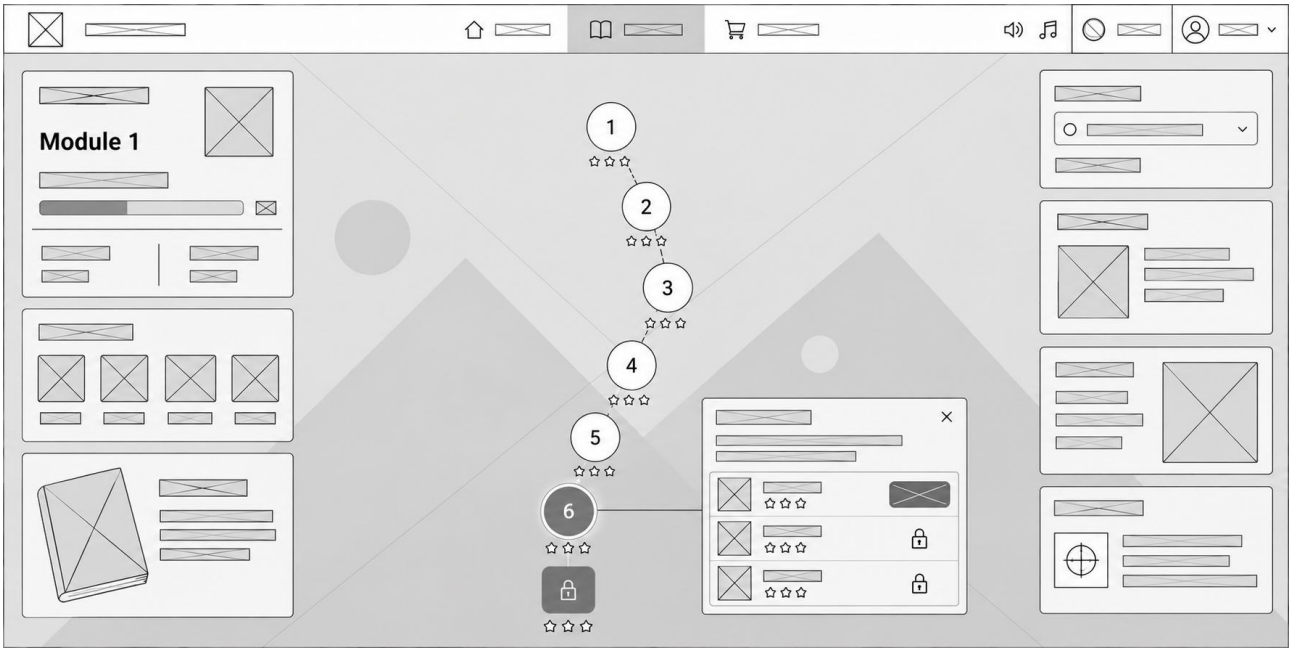
Field	Description
Use Case ID	UC-LRF-06
Use Case Name	Practice Unstaging and Discarding Changes
Curriculum SO	SO 1.6
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Unstage and discard are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Unstage and discard.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing unstaging files and discarding working-tree changes safely.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel

	after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced unstaging files and discarding working-tree changes safely in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

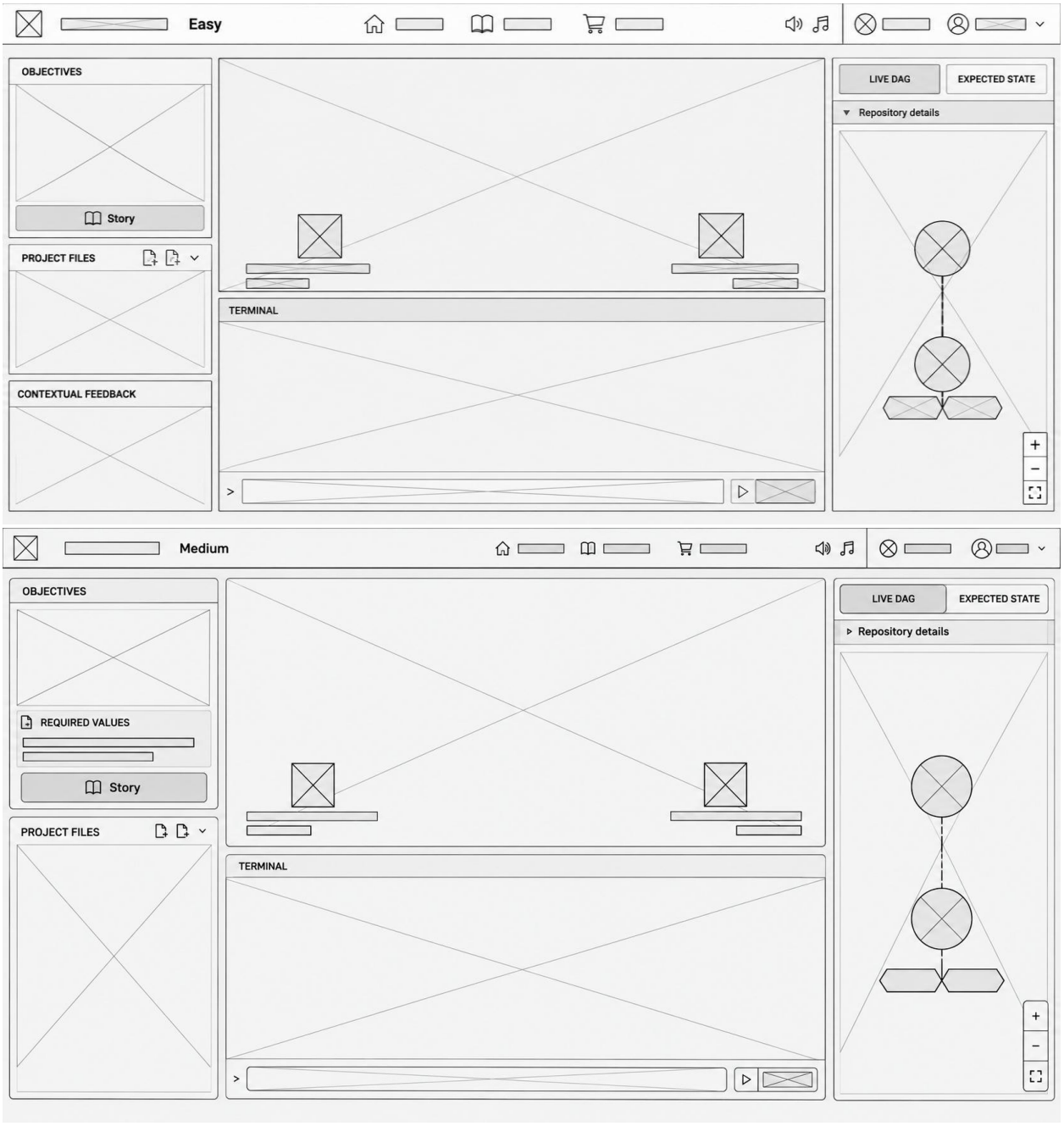
Activity Diagram

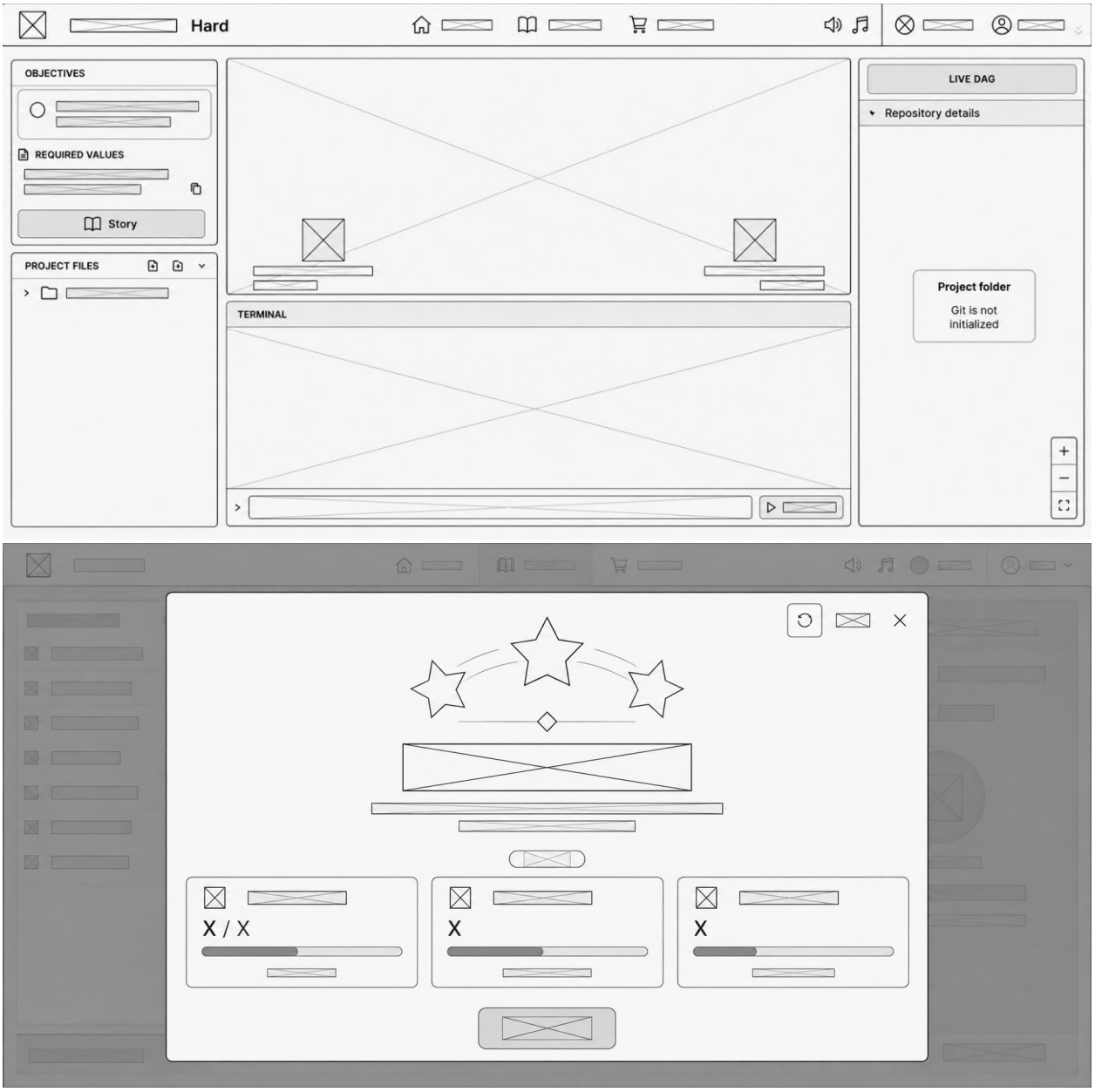


Wireframe



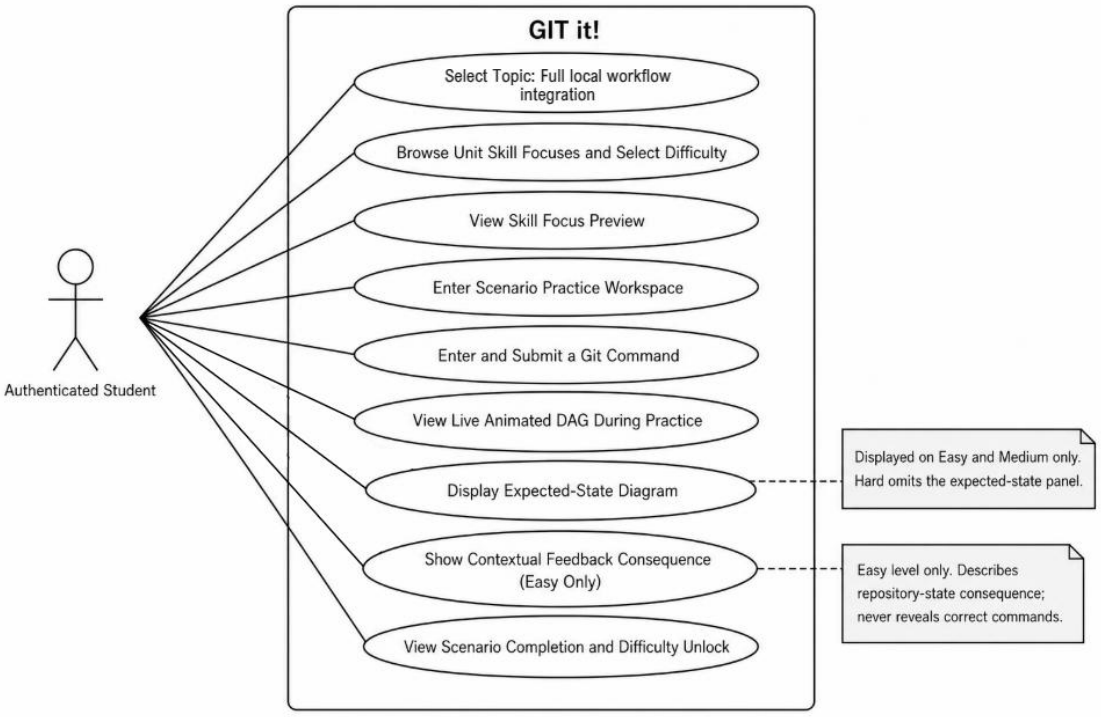



Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized
X / X
X
X

1.7 Complete Hard-Level Independent Local Repository Scenarios

Use Case Diagram

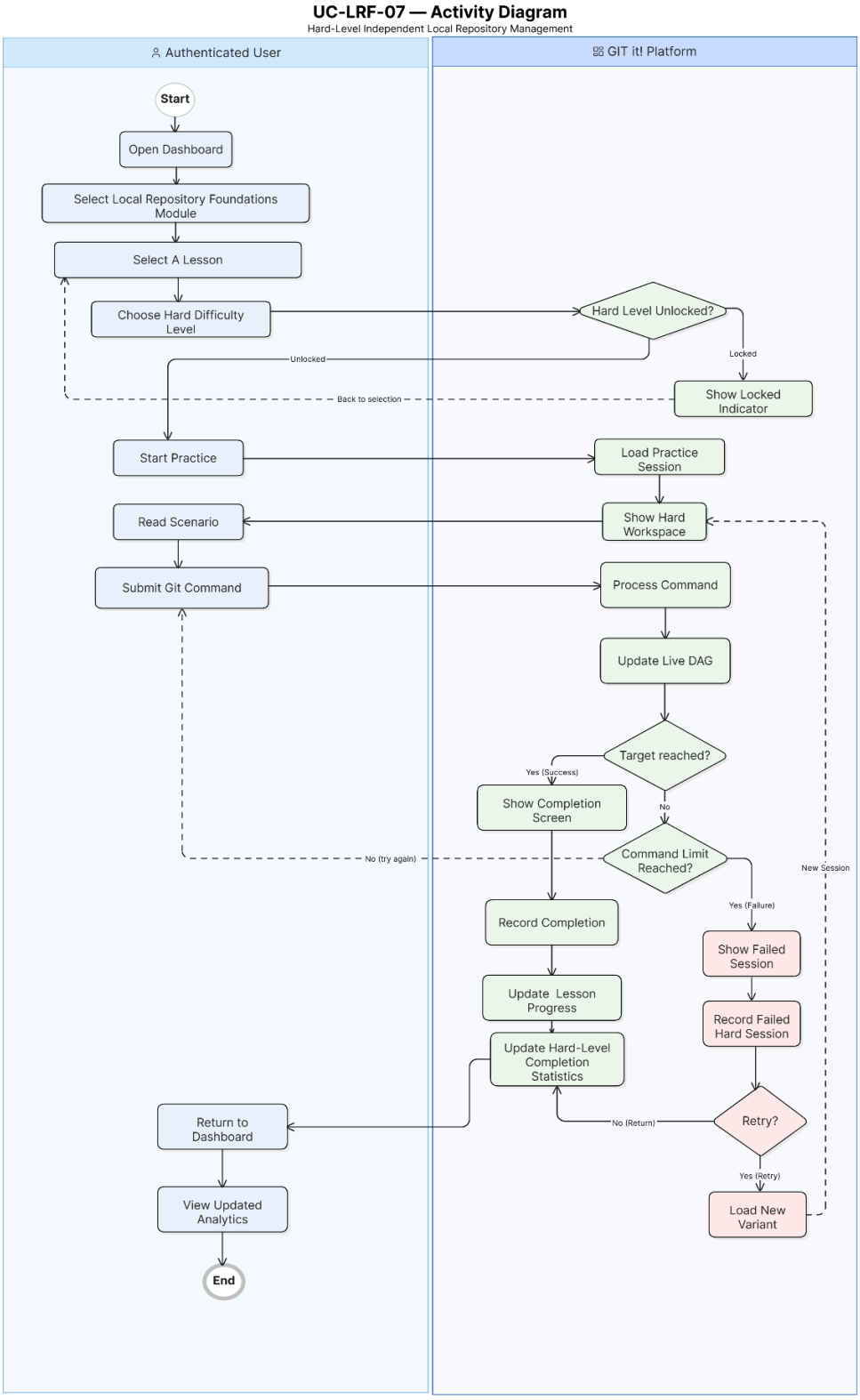


Use Case Description

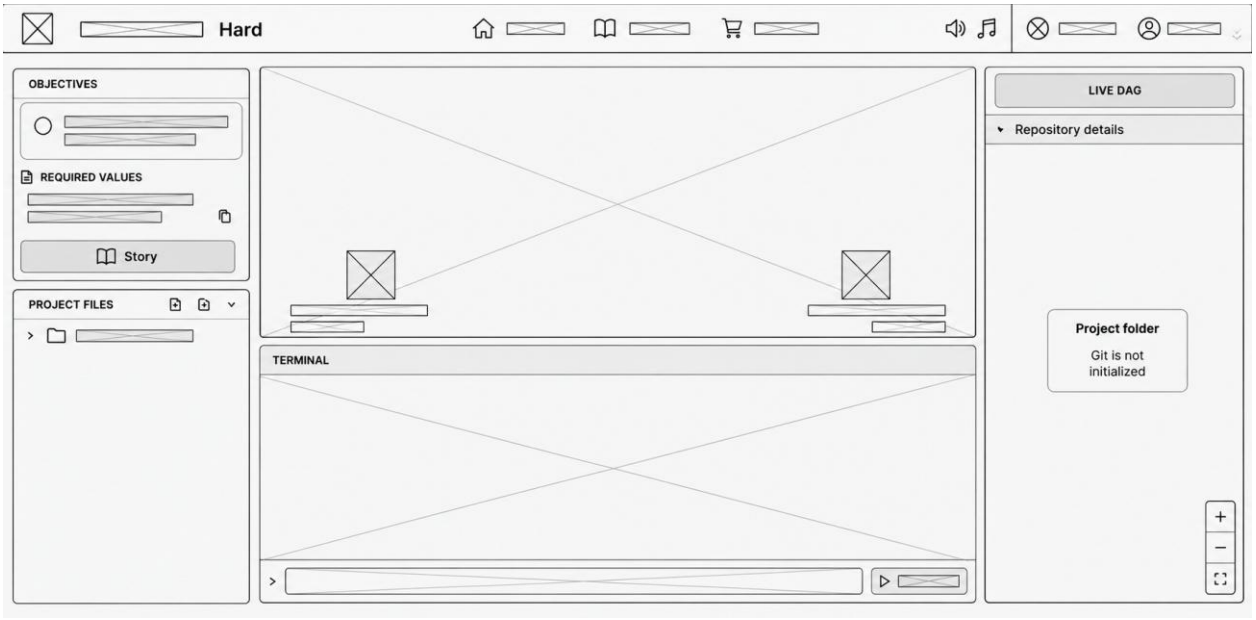
Field	Description
Use Case ID	UC-LRF-07
Use Case Name	Complete Hard-Level Independent Local Repository Scenarios
Curriculum SO	SO 1.7
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Medium difficulty is completed for the integrated skill focus in Local Repository Foundations. Hard difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and expands the Local Repository Foundations chapter entry.2. Student selects the integrated learning objective for multi-step local repository management without scaffolding support.3. Student confirms Medium is completed and clicks Start or Retry on the unlocked Hard difficulty action.4. Student reads the mission briefing and starts the tier.5. Student reads the multi-step Hard-level narrative (no target-state guidance; Live DAG and task prompt only).6. Student types Git commands and observes terminal output and Live DAG updates across multiple repository-state steps.7. Student checks the counted-command allowance throughout the session.8. Student applies repository-state reasoning without expected-state scaffolding to reach each step goal.9. Student sees the completion screen when all required target states are reached, or clicks Retry after failure.10. Student sees updated Hard completion status on the learning objective card when successful.

Postconditions	Student has completed or attempted the Hard-level integrated scenario. HLCR-relevant completion status is visible on the dashboard and chapter entry.
Alternative Flows	AF1 — Hard locked: Student sees Hard disabled until Medium is completed.AF2 — Session failed: Student clicks Retry without automatically receiving a scenario-specific solution sequence.AF3 — Counted-command limit: Student sees Failed state and may retry or exit.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19Note: Emphasizes FR-SCENE-03 (Hard scaffolding) and FR-DASH-07 (HLCR indicators).

Activity Diagram

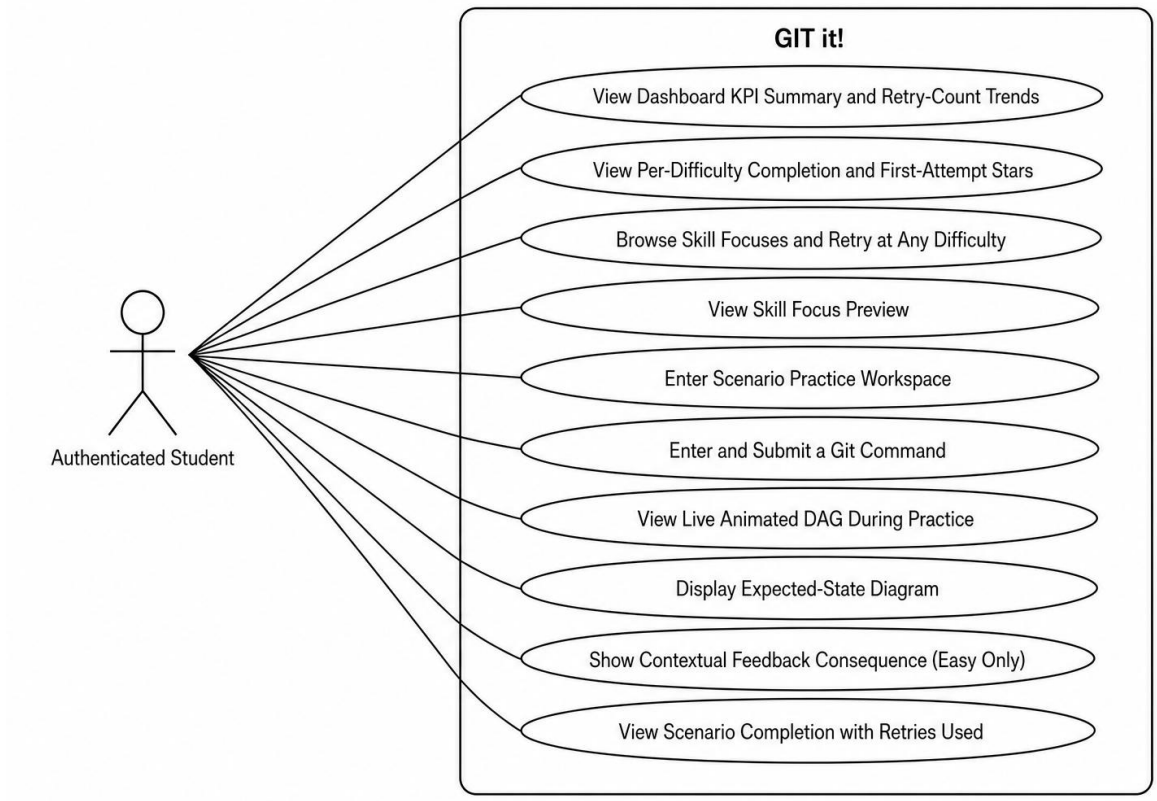


Wireframe



1.8 Monitor Retry Practice and Apply Repository-State Reasoning

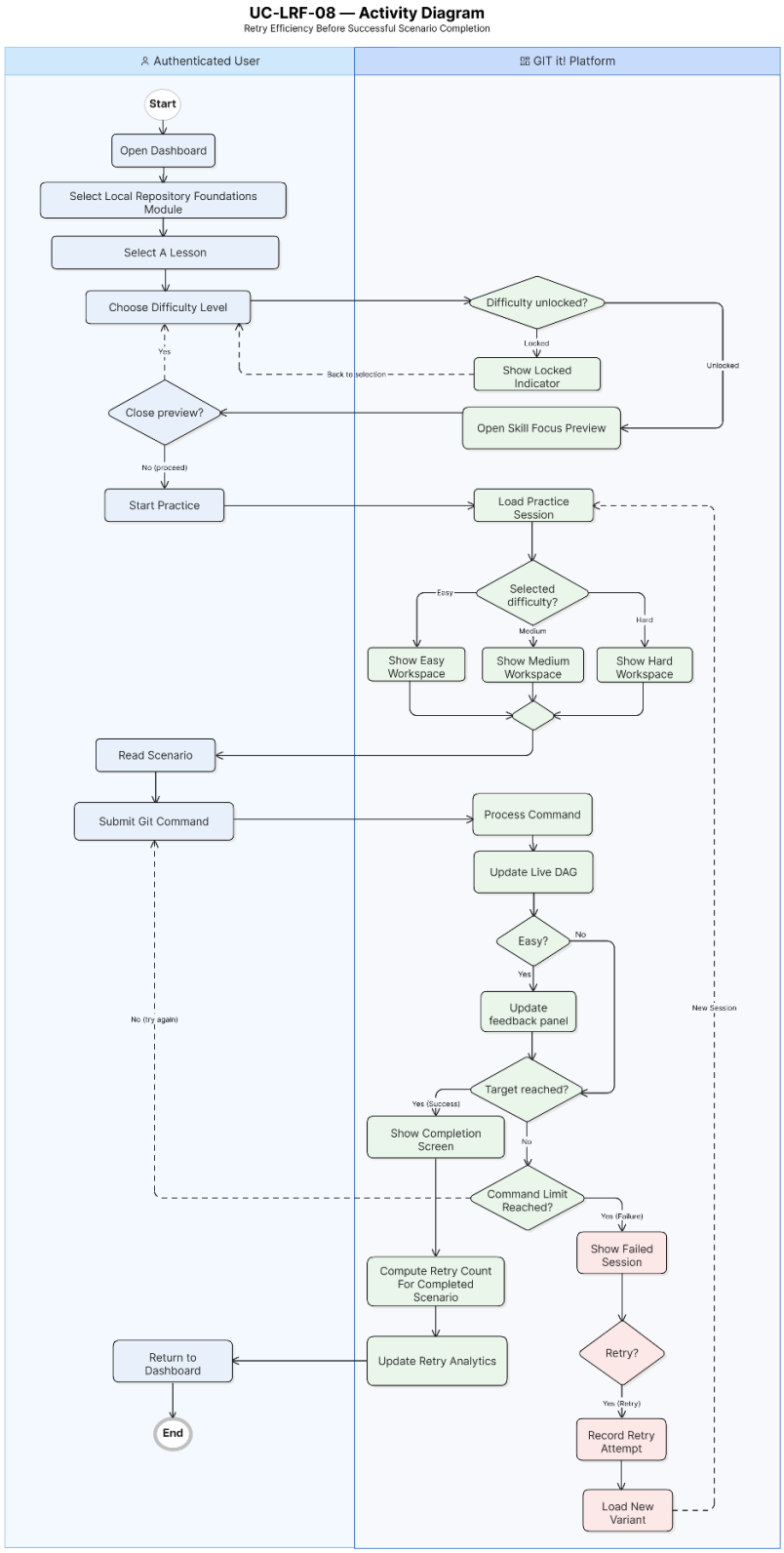
Use Case Diagram



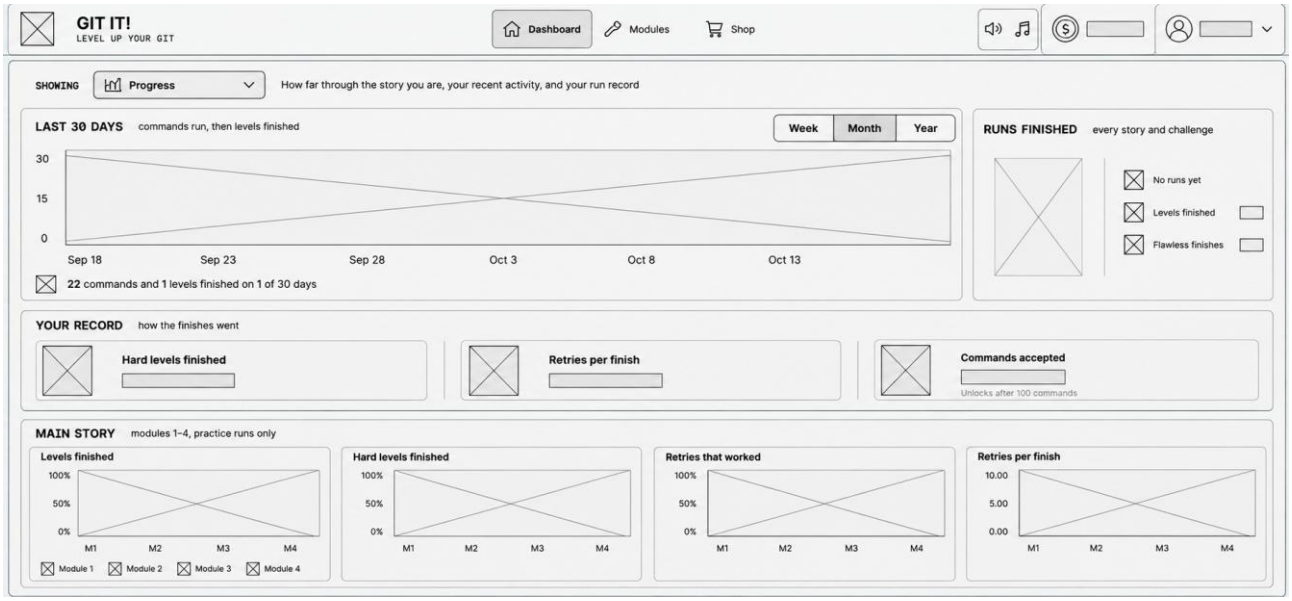
Use Case Description

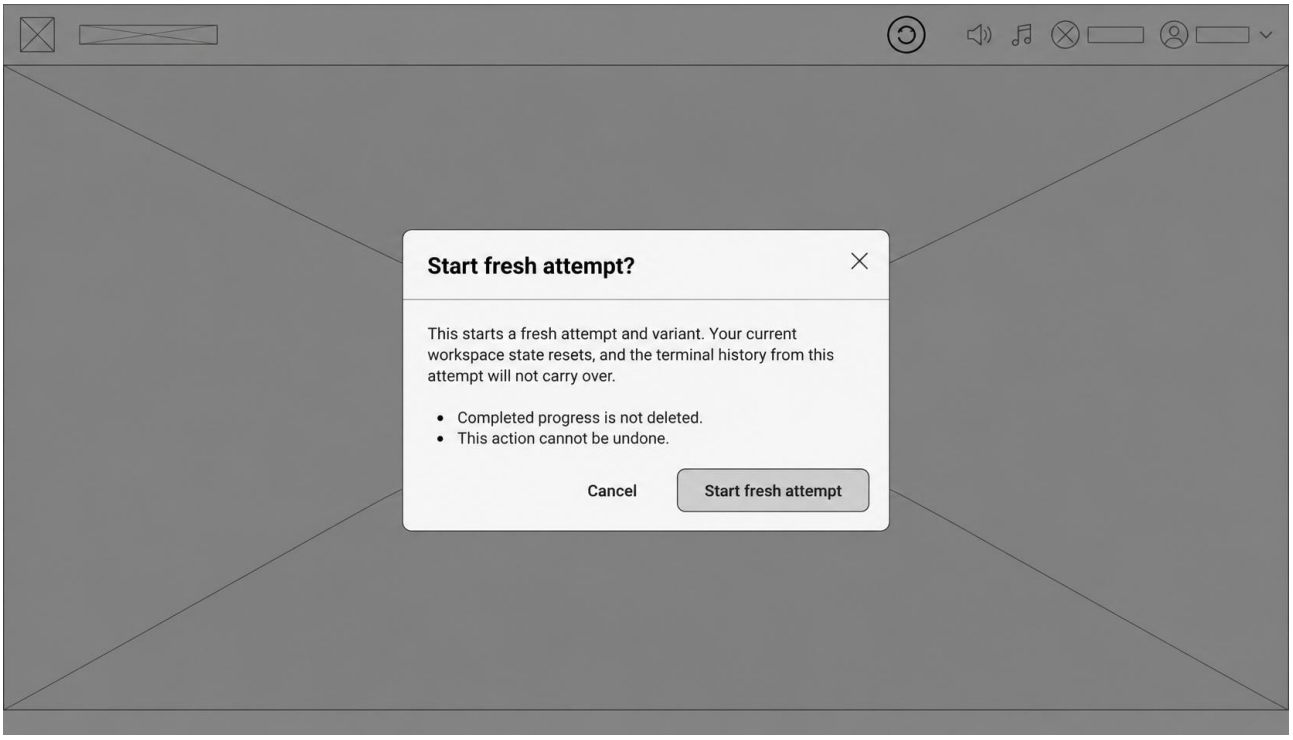
Field	Description
Use Case ID	UC-LRF-08
Use Case Name	Monitor Retry Practice and Apply Repository-State Reasoning
Curriculum SO	SO 1.8
Actor(s)	Authenticated Student
Preconditions	Student is logged in and has access to the Home and Module scenario cards.
Main Flow	1. Student navigates to the Home.2. Student views KPI summary indicators and per-scenario retry-count trends for Module 1 — Local Repository Foundations.3. Student opens a chapter and reviews the Easy/Medium/Hard states of its levels on the story map.4. Student opens a scenario practice session (Start, Continue, or Retry).5. Student reads the narrative, inspects the Live DAG, and plans the next command before submitting.6. Student submits commands and monitors terminal output, DAG changes, and the counted-command allowance.7. If the session fails, student clicks Retry and returns to practice with a fresh attempt.8. Student sees retries used on the completion screen and updated retry trends on the dashboard after finishing.
Postconditions	Student has reviewed retry trends and completed or retried practice sessions, supporting ARC monitoring for the module.
Alternative Flows	AF1 — No trend data yet: Student sees 'No trend available' on dashboard. AF2 — Student abandons mid-session: Partial progress not persisted; card shows Abandoned.
Functional Requirements	FR-DASH-02, FR-DASH-04, FR-DASH-05, FR-DASH-07, FR-SCENE-03, FR-SCENE-19

Activity Diagram



Wireframe

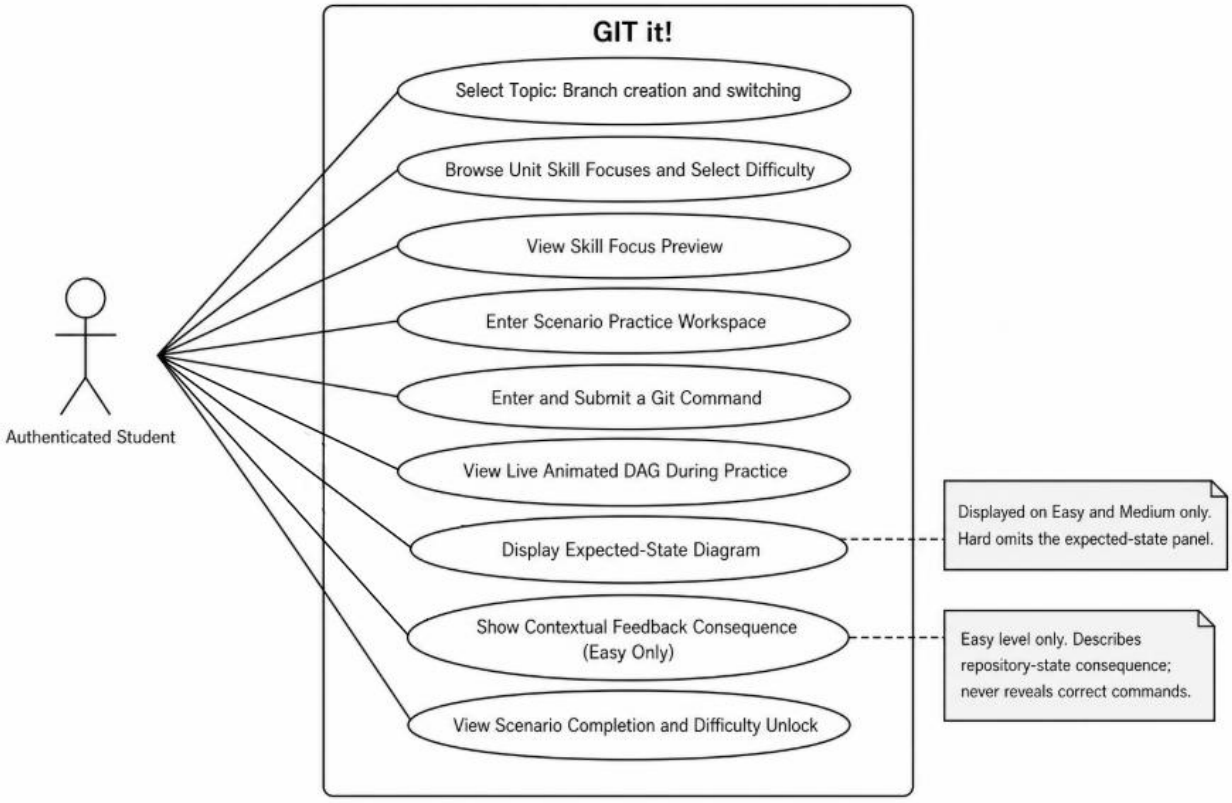

GIT IT!
LEVEL UP YOUR GIT
Dashboard
Modules
Shop
SHOWING
Progress
How far through the story you are, your recent activity, and your run record
LAST 30 DAYS
commands run, then levels finished
Week
Month
Year
30
15
0
Sep 18
Sep 23
Sep 28
Oct 3
Oct 8
Oct 13
22 commands and 1 levels finished on 1 of 30 days
RUNS FINISHED
every story and challenge
No runs yet
Levels finished
Flawless finishes
YOUR RECORD
how the finishes went
Hard levels finished
Retries per finish
Commands accepted
Unlocks after 100 commands
MAIN STORY
modules 1–4, practice runs only
Levels finished
Hard levels finished
Retries that worked
Retries per finish
100%
50%
0%
10.00
5.00
0.00
M1
M2
M3
M4
Module 1
Module 2
Module 3
Module 4


Start fresh attempt?
This starts a fresh attempt and variant. Your current workspace state resets, and the terminal history from this attempt will not carry over.
Completed progress is not deleted.
This action cannot be undone.
Cancel
Start fresh attempt

Module 2 — Branching and Collaboration

2.1 Practice Creating and Switching Branches

Use Case Diagram

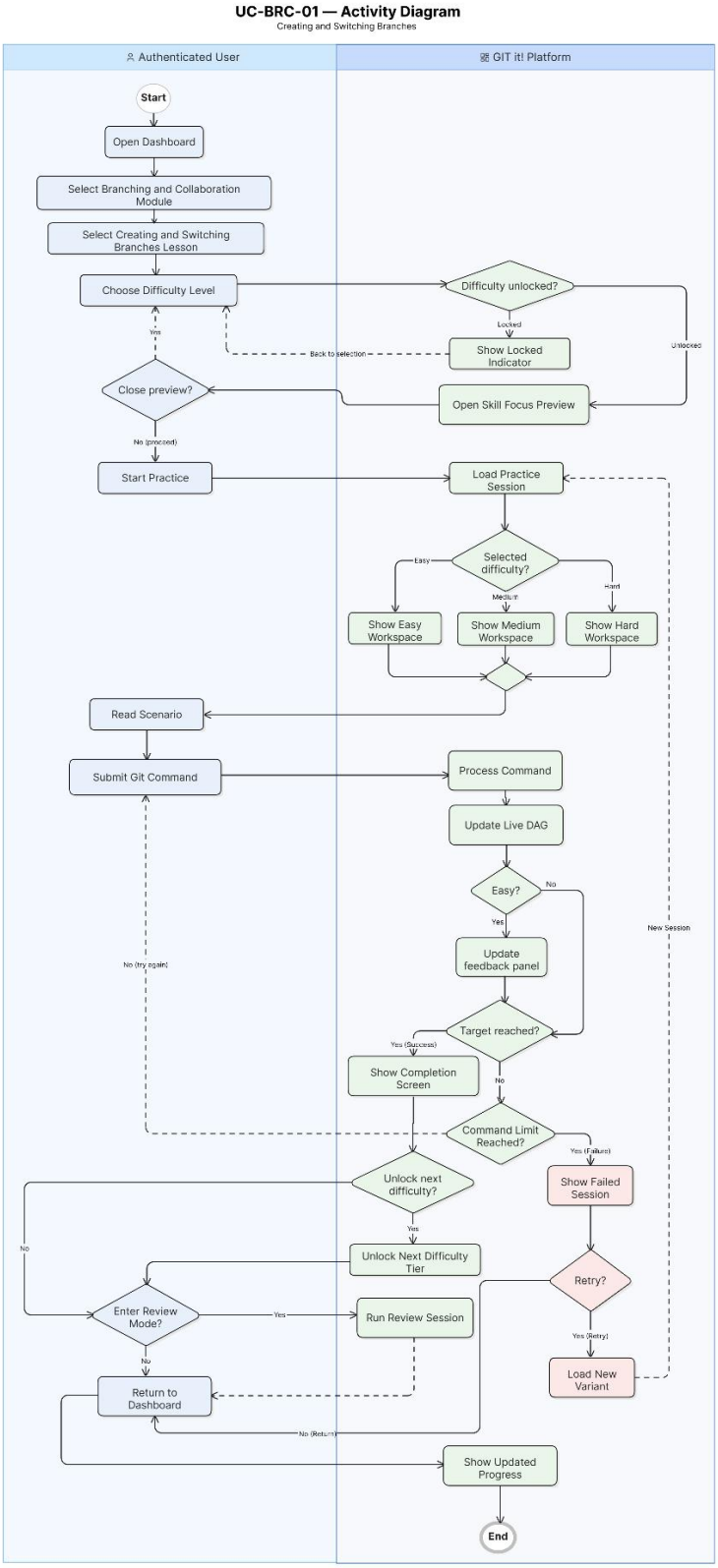


Use Case Description

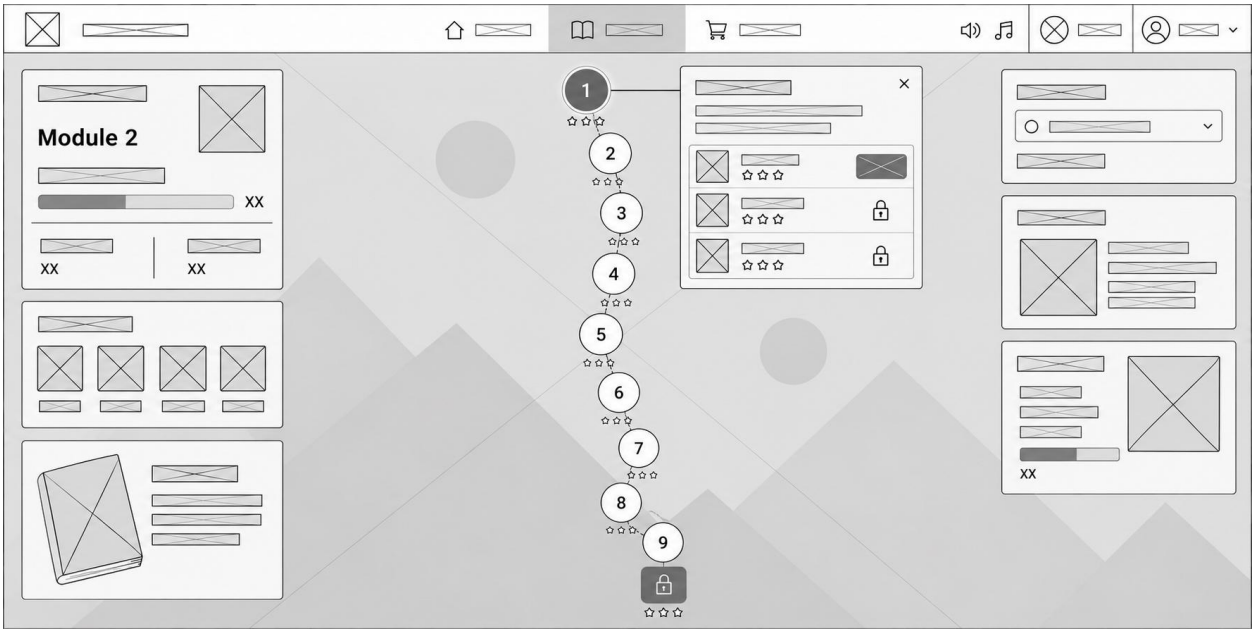
Field	Description
Use Case ID	UC-BRC-01
Use Case Name	Practice Creating and Switching Branches
Curriculum SO	SO 2.1
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Branch creation and switching are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Branch creation and switching.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the

	scenario narrative and task prompt describing creating and switching branches across varied topologies.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced creating and switching branches across varied topologies in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

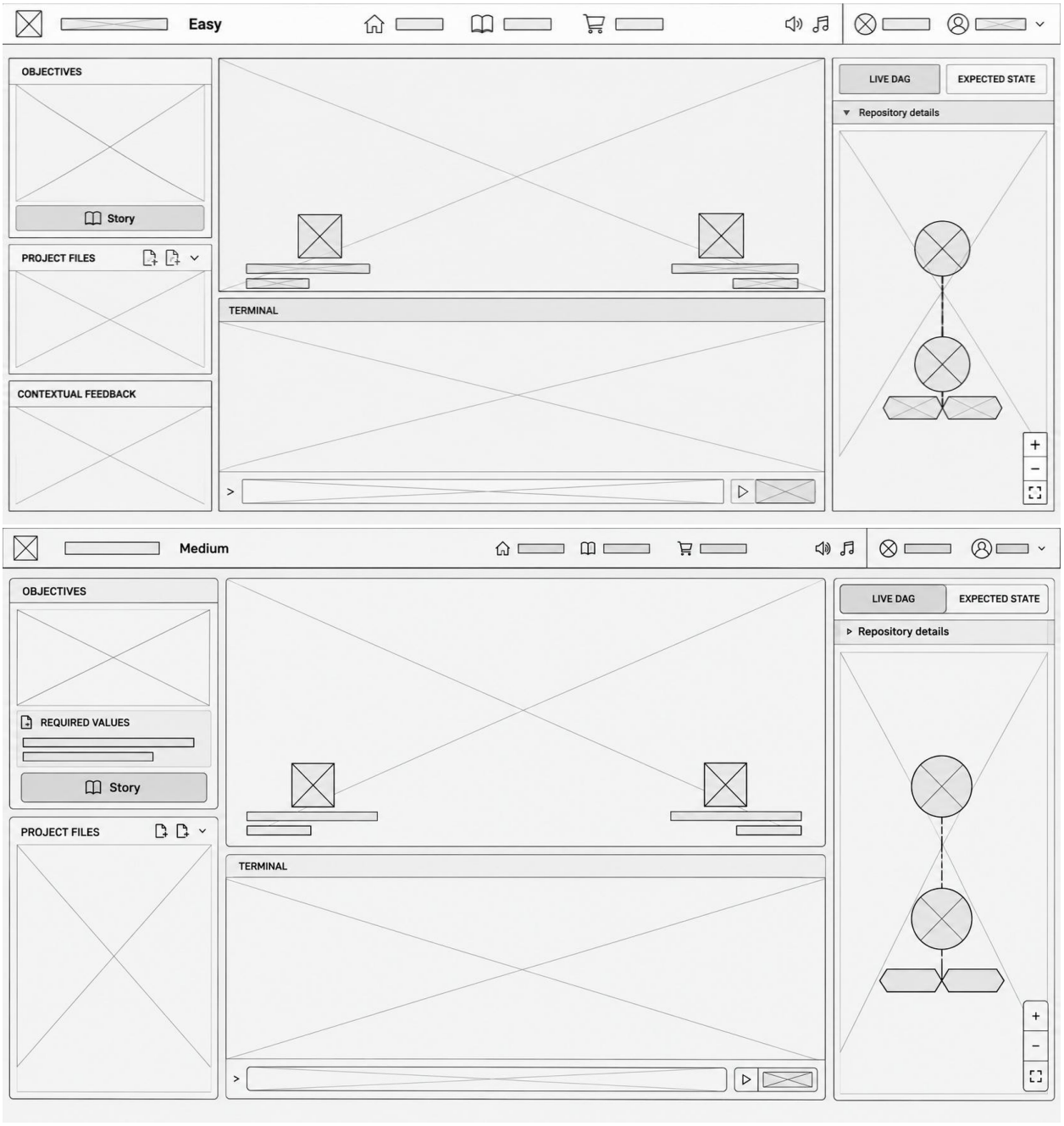
Activity Diagram

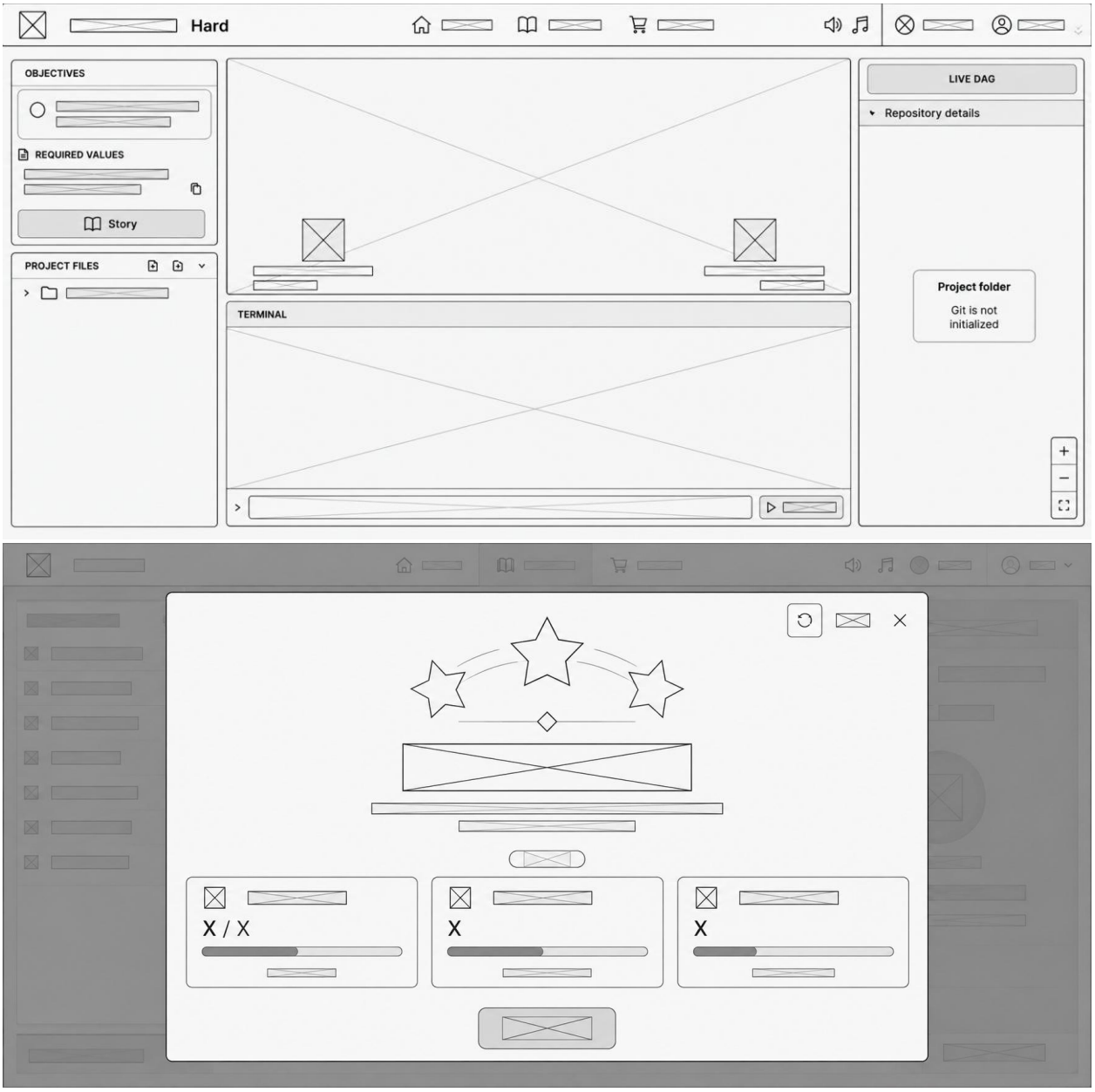


Wireframe



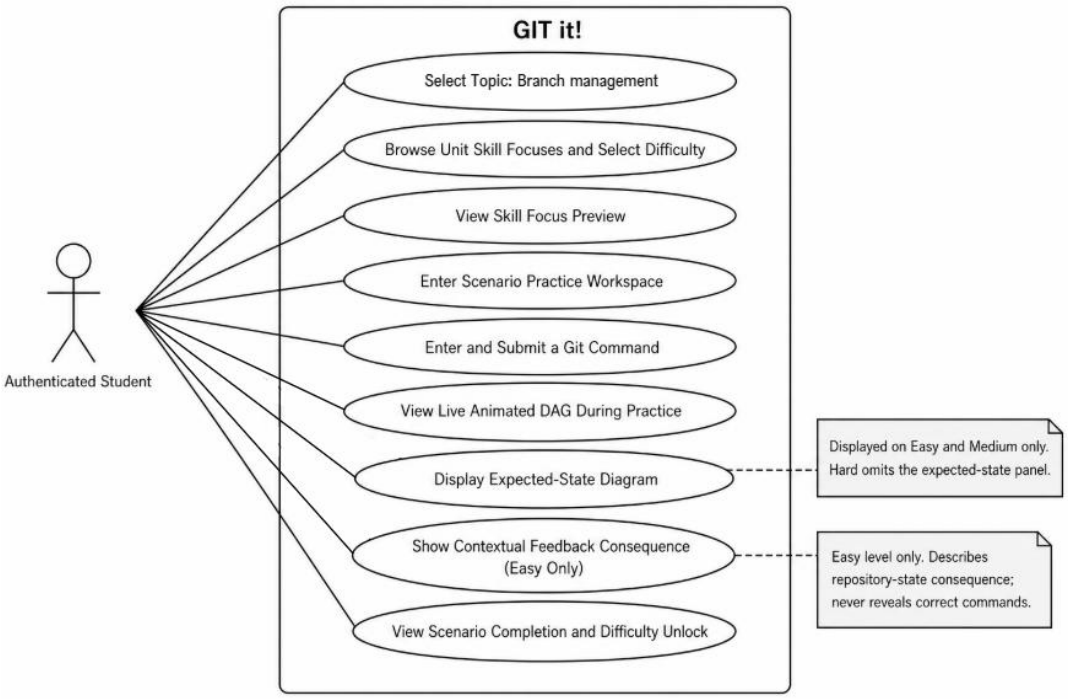



Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized
X / X
X
X

2.2 Practice Branch Naming and Housekeeping

Use Case Diagram

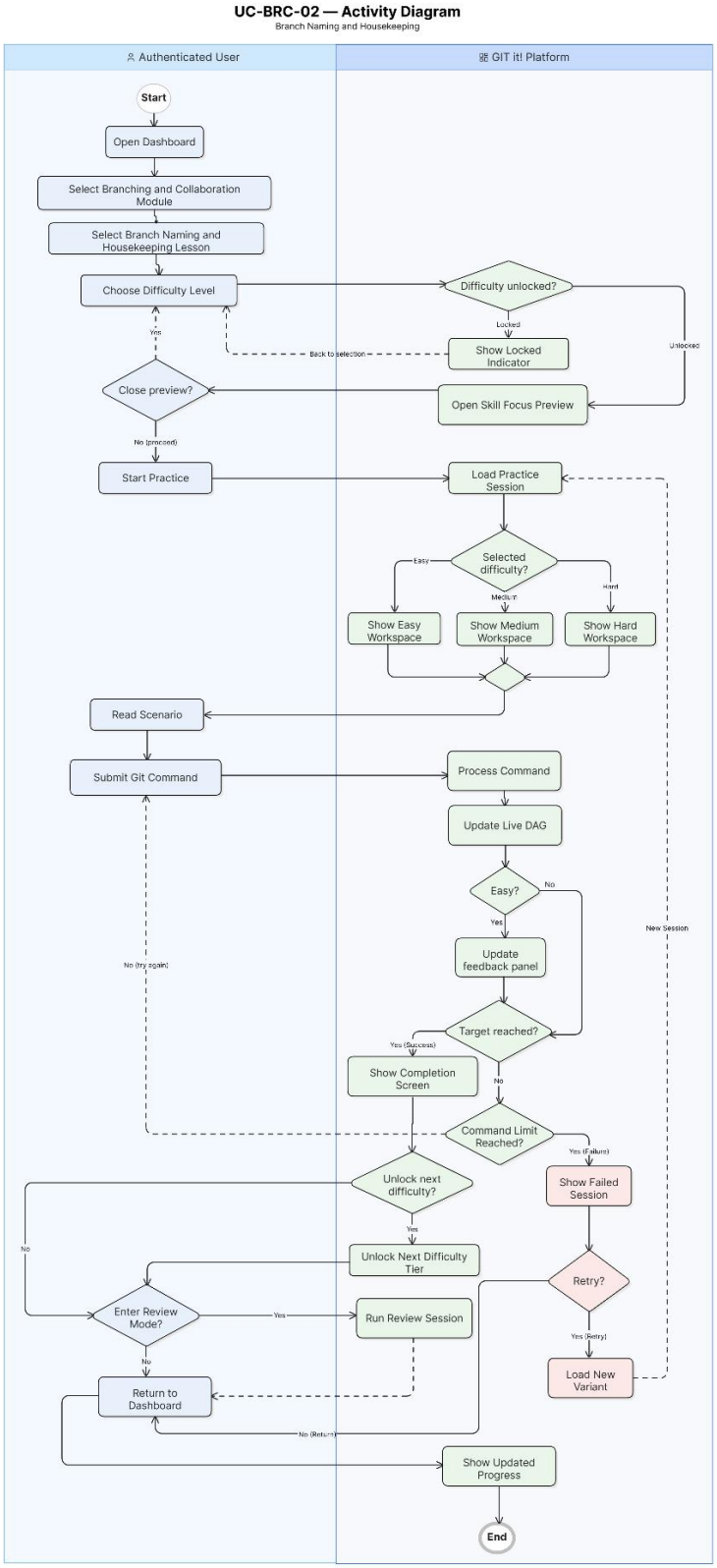


Use Case Description

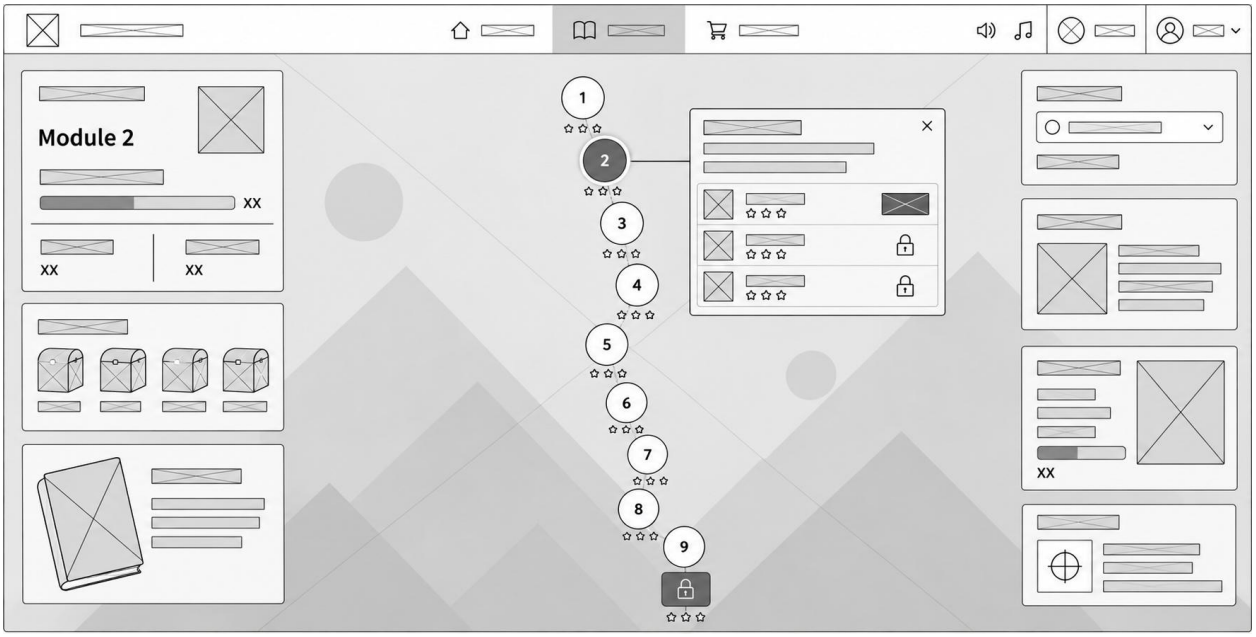
Field	Description
Use Case ID	UC-BRC-02
Use Case Name	Practice Branch Naming and Housekeeping
Curriculum SO	SO 2.2
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Branch management are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Branch management.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing applying valid branch naming and removing stale local branches.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the

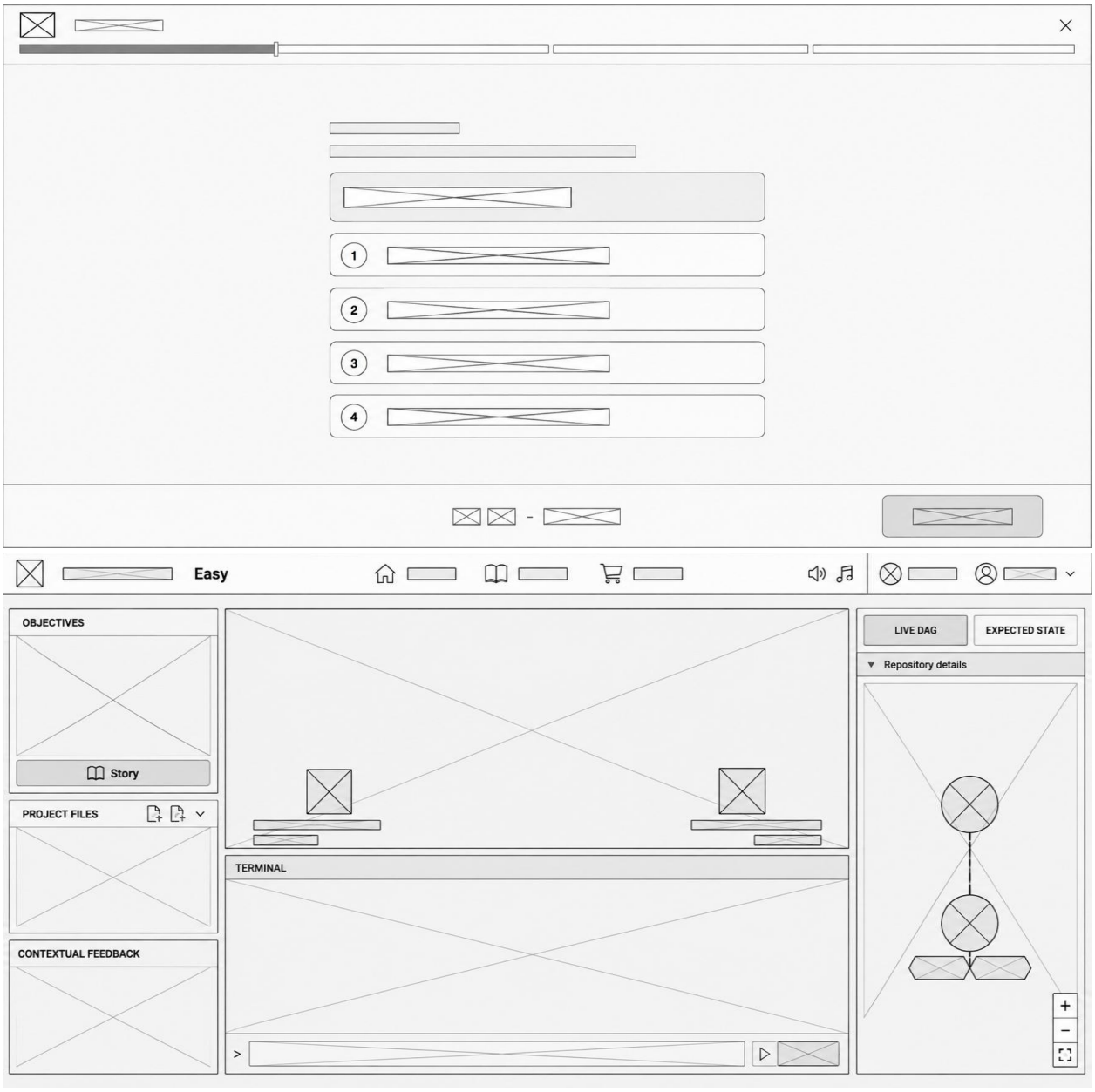
	Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced applying valid branch naming and removing stale local branches in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

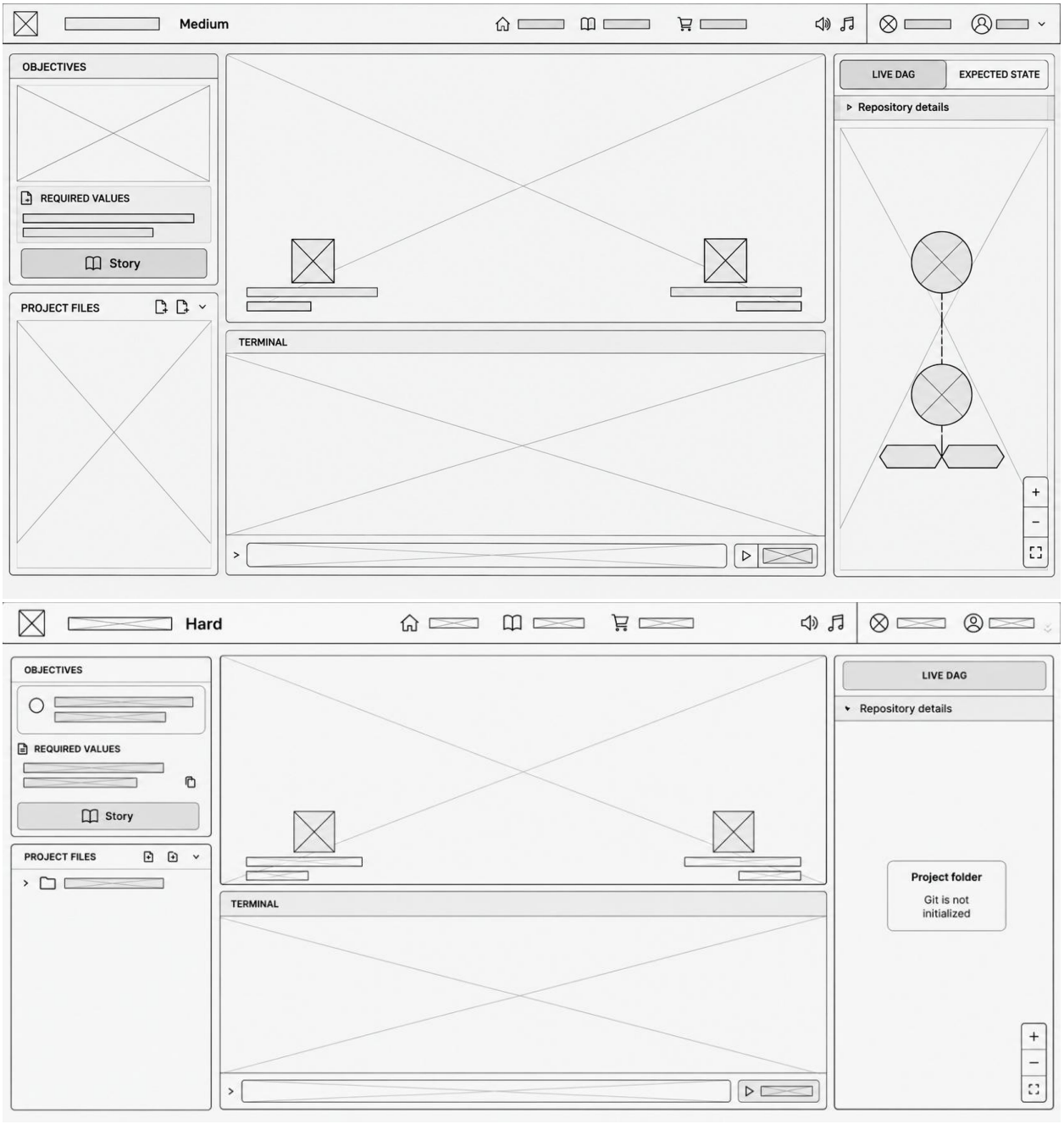
Activity Diagram

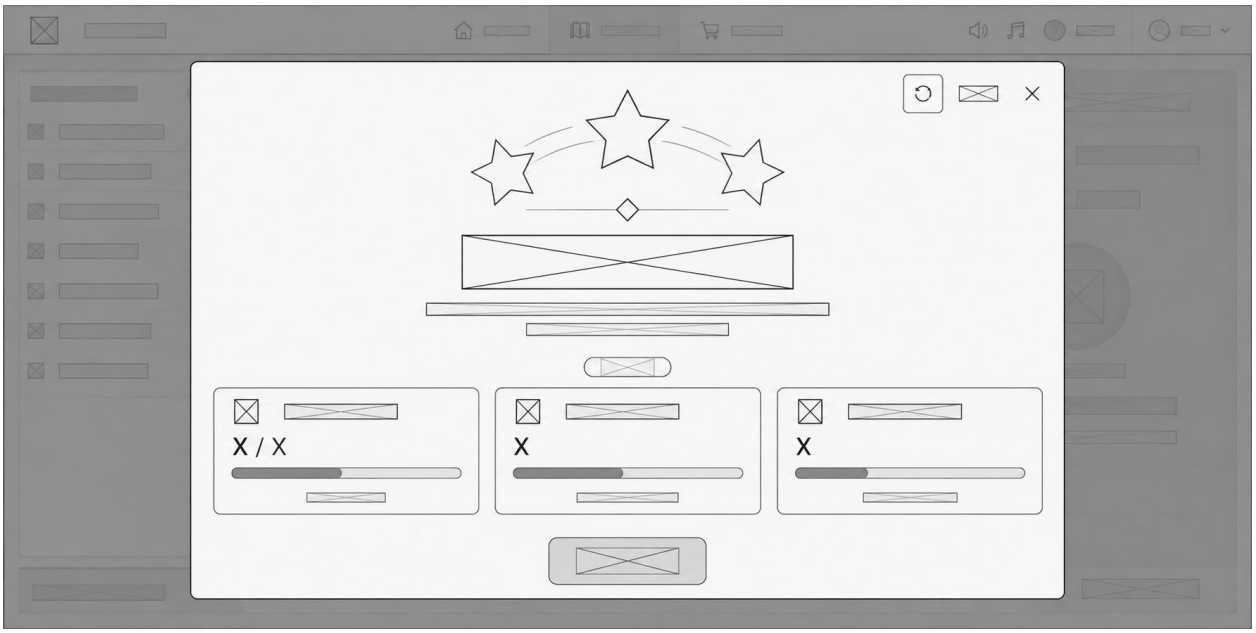


Wireframe



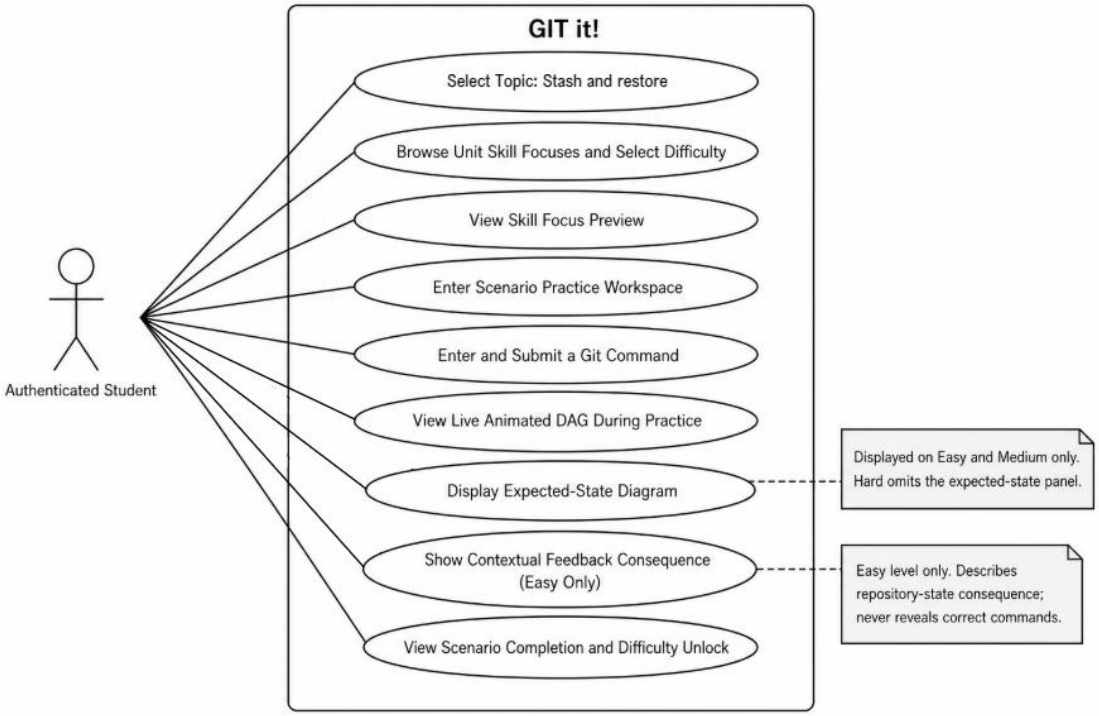

1
2
3
4
Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized



2.3 Practice Stashing and Restoring Work in Progress

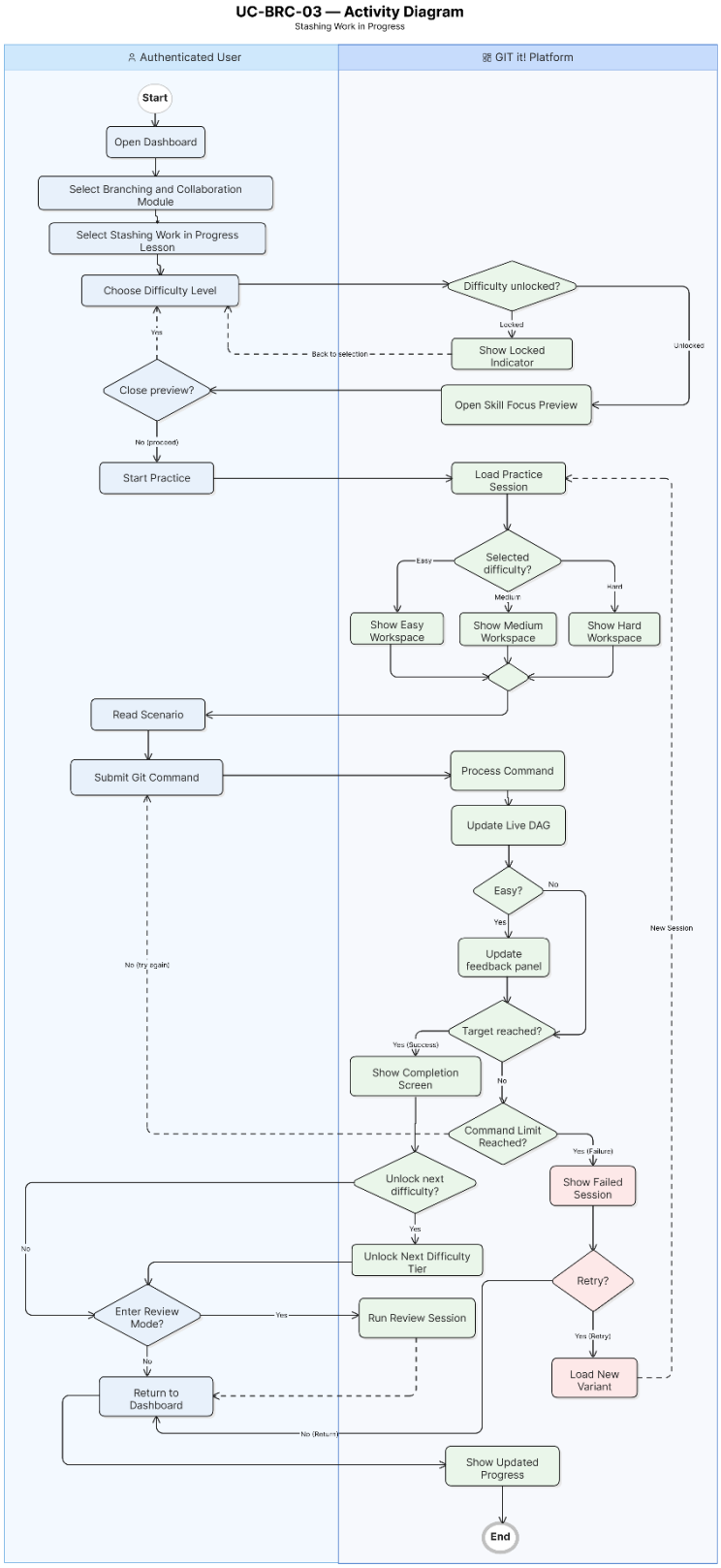
Use Case Diagram



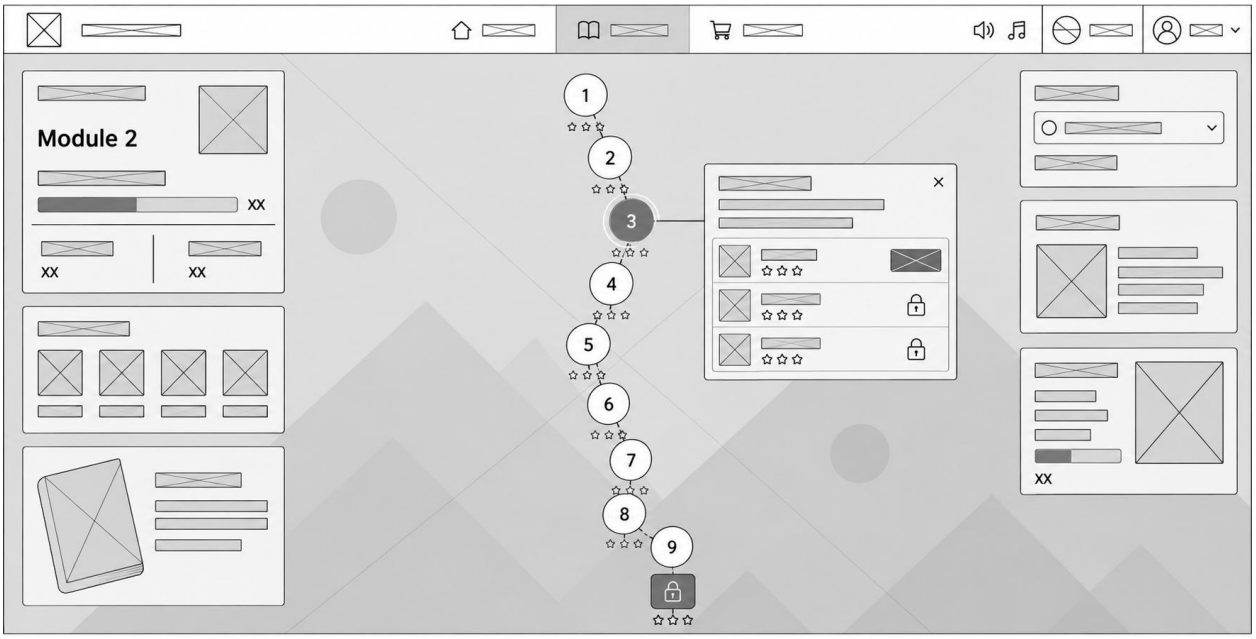
Use Case Description

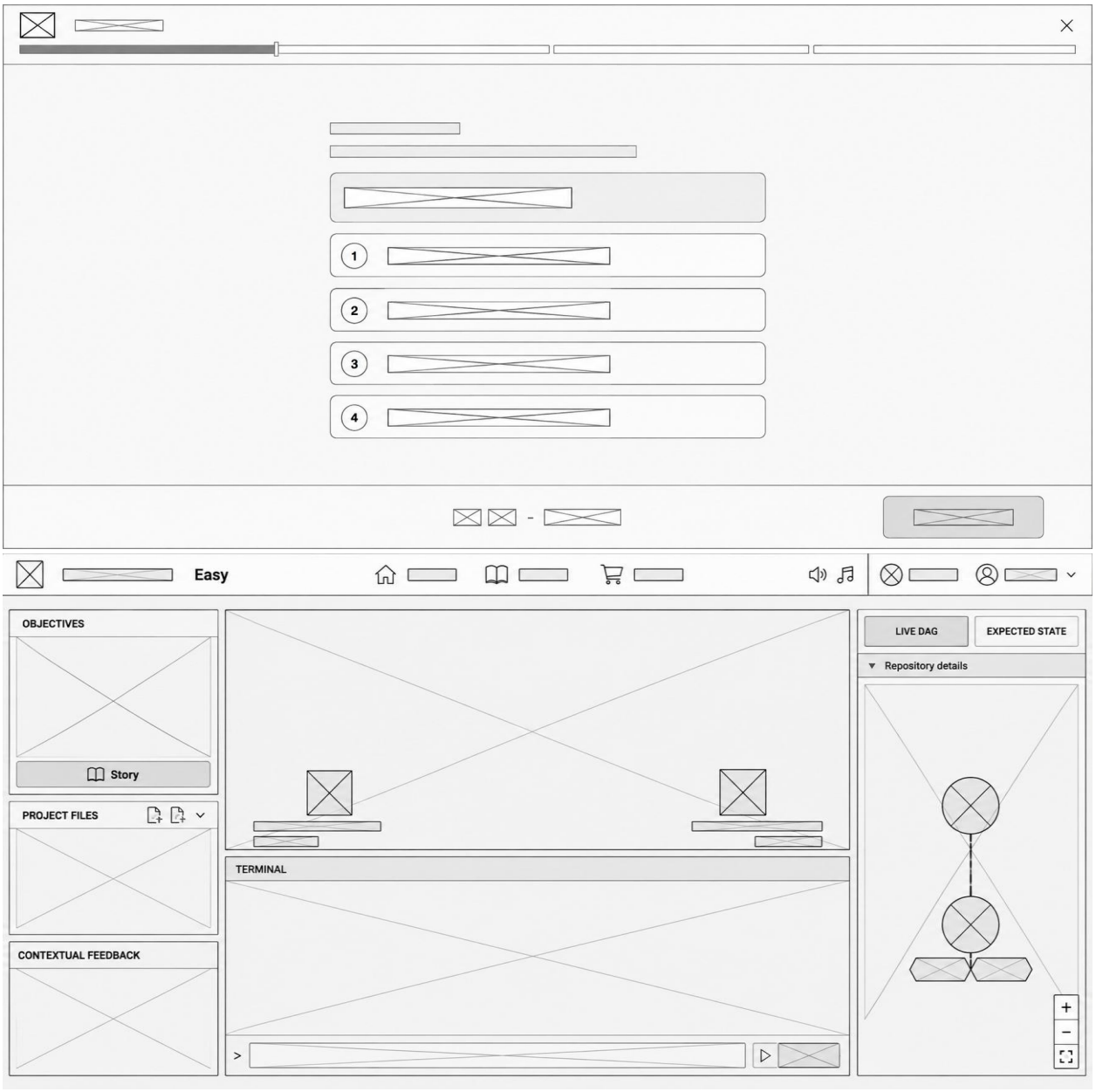
Field	Description
Use Case ID	UC-BRC-03
Use Case Name	Practice Stashing and Restoring Work in Progress
Curriculum SO	SO 2.3
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Stash and restore are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Stash and restore.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing stashing and restoring work during context switches.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced stashing and restoring work during context switches in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

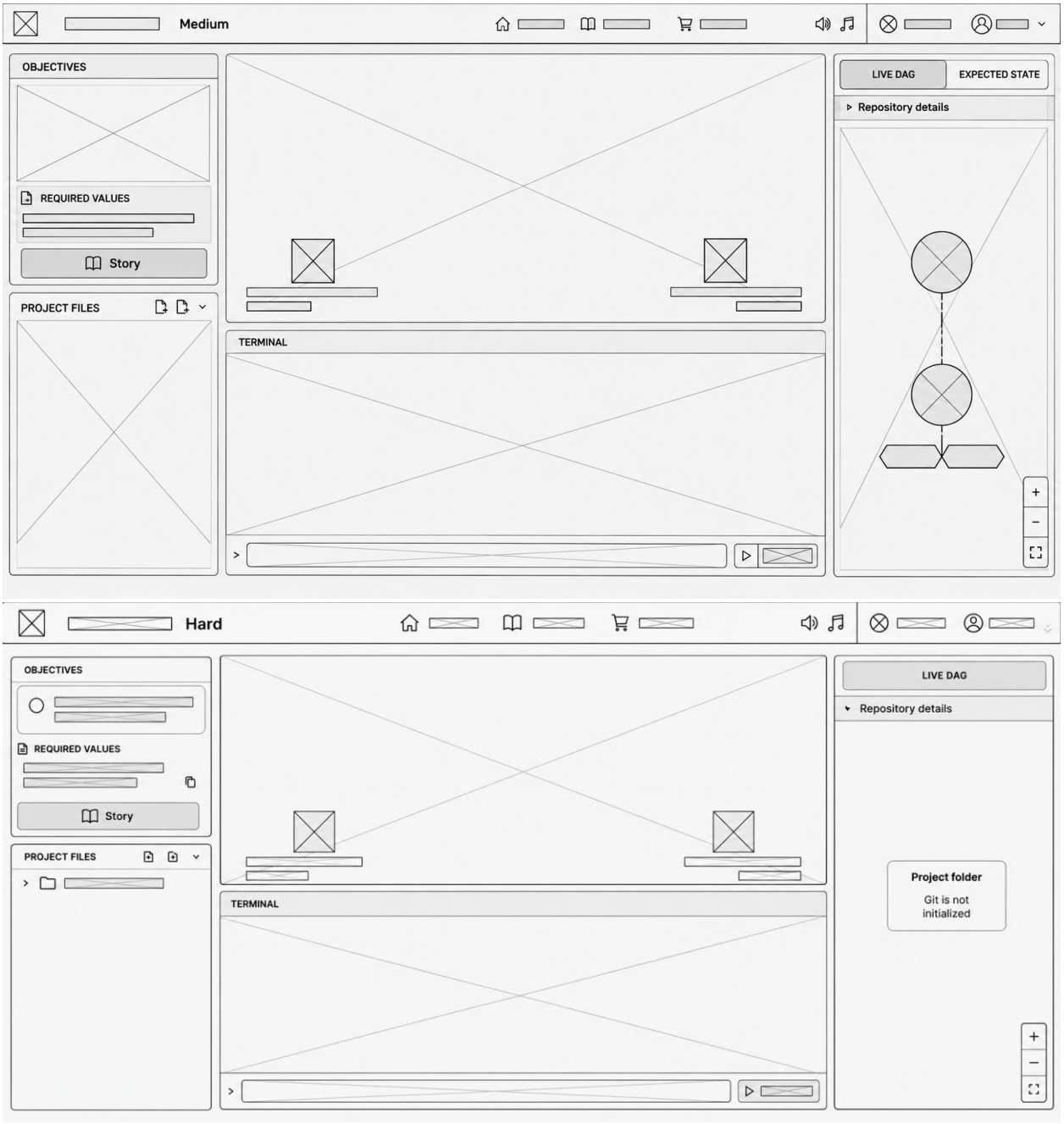
Activity Diagram

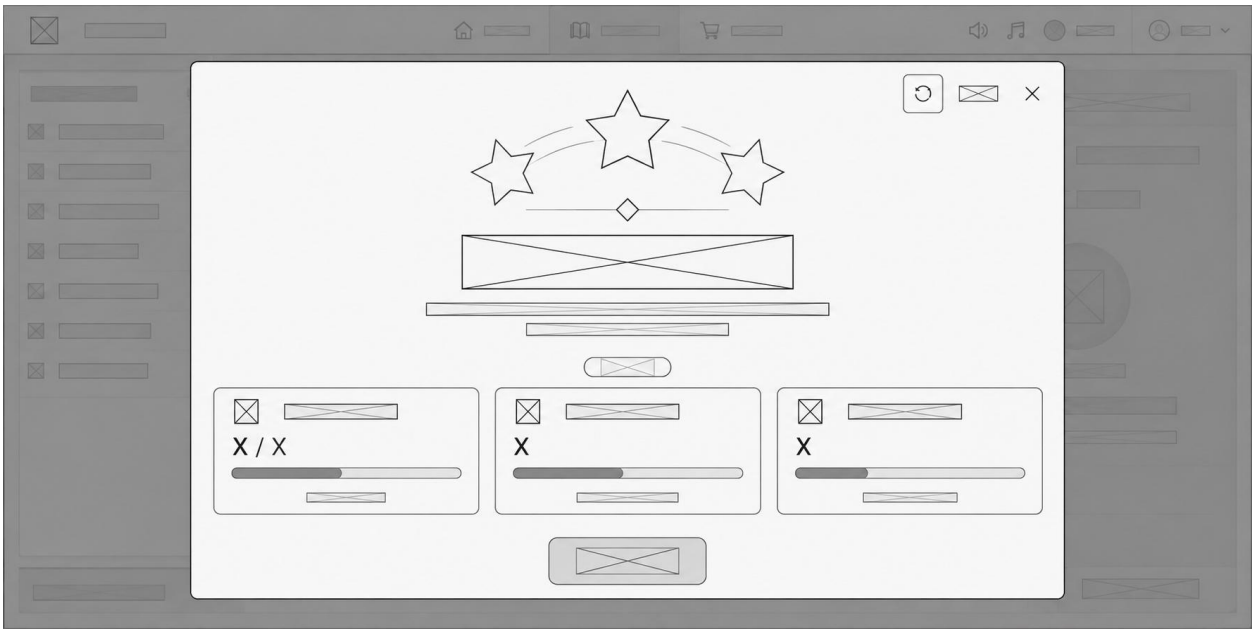


Wireframe



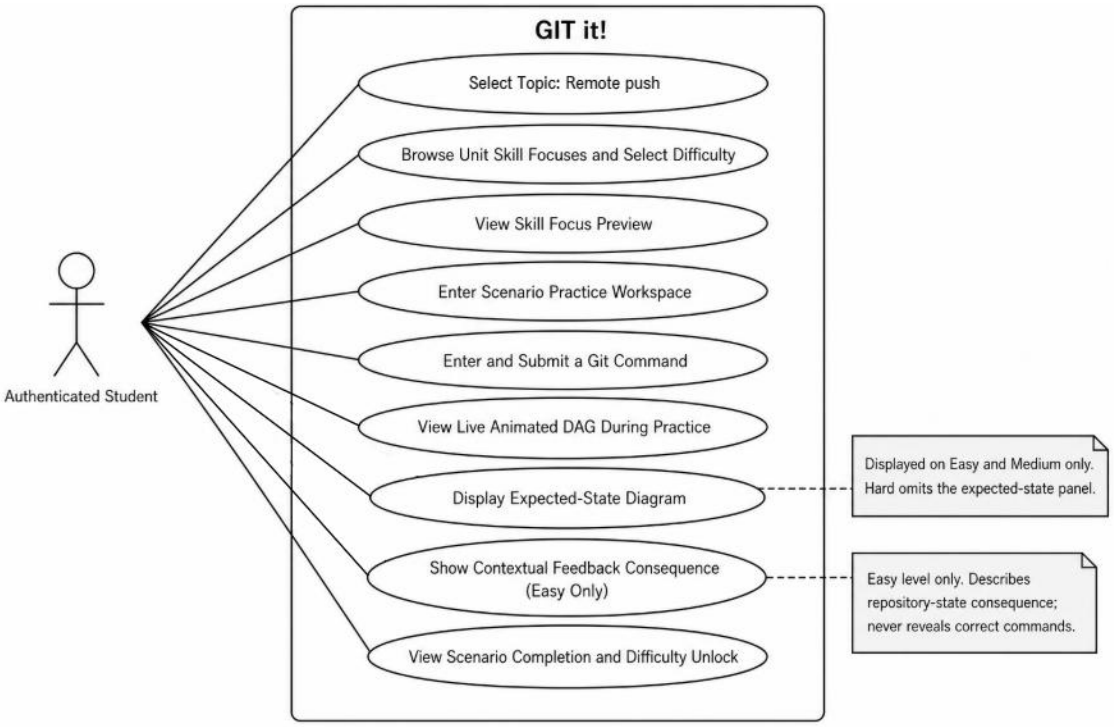

1
2
3
4
Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

2.4 Practice Pushing to a Remote

Use Case Diagram

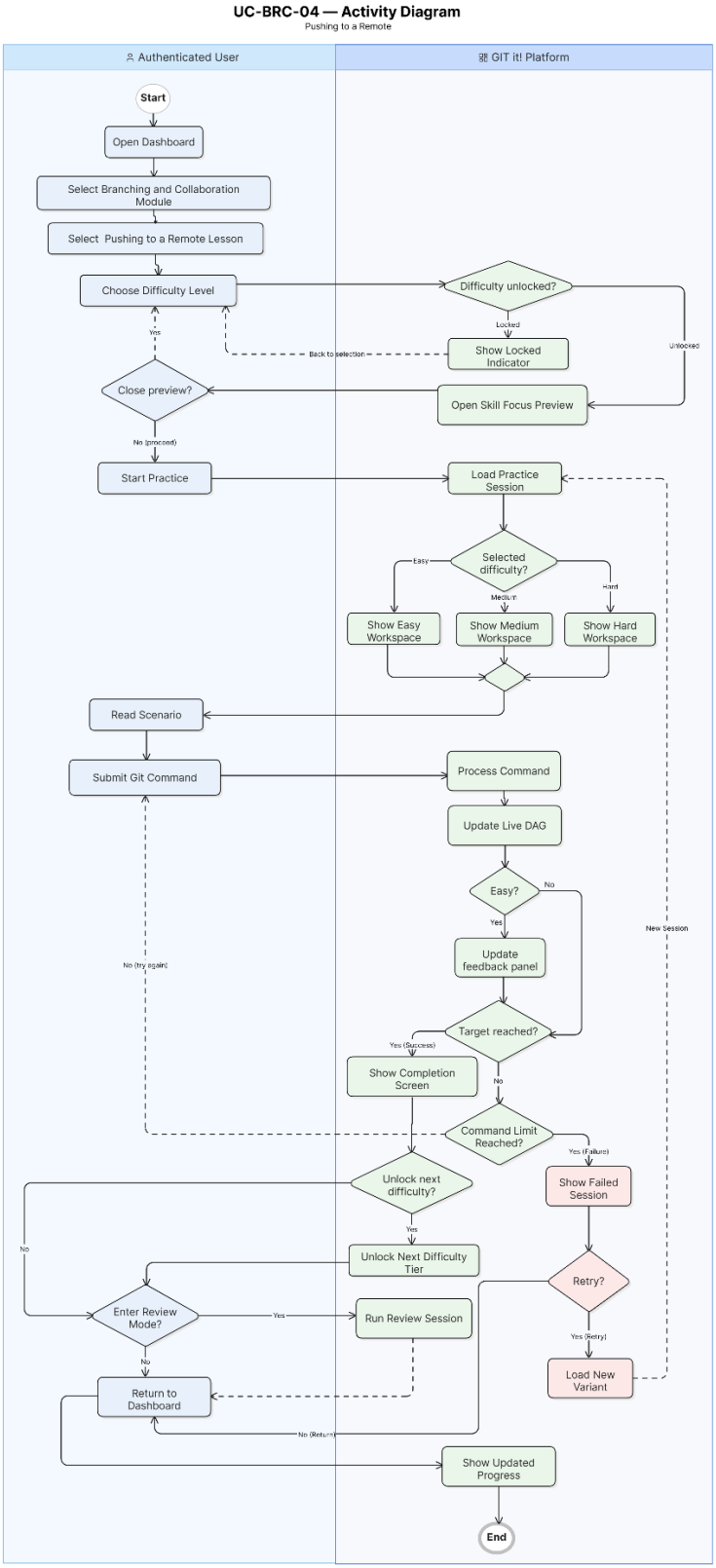


Use Case Description

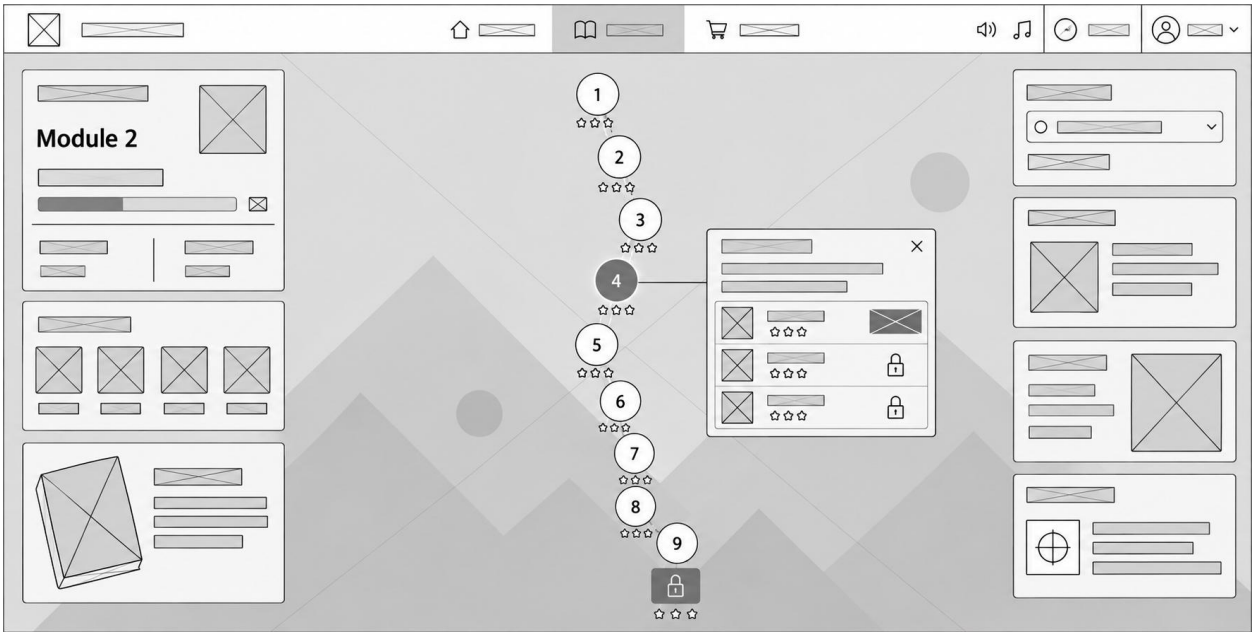
Field	Description
Use Case ID	UC-BRC-04
Use Case Name	Practice Pushing to a Remote
Curriculum SO	SO 2.4
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Remote push are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Remote push.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing pushing local branches and setting upstream tracking.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel

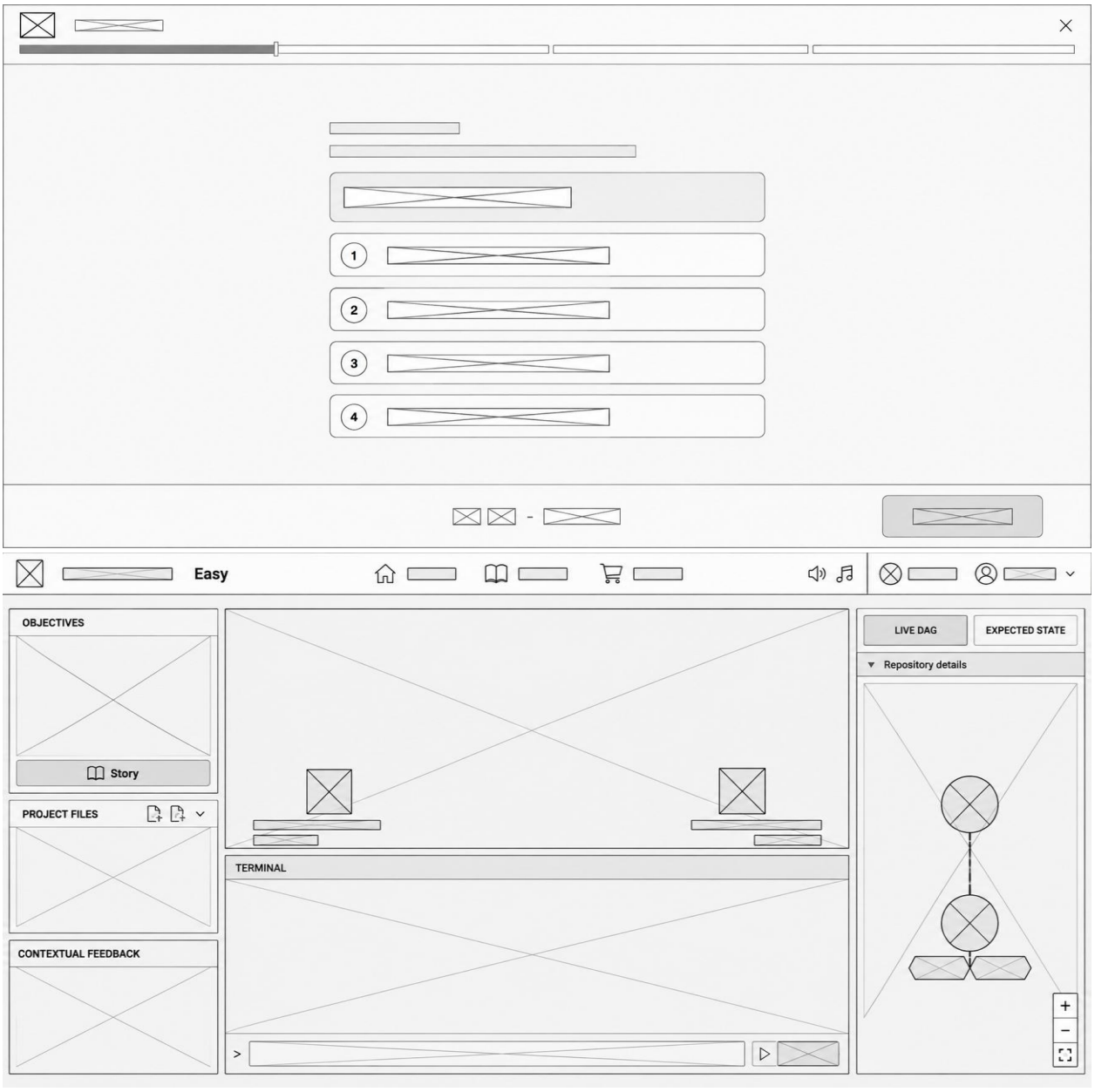
	after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced pushing local branches and setting upstream tracking in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

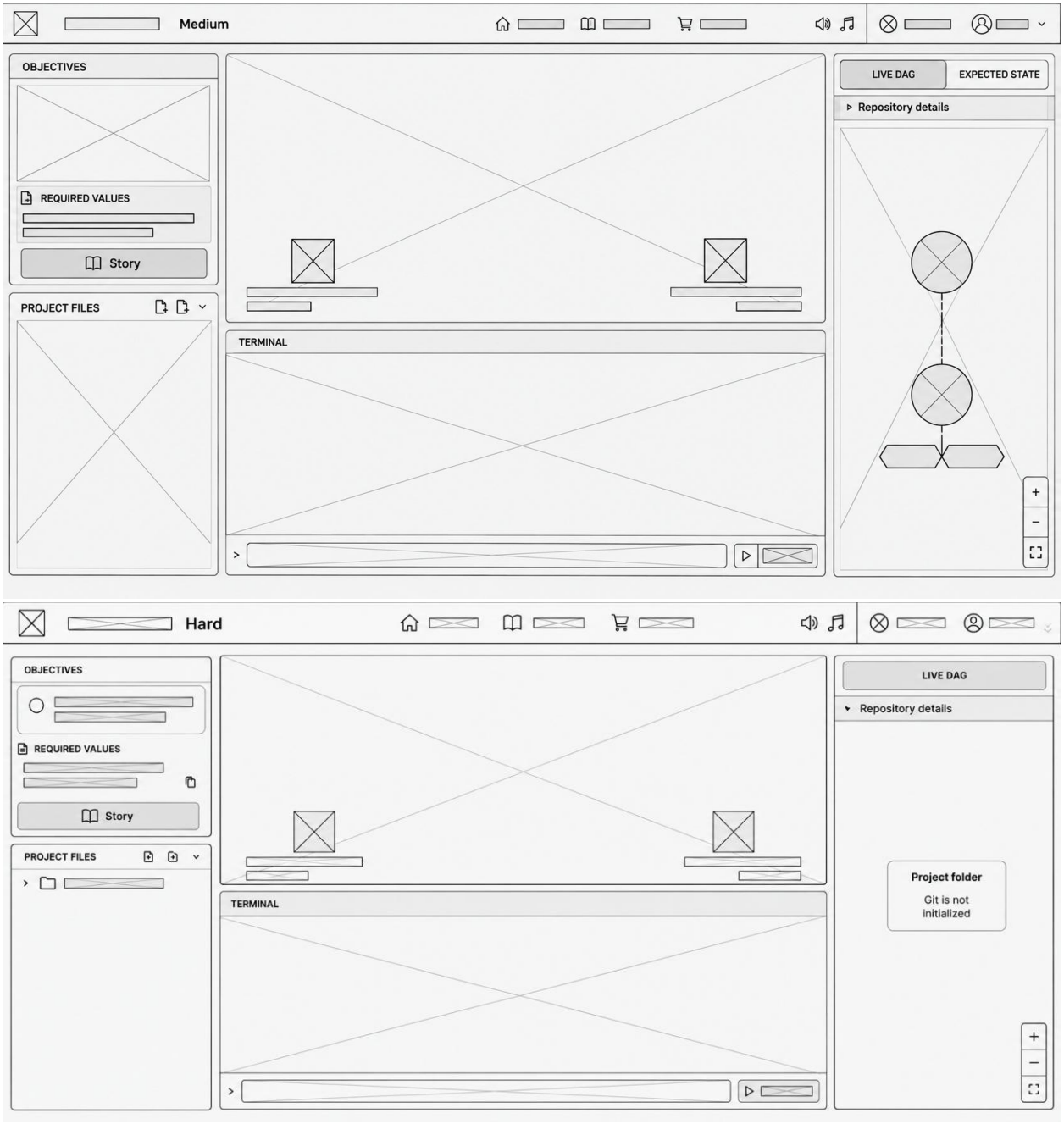
Activity Diagram

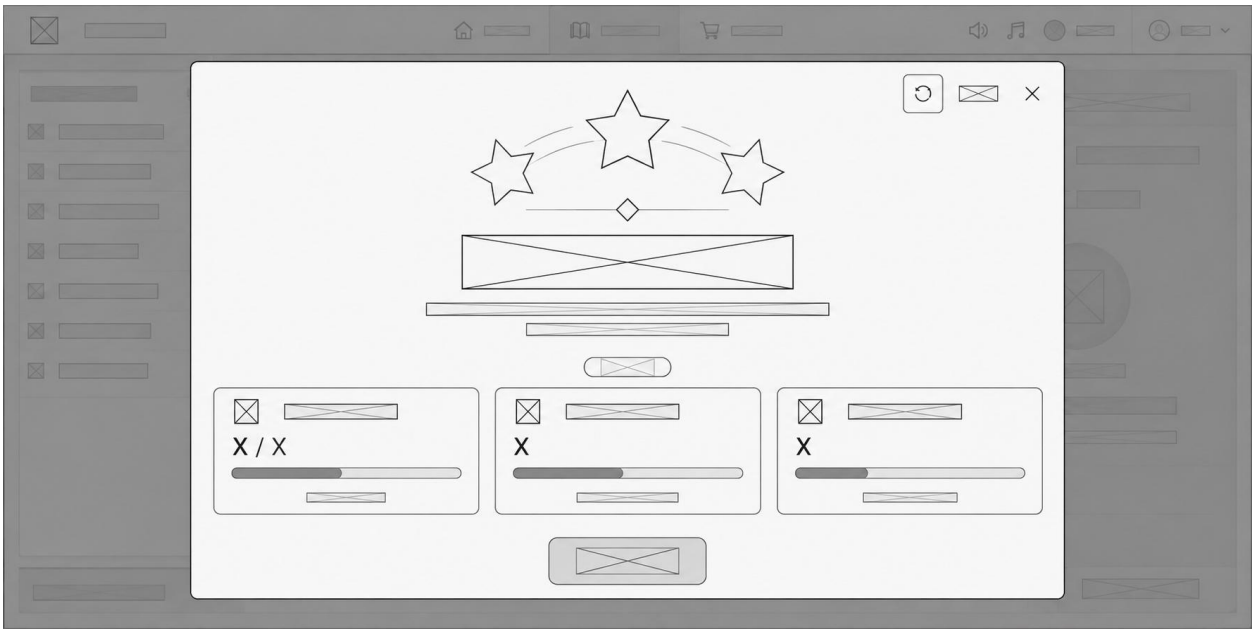


Wireframe



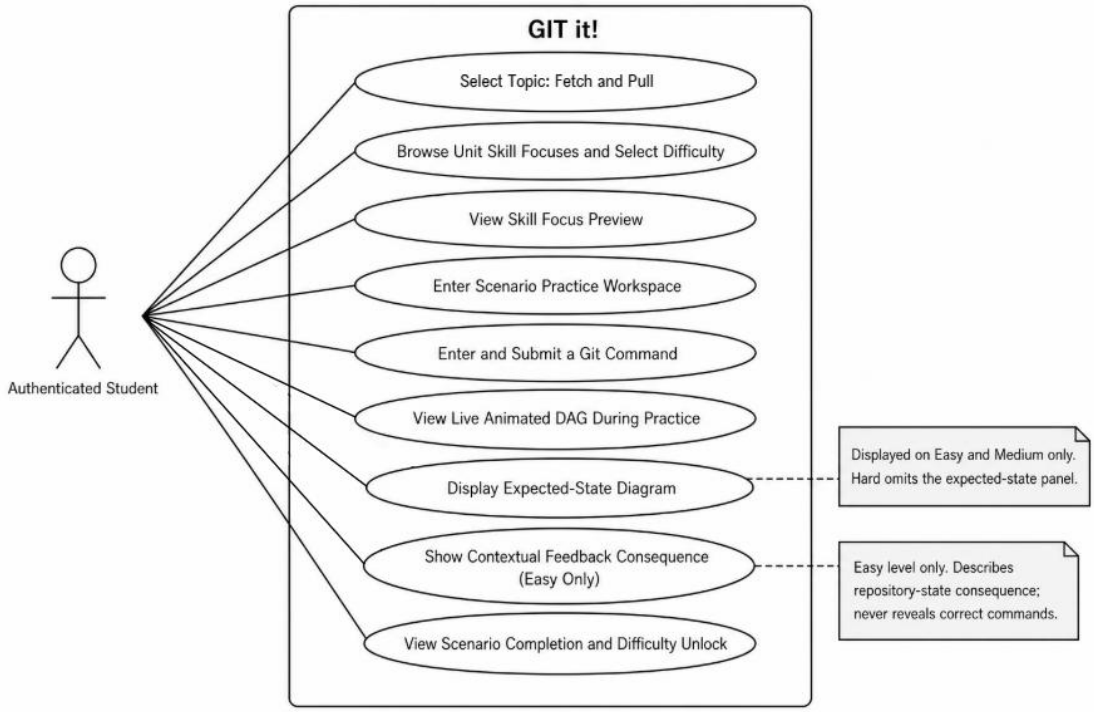

1
2
3
4
Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

2.5 Practice Fetching and Pulling from a Remote

Use Case Diagram

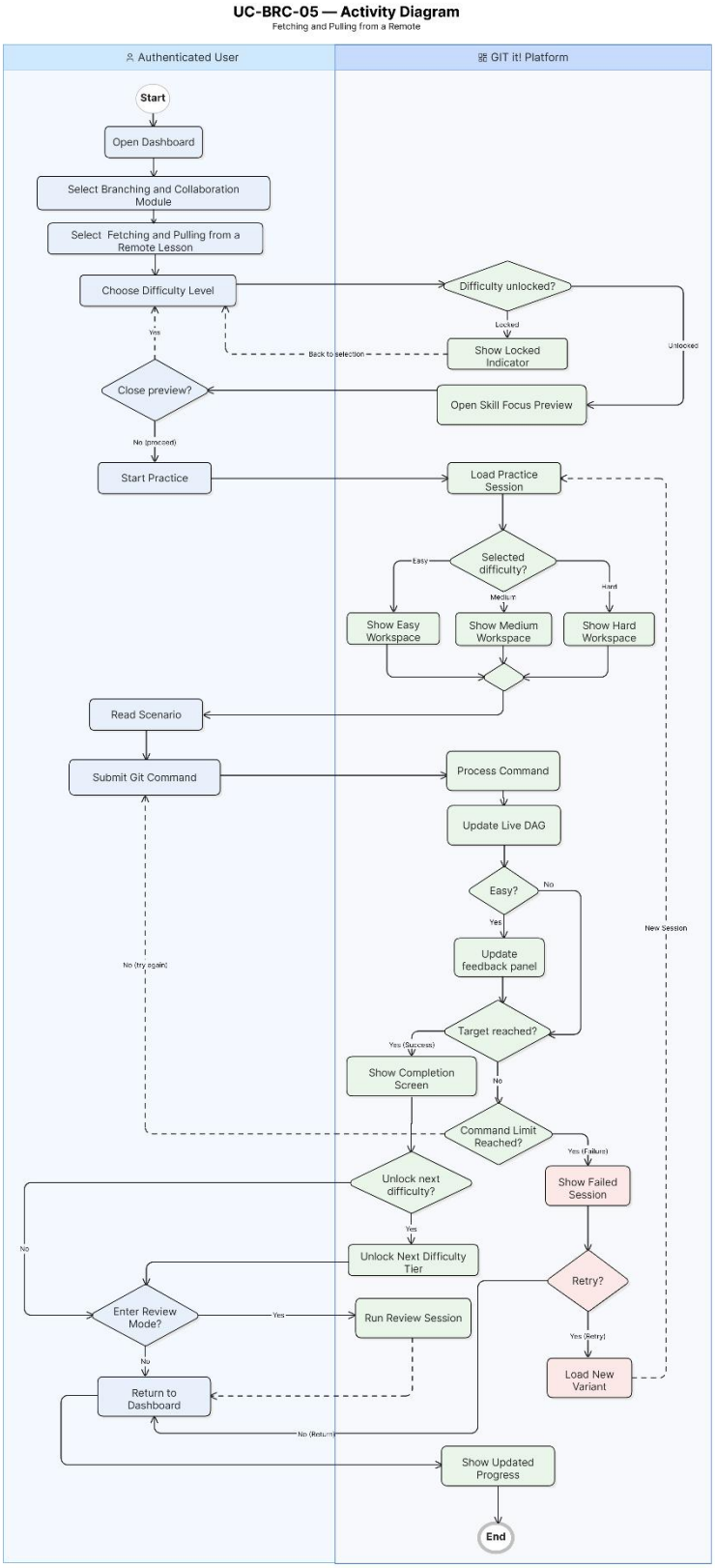


Use Case Description

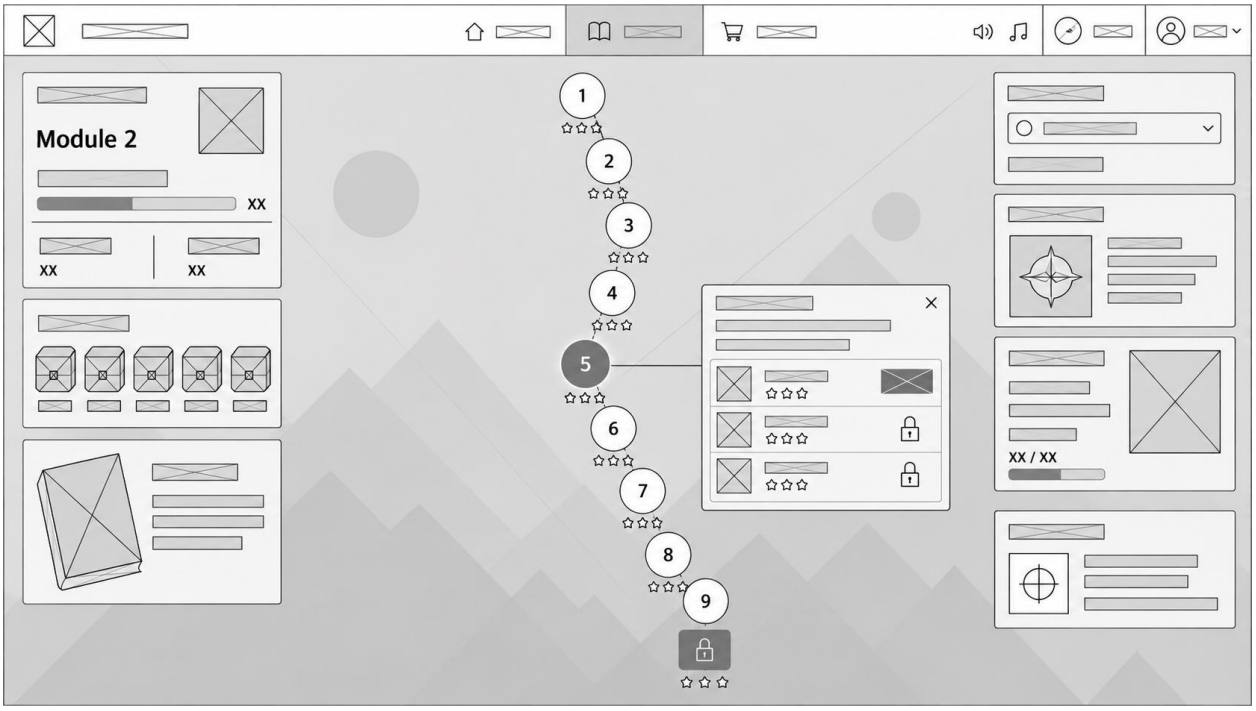
Field	Description
Use Case ID	UC-BRC-05
Use Case Name	Practice Fetching and Pulling from a Remote
Curriculum SO	SO 2.5
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Fetch and pull are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Fetch and pull.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing distinguishing fetch-only from fetch-with-integrate workflows.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel

	after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced distinguishing fetch-only from fetch-with-integrate workflows in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

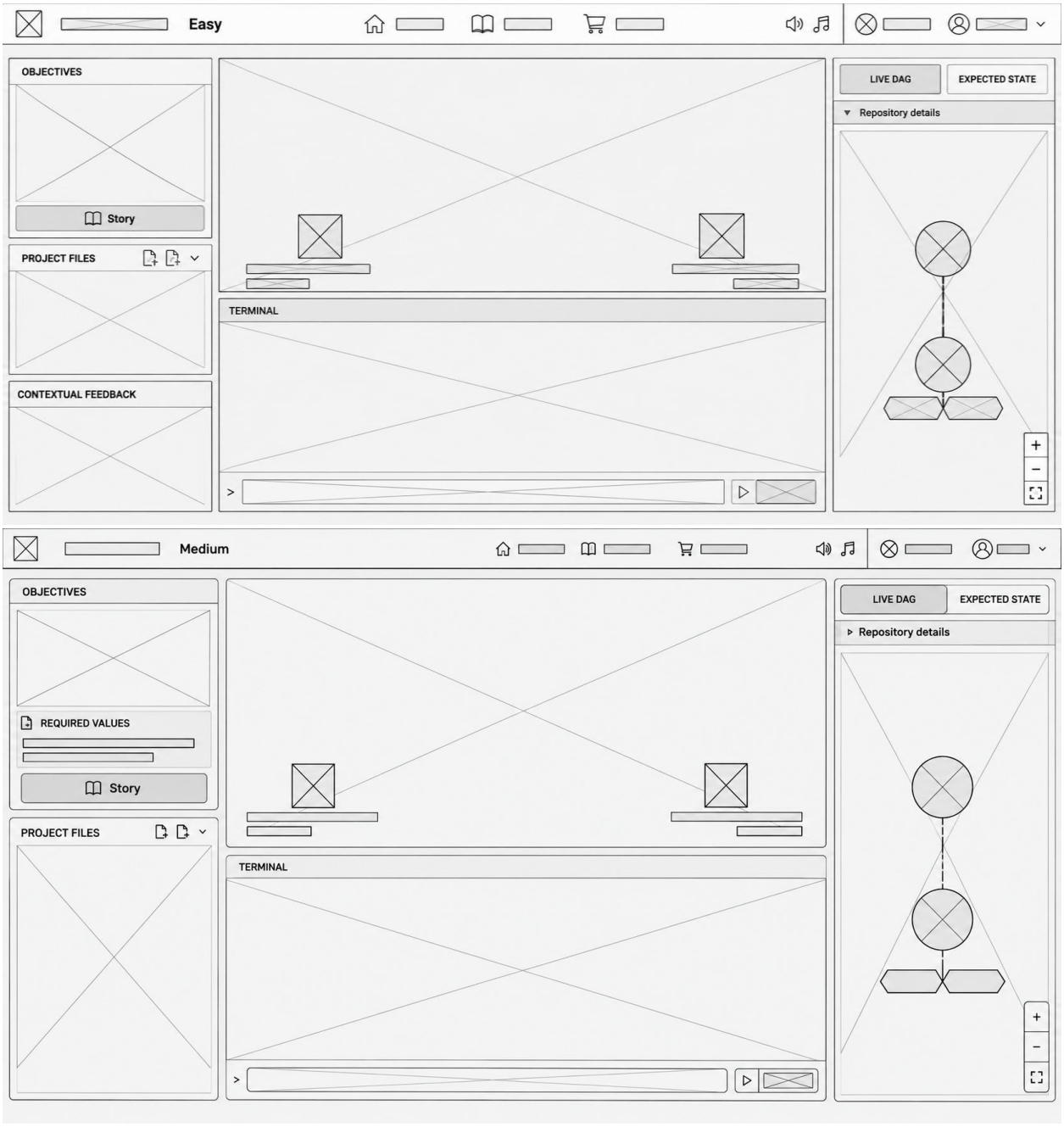
Activity Diagram

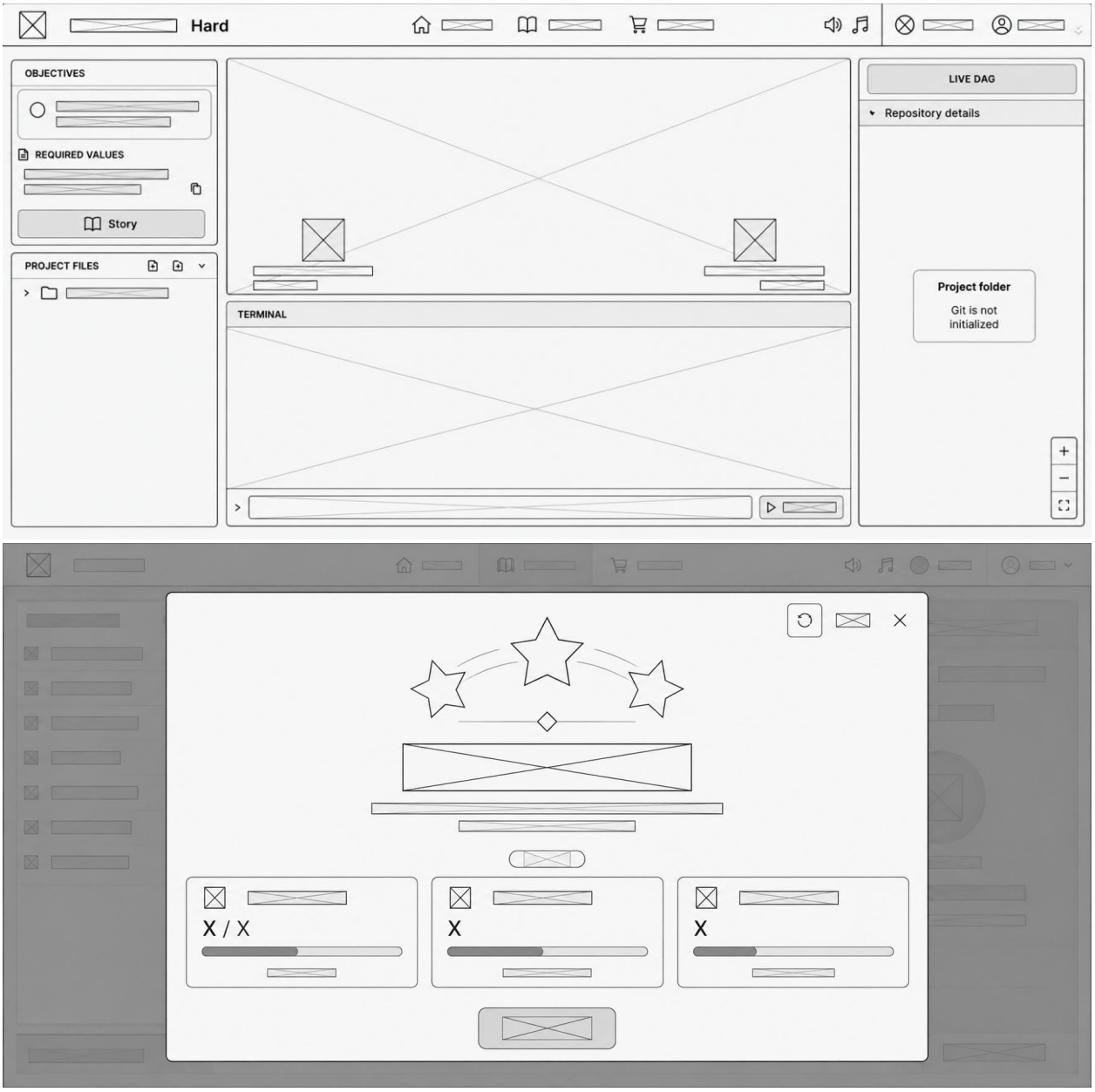


Wireframe



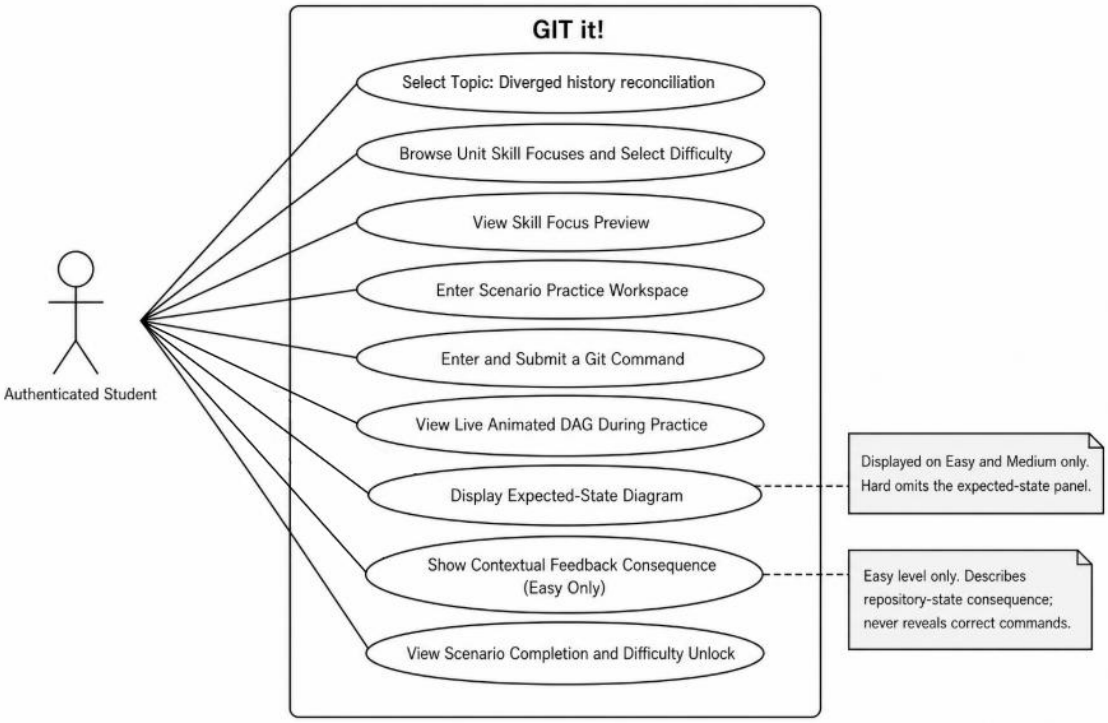



Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized
X / X
X
X

2.6 Practice Reconciling Diverged Histories

Use Case Diagram

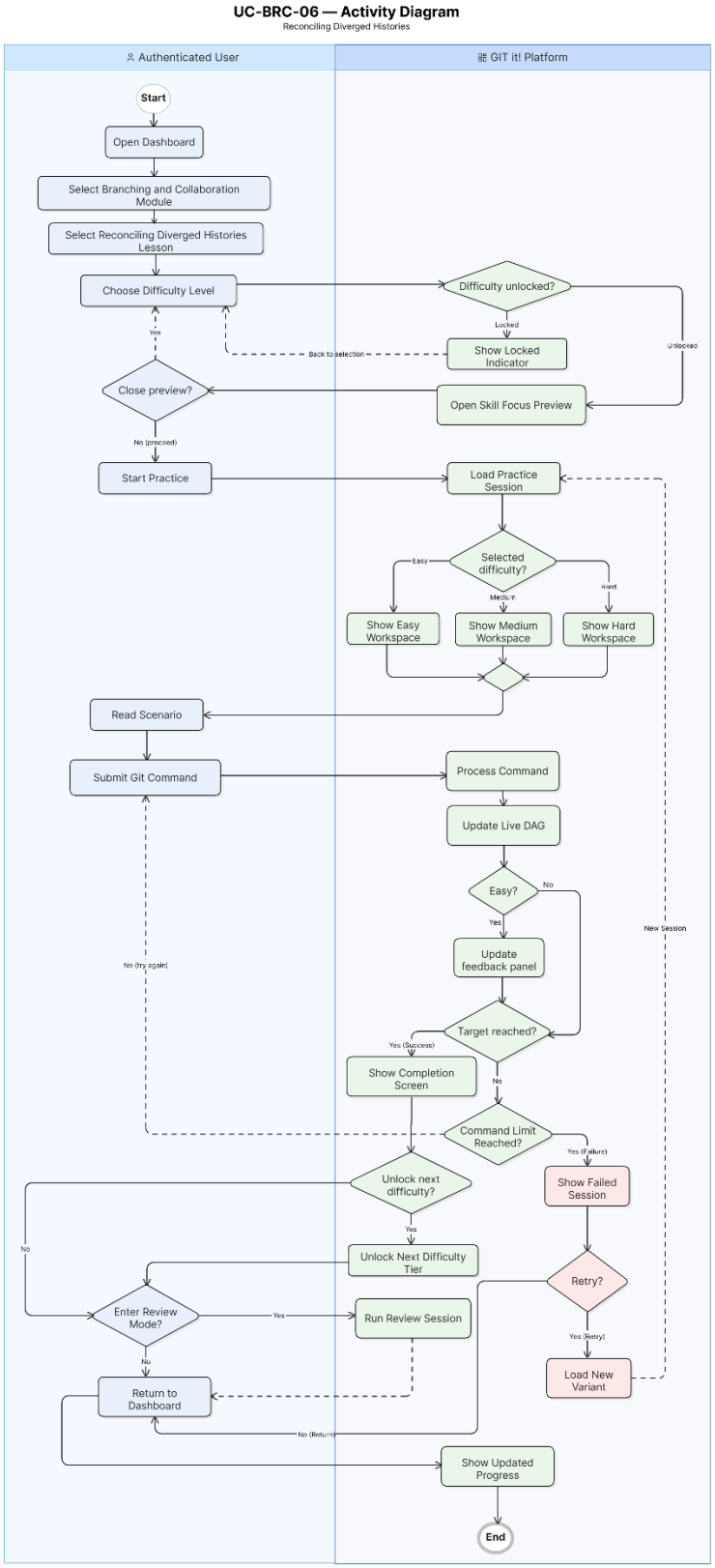


Use Case Description

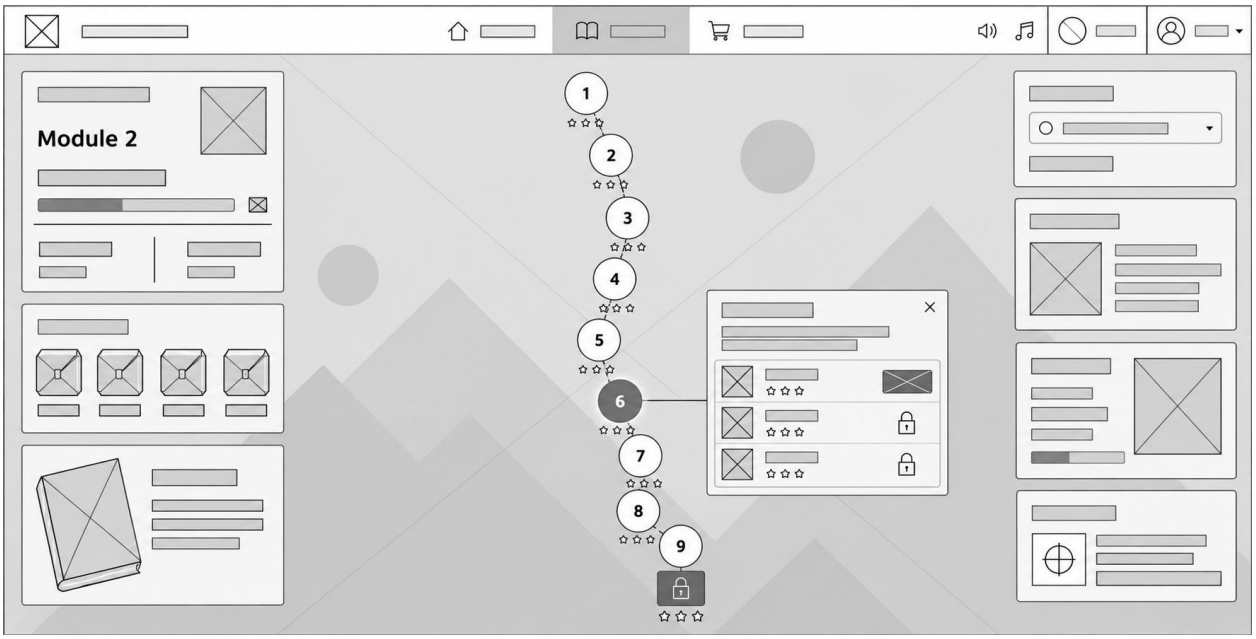
Field	Description
Use Case ID	UC-BRC-06
Use Case Name	Practice Reconciling Diverged Histories
Curriculum SO	SO 2.6
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Diverged history reconciliation are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Diverged history reconciliation.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing reconciling local and remote histories before a successful push.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the

	Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced reconciling local and remote histories before a successful push in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

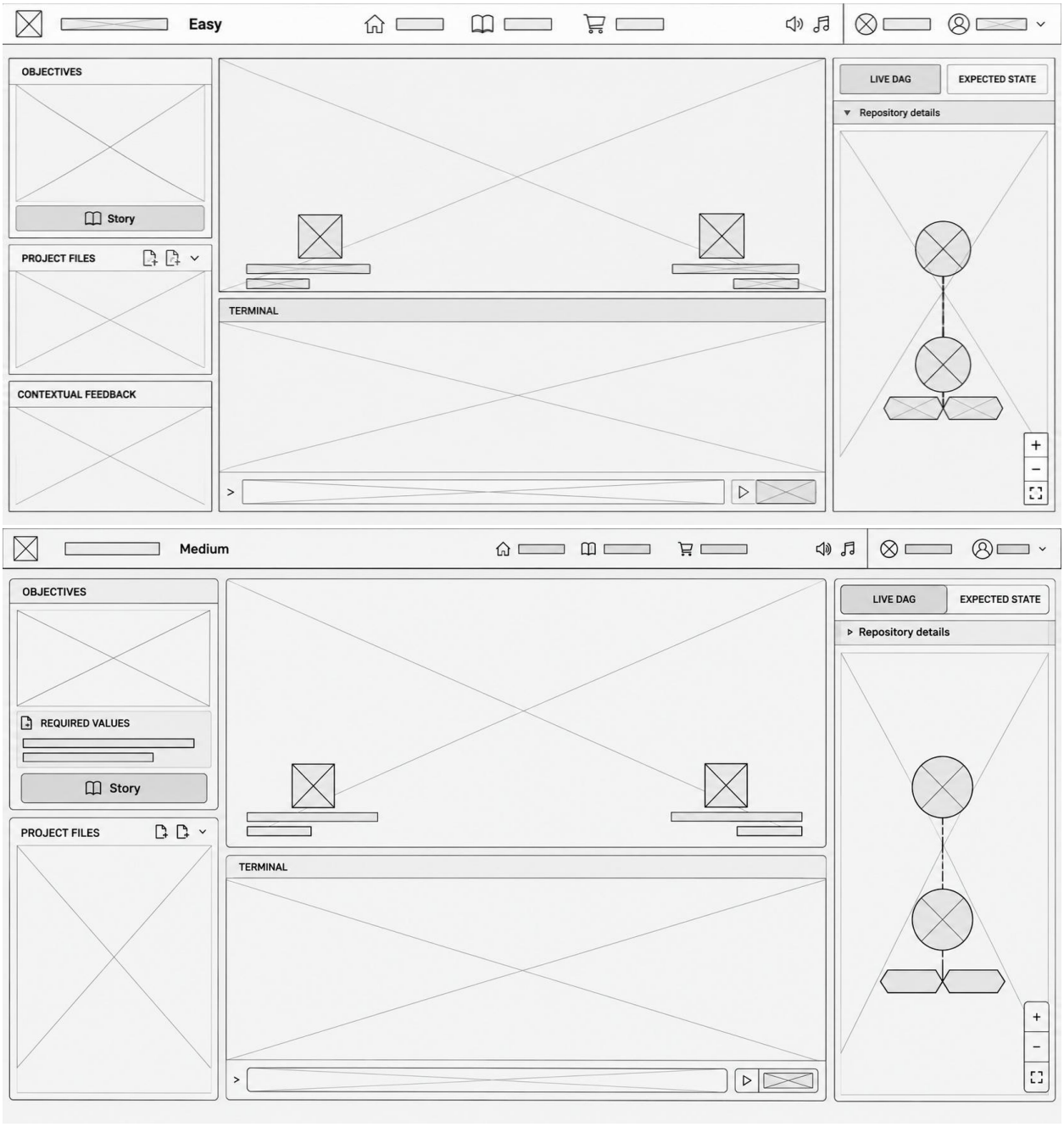
Activity Diagram

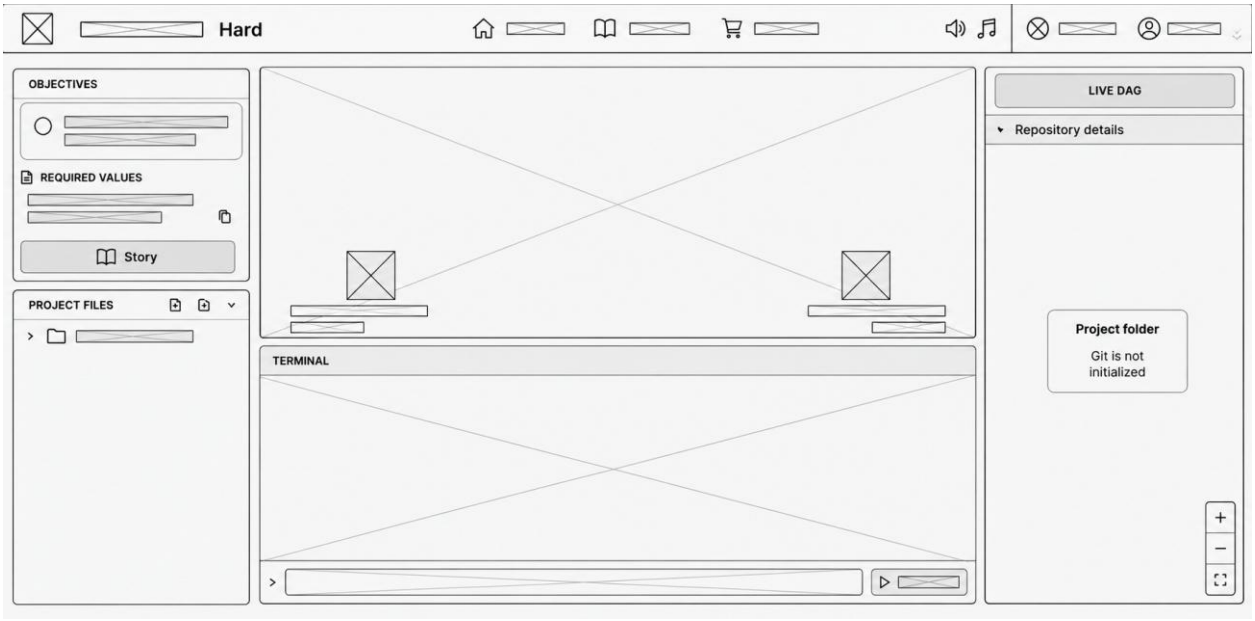


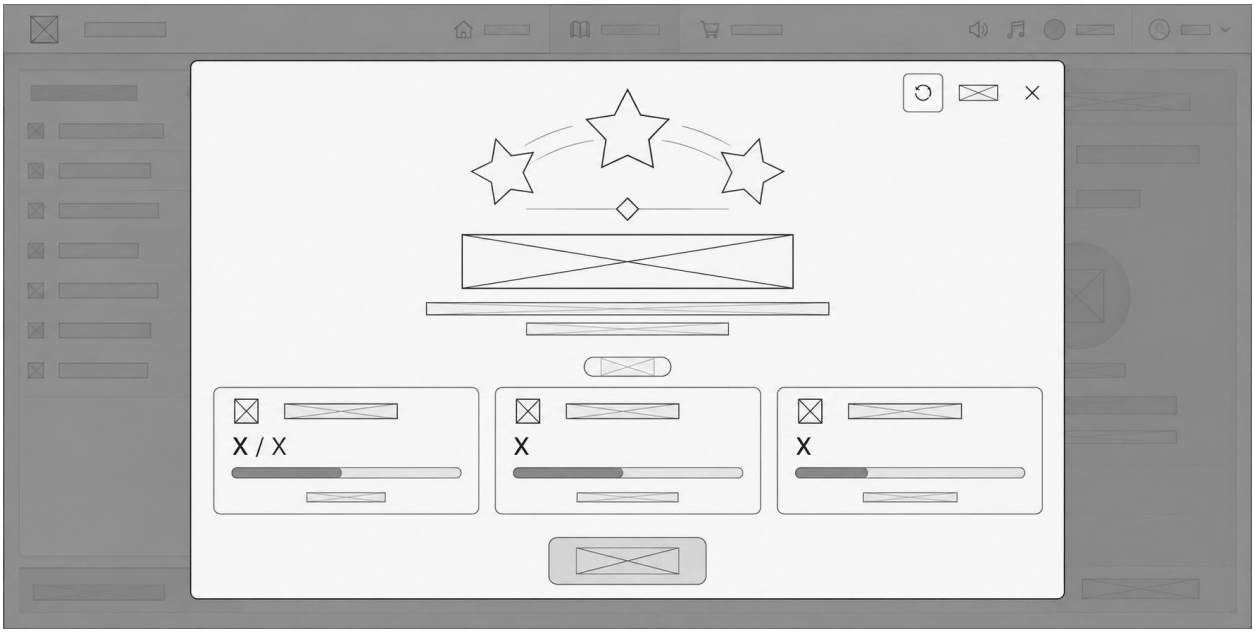
Wireframe





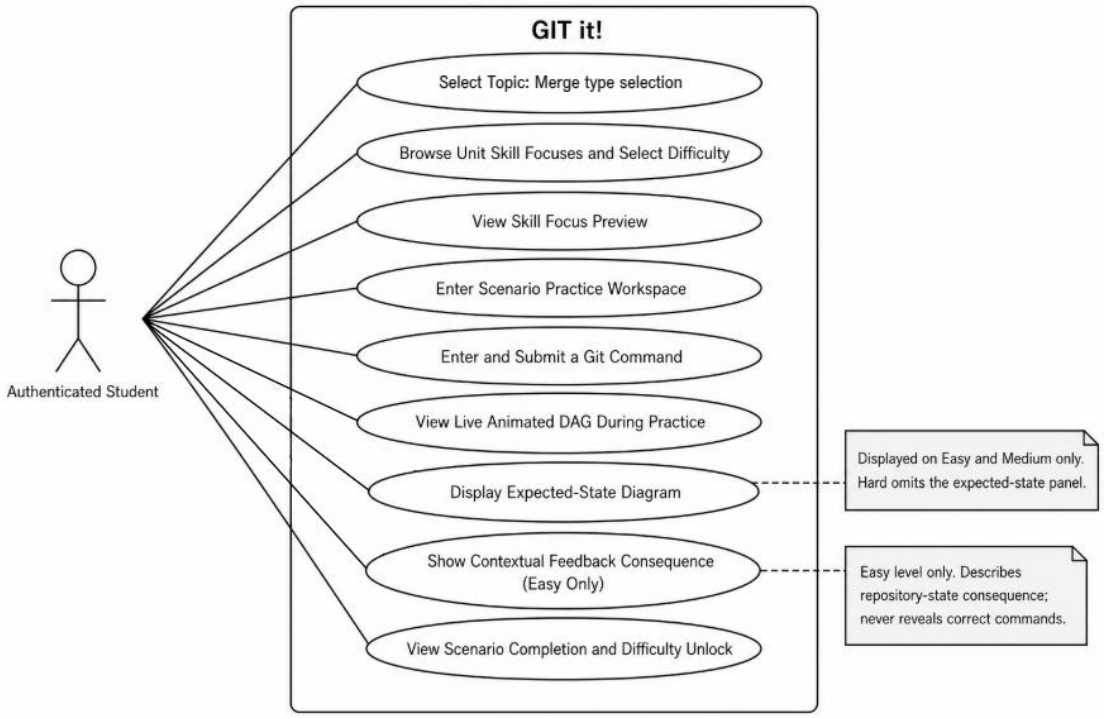

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

2.7 Practice Completing Branch Merges

Use Case Diagram

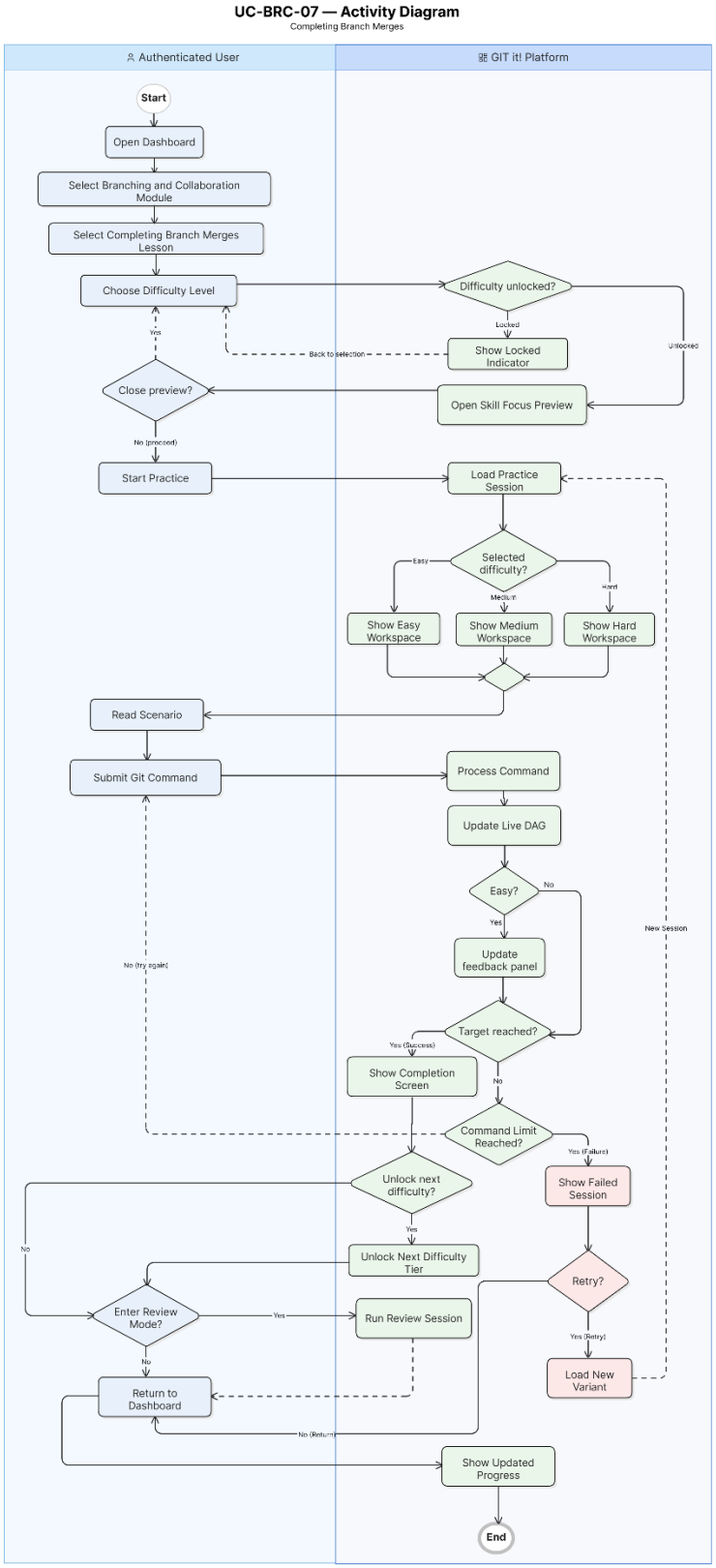


Use Case Description

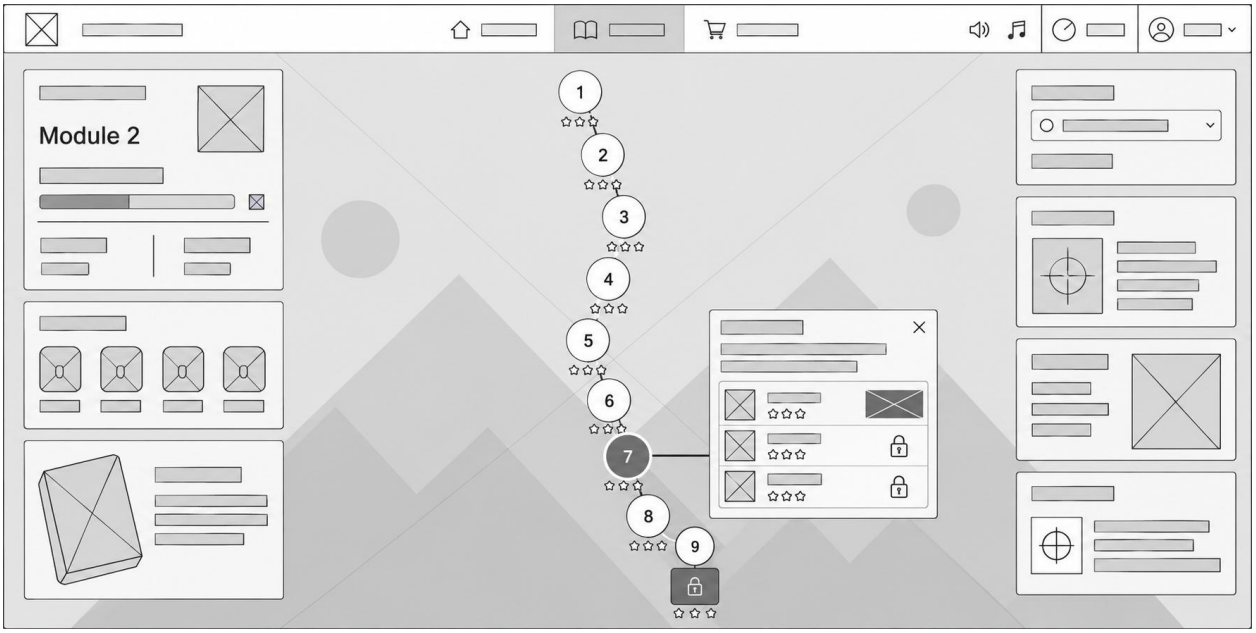
Field	Description
Use Case ID	UC-BRC-07
Use Case Name	Practice Completing Branch Merges
Curriculum SO	SO 2.7
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Merge type selection are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Merge type selection.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing fast-forward and three-way merges.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each

	processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced fast-forward and three-way merges in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

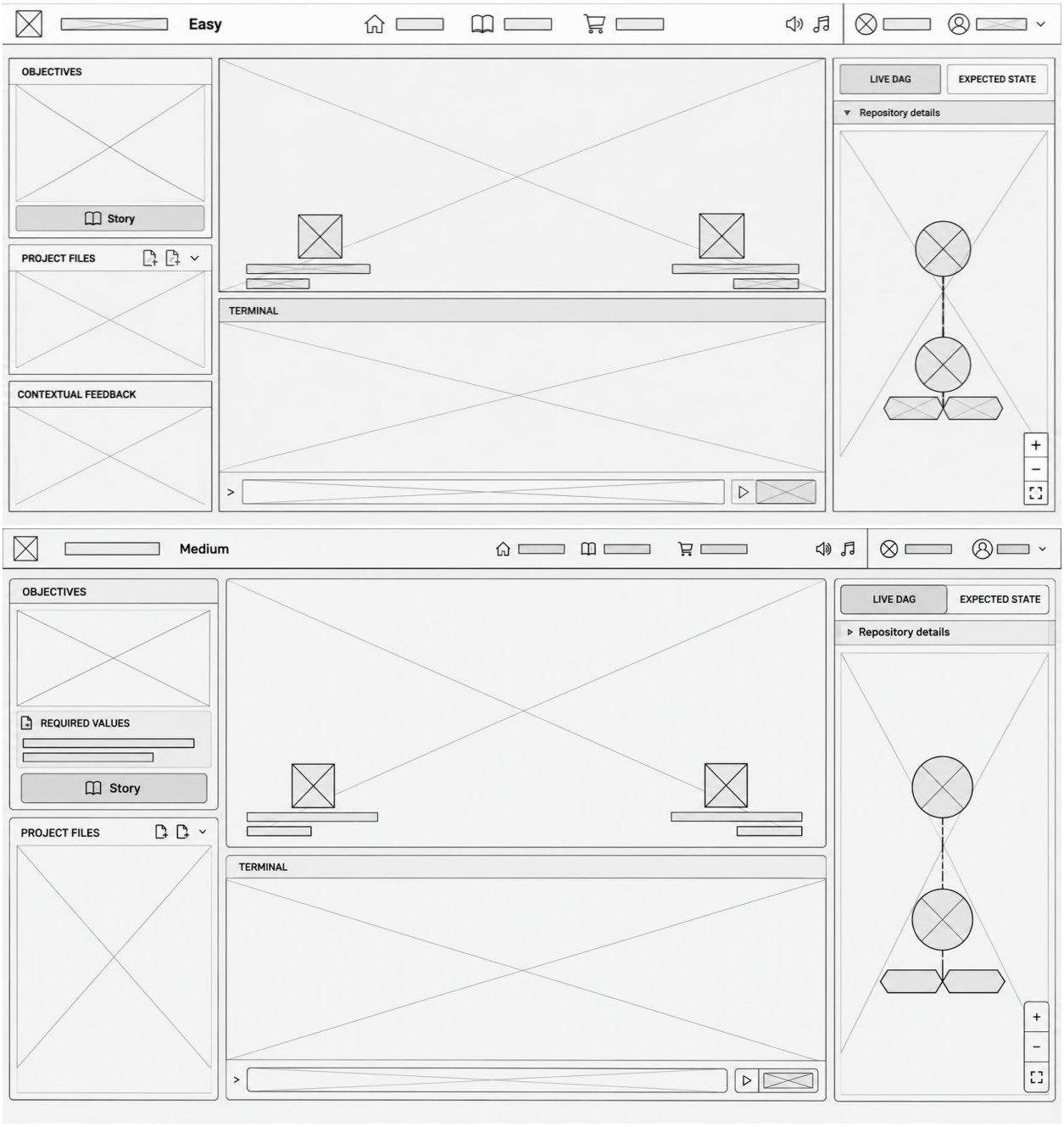
Activity Diagram

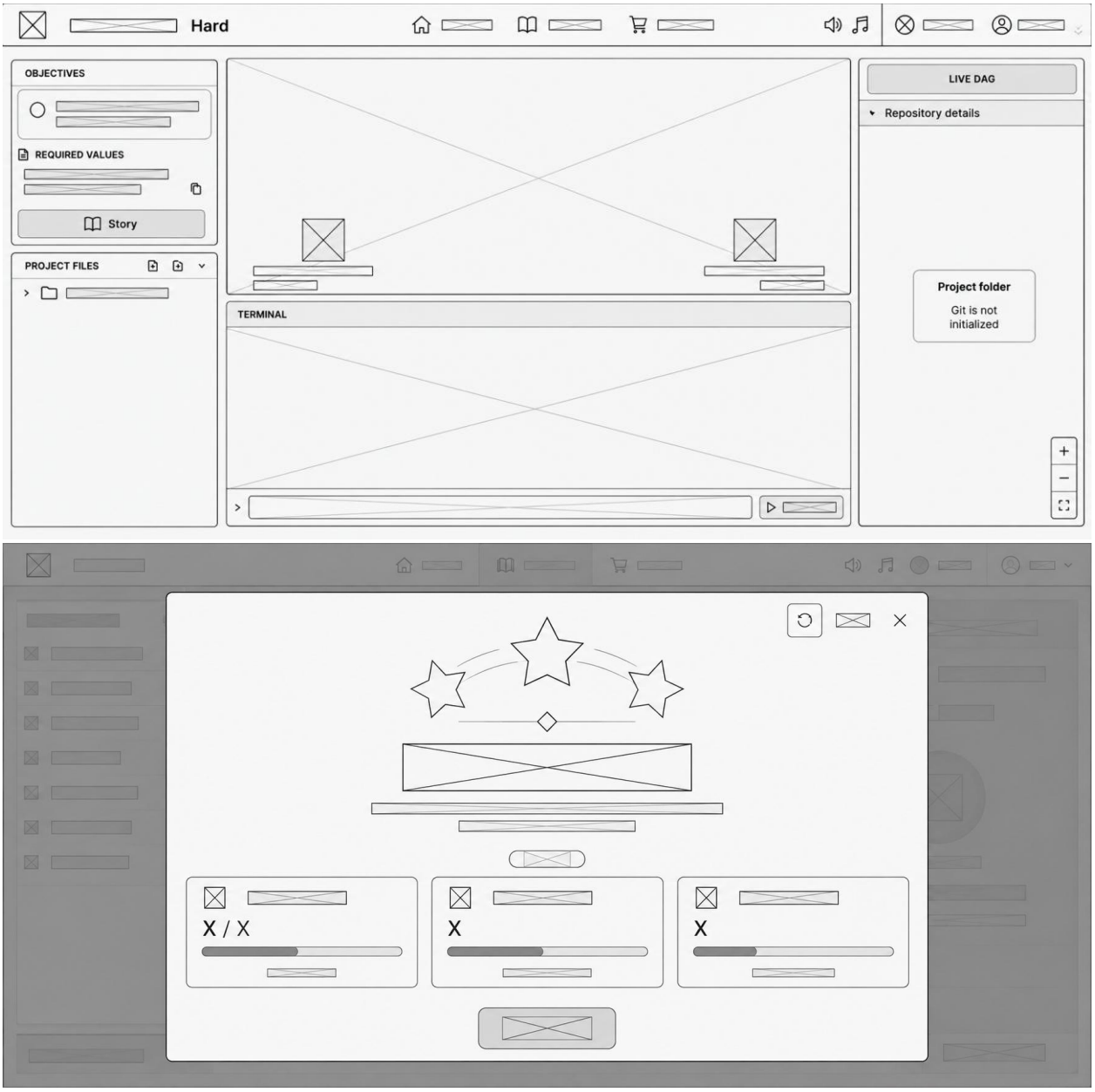


Wireframe



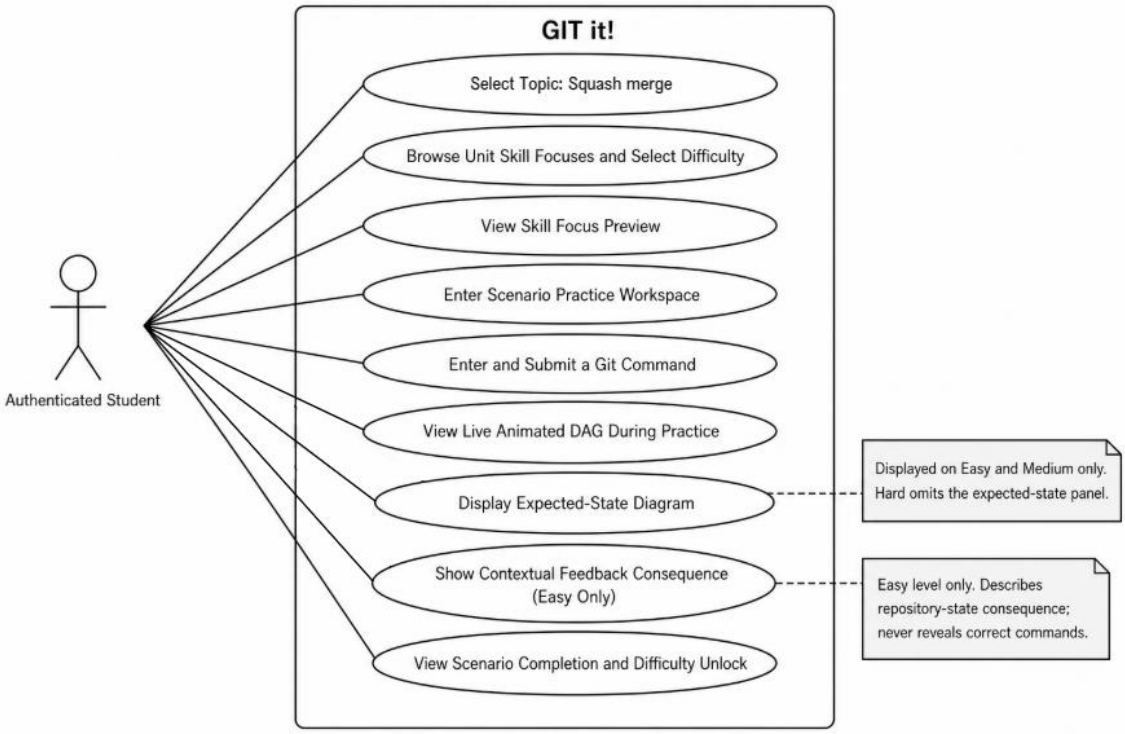



Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized
X / X
X
X

2.8 Practice Squash Merging

Use Case Diagram

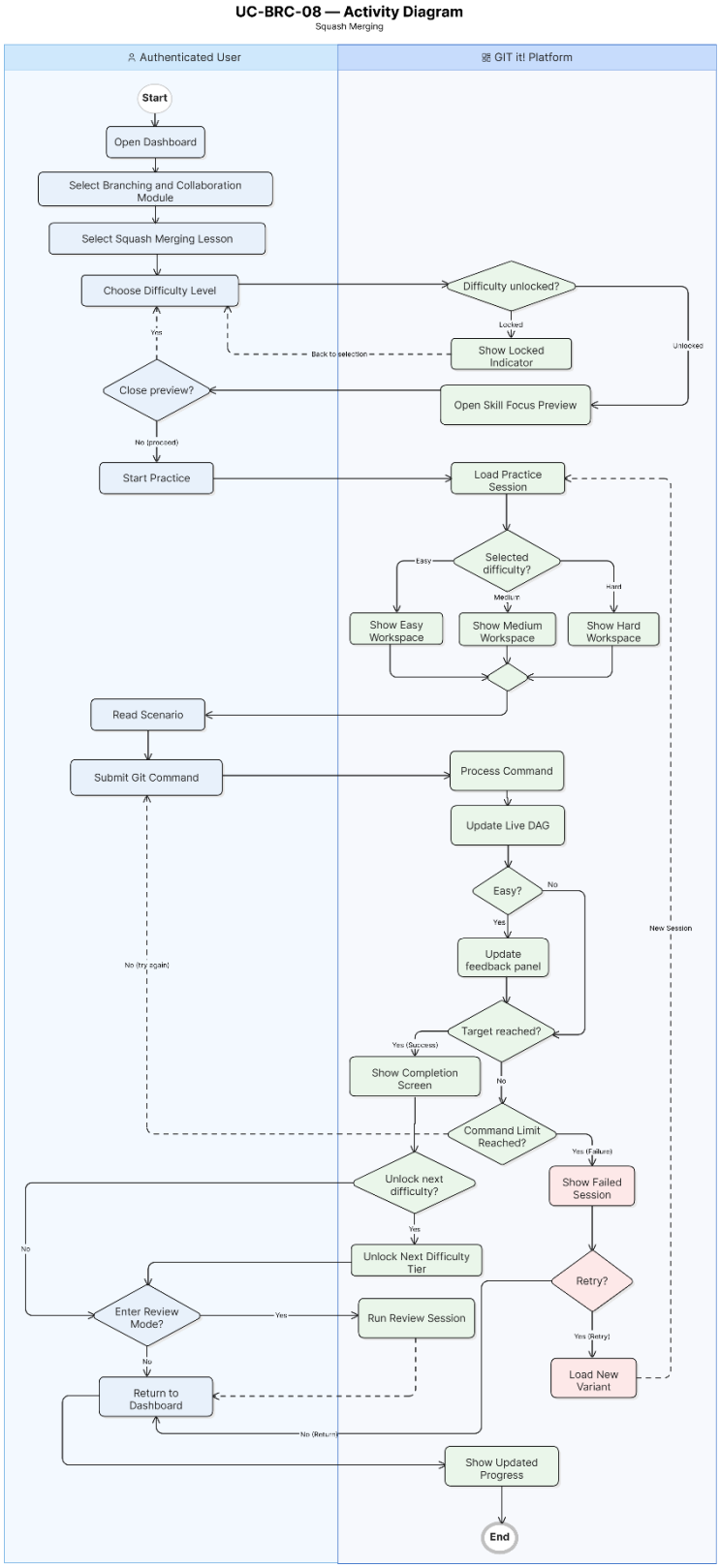


Use Case Description

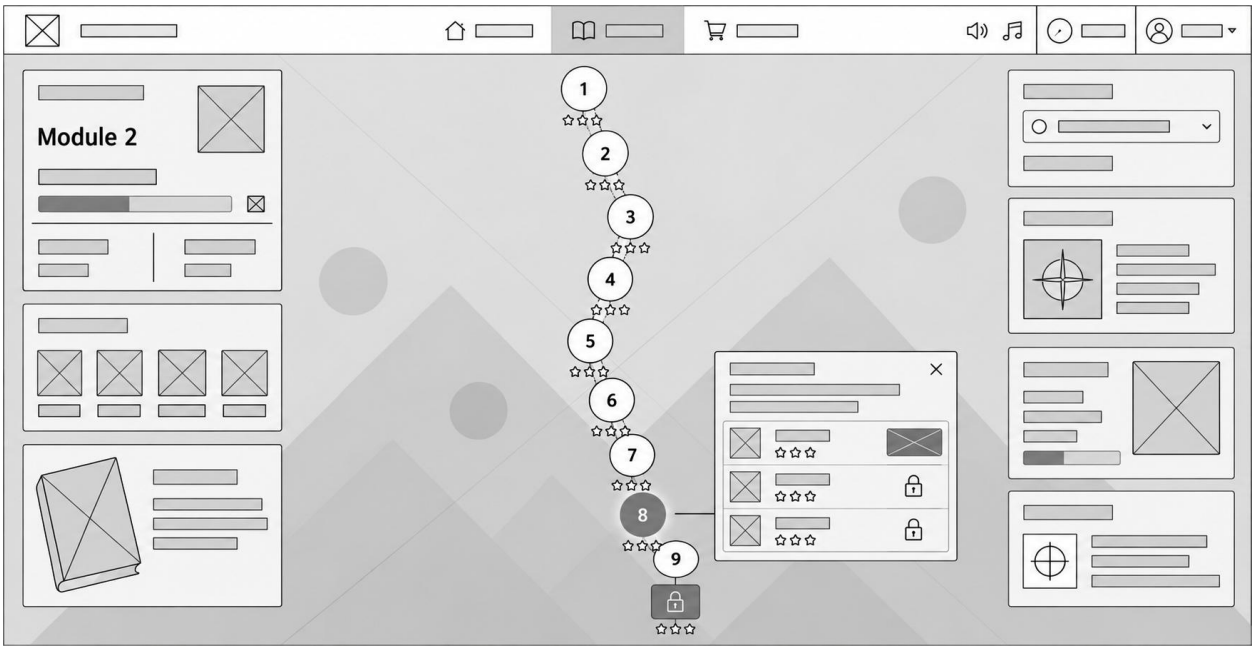
Field	Description
Use Case ID	UC-BRC-08
Use Case Name	Practice Squash Merging
Curriculum SO	SO 2.8
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Squash merge are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Squash merge.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing consolidating feature-branch history with squash merge.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On

	Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced consolidating feature-branch history with squash merge in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

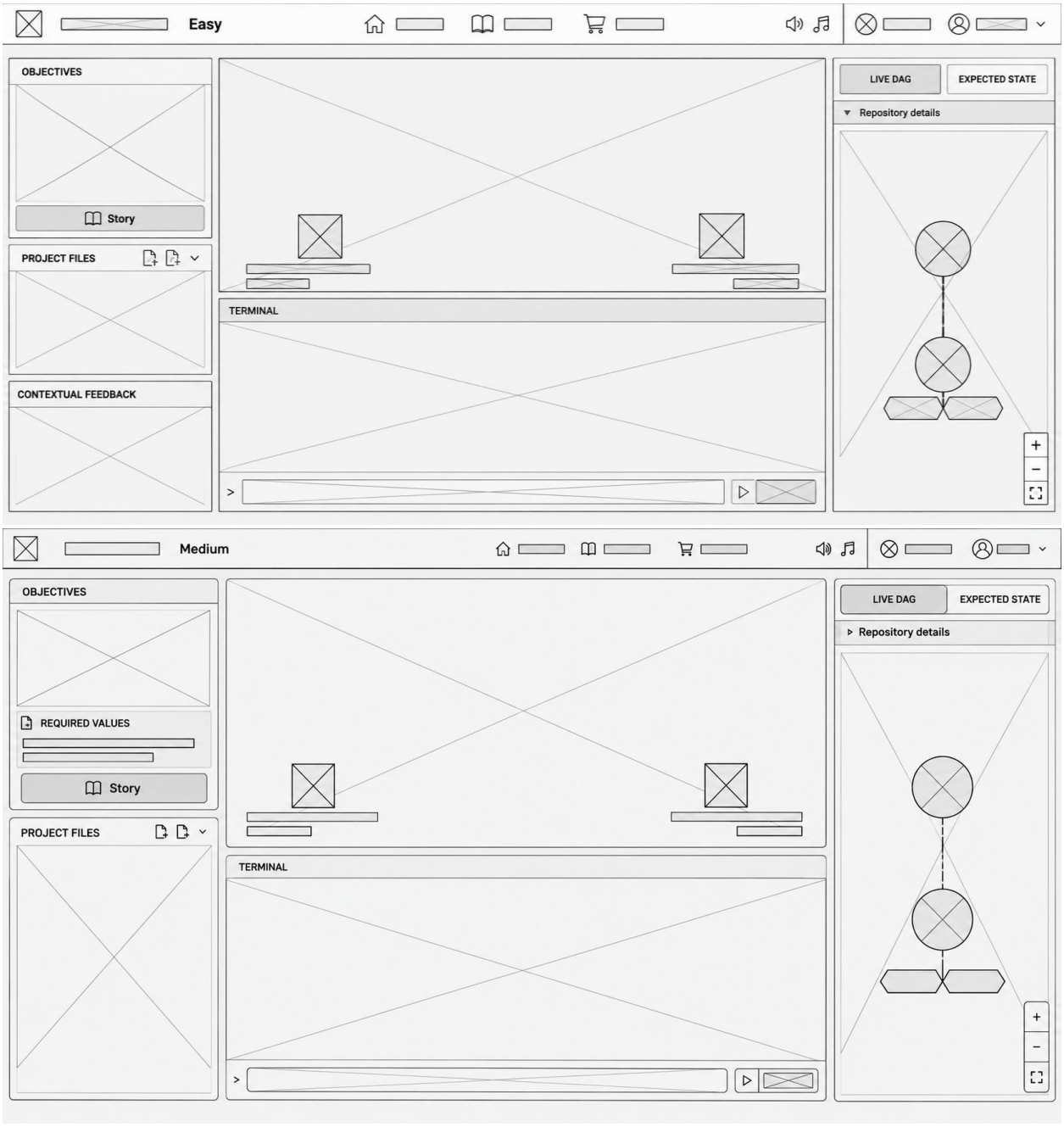
Activity Diagram

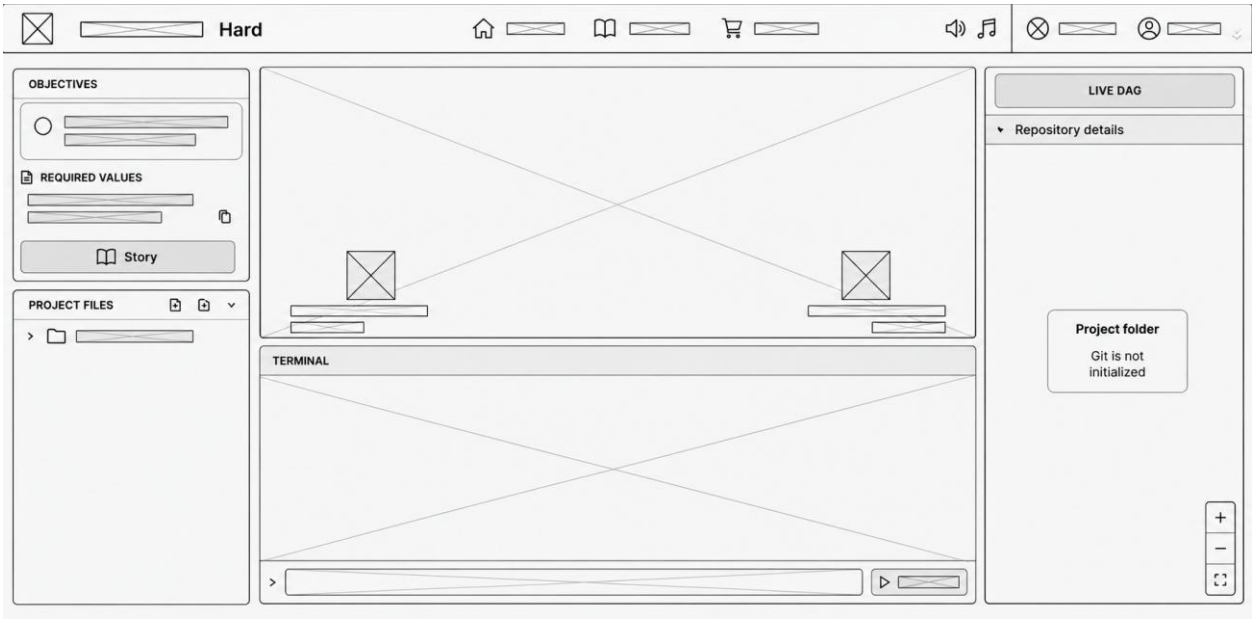


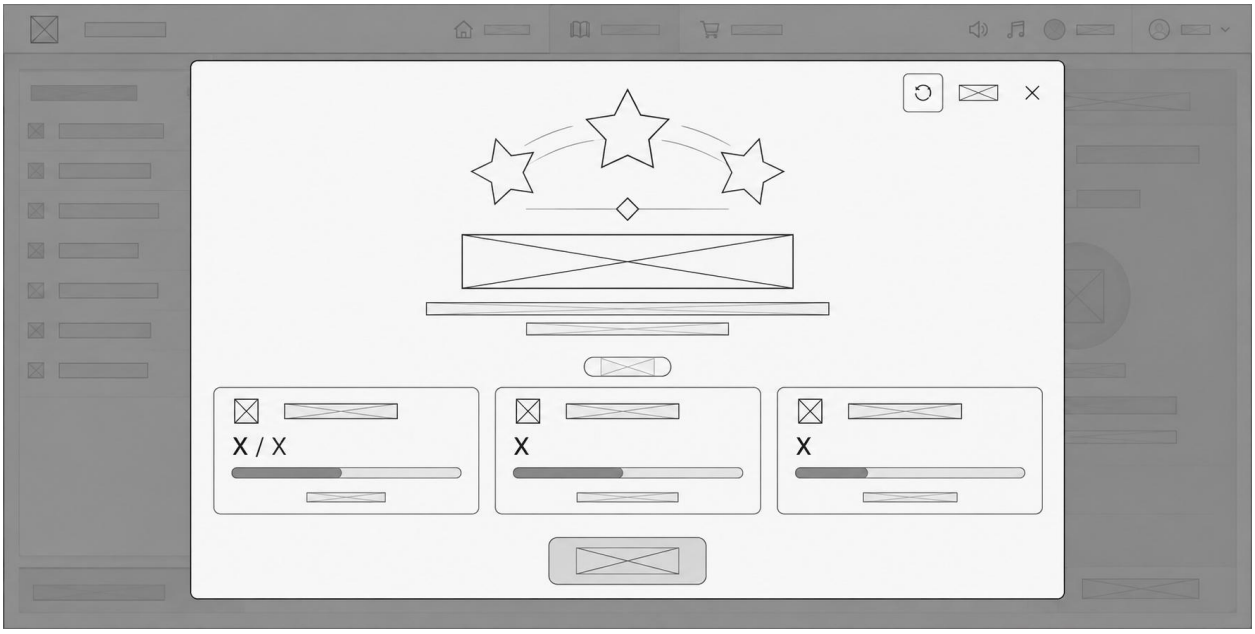
Wireframe





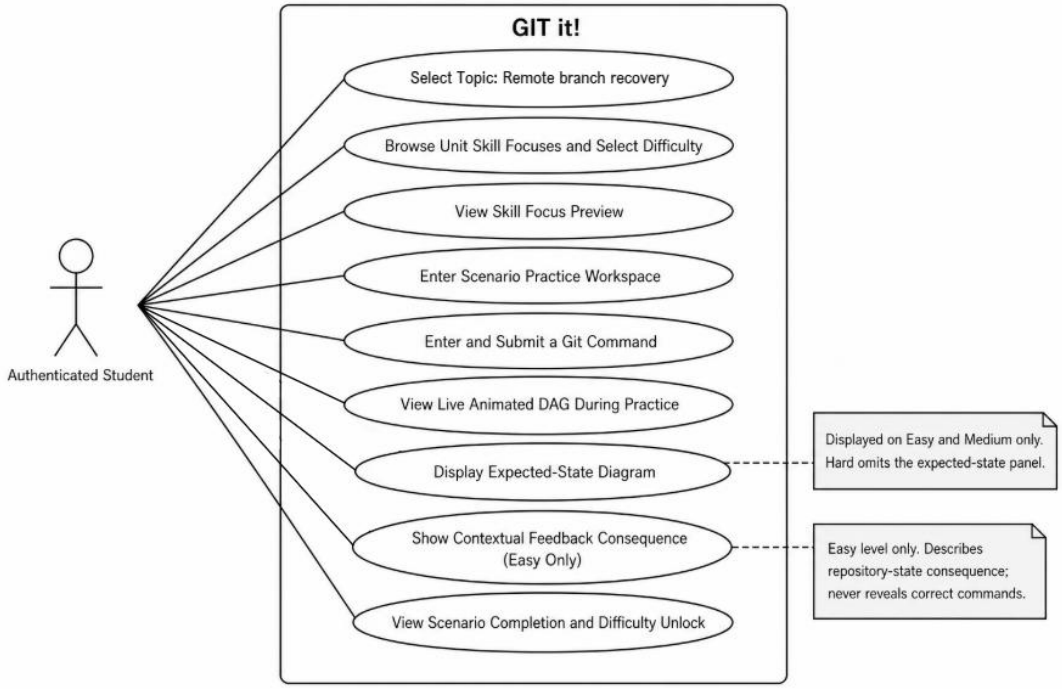

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

2.9 Practice Deleting and Recovering Remote Branches

Use Case Diagram

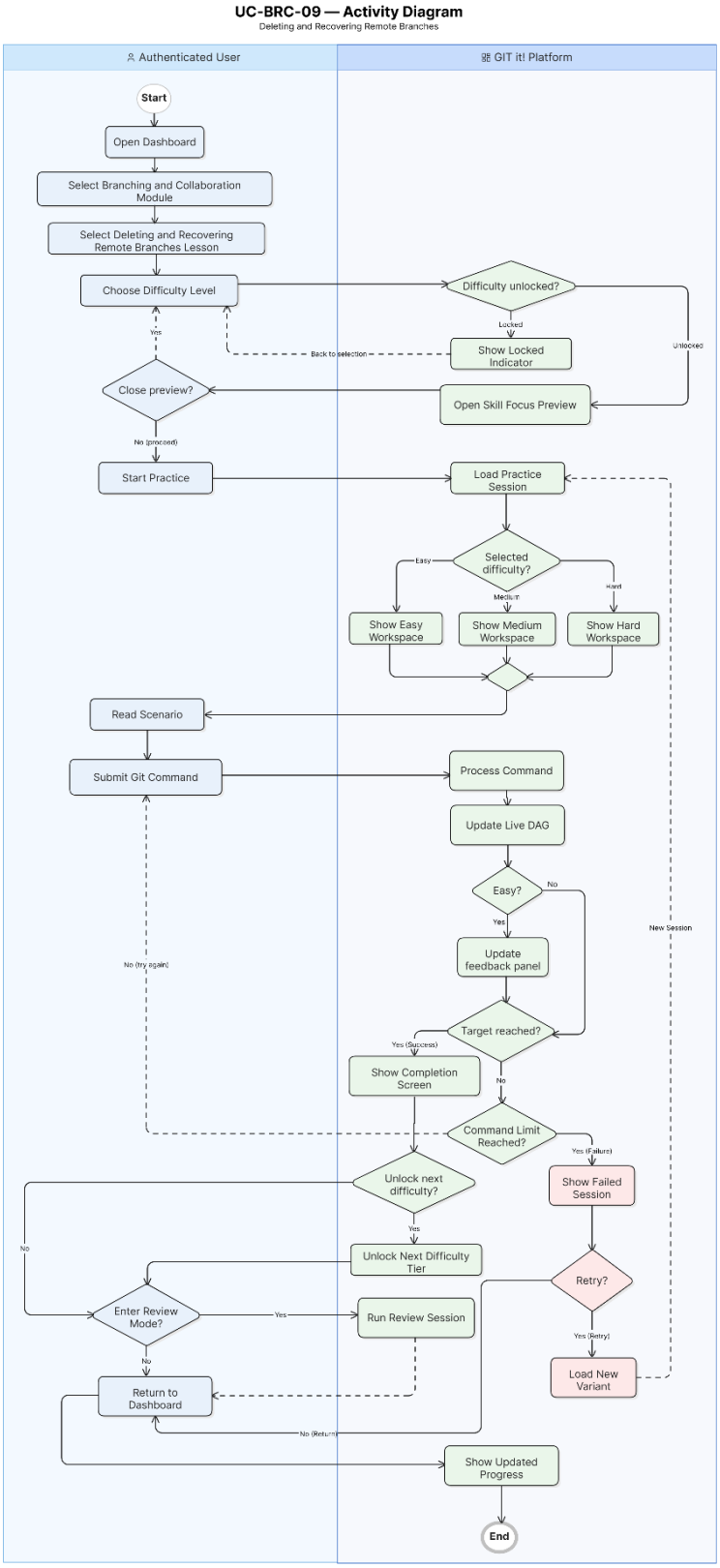


Use Case Description

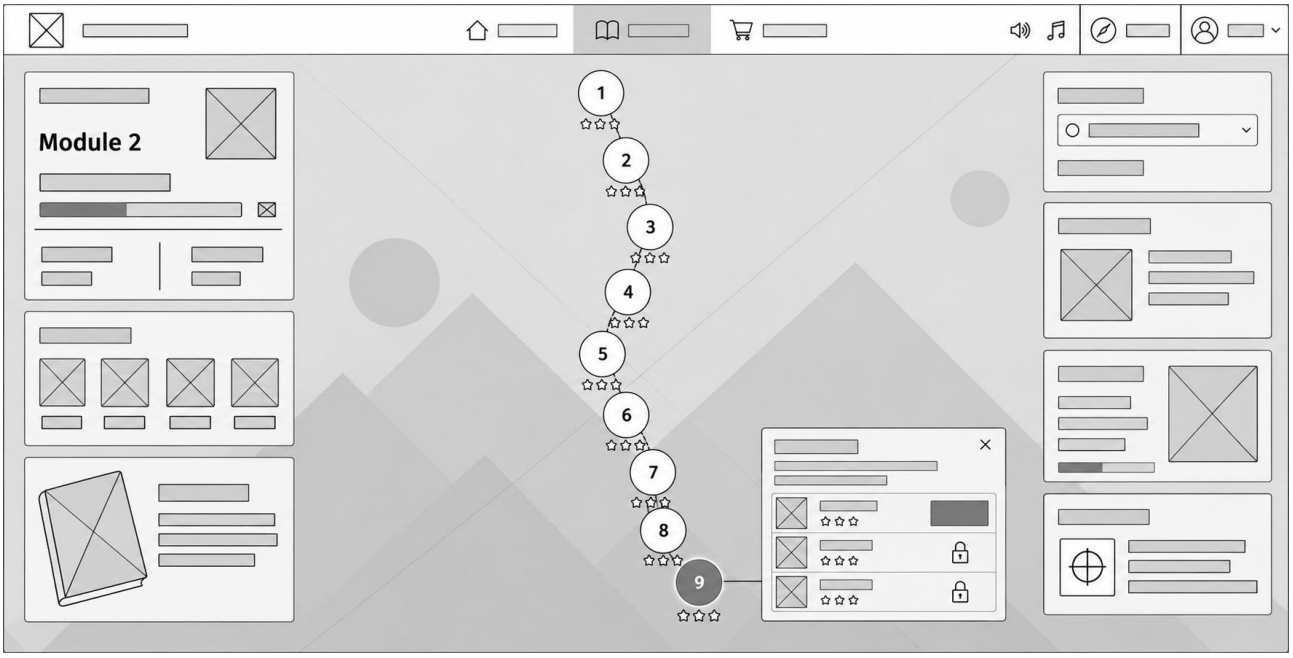
Field	Description
Use Case ID	UC-BRC-09
Use Case Name	Practice Deleting and Recovering Remote Branches
Curriculum SO	SO 2.9
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Remote branch recovery are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Remote branch recovery.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing deleting stale remote branches and recovering deleted remotes.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student

	views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced deleting stale remote branches and recovering deleted remotes in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

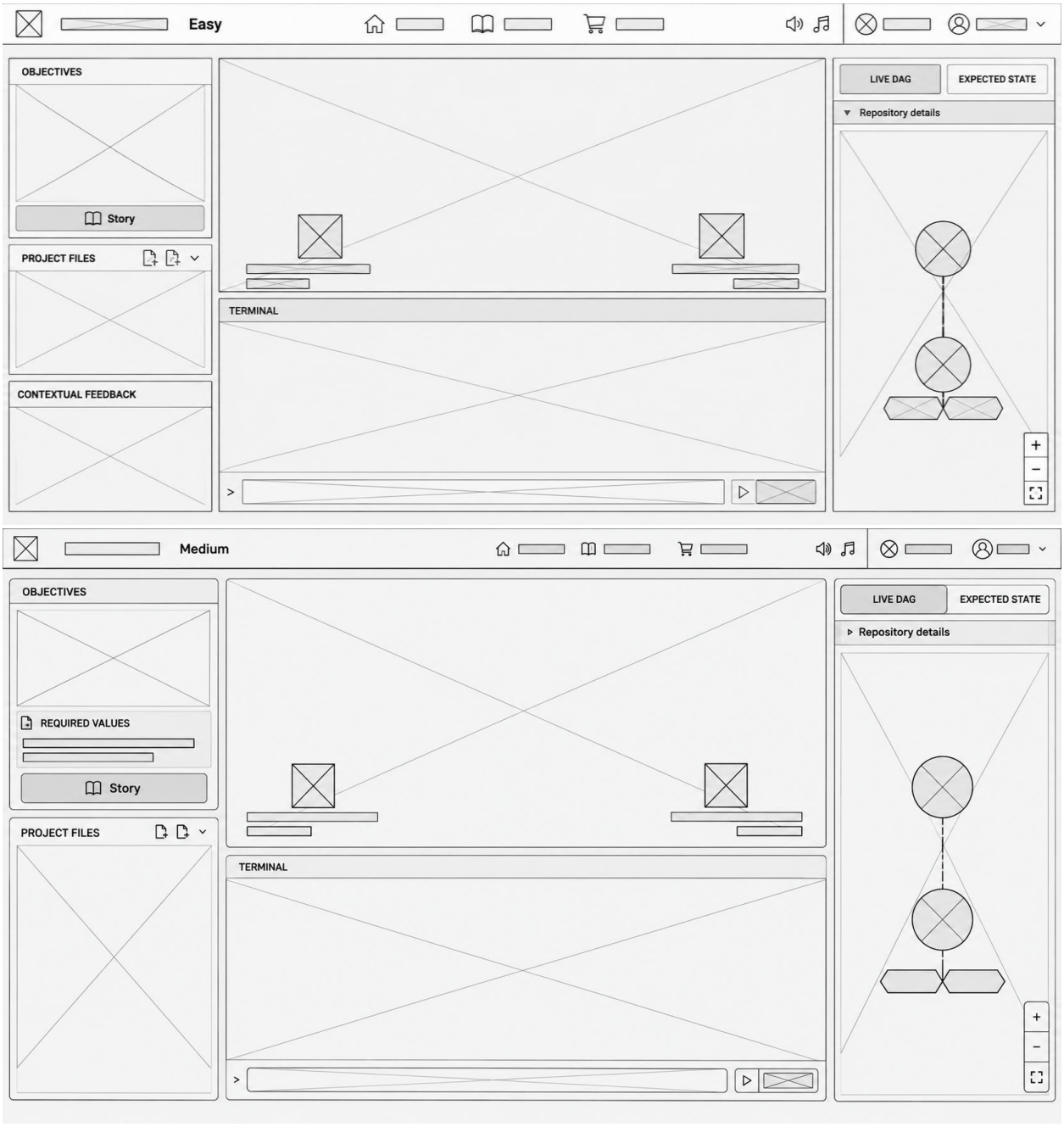
Activity Diagram

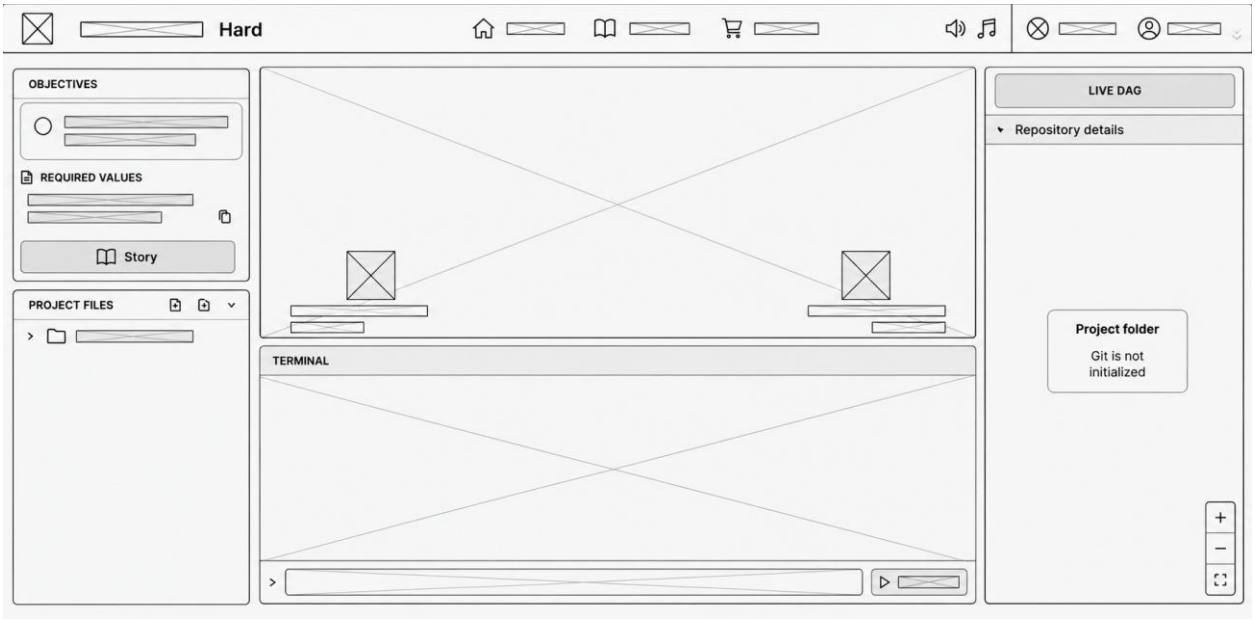


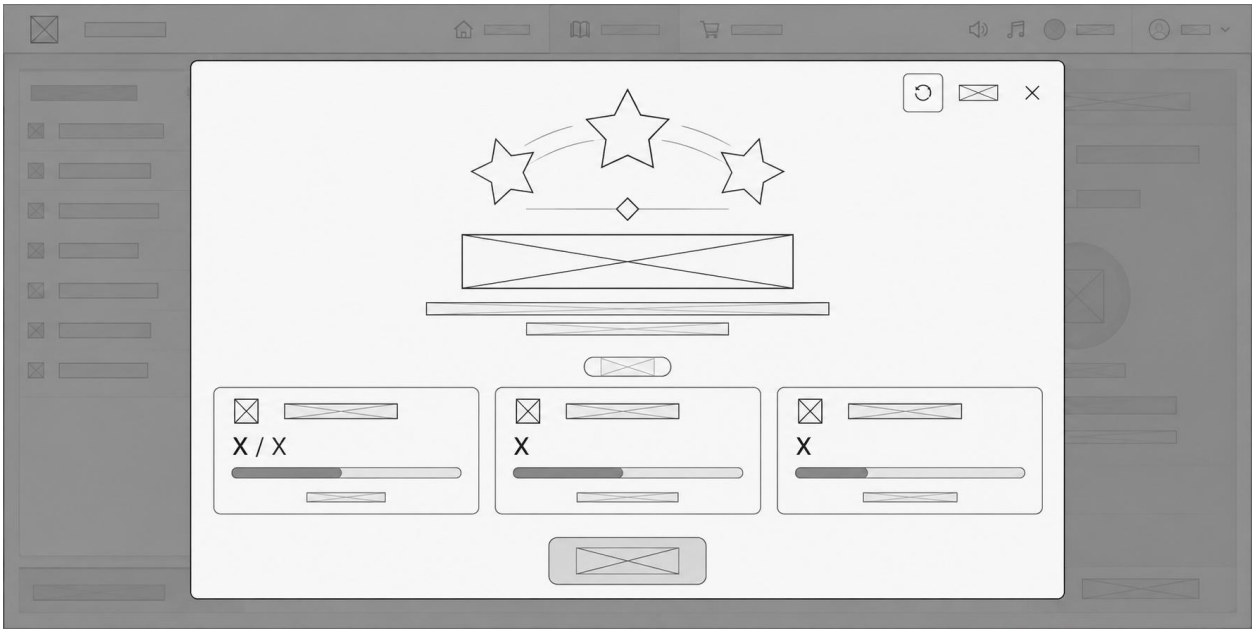
Wireframe





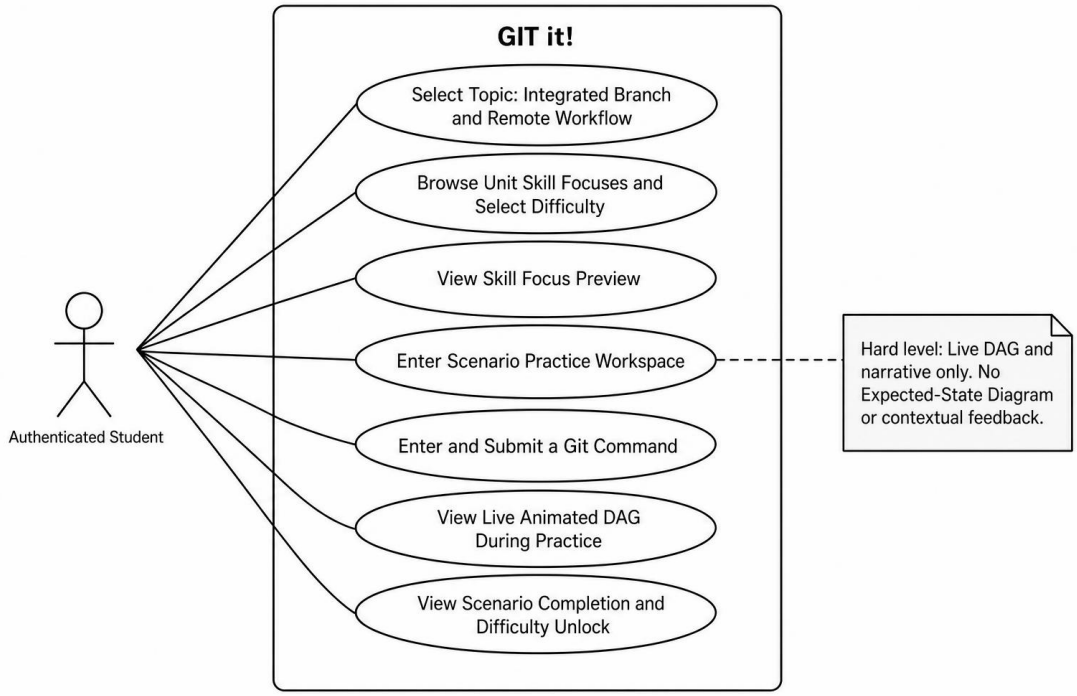

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

2.10 Complete Hard-Level Independent Branch and Collaboration Scenarios

Use Case Diagram

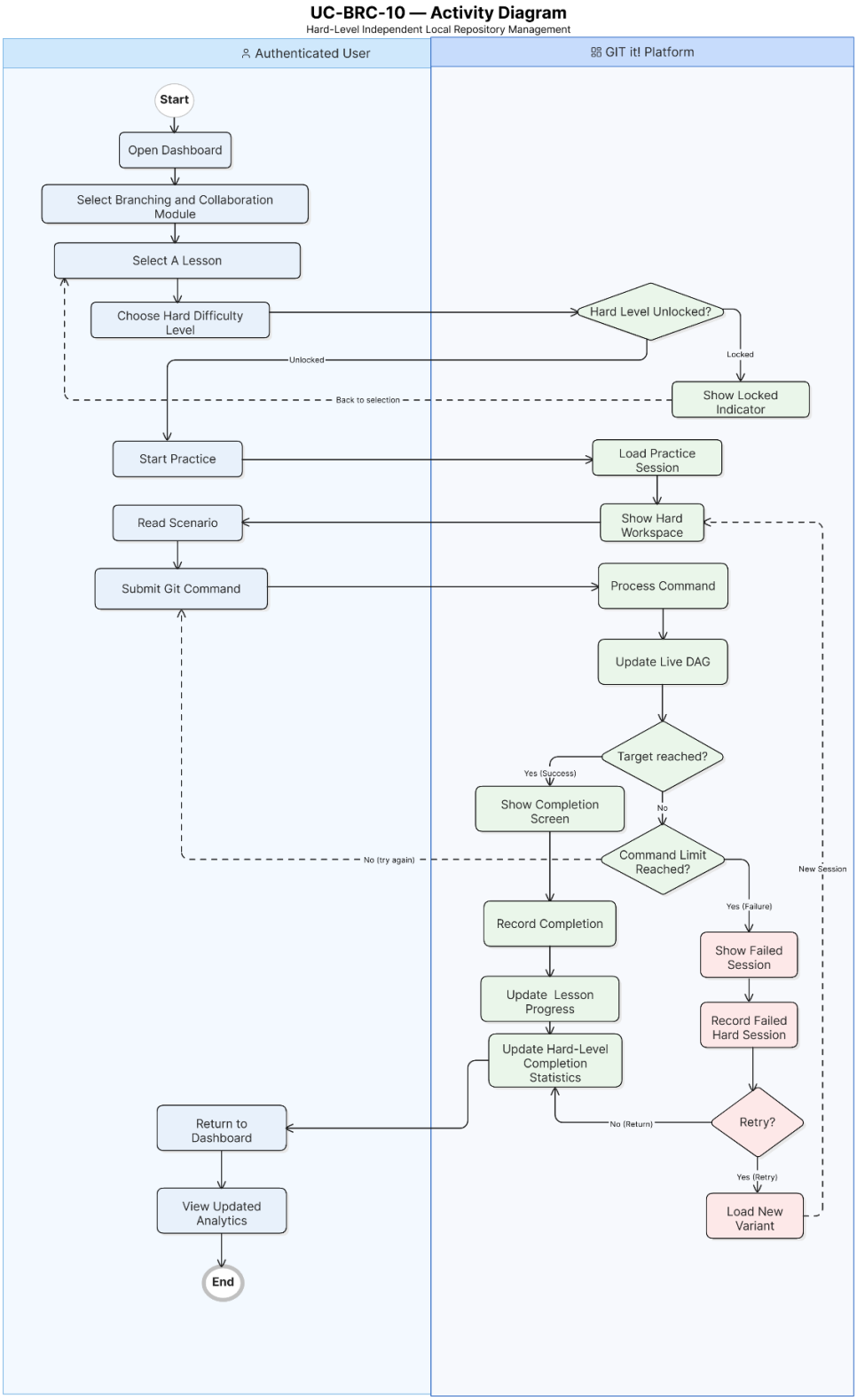


Use Case Description

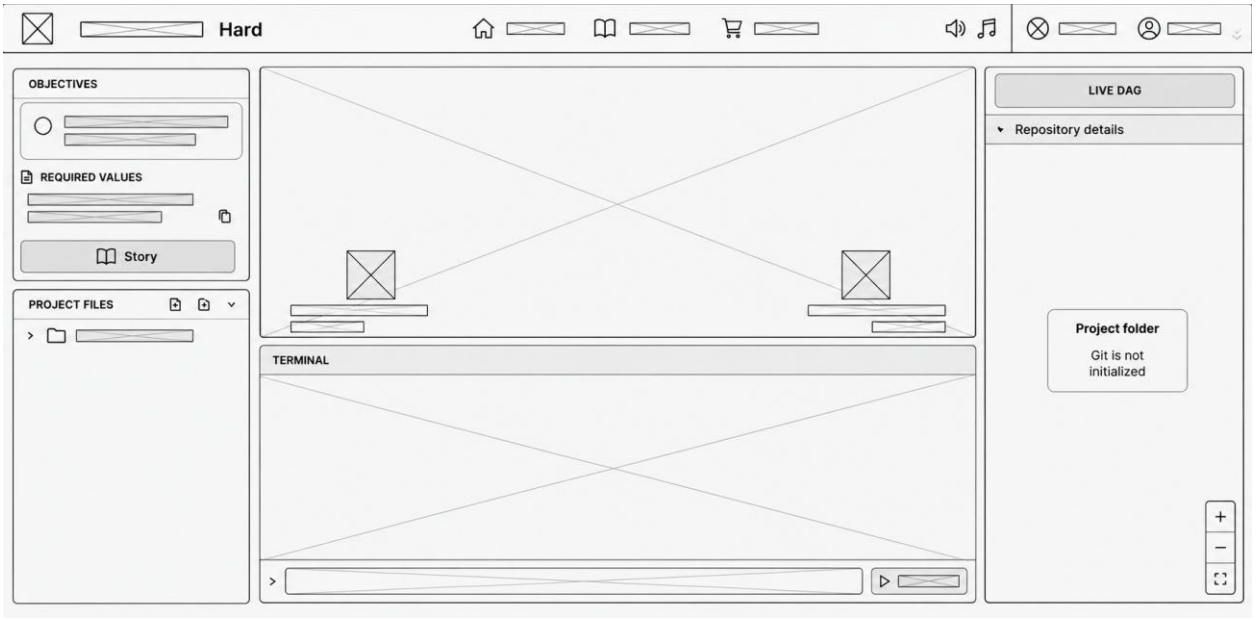
Field	Description
Use Case ID	UC-BRC-10
Use Case Name	Complete Hard-Level Independent Branch and Collaboration Scenarios
Curriculum SO	SO 2.10
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Medium difficulty is completed for the integrated skill focus in Branching and Collaboration. Hard difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and expands the Branching and Collaboration chapter entry.2. Student selects the integrated learning objective for independent branch creation, push/pull, reconciliation, and merge without scaffolding.3. Student confirms Medium is completed and clicks Start or Retry on the unlocked Hard difficulty action.4. Student reads the mission briefing and starts the tier.5. Student reads the multi-step Hard-level narrative (no target-state guidance; Live DAG and task prompt only).6. Student types Git commands and observes terminal output and Live DAG updates across multiple repository-state steps.7. Student checks the counted-command allowance throughout the session.8. Student applies repository-state reasoning without expected-state scaffolding to reach each step goal.9. Student sees the completion screen when all required target states are

	reached, or clicks Retry after failure.10. Student sees updated Hard completion status on the learning objective card when successful.
Postconditions	Student has completed or attempted the Hard-level integrated scenario. HLCR-relevant completion status is visible on the dashboard and chapter entry.
Alternative Flows	AF1 — Hard locked: Student sees Hard disabled until Medium is completed.AF2 — Session failed: Student clicks Retry without automatically receiving a scenario-specific solution sequence.AF3 — Counted-command limit: Student sees Failed state and may retry or exit.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19Note: Emphasizes FR-SCENE-03 (Hard scaffolding) and FR-DASH-07 (HLCR indicators).

Activity Diagram

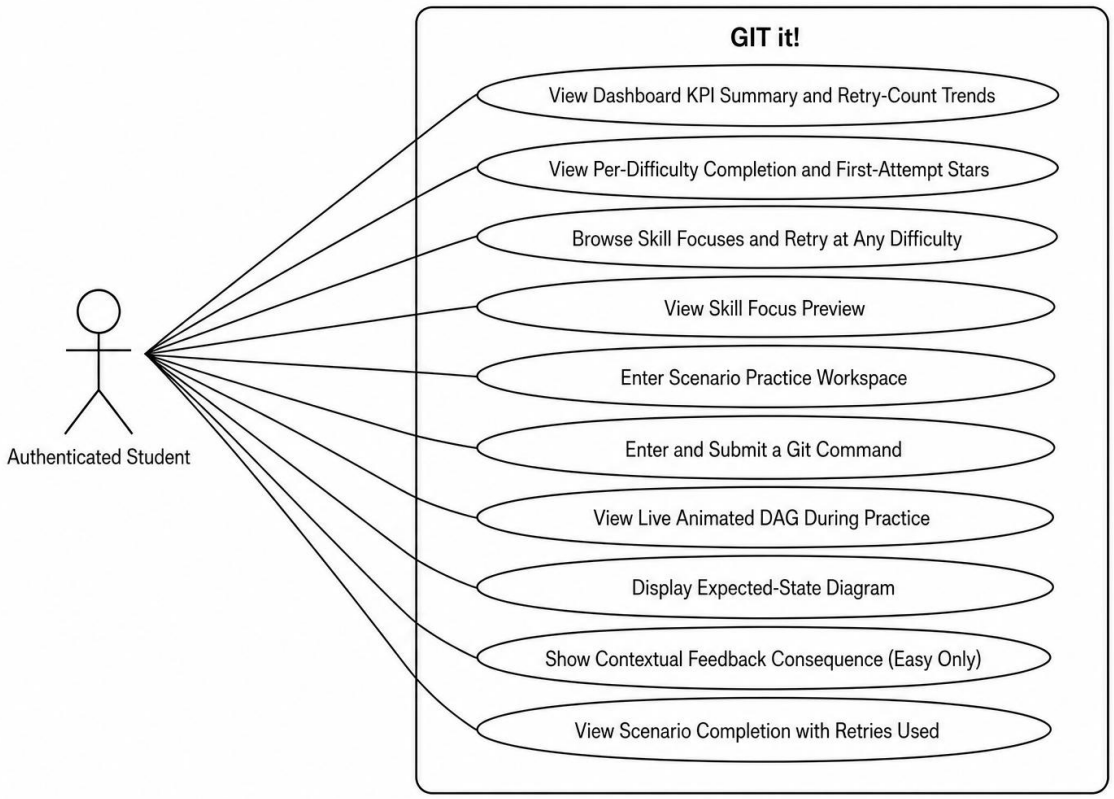


Wireframe


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized

2.11 Monitor Retry Practice and Apply Consequence-Aware Branching

Use Case Diagram

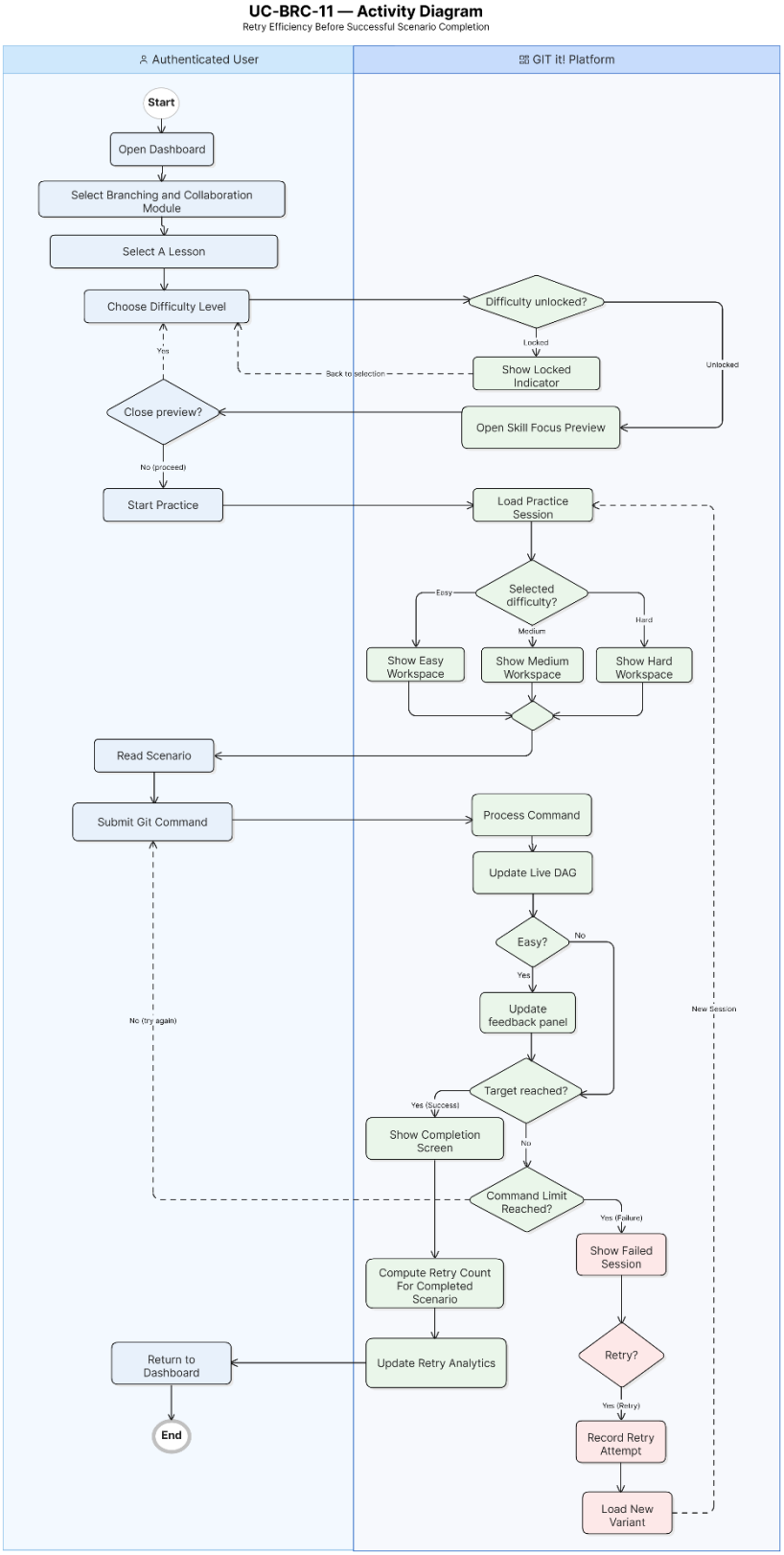


Use Case Description

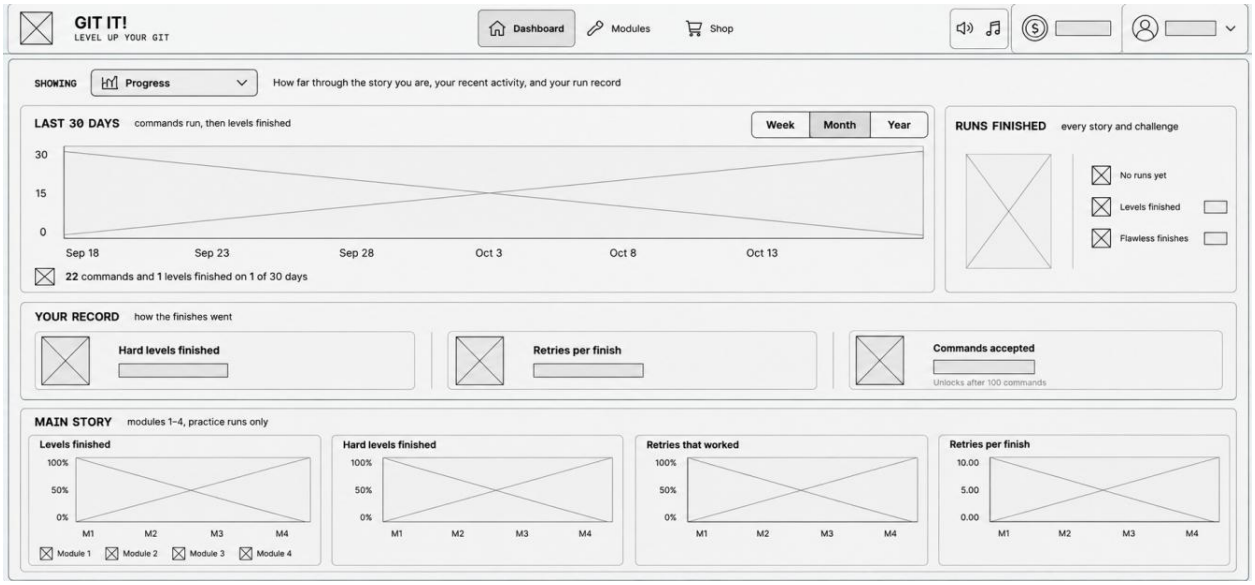
Field	Description
Use Case ID	UC-BRC-11
Use Case Name	Monitor Retry Practice and Apply Consequence-Aware Branching
Curriculum SO	SO 2.11
Actor(s)	Authenticated Student
Preconditions	Student is logged in and has access to the Home and Module scenario cards.
Main Flow	1. Student navigates to the Home.2. Student views KPI summary indicators and per-scenario retry-count trends for Module 2 — Branching and Collaboration.3. Student opens a chapter and reviews the Easy/Medium/Hard states of its levels on the story map.4. Student opens a scenario practice session (Start, Continue, or Retry).5. Student reads the narrative, inspects the Live DAG, and plans the next command before submitting.6. Student submits commands and monitors terminal output, DAG changes, and the counted-command allowance.7. If the session fails, student clicks Retry and returns to practice with a fresh attempt.8. Student sees retries used on the completion screen and updated retry trends on the dashboard after finishing.

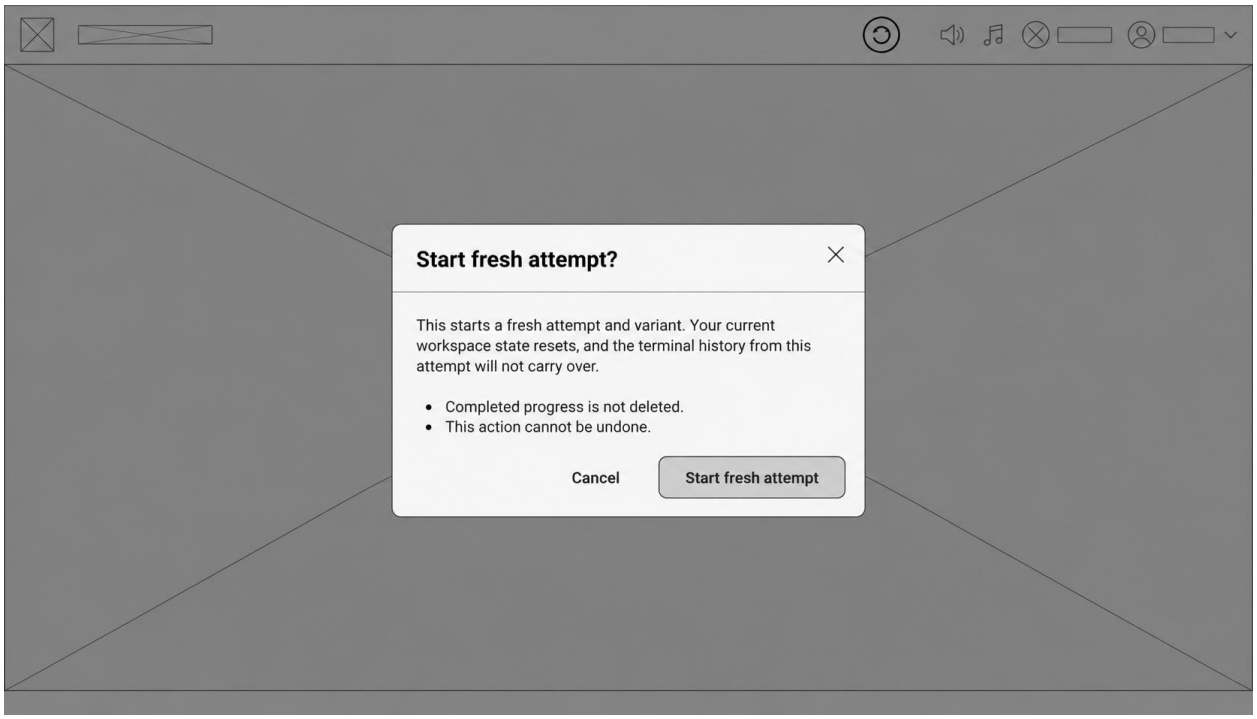
Postconditions	Student has reviewed retry trends and completed or retried practice sessions, supporting ARC monitoring for the module.
Alternative Flows	AF1 — No trend data yet: Student sees 'No trend available' on dashboard. AF2 — Student abandons mid-session: Partial progress not persisted; card shows Abandoned.
Functional Requirements	FR-DASH-02, FR-DASH-04, FR-DASH-05, FR-DASH-07, FR-SCENE-03, FR-SCENE-19

Activity Diagram



Wireframe

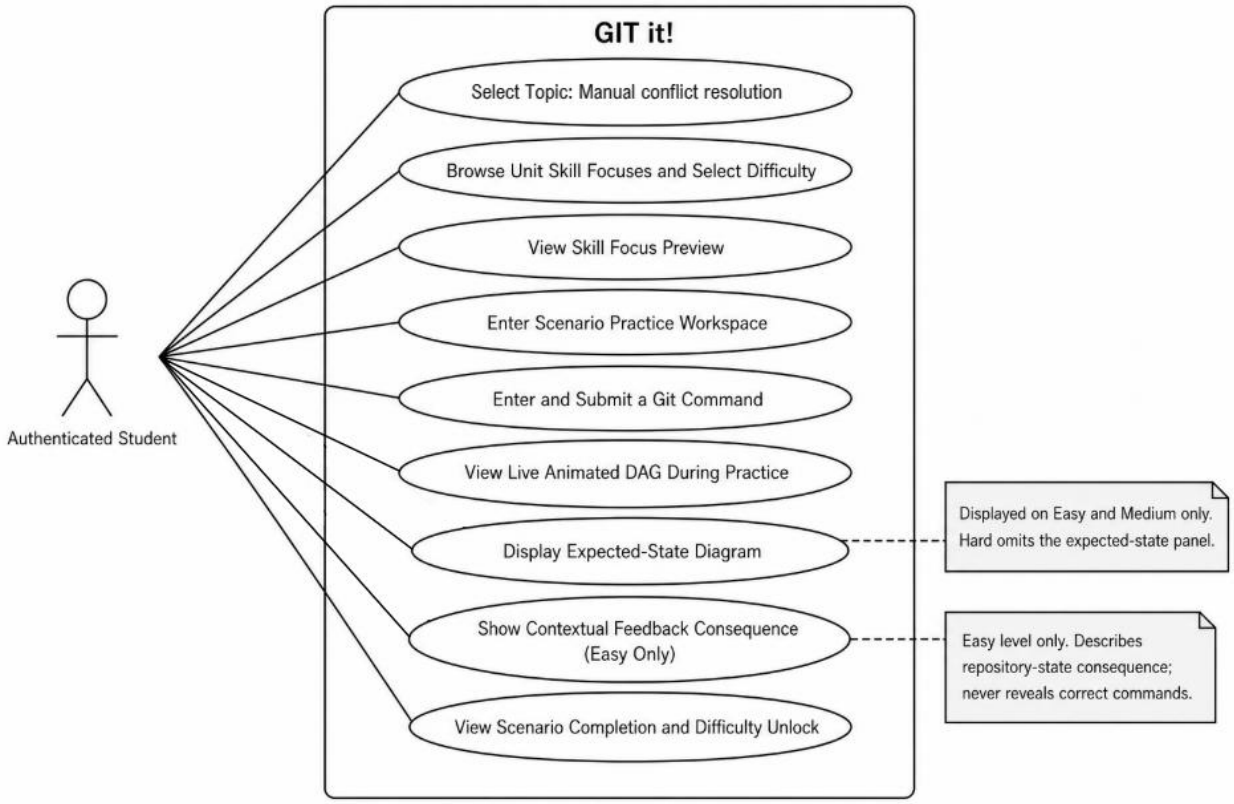

GIT IT!
LEVEL UP YOUR GIT
Dashboard
Modules
Shop
SHOWING
Progress
How far through the story you are, your recent activity, and your run record
LAST 30 DAYS
commands run, then levels finished
Week
Month
Year
30
15
0
Sep 18
Sep 23
Sep 28
Oct 3
Oct 8
Oct 13
22 commands and 1 levels finished on 1 of 30 days
RUNS FINISHED
every story and challenge
No runs yet
Levels finished
Flawless finishes
YOUR RECORD
how the finishes went
Hard levels finished
Retries per finish
Commands accepted
Unlocks after 100 commands
MAIN STORY
modules 1–4, practice runs only
Levels finished
Hard levels finished
Retries that worked
Retries per finish
100%
50%
0%
10.00
5.00
0.00
M1
M2
M3
M4
Module 1
Module 2
Module 3
Module 4


Start fresh attempt?
This starts a fresh attempt and variant. Your current workspace state resets, and the terminal history from this attempt will not carry over.
Completed progress is not deleted.
This action cannot be undone.
Cancel
Start fresh attempt

Module 3 — Conflict Resolution

3.1 Practice Resolving Merge Conflicts Manually

Use Case Diagram

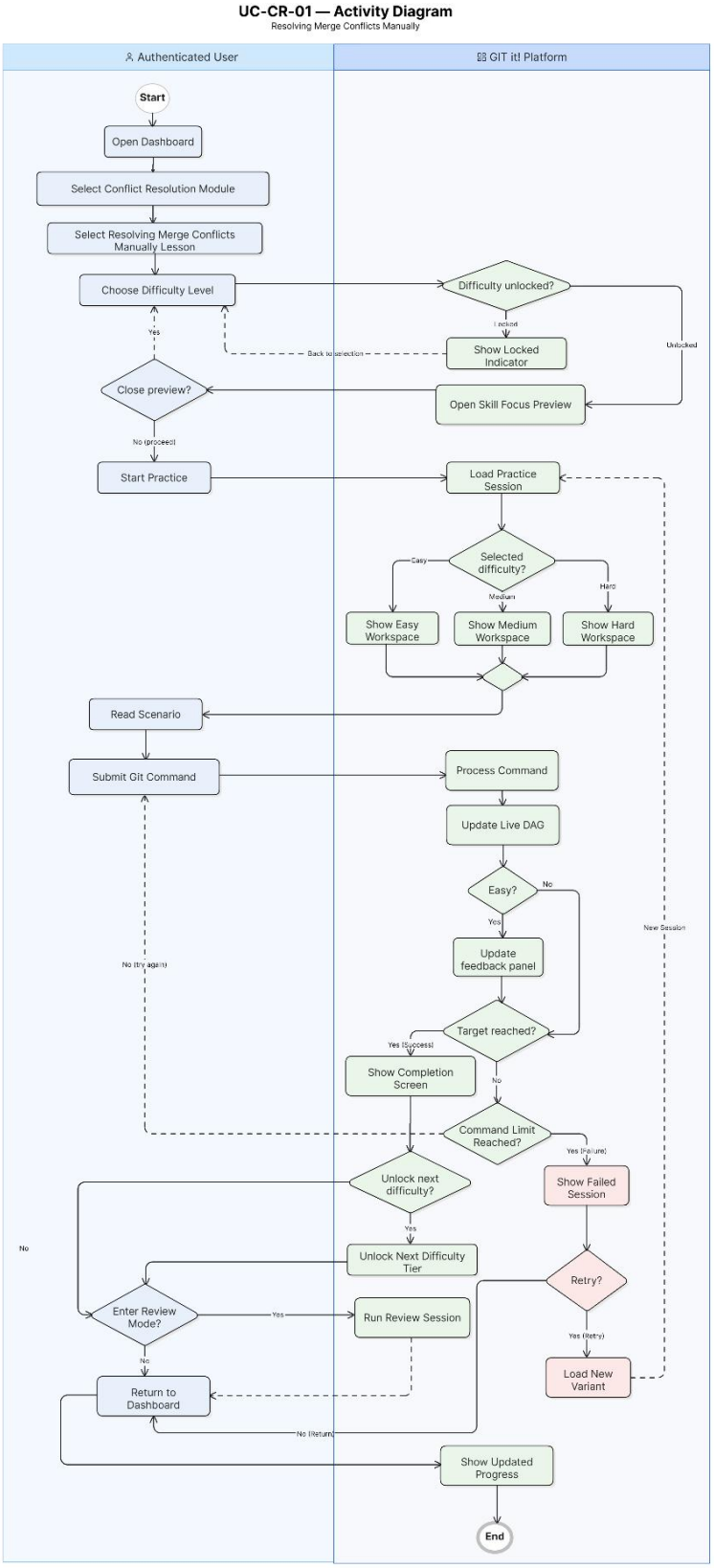


Use Case Description

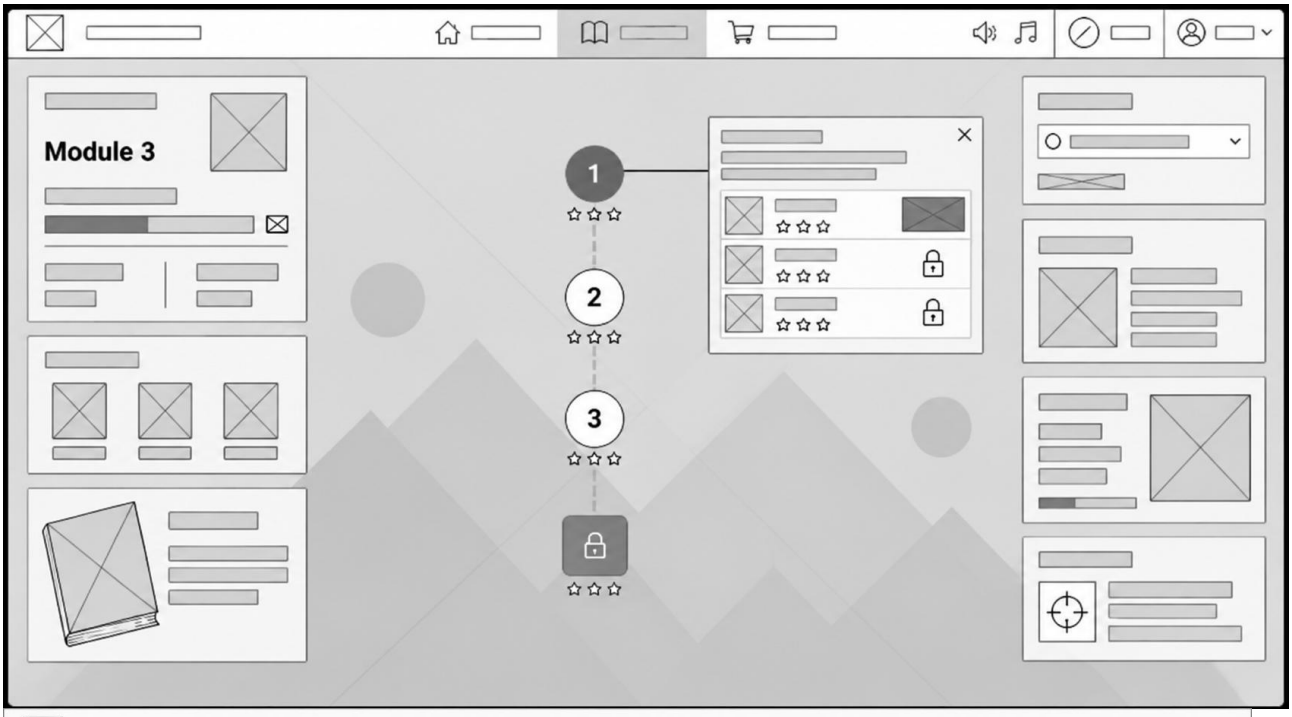
Field	Description
Use Case ID	UC-CR-01
Use Case Name	Practice Resolving Merge Conflicts Manually
Curriculum SO	SO 3.1
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Manual conflict resolution are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Manual conflict resolution.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the

	scenario narrative and task prompt describing identifying conflict markers, editing files, and completing the merge.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced identifying conflict markers, editing files, and completing the merge in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

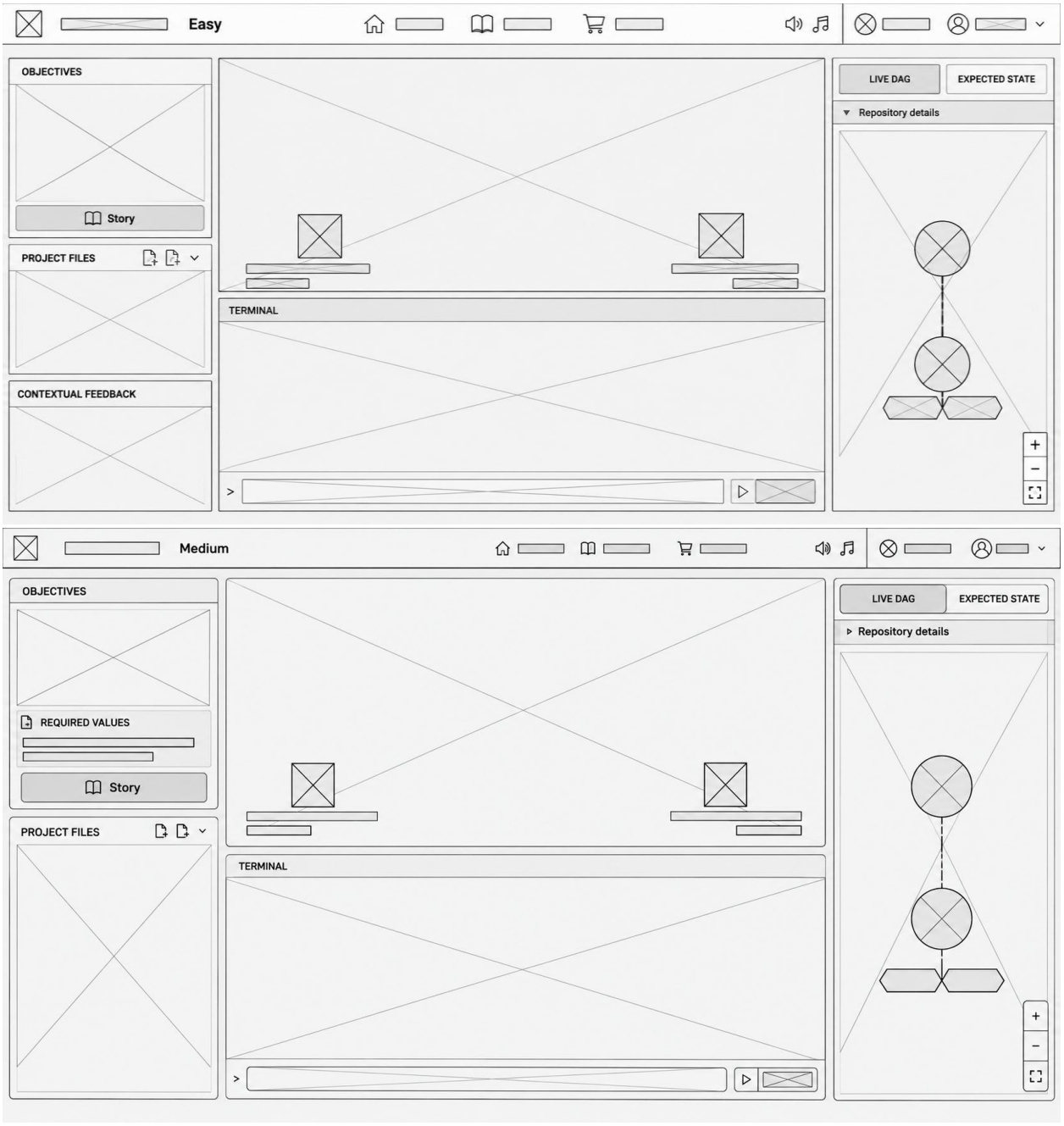
Activity Diagram

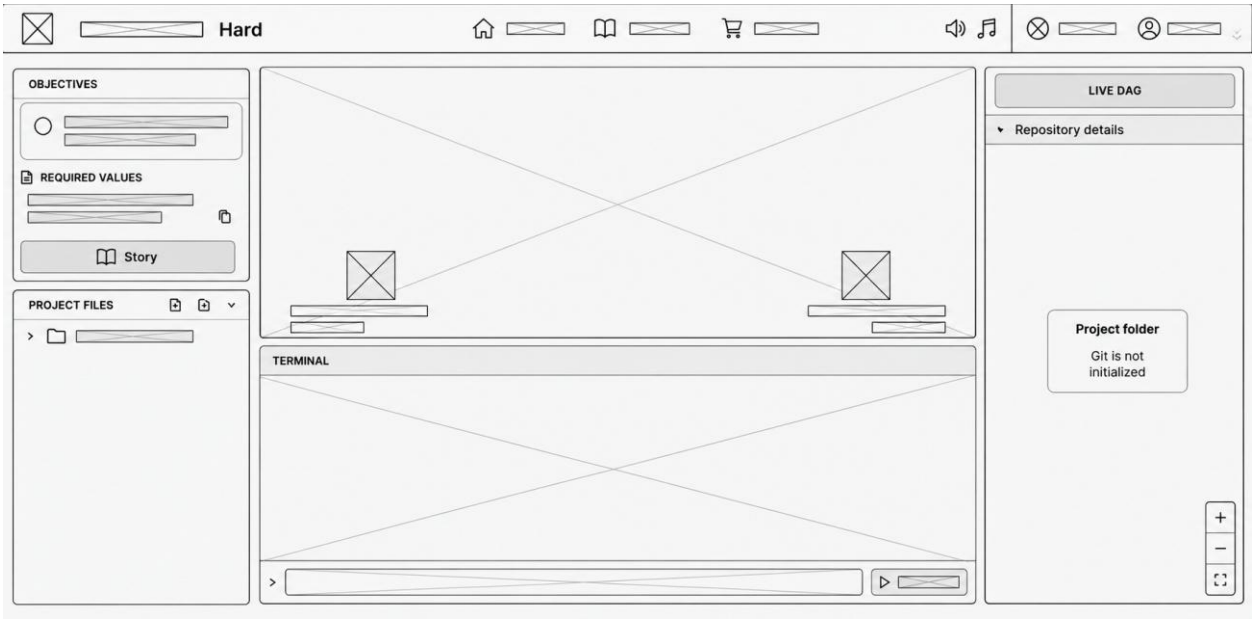


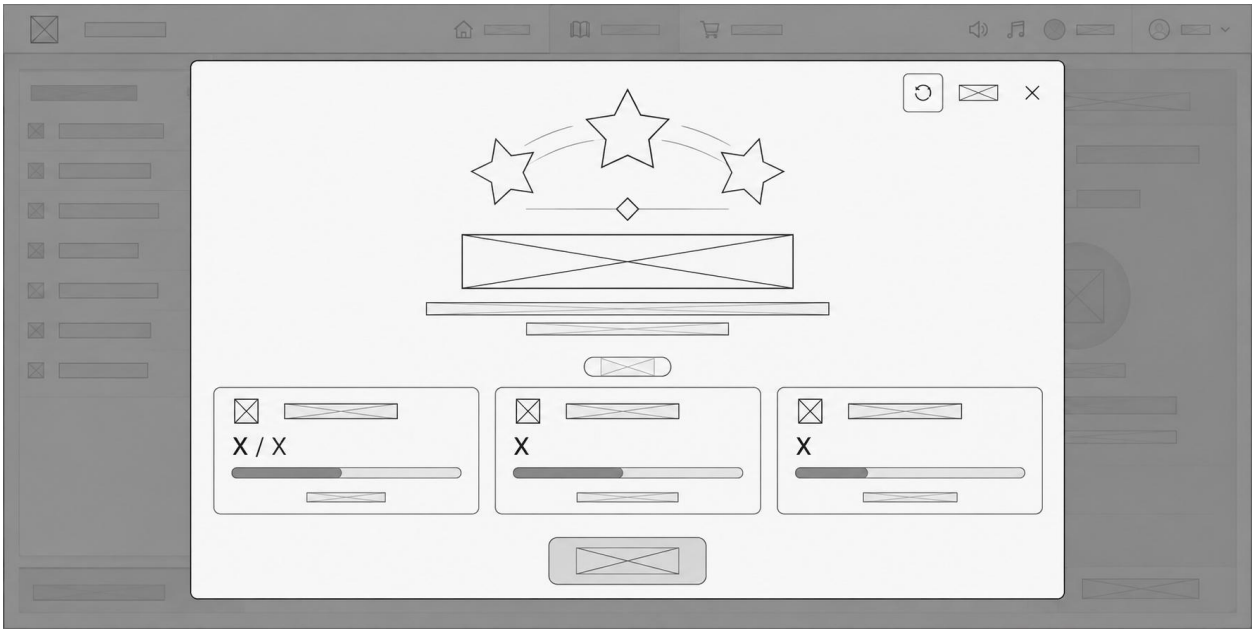
Wireframe





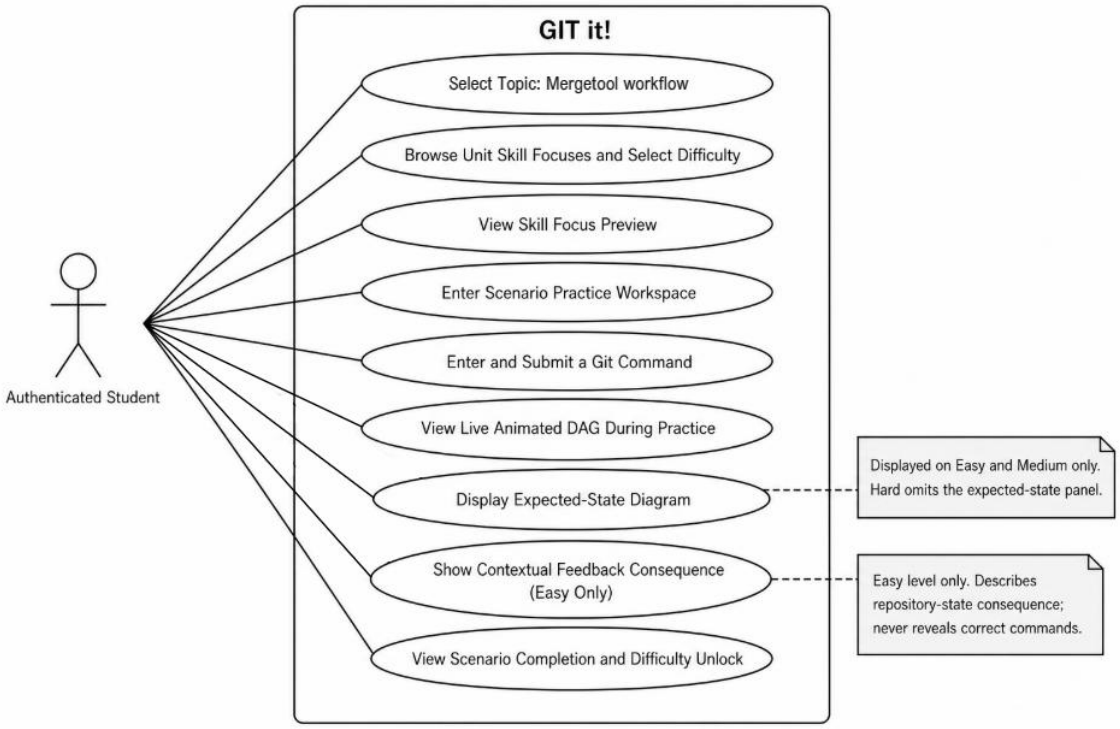

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

3.2 Practice Resolving Conflicts Using a Merge Tool

Use Case Diagram

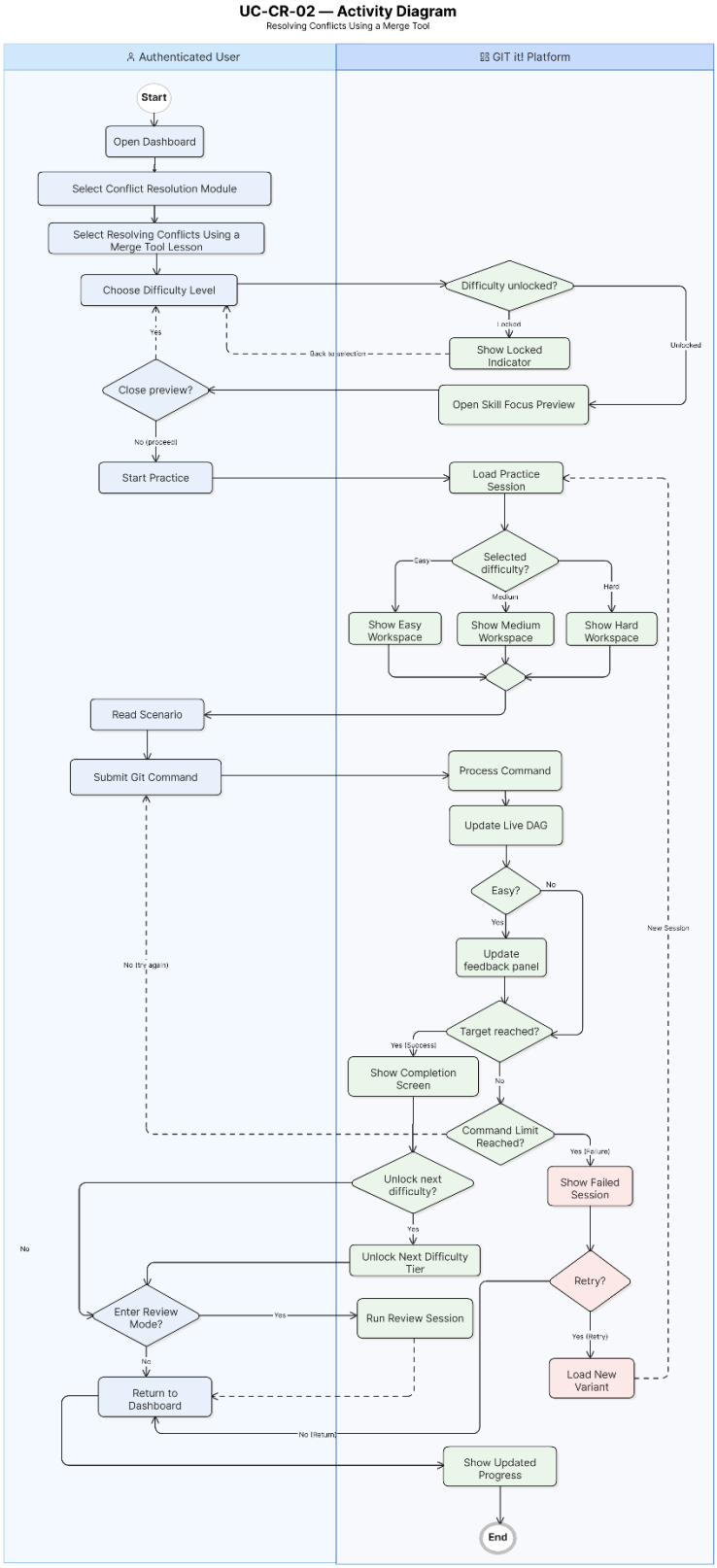


Use Case Description

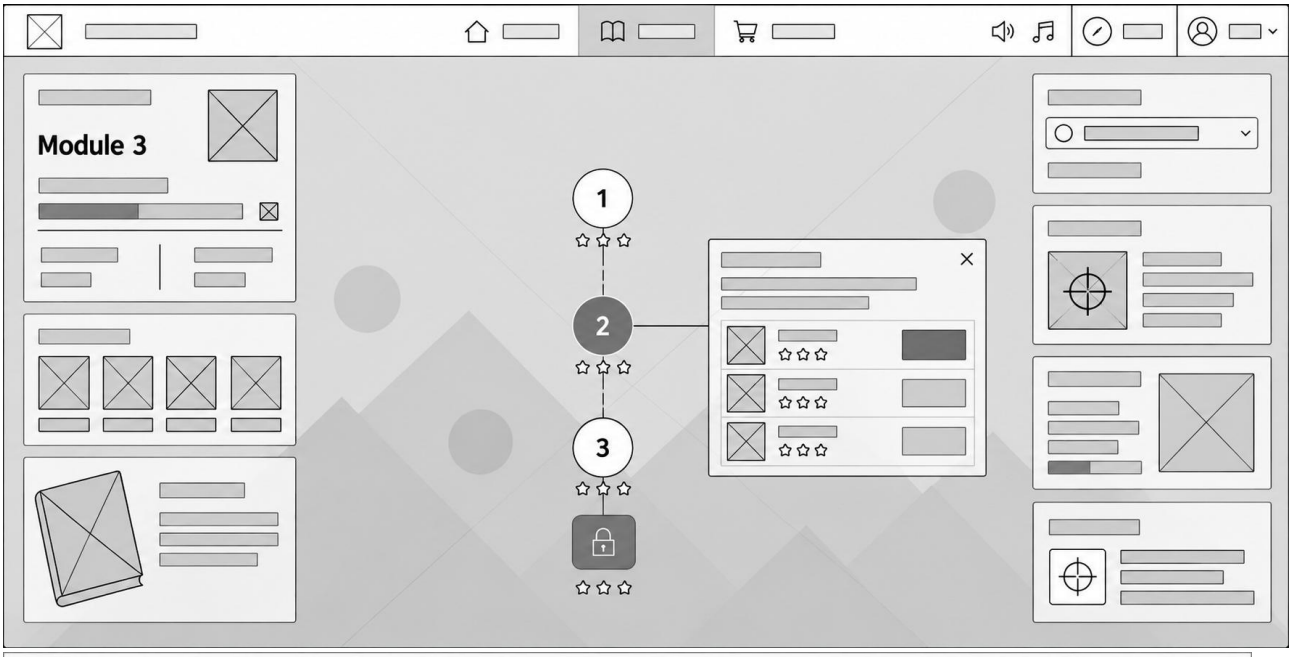
Field	Description
Use Case ID	UC-CR-02
Use Case Name	Practice Resolving Conflicts Using a Merge Tool
Curriculum SO	SO 3.2
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Mergetool workflow are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Mergetool workflow.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing configuring merge.tool and completing mergetool-driven resolution.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style

	feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced configuring merge.tool and completing mergetool-driven resolution in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

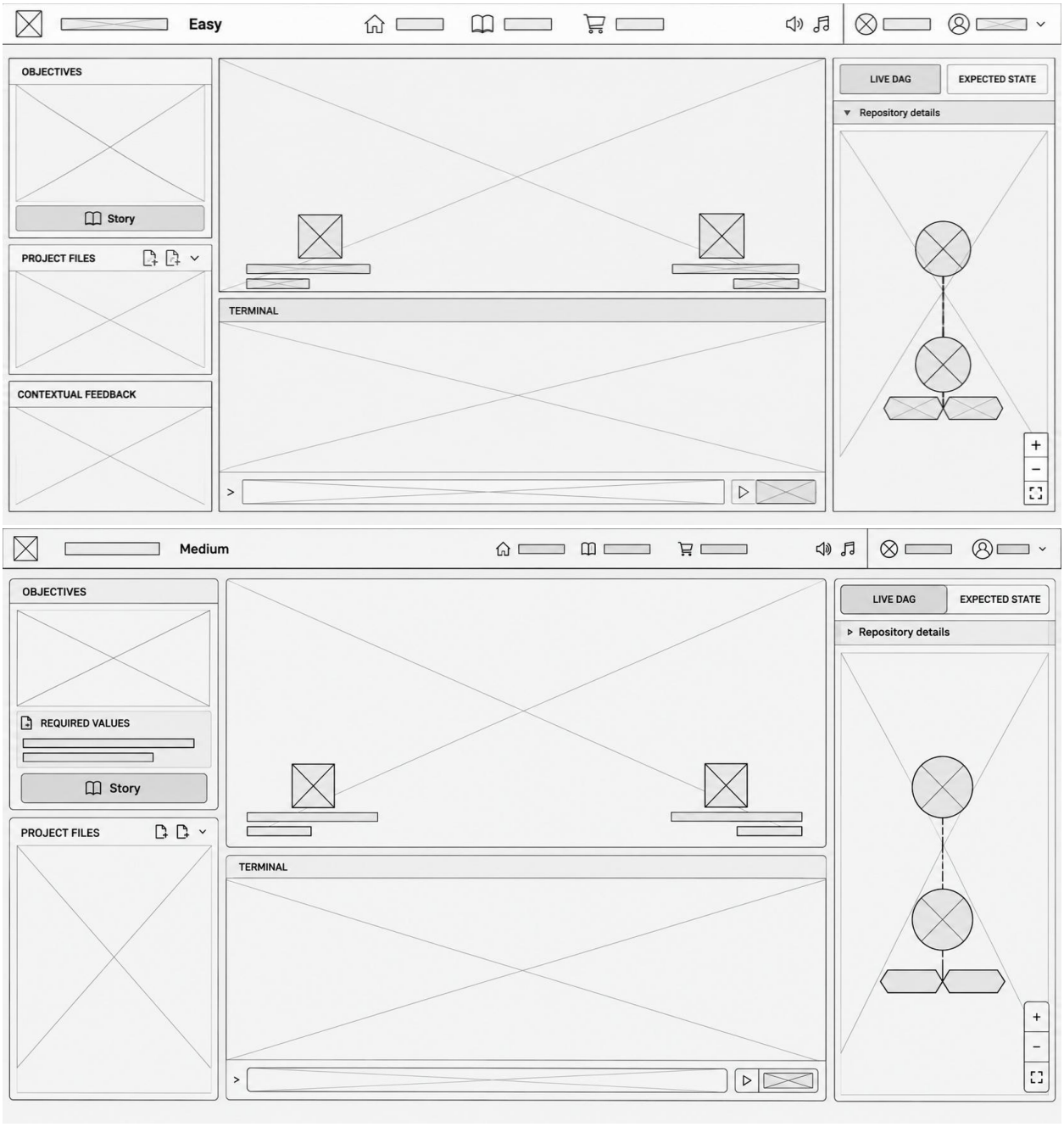
Activity Diagram

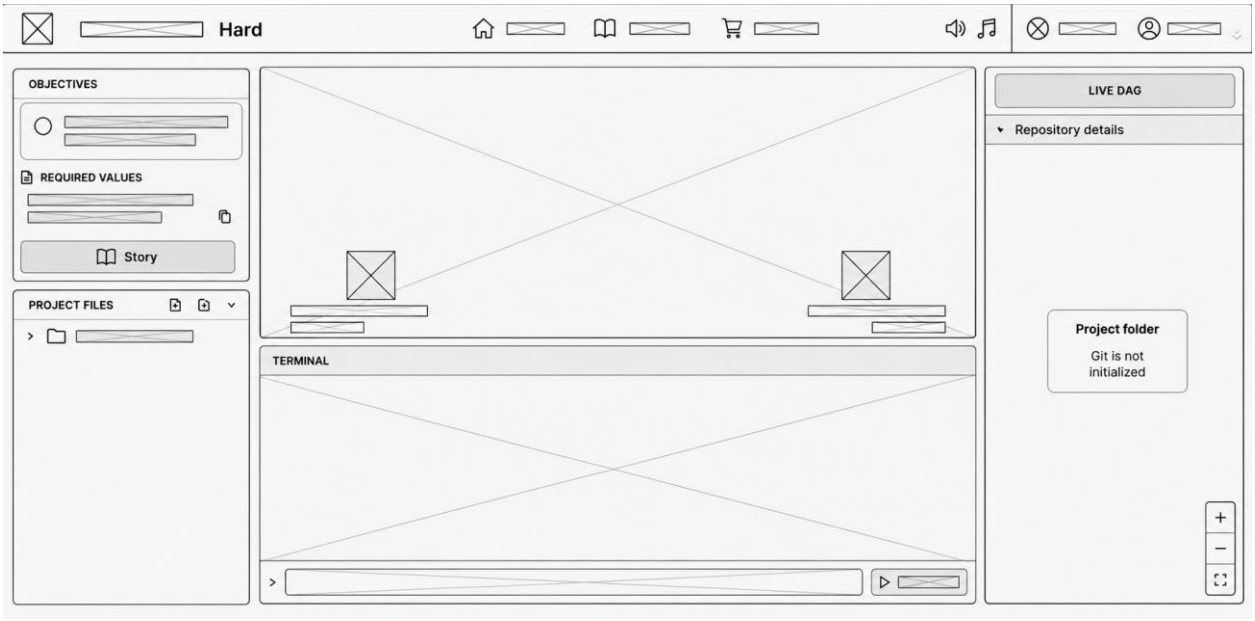


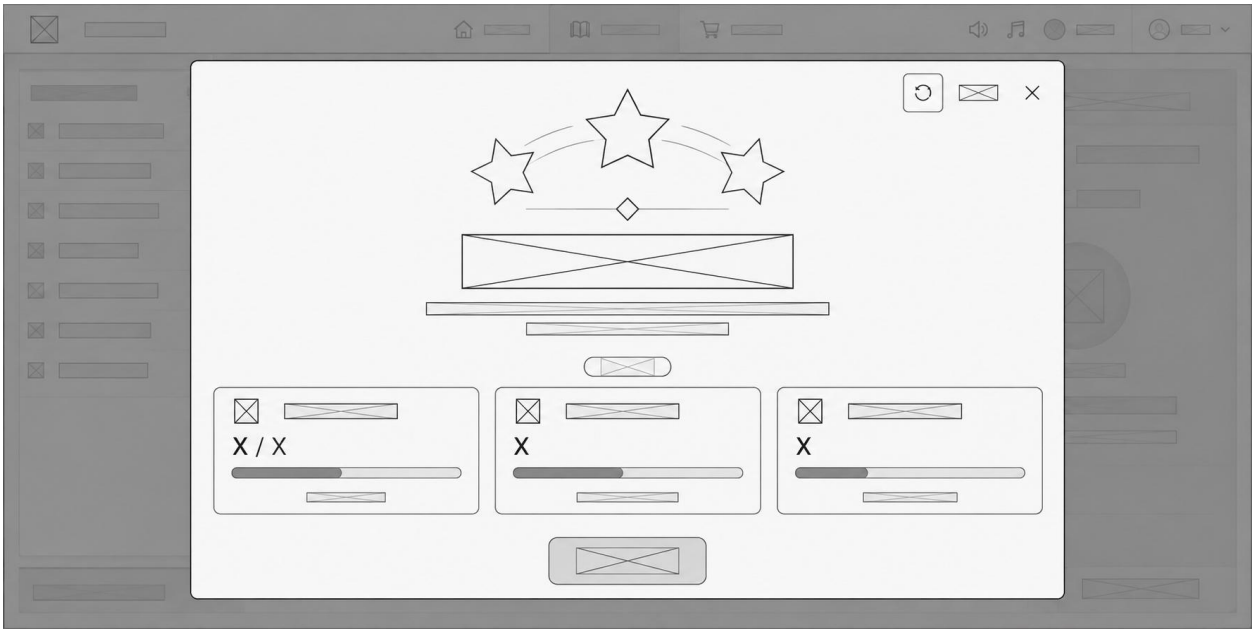
Wireframe





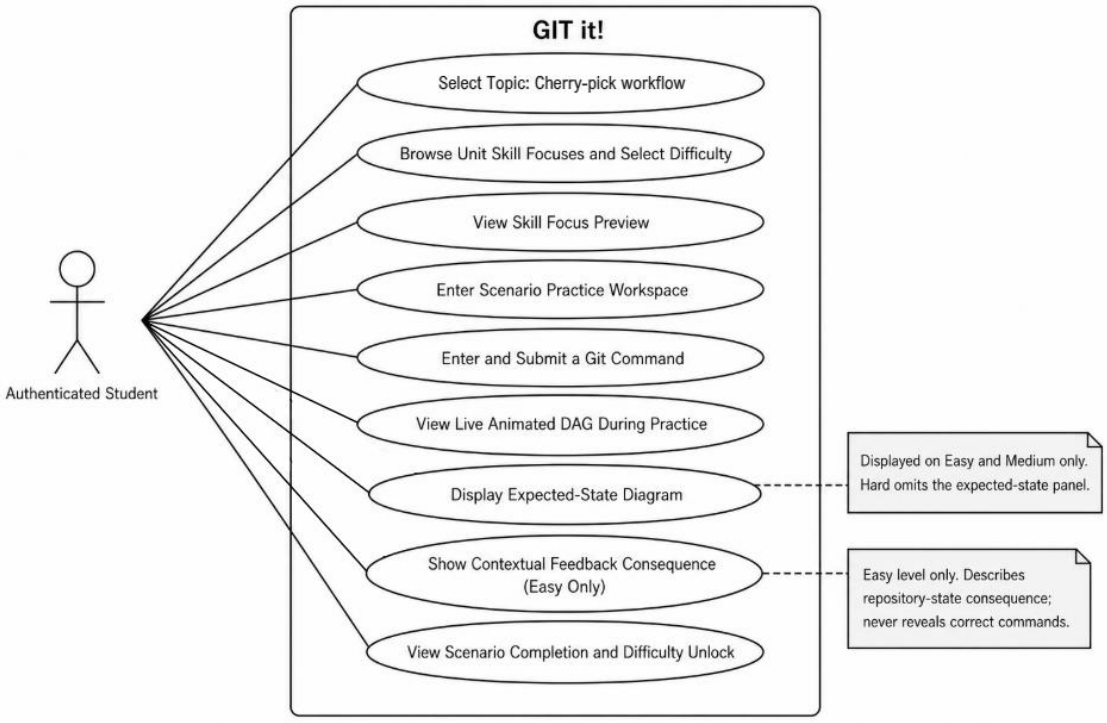

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

3.3 Practice Cherry-Picking Commits

Use Case Diagram

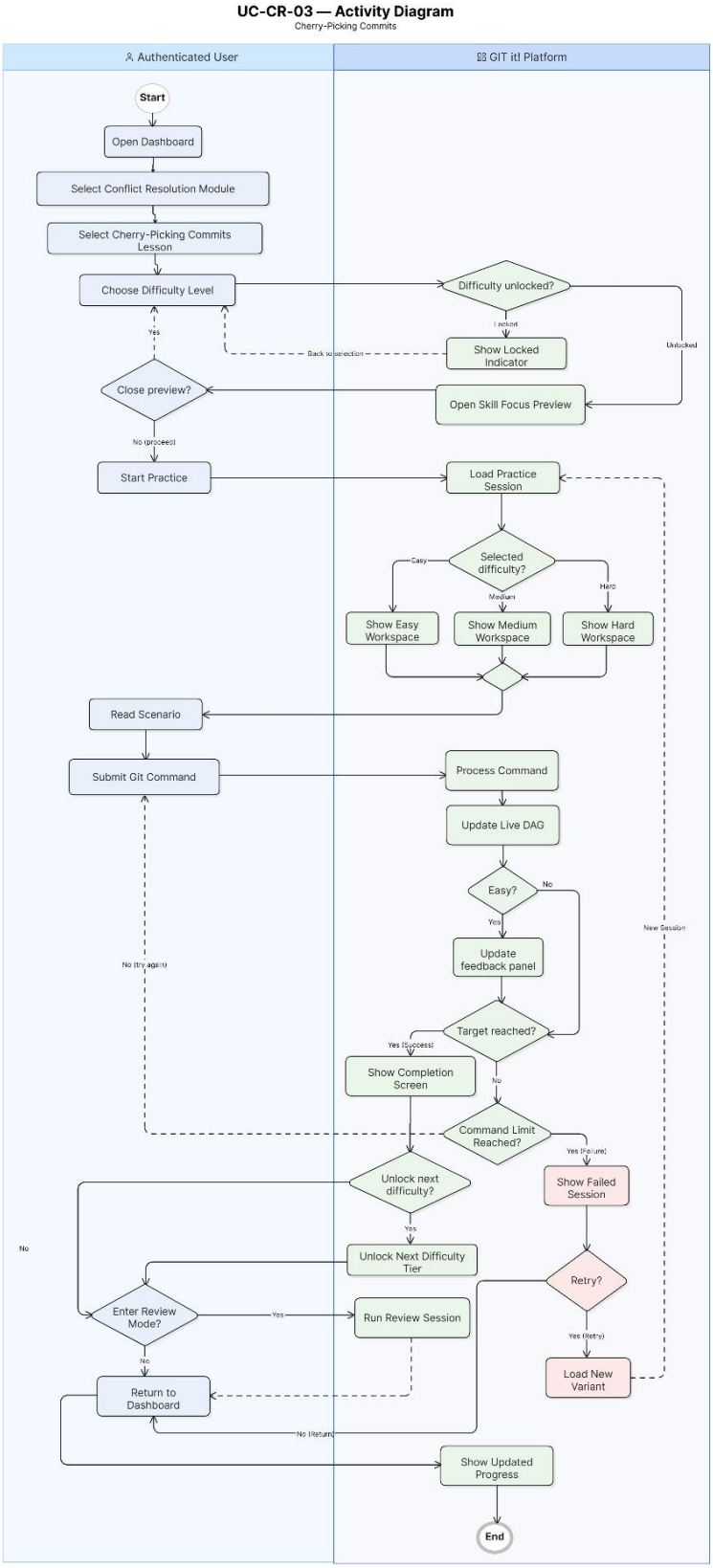


Use Case Description

Field	Description
Use Case ID	UC-CR-03
Use Case Name	Practice Cherry-Picking Commits
Curriculum SO	SO 3.3
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Cherry-pick workflow are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Cherry-pick workflow.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing selectively applying commits including no-commit and abort cases.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student

	views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced selectively applying commits including no-commit and abort cases in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

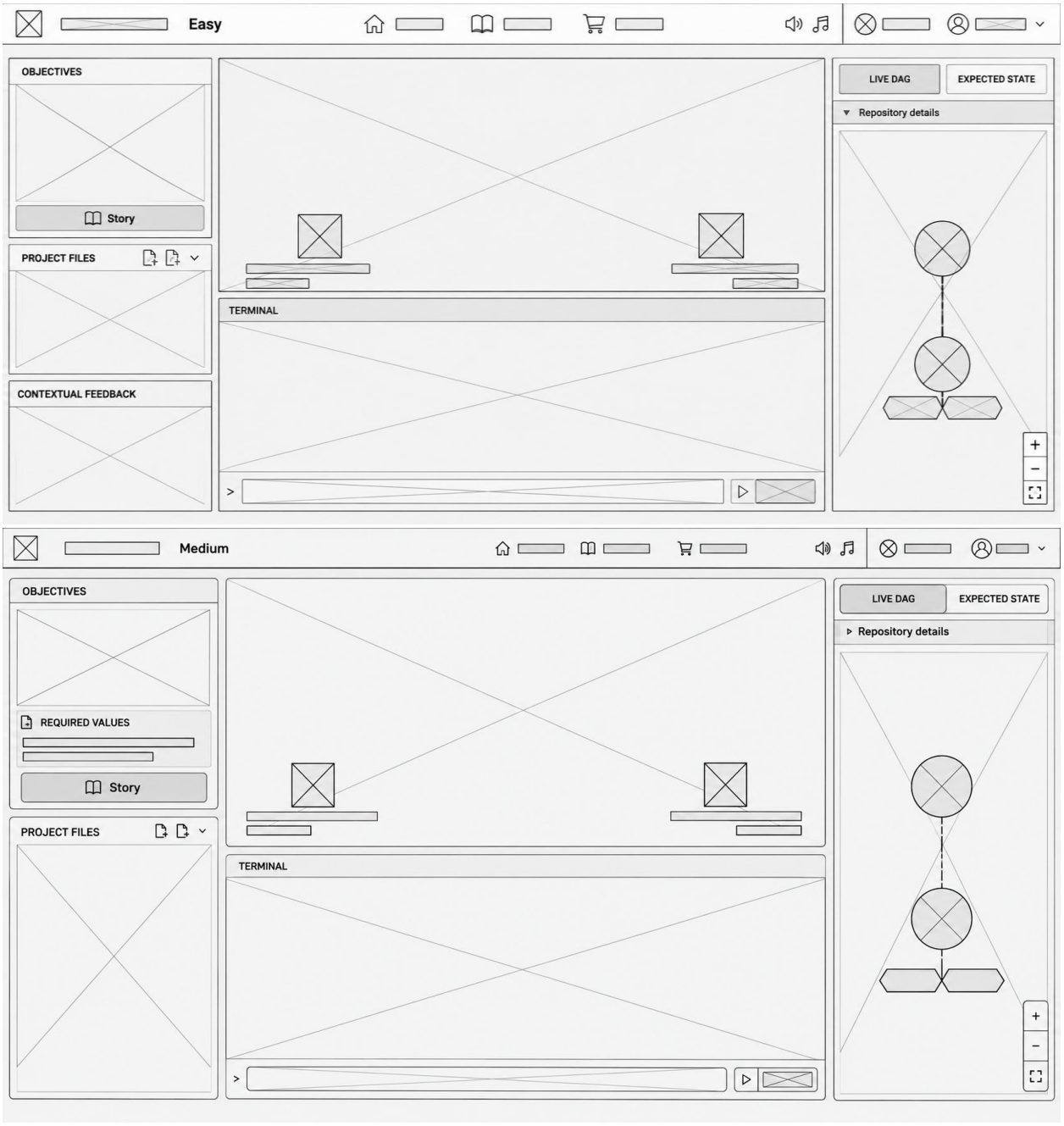
Activity Diagram

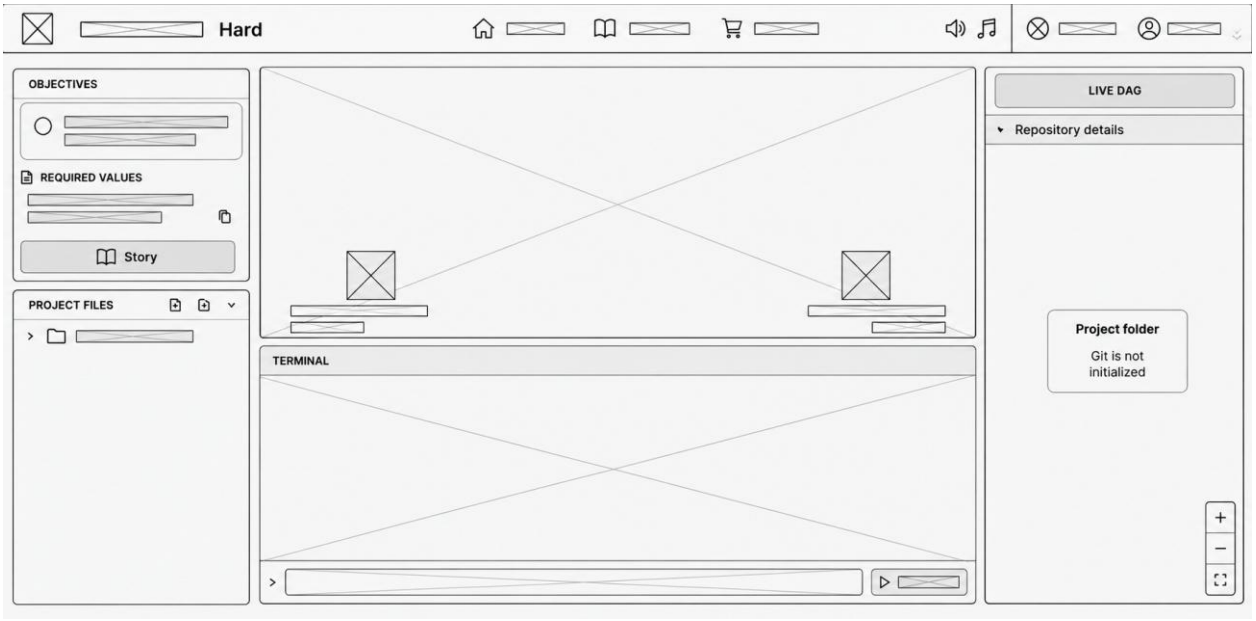


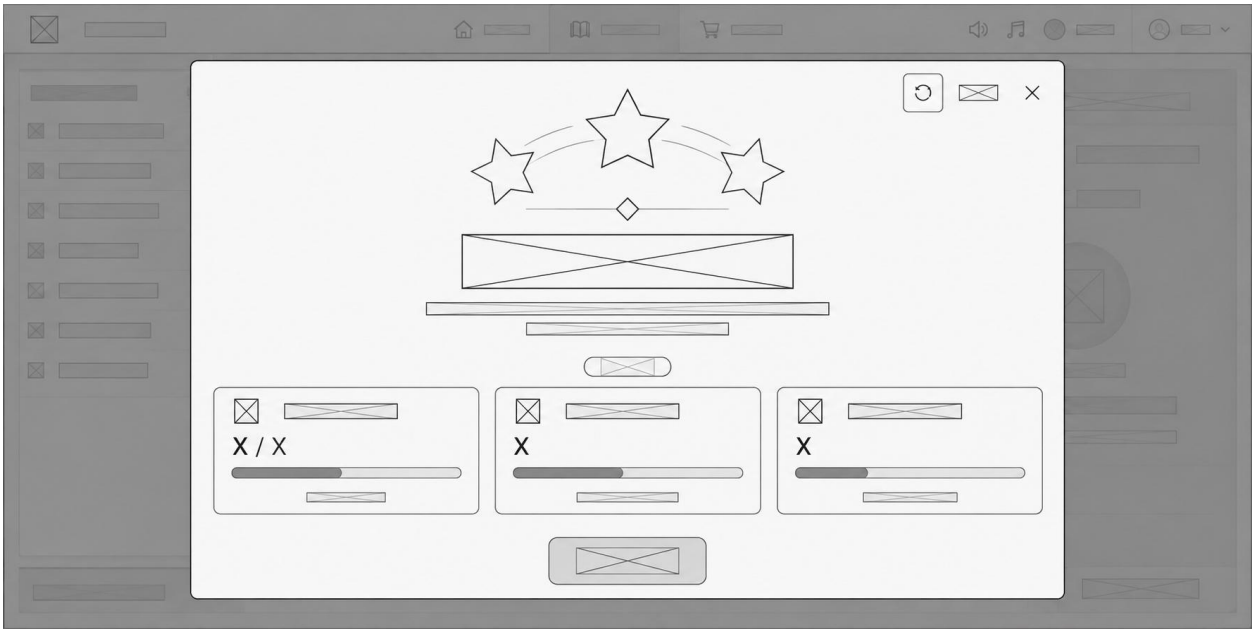
Wireframe





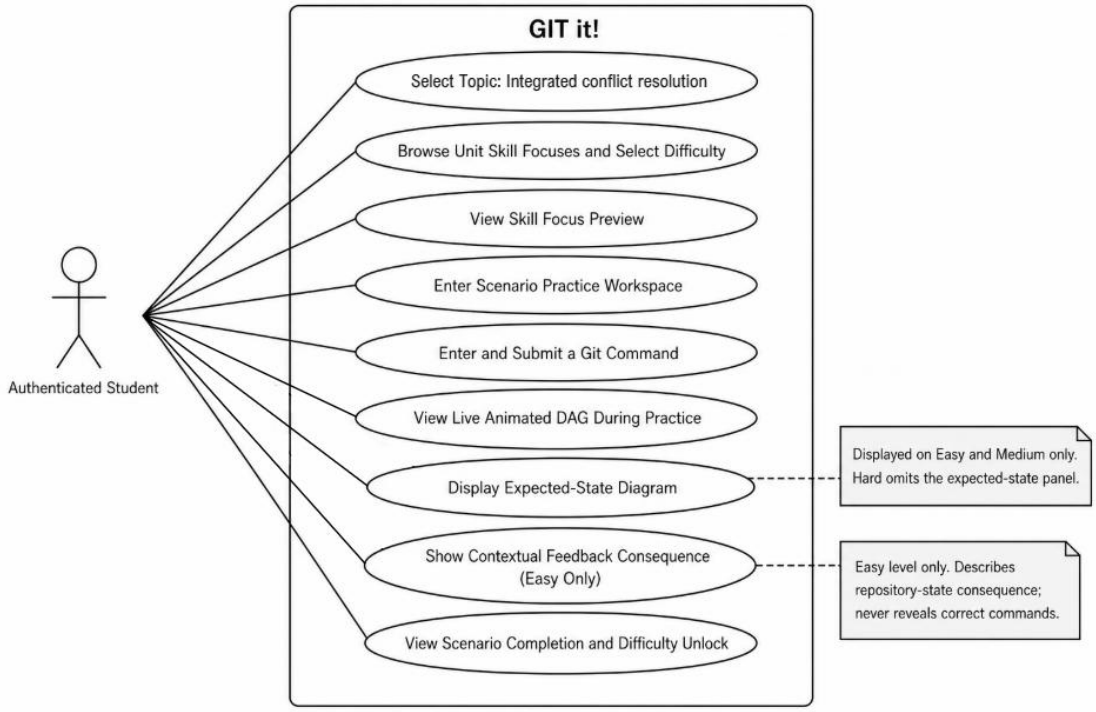

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

3.4 Complete Hard-Level Independent Conflict Resolution Scenarios

Use Case Diagram

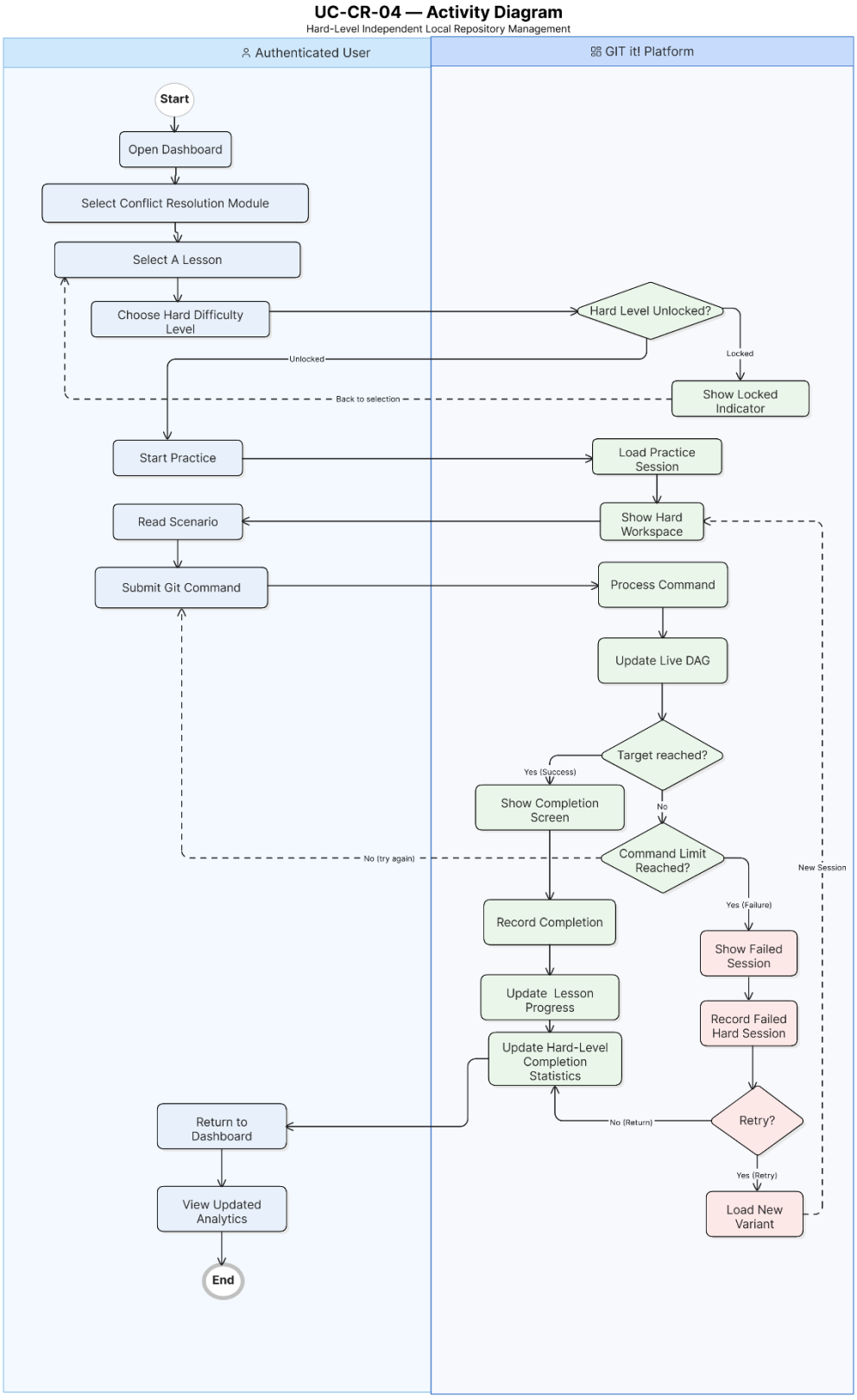


Use Case Description

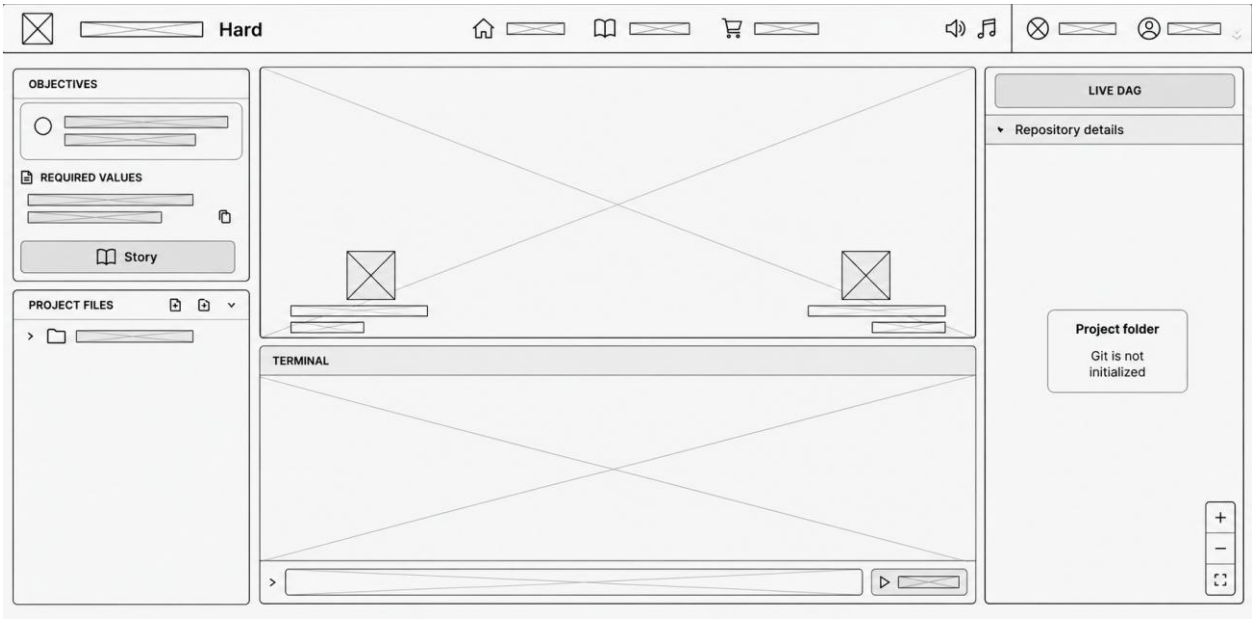
Field	Description
Use Case ID	UC-CR-04
Use Case Name	Complete Hard-Level Independent Conflict Resolution Scenarios
Curriculum SO	SO 3.4
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Medium difficulty is completed for the integrated skill focus in Conflict Resolution. Hard difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and expands the Conflict Resolution chapter entry.2. Student selects the integrated learning objective for independent manual resolution, mergetool workflow, and cherry-pick without scaffolding.3. Student confirms Medium is completed and clicks Start or Retry on the unlocked Hard difficulty action.4. Student reads the mission briefing and starts the tier.5. Student reads the multi-step Hard-level narrative (no target-state guidance; Live DAG and task prompt only).6. Student types Git commands and observes terminal output and Live DAG updates across multiple repository-state steps.7. Student checks the counted-command allowance throughout the session.8. Student applies repository-state reasoning without expected-state scaffolding to reach each step goal.9. Student sees the completion screen when all required target states are reached, or clicks Retry after failure.10. Student sees updated Hard completion status on the learning objective card when successful.

Postconditions	Student has completed or attempted the Hard-level integrated scenario. HLCR-relevant completion status is visible on the dashboard and chapter entry.
Alternative Flows	AF1 — Hard locked: Student sees Hard disabled until Medium is completed.AF2 — Session failed: Student clicks Retry without automatically receiving a scenario-specific solution sequence.AF3 — Counted-command limit: Student sees Failed state and may retry or exit.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19Note: Emphasizes FR-SCENE-03 (Hard scaffolding) and FR-DASH-07 (HLCR indicators).

Activity Diagram

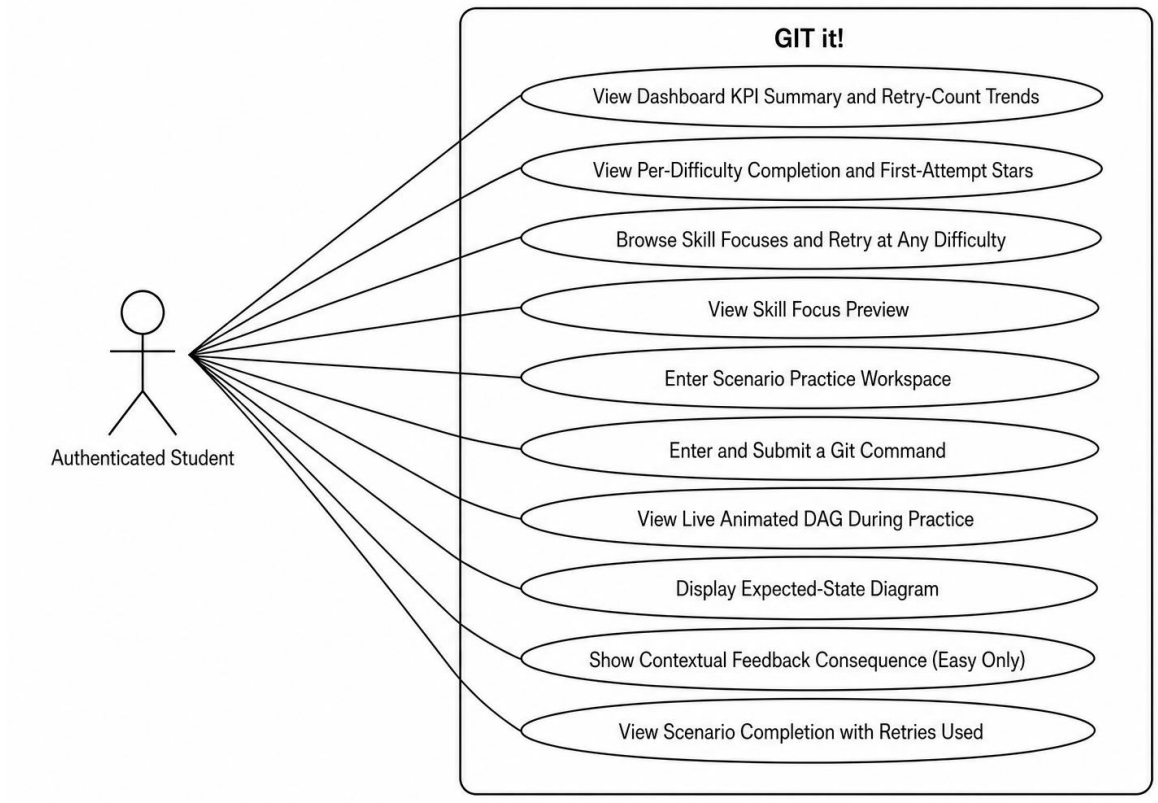


Wireframe



3.5 Retry Changed Variants and Monitor Conflict-Resolution Transfer

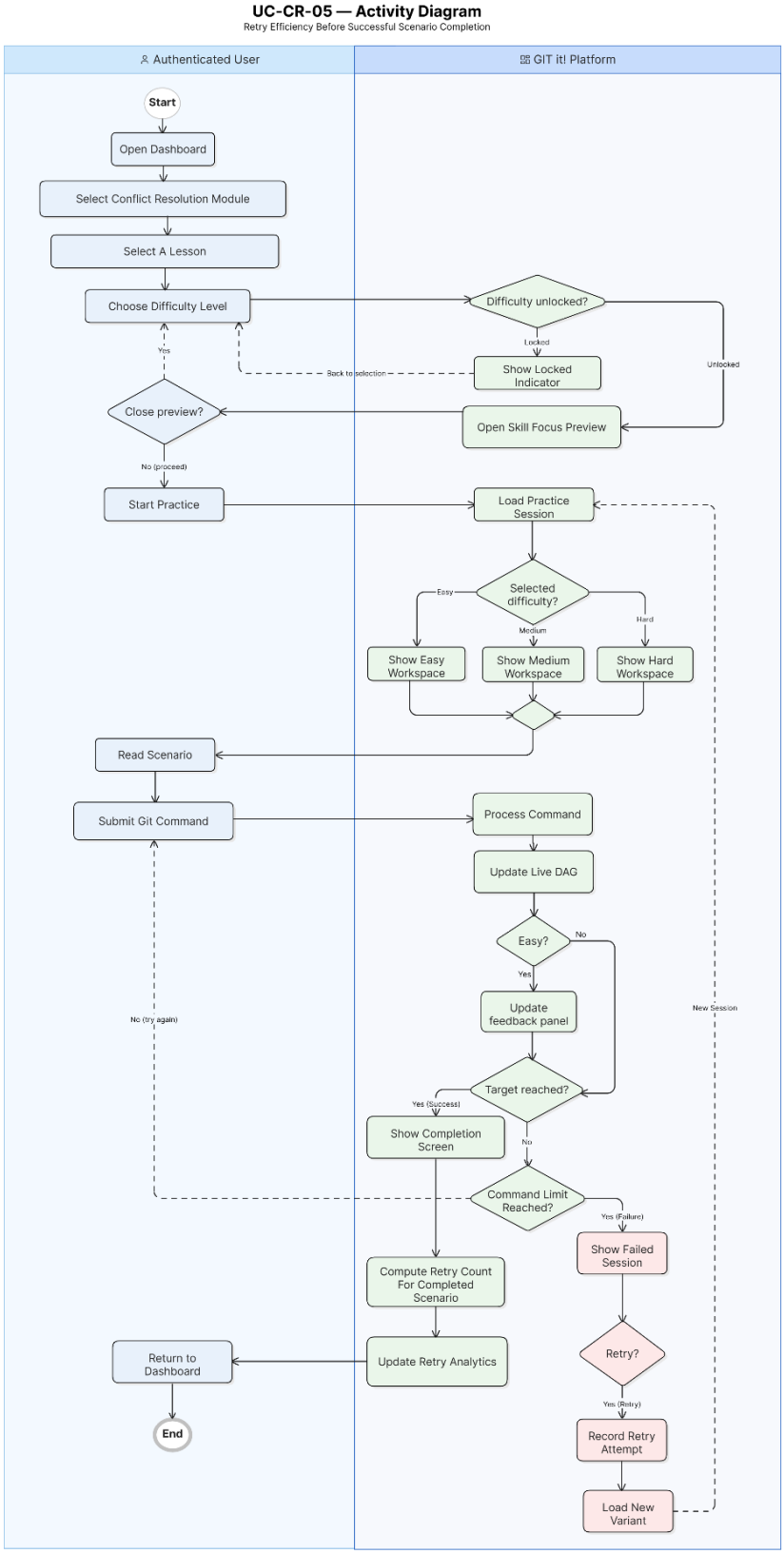
Use Case Diagram



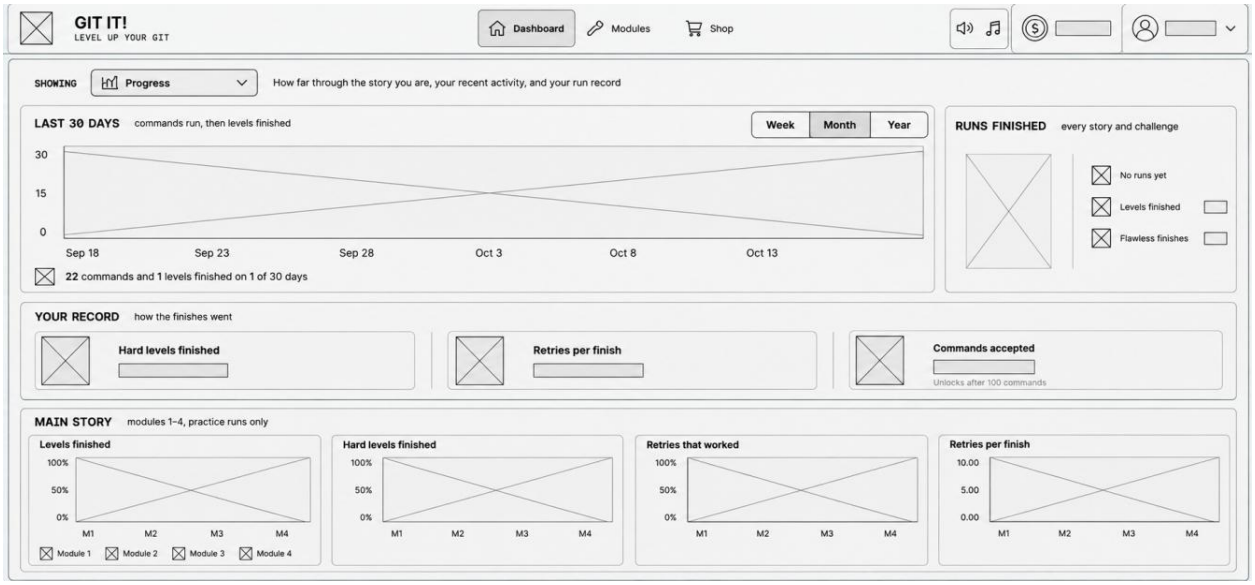
Use Case Description

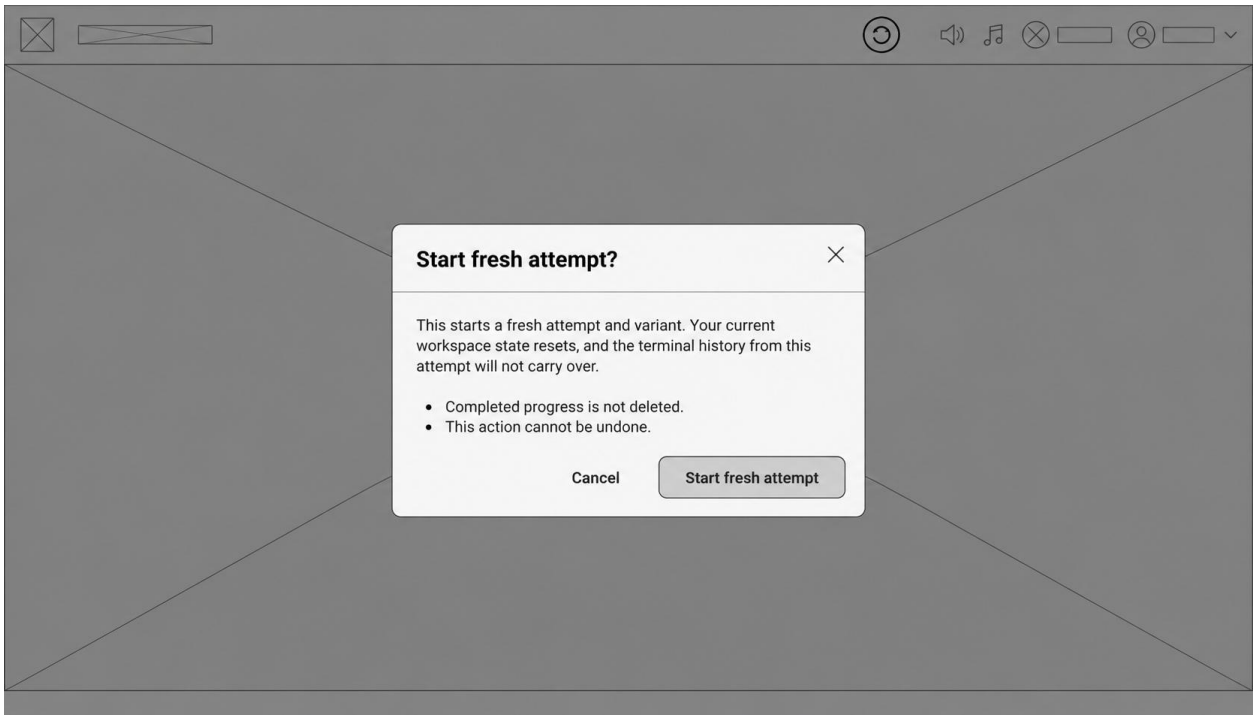
Field	Description
Use Case ID	UC-CR-05
Use Case Name	Retry Changed Variants and Monitor Conflict-Resolution Transfer
Curriculum SO	SO 3.5
Actor(s)	Authenticated Student
Preconditions	Student is logged in and has a prior Failed or Abandoned session for the same learning objective and difficulty level.
Main Flow	1. Student completes or abandons a scenario session in Module 3 — Conflict Resolution without reaching all targets.2. Student clicks Retry on the Failed or Abandoned difficulty action.3. Student reads the mission briefing and starts the tier.4. Student reads a retry narrative with changed branch names, file context, or repository layout (structurally different variant).5. Student applies the same Git skill focus using consequence-aware reasoning rather than repeating prior commands blindly.6. Student types commands, sees terminal output and Live DAG updates, and checks the counted-command allowance.7. Student reaches the target repository state or chooses to abandon/retry again.8. Student views dashboard indicators reflecting practice progress (retry trends, KPI summary when data is available).
Postconditions	Student has attempted a retry with visibly changed scenario context. Dashboard reflects updated practice status; RTA is computed internally, not shown as a separate student action.
Alternative Flows	AF1 — First-ever attempt: Student practices without a prior Failed session (not RTA-eligible). AF2 — Student completes changed variant on first retry try: Transfer practice succeeded from student view. AF3 — Student abandons retry: Card shows Abandoned; student may retry again.
Functional Requirements	FR-VAR-01, FR-VAR-02, FR-VAR-03, FR-VAR-04, FR-DASH-05, FR-DASH-07, FR-DASH-17, FR-DASH-19, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-17, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-18, FR-SCENE-19

Activity Diagram



Wireframe

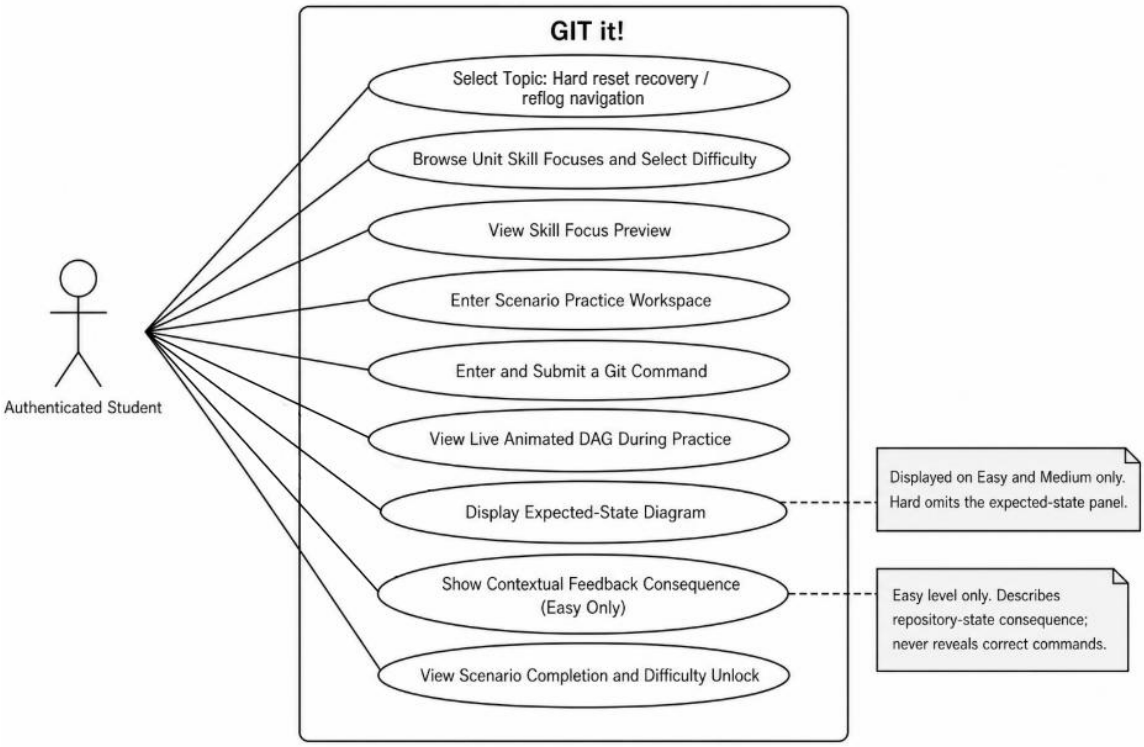

GIT IT!
LEVEL UP YOUR GIT
Dashboard
Modules
Shop
SHOWING
Progress
How far through the story you are, your recent activity, and your run record
LAST 30 DAYS
commands run, then levels finished
Week
Month
Year
30
15
0
Sep 18
Sep 23
Sep 28
Oct 3
Oct 8
Oct 13
22 commands and 1 levels finished on 1 of 30 days
RUNS FINISHED
every story and challenge
No runs yet
Levels finished
Flawless finishes
YOUR RECORD
how the finishes went
Hard levels finished
Retries per finish
Commands accepted
Unlocks after 100 commands
MAIN STORY
modules 1–4, practice runs only
Levels finished
Hard levels finished
Retries that worked
Retries per finish
100%
50%
0%
10.00
5.00
0.00
M1
M2
M3
M4
Module 1
Module 2
Module 3
Module 4


Start fresh attempt?
This starts a fresh attempt and variant. Your current workspace state resets, and the terminal history from this attempt will not carry over.
Completed progress is not deleted.
This action cannot be undone.
Cancel
Start fresh attempt

Module 4 — Advanced Recovery and History

4.1 Practice Recovering from Hard Resets

Use Case Diagram

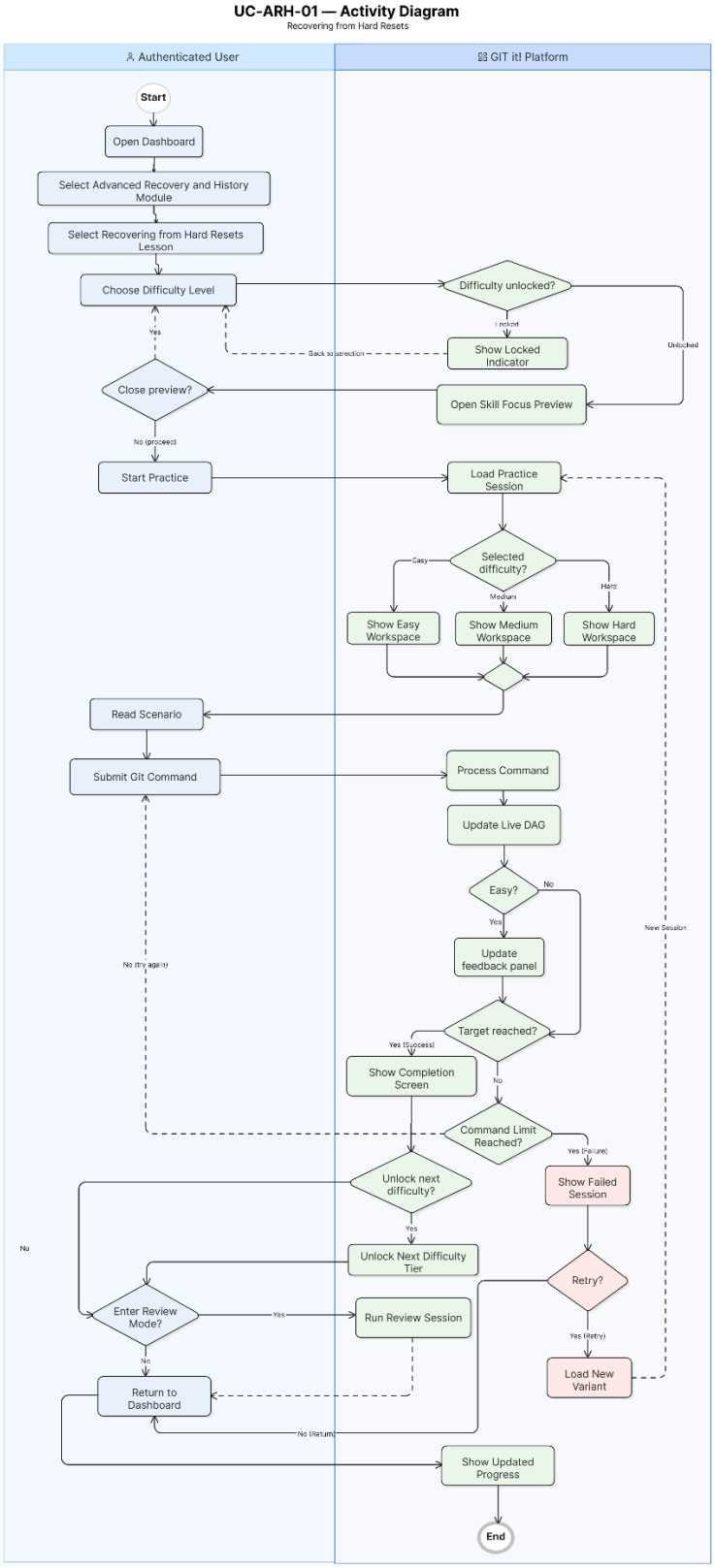


Use Case Description

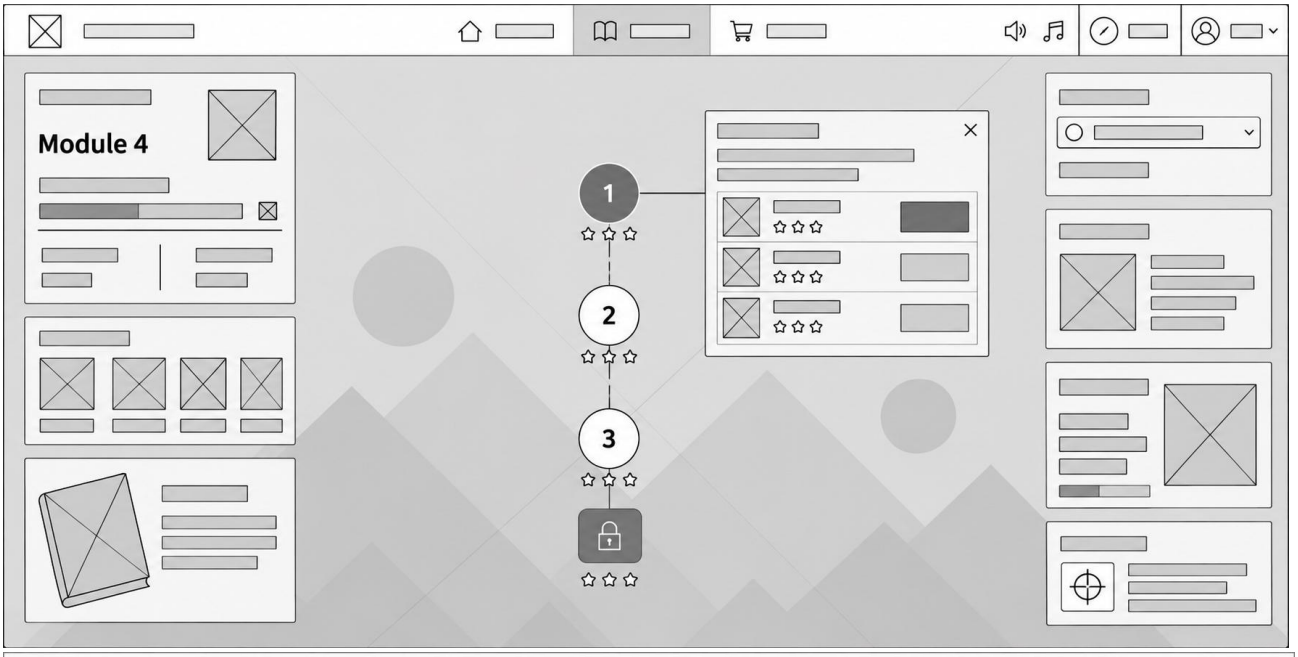
Field	Description
Use Case ID	UC-ARH-01
Use Case Name	Practice Recovering from Hard Resets
Curriculum SO	SO 4.1
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Hard reset recovery / reflog navigation are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Hard reset recovery / reflog navigation.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing restoring lost work after destructive reset

	operations.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced restoring lost work after destructive reset operations in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

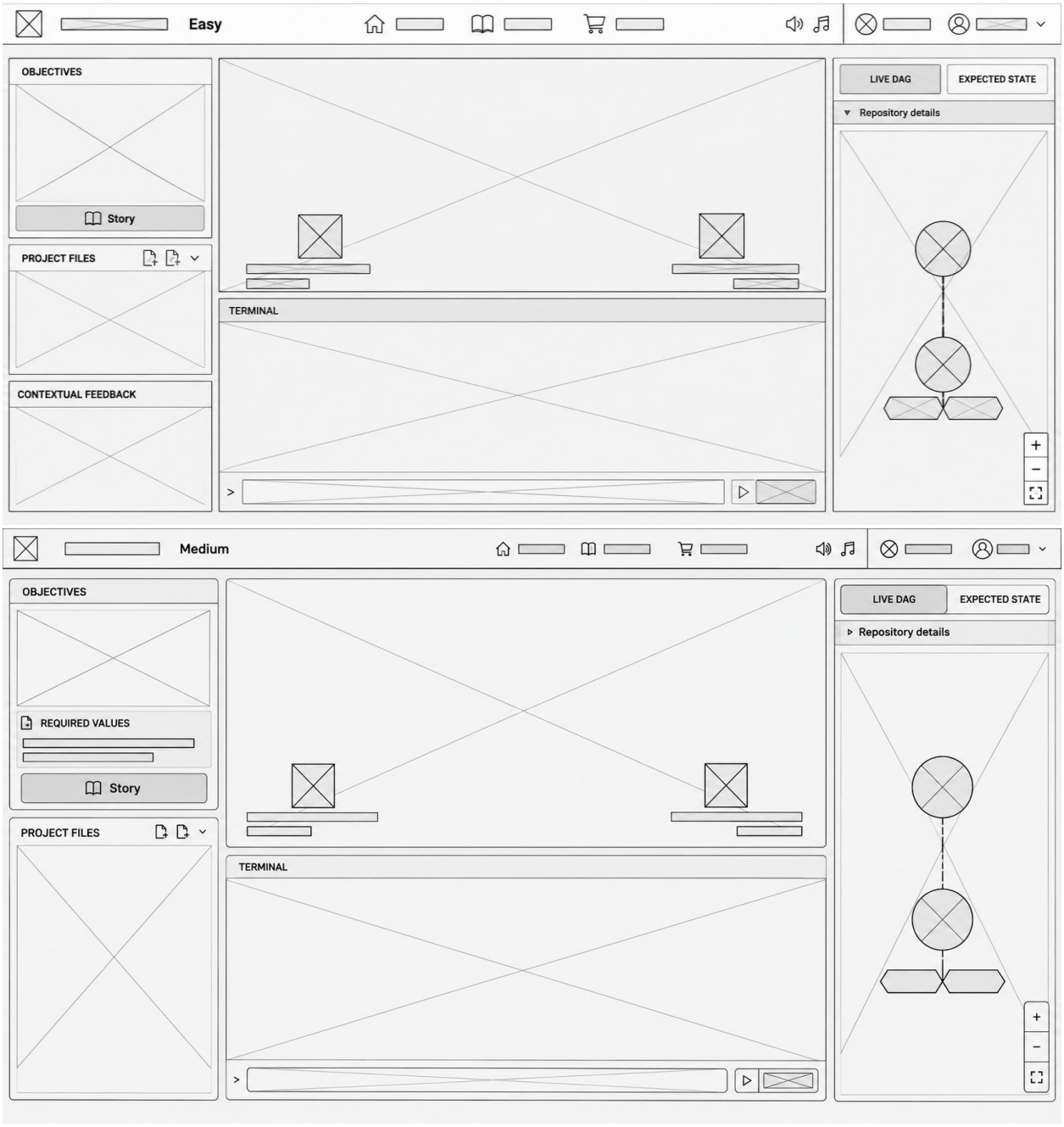
Activity Diagram

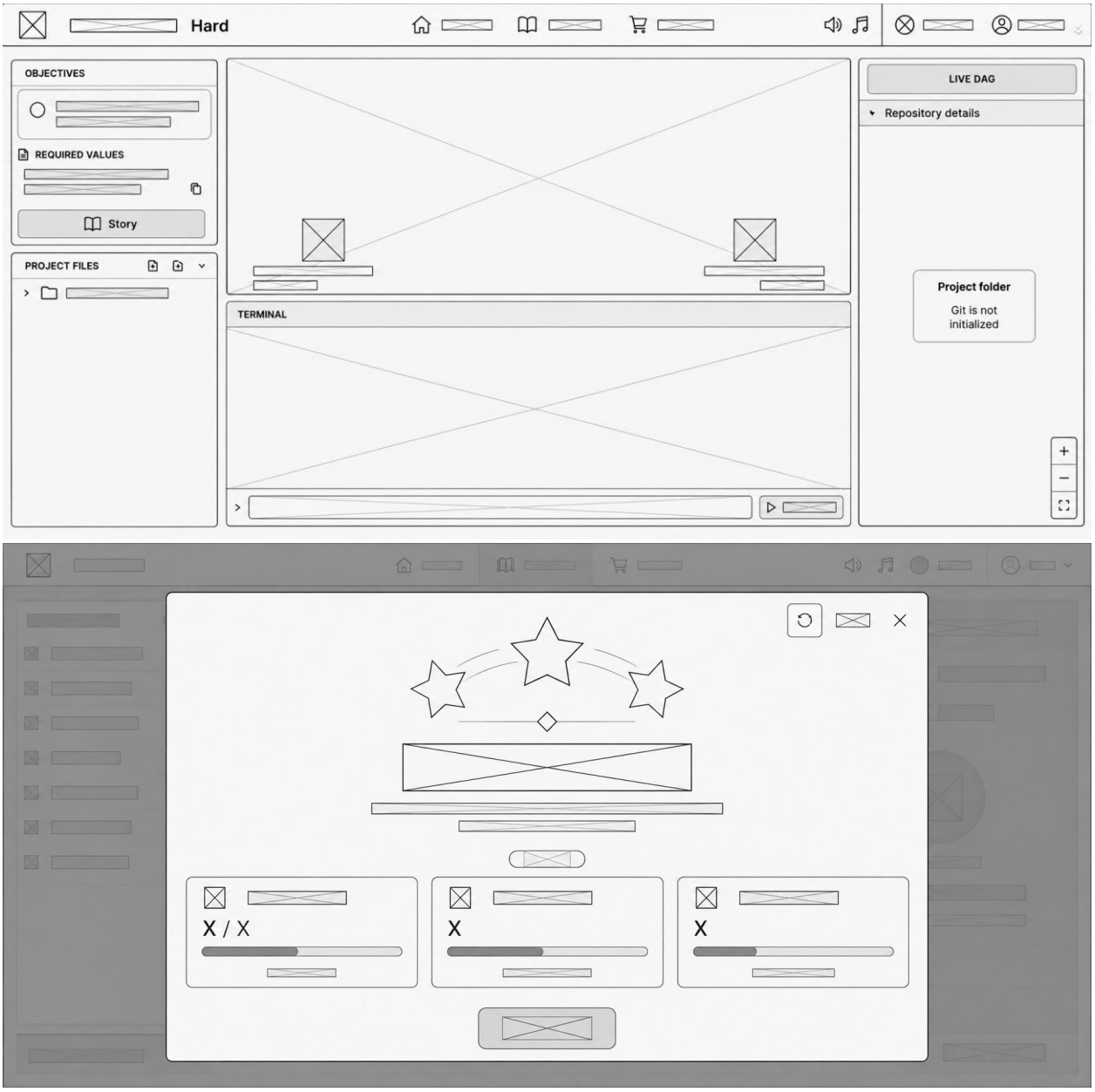


Wireframe



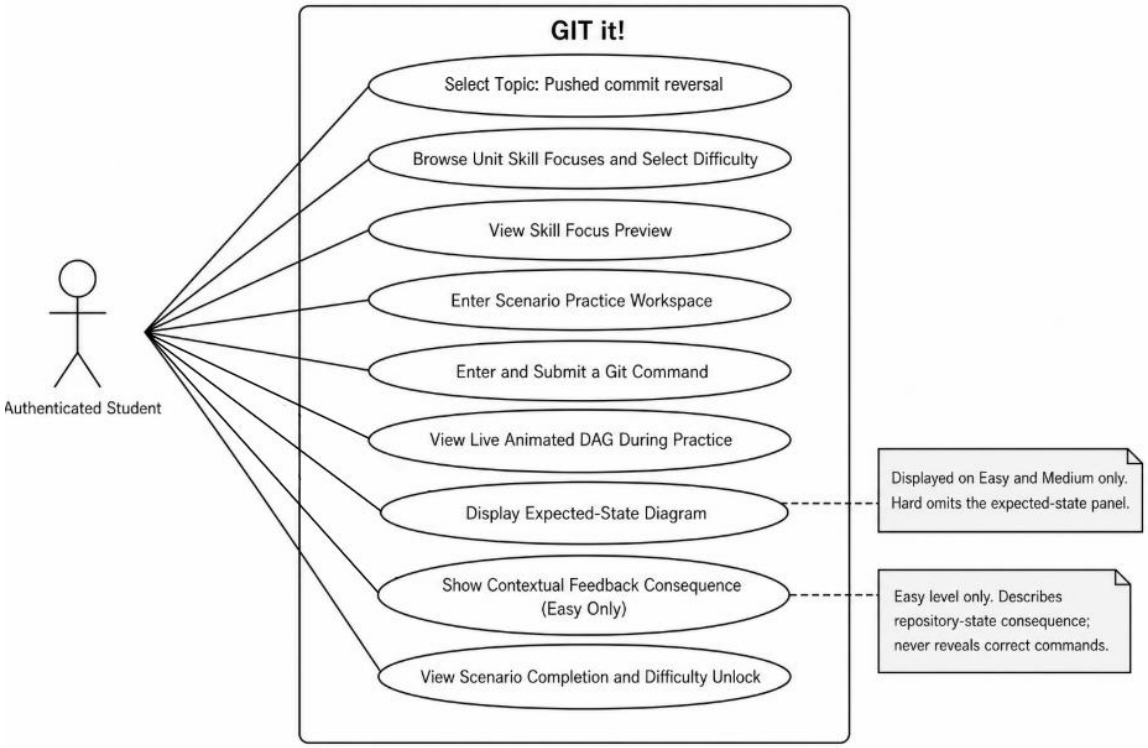



Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details



4.2 Practice Safely Reversing Pushed Commits

Use Case Diagram

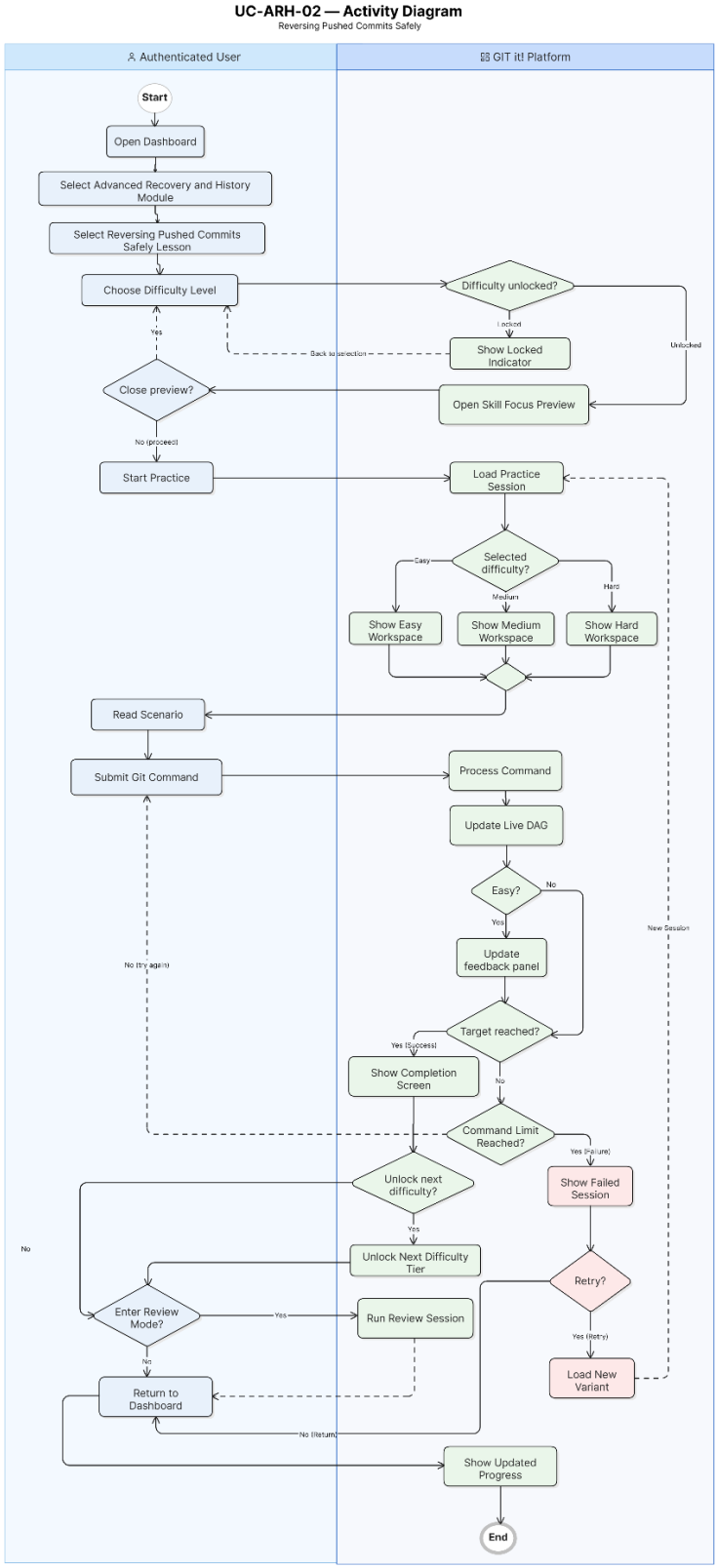


Use Case Description

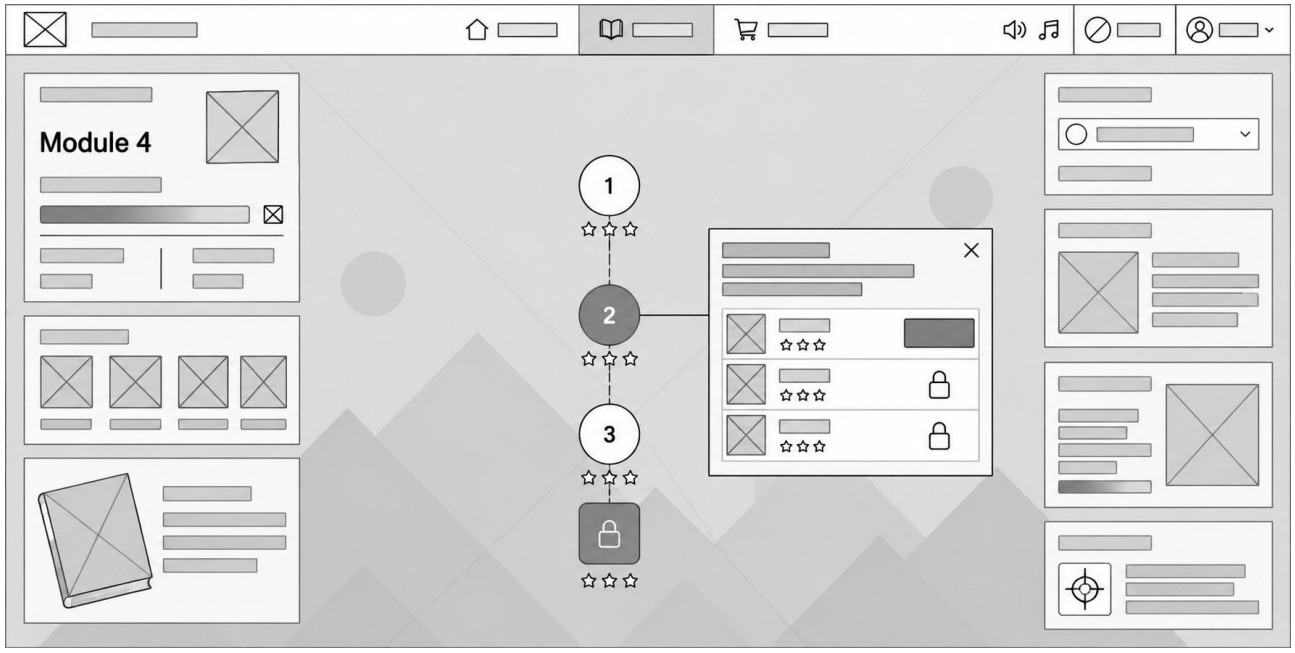
Field	Description
Use Case ID	UC-ARH-02
Use Case Name	Practice Safely Reversing Pushed Commits
Curriculum SO	SO 4.2
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Pushed commit reversal are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Pushed commit reversal.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing choosing revert vs reset when shared history must be preserved.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student

	checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced choosing revert vs reset when shared history must be preserved in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

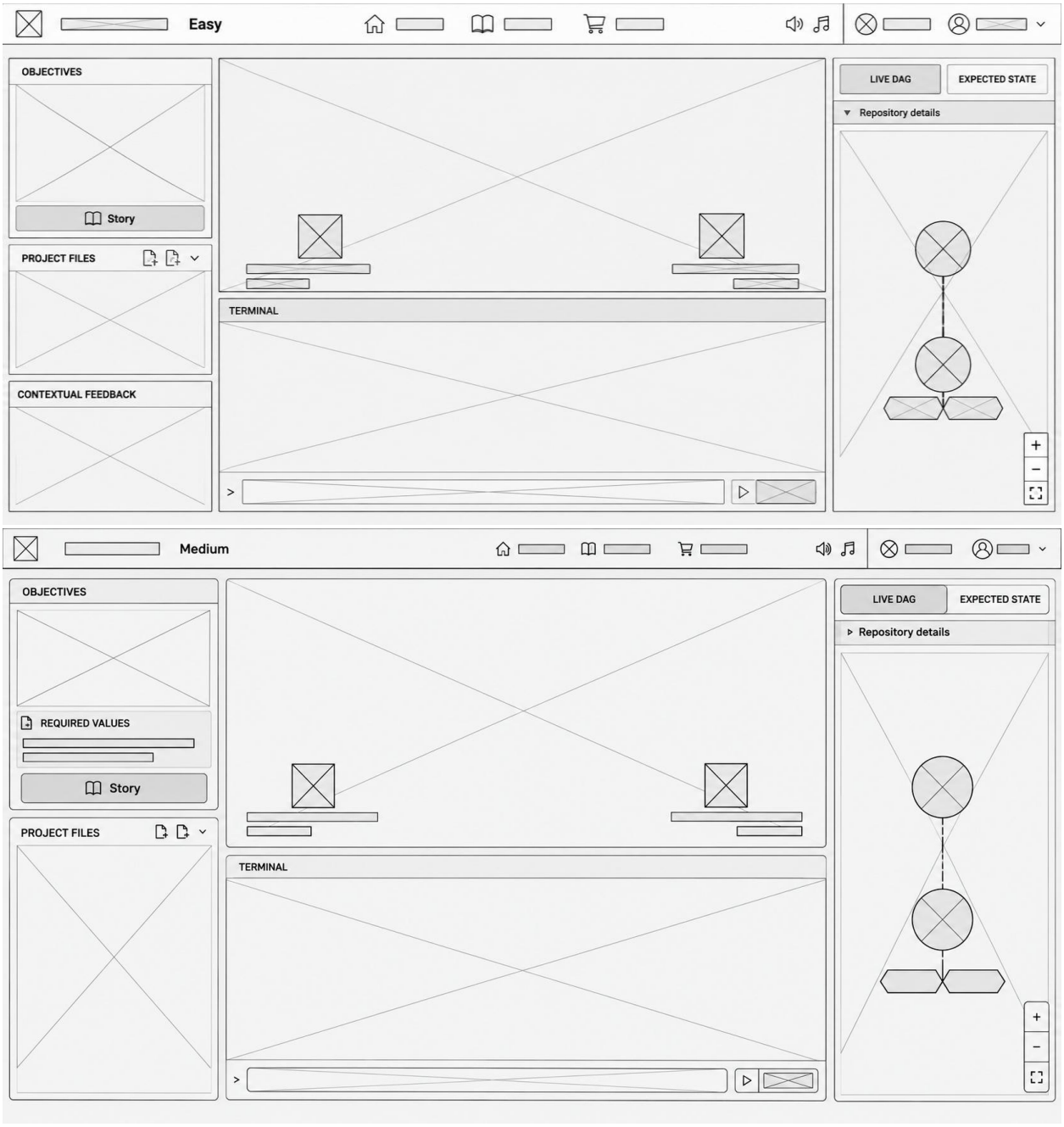
Activity Diagram

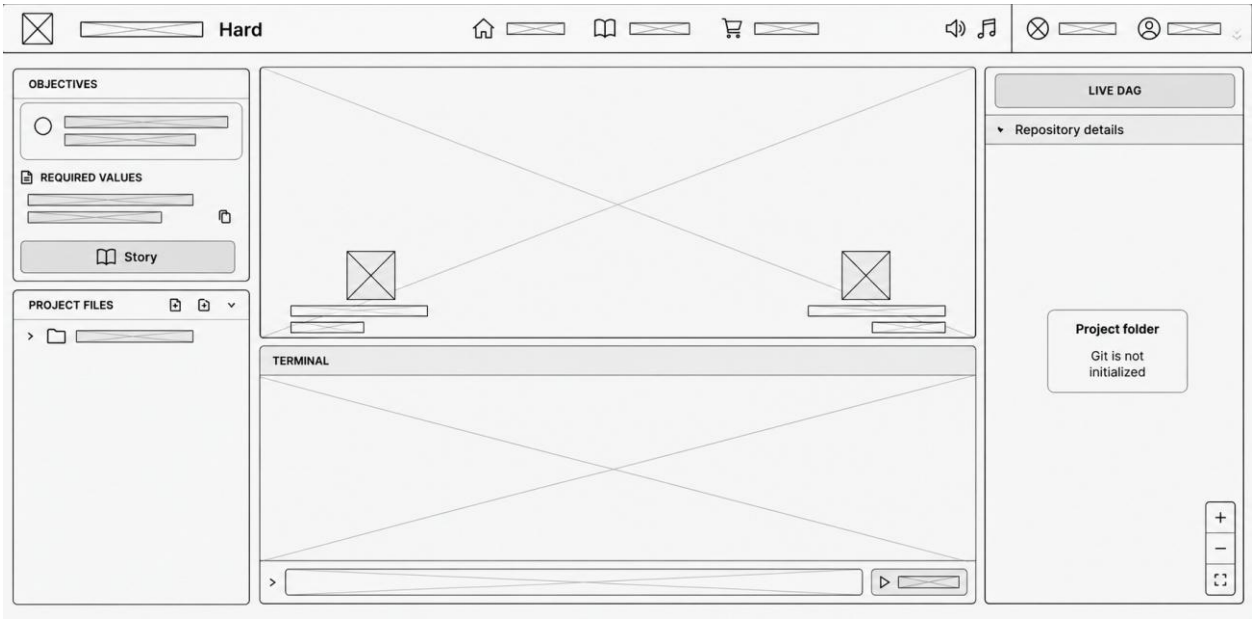


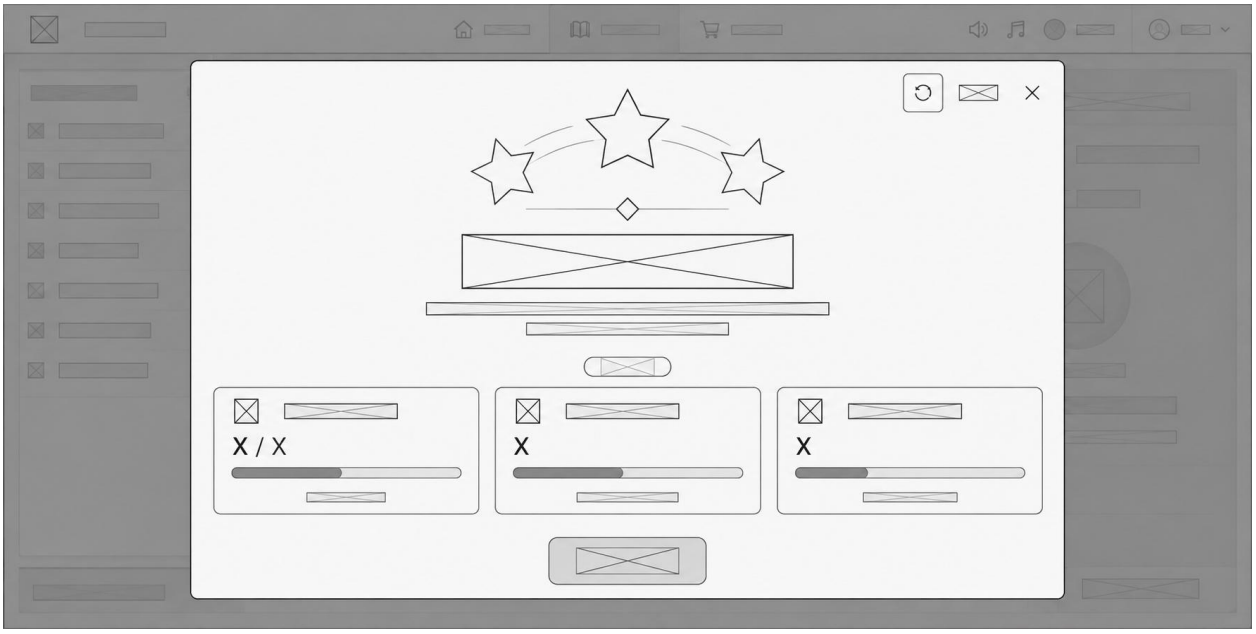
Wireframe





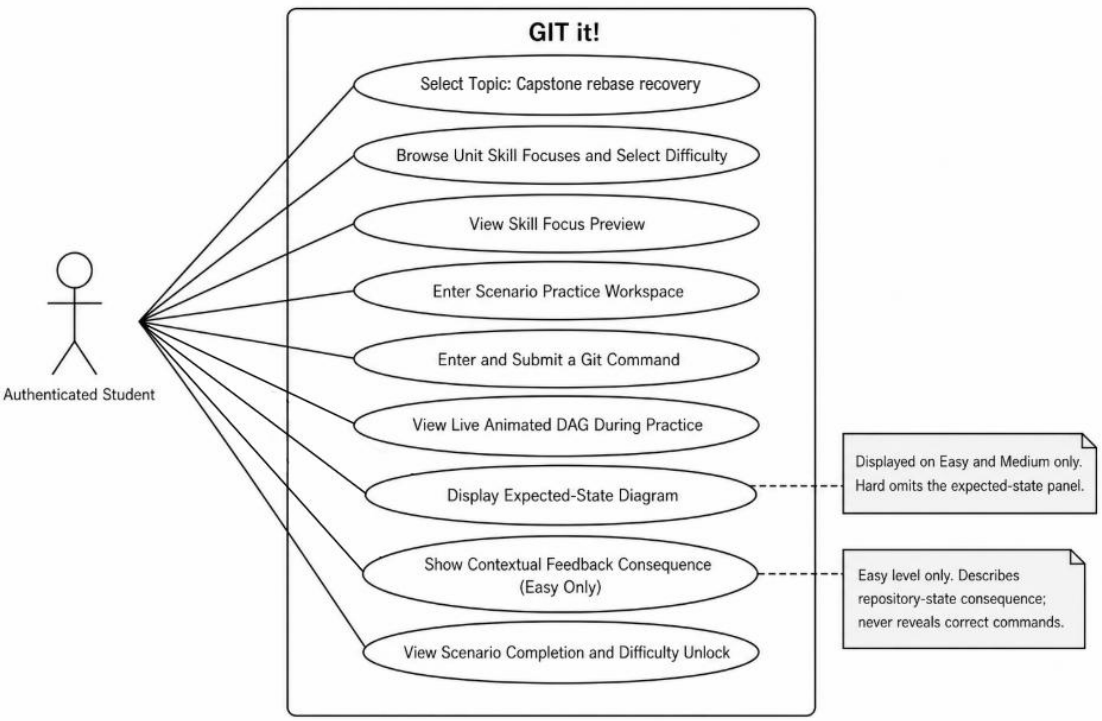

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

4.3 Practice Completing Rebase Recovery Sequences

Use Case Diagram

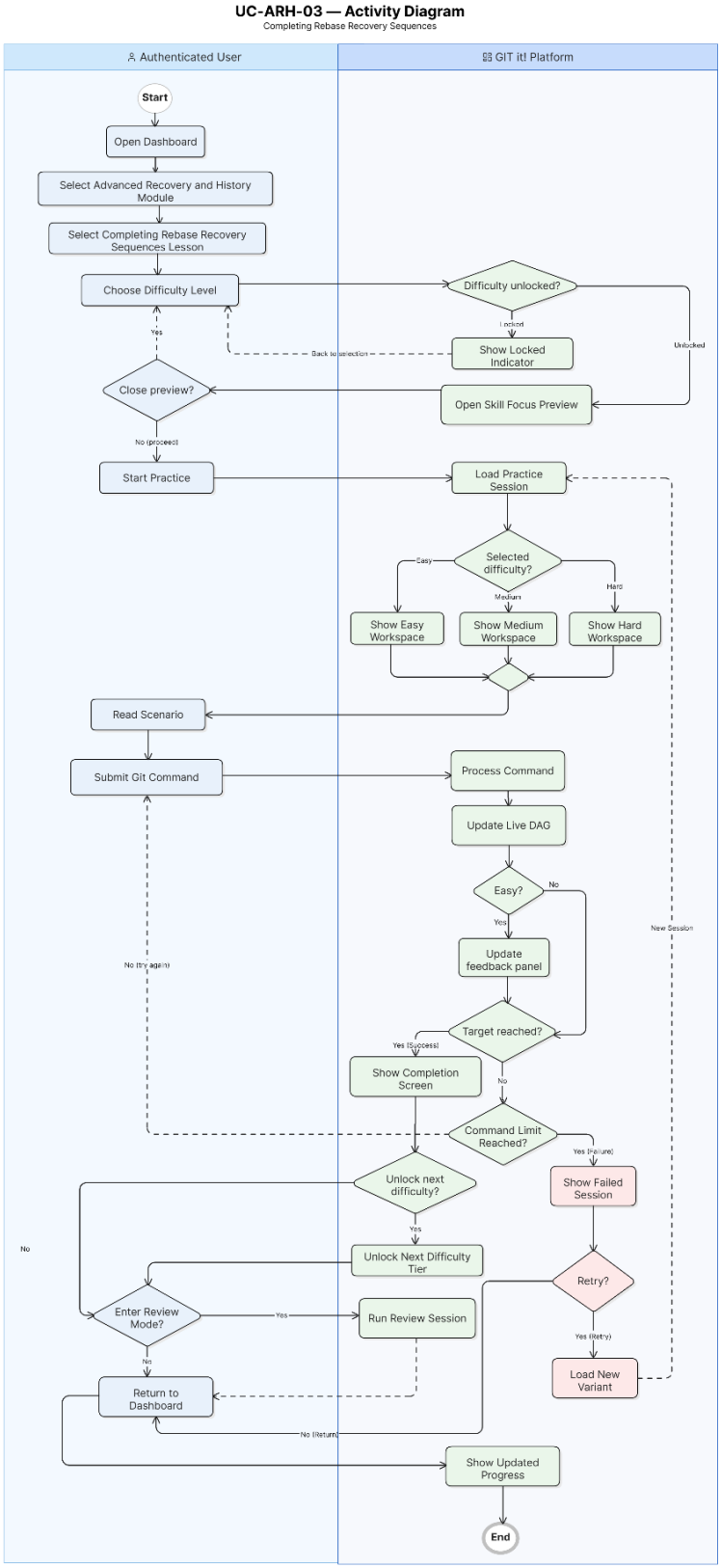


Use Case Description

Field	Description
Use Case ID	UC-ARH-03
Use Case Name	Practice Completing Rebase Recovery Sequences
Curriculum SO	SO 4.3
Actor(s)	Authenticated Student
Preconditions	Student is logged in. The chapter and learning objective for Capstone rebase recovery are published. At least one difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and opens the Stories and chapter map.2. Student expands the chapter entry for Local Repository Foundations, Branching and Collaboration, Conflict Resolution, or Advanced Recovery and History (as applicable).3. Student selects the level for Capstone rebase recovery.4. Student clicks Start, Continue, or Retry on an unlocked Easy, Medium, or Hard difficulty action.5. Student reads the mission briefing, which opens the first time the level is played and can be reopened from Story.6. System opens the level workspace on the tier's next uncleared scenario.7. Student reads the scenario narrative and task prompt describing full rebase recovery without dropping, duplicating, or orphaning commits.8. Student types a Git command in the terminal input and presses Enter.9. Student sees the command echoed in the terminal output area with simulator-style feedback.10. Student sees the repository graph update when the command changes repository state.11. On Easy: student views the target-state guidance and reads the Contextual Feedback panel after each processable command.12. On Medium: student

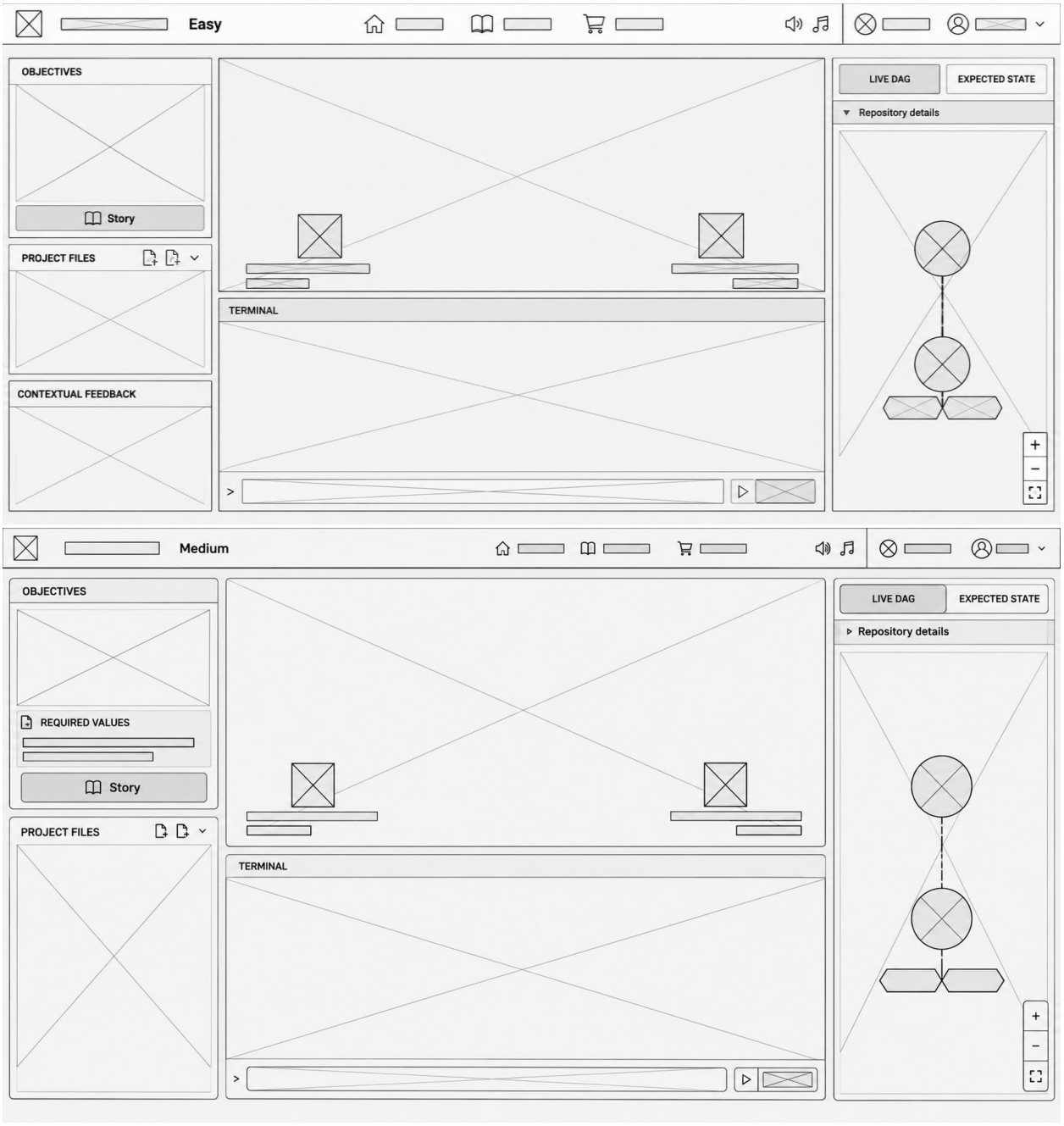
	views the target-state guidance; contextual feedback panel is not shown.13. On Hard: student relies on the narrative and Live DAG only (no target-state guidance).14. Student checks the counted-command allowance after counted action commands.15. Student repeats steps 8–14 until the practice goal is reached or the session ends.16. Student sees the completion screen (scenario name, difficulty, retries used, star status, unlock indicators) when the target repository state is reached; the screen does not reveal correct commands or sequences.
Postconditions	Student has practiced full rebase recovery without dropping, duplicating, or orphaning commits in the practice workspace. Completion status and difficulty unlock states are visible on the chapter entry when applicable.
Alternative Flows	AF1 — Locked difficulty: Student sees the Medium or Hard action disabled until the prior level is completed.AF2 — Target not yet reached: Student continues practicing from the current repository state shown in the Live DAG.AF3 — Counted-command limit reached: Student sees the session marked Failed and may click Retry or return to the chapter entry.AF4 — Student exits the run: the system asks for confirmation, ends the run and returns to the story map.AF5 — Unprocessable command: Student sees neutral terminal output; Live DAG unchanged; student tries another command.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19, FR-ACC-01, FR-ACC-02

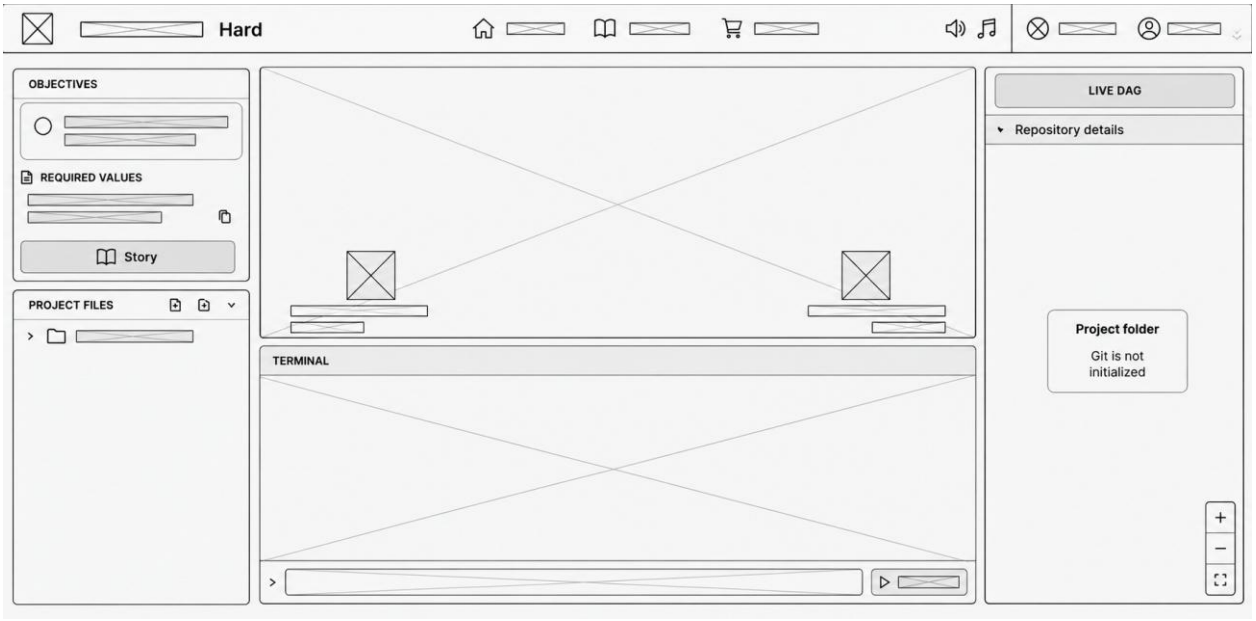
Activity Diagram

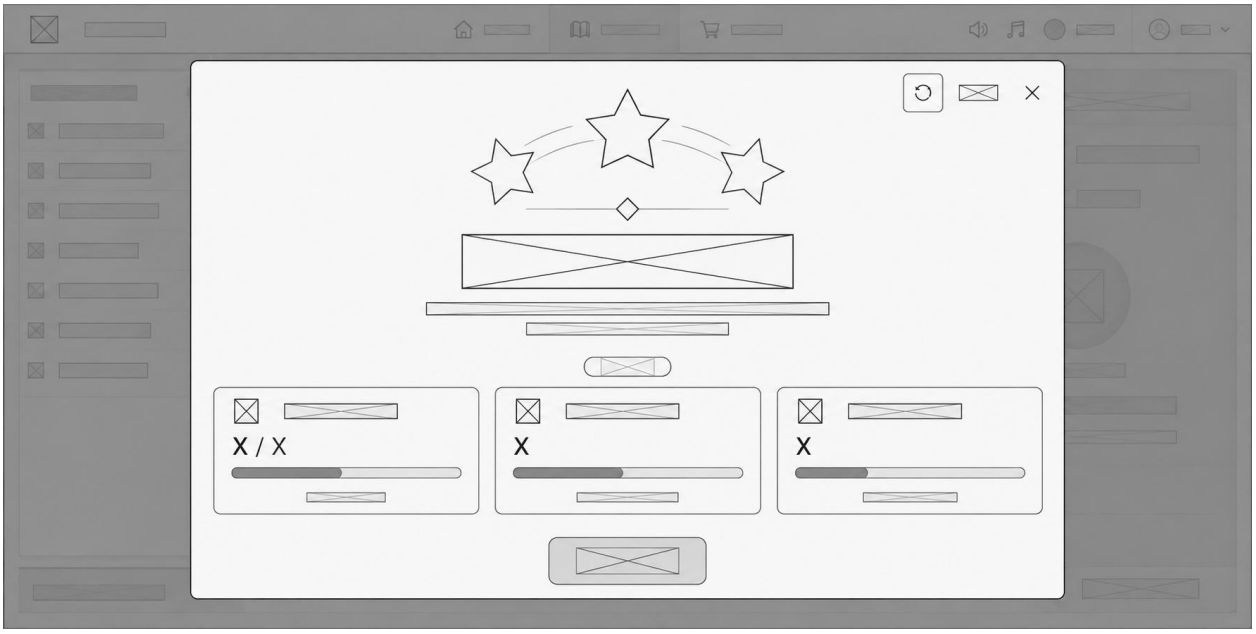


Wireframe



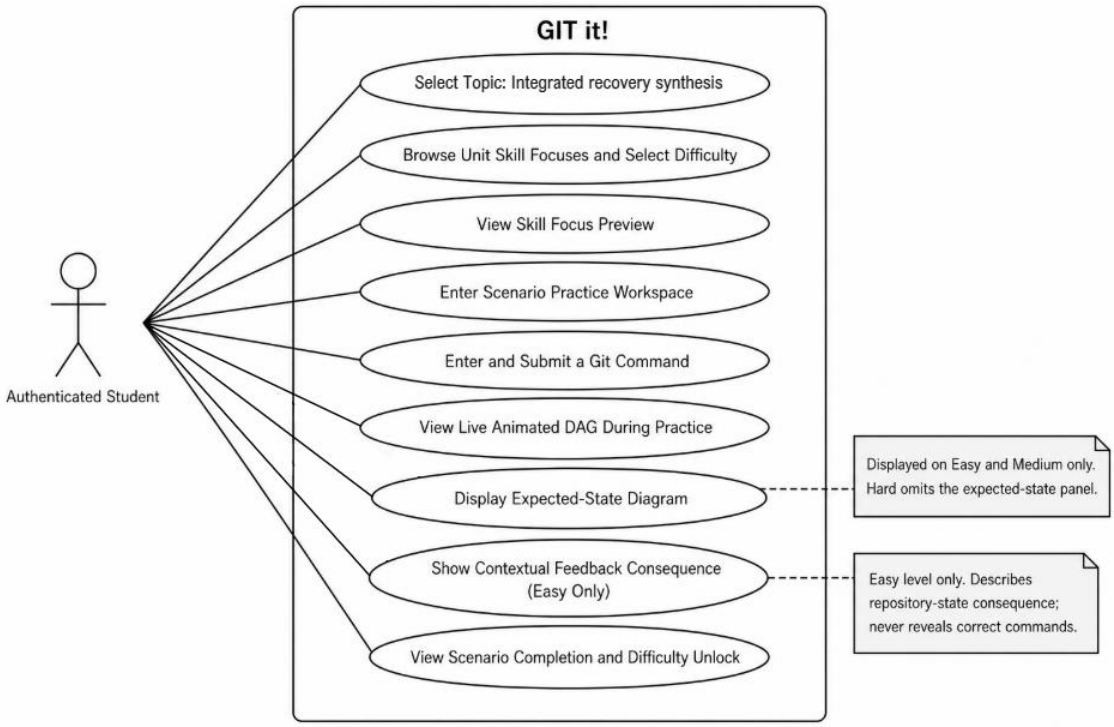

Easy
OBJECTIVES
Story
PROJECT FILES
CONTEXTUAL FEEDBACK
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details
Medium
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
EXPECTED STATE
Repository details


Hard
OBJECTIVES
REQUIRED VALUES
Story
PROJECT FILES
TERMINAL
LIVE DAG
Repository details
Project folder
Git is not initialized


X / X
X
X

4.4 Complete Hard-Level Independent Recovery Scenarios

Use Case Diagram

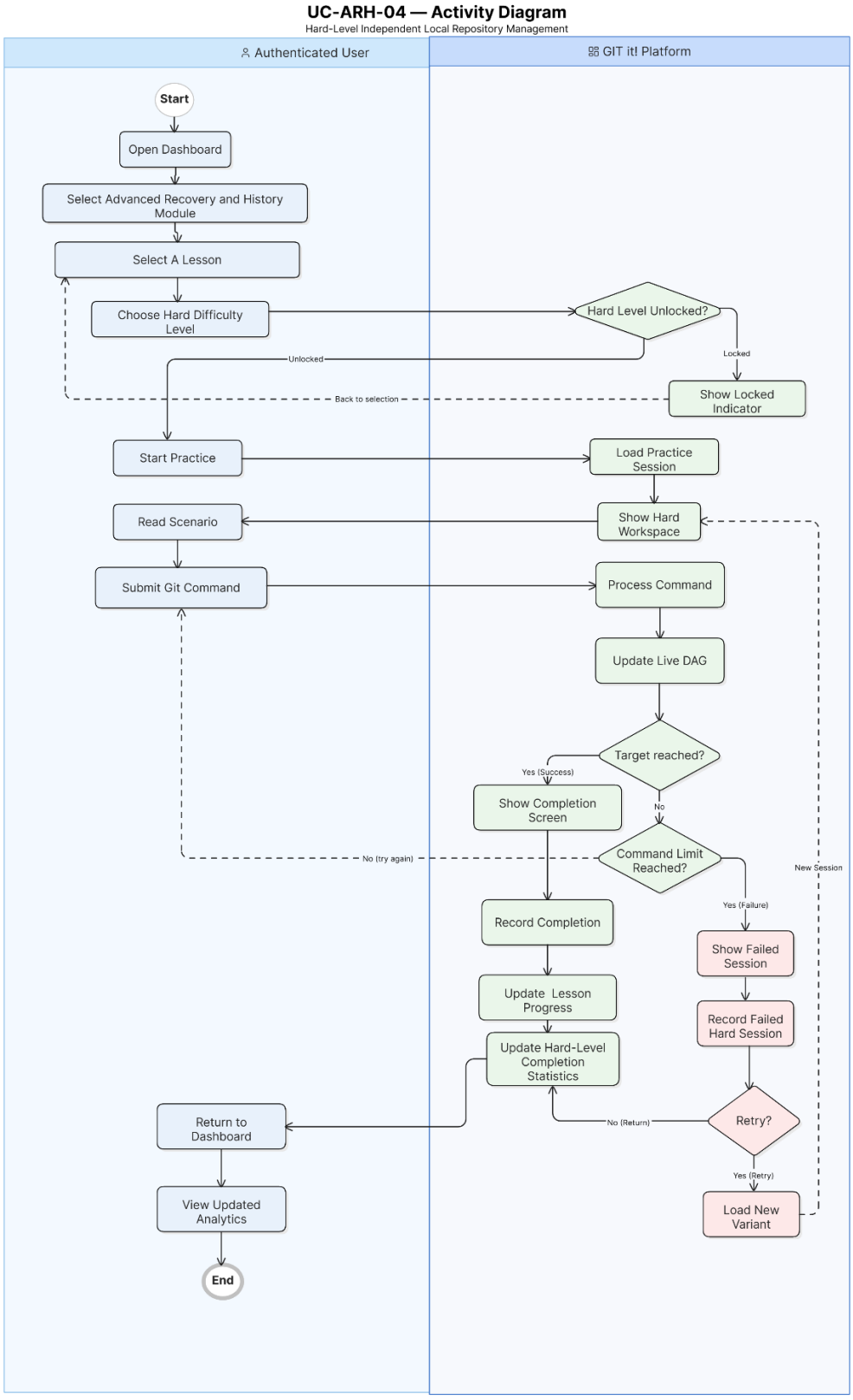


Use Case Description

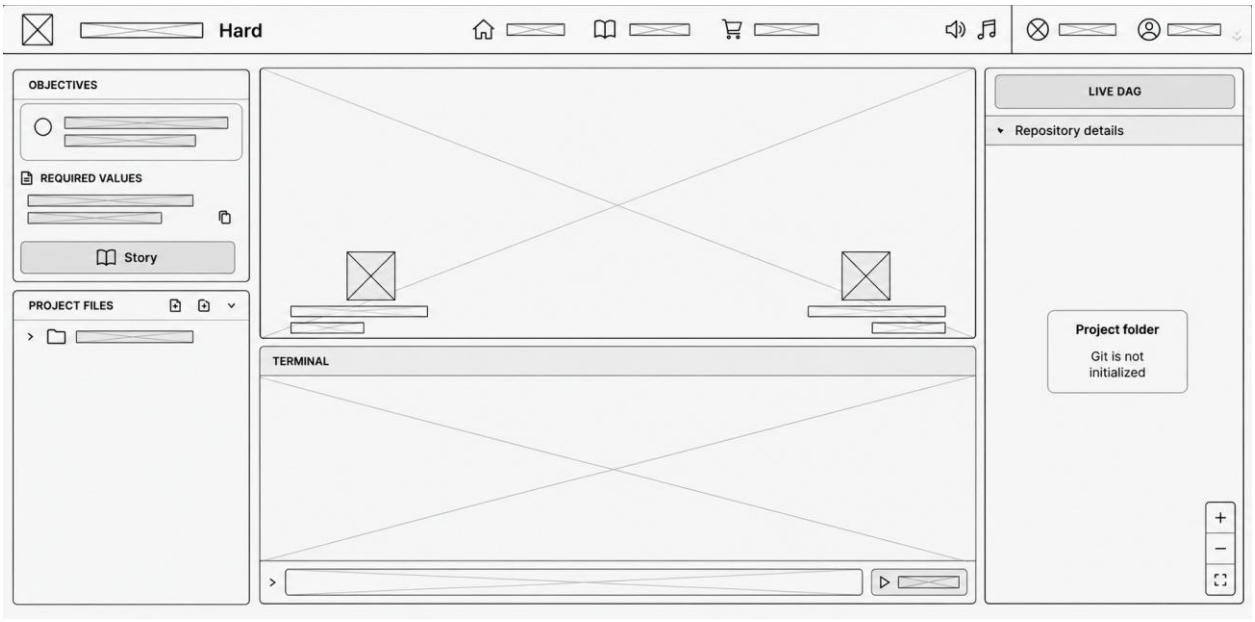
Field	Description
Use Case ID	UC-ARH-04
Use Case Name	Complete Hard-Level Independent Recovery Scenarios
Curriculum SO	SO 4.4
Actor(s)	Authenticated Student
Preconditions	Student is logged in. Medium difficulty is completed for the integrated skill focus in Advanced Recovery and History. Hard difficulty action is unlocked.
Main Flow	1. Student navigates to the Home and expands the Advanced Recovery and History chapter entry.2. Student selects the integrated learning objective for independent hard reset recovery, pushed-commit reversal, and rebase recovery.3. Student confirms Medium is completed and clicks Start or Retry on the unlocked Hard difficulty action.4. Student reads the mission briefing and starts the tier.5. Student reads the multi-step Hard-level narrative (no target-state guidance; Live DAG and task prompt only).6. Student types Git commands and observes terminal output and Live DAG updates across multiple repository-state steps.7. Student checks the counted-command allowance throughout the session.8. Student applies repository-state reasoning without expected-state scaffolding to reach each step goal.9. Student sees the completion screen when all required target states are reached, or clicks Retry after failure.10. Student sees updated Hard completion status on the learning objective card when successful.

Postconditions	Student has completed or attempted the Hard-level integrated scenario. HLCR-relevant completion status is visible on the dashboard and chapter entry.
Alternative Flows	AF1 — Hard locked: Student sees Hard disabled until Medium is completed.AF2 — Session failed: Student clicks Retry without automatically receiving a scenario-specific solution sequence.AF3 — Counted-command limit: Student sees Failed state and may retry or exit.
Functional Requirements	FR-DASH-15, FR-DASH-16, FR-DASH-17, FR-DASH-18, FR-DASH-19, FR-DASH-20, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-02, FR-SCENE-03, FR-SCENE-04, FR-SCENE-05, FR-SCENE-05A, FR-SCENE-05B, FR-SCENE-05C, FR-TERM-01, FR-TERM-02, FR-TERM-03, FR-TERM-04, FR-TERM-05, FR-TERM-06, FR-TERM-07, FR-TERM-08, FR-TERM-09, FR-SCENE-06, FR-SCENE-07, FR-SCENE-08, FR-SCENE-09, FR-SCENE-10, FR-SCENE-11, FR-SCENE-12, FR-SCENE-12A, FR-SCENE-12B, FR-SCENE-12C, FR-DAG-01, FR-DAG-02, FR-DAG-03, FR-DAG-04, FR-DAG-05, FR-DAG-06, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-17, FR-SCENE-18, FR-SCENE-19Note: Emphasizes FR-SCENE-03 (Hard scaffolding) and FR-DASH-07 (HLCR indicators).

Activity Diagram

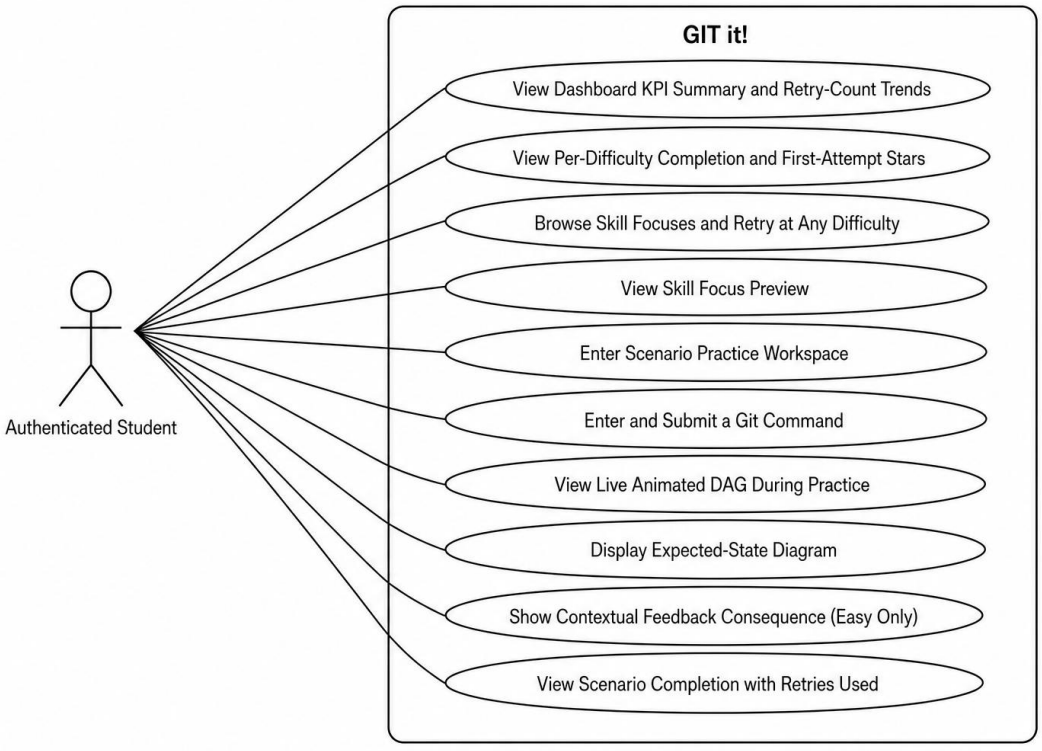


Wireframe



4.5 Retry Changed Variants and Monitor Recovery Transfer

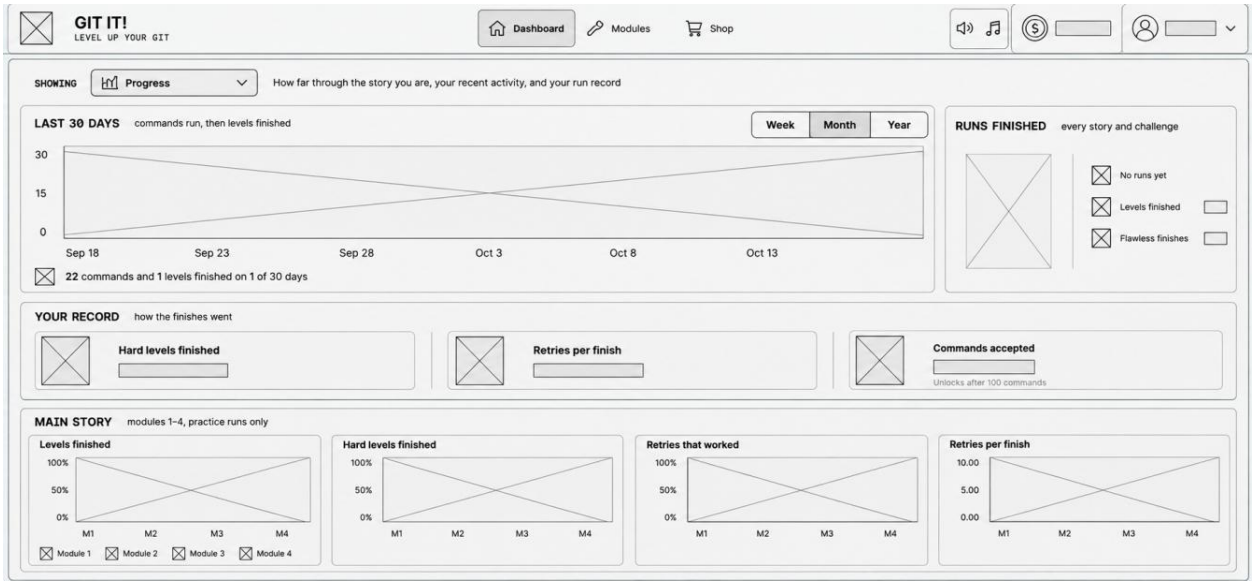
Use Case Diagram

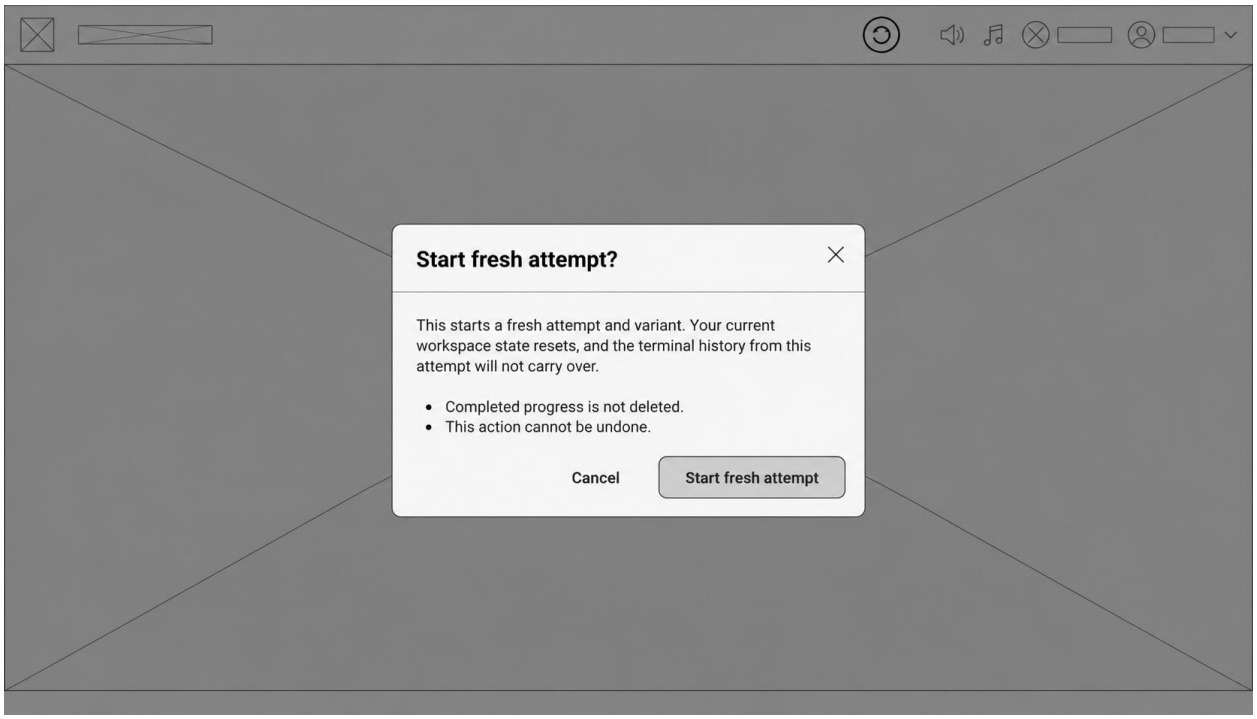


Use Case Description

Field	Description
Use Case ID	UC-ARH-05
Use Case Name	Retry Changed Variants and Monitor Recovery Transfer
Curriculum SO	SO 4.5
Actor(s)	Authenticated Student
Preconditions	Student is logged in and has a prior Failed or Abandoned session for the same learning objective and difficulty level.
Main Flow	1. Student completes or abandons a scenario session in Module 4 — Advanced Recovery and History without reaching all targets.2. Student clicks Retry on the Failed or Abandoned difficulty action.3. Student reads the mission briefing and starts the tier.4. Student reads a retry narrative with changed branch names, file context, or repository layout (structurally different variant).5. Student applies the same Git skill focus using consequence-aware reasoning rather than repeating prior commands blindly.6. Student types commands, sees terminal output and Live DAG updates, and checks the counted-command allowance.7. Student reaches the target repository state or chooses to abandon/retry again.8. Student views dashboard indicators reflecting practice progress (retry trends, KPI summary when data is available).
Postconditions	Student has attempted a retry with visibly changed scenario context. Dashboard reflects updated practice status; RTA is computed internally, not shown as a separate student action.
Alternative Flows	AF1 — First-ever attempt: Student practices without a prior Failed session (not RTA-eligible). AF2 — Student completes changed variant on first retry try: Transfer practice succeeded from student view. AF3 — Student abandons retry: Card shows Abandoned; student may retry again.
Functional Requirements	FR-VAR-01, FR-VAR-02, FR-VAR-03, FR-VAR-04, FR-DASH-05, FR-DASH-07, FR-DASH-17, FR-DASH-19, FR-PREVIEW-02, FR-PREVIEW-03, FR-PREVIEW-07, FR-SCENE-01, FR-SCENE-17, FR-SCENE-13, FR-SCENE-14, FR-SCENE-15, FR-SCENE-18, FR-SCENE-19

Wireframe





Cross-Cutting Functional Requirements

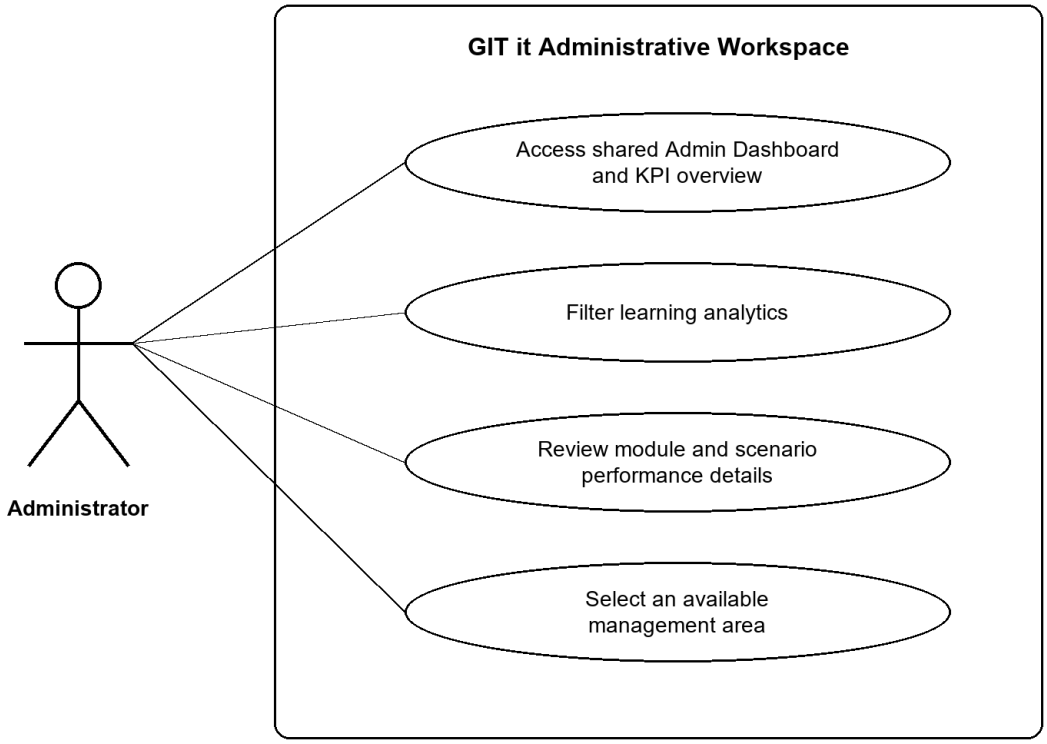
Requirements in this section apply to all student-facing interface surfaces across the entire platform and are not specific to any single module or use case.

FR ID	Summary	Requirement Statement
FR-ACC-01	Keyboard accessibility	The system shall ensure that all primary student actions — including page navigation, orientation lesson steps, tier selection, command submission, command guide and the AI tutor interactions, dialogs, and Squire's Drill — are operable via keyboard input alone, without requiring a pointer device.
FR-ACC-02	Supported screen sizes	The system shall render all primary interface components — including the landing page, Home, story map, orientation lessons, level workspace, shop and settings — without horizontal scrolling from a 390 × 844 phone screen up to desktop sizes.

Administrative Management

AM.1 Access Admin Dashboard and Review Learning Analytics

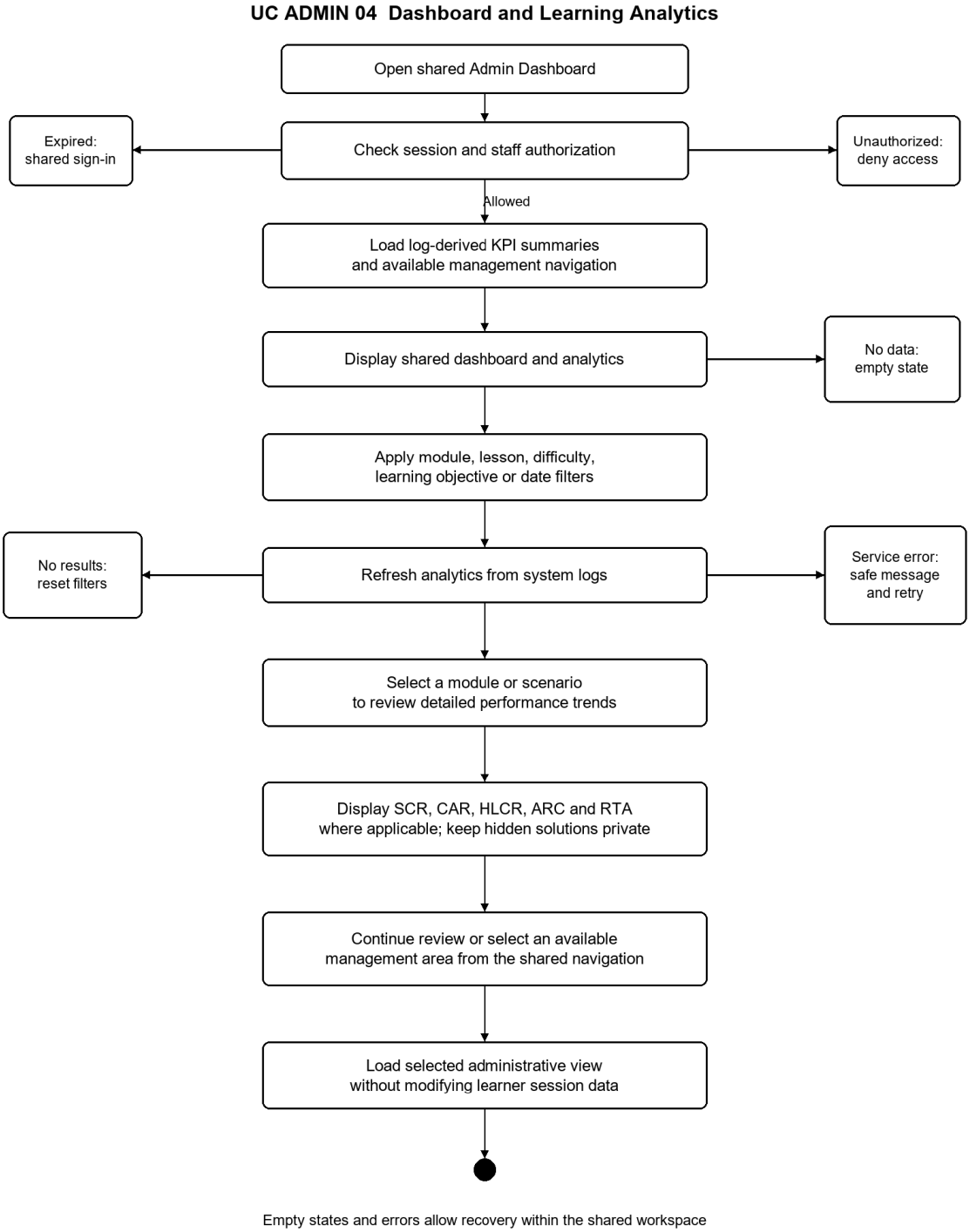
Use Case Diagram



Use Case Description

Field	Description
Use Case ID	UC-ADMIN-04
Use Case Name	Access Admin Dashboard and Review Learning Analytics
Actor(s)	Administrator
Preconditions	Administrator is authenticated through shared sign-in with an active staff-authorized session and analytics permission. Analytics are derived from available session, step, completion, and progress logs; missing data is handled through an empty state.
Main Flow	1. Administrator opens the shared Admin Dashboard from the navigation menu. 2. System verifies staff authorization and displays the dashboard with its KPI overview and available management navigation. Learning analytics are reviewed within this shared interface. 3. System displays module-level and scenario-level KPI summaries, including SCR, CAR, HLCR, ARC, and RTA where applicable. 4. Administrator sets a date range and opens a module's detail. 5. System updates the analytics view using system-generated logs. 6. Administrator selects a module or scenario to view detailed performance trends within the shared workspace. 7. System displays log-derived summaries without exposing hidden solution commands or correct command sequences. 8. Administrator continues the review or selects an available management area from the shared navigation; the system loads the selected administrative view.
Postconditions	Administrator has accessed the shared administrative workspace and reviewed available learning analytics. No student scenario session data is modified by dashboard access or analytics review.
Alternative Flows	AF1 - Unauthorized role or missing analytics permission: A learner opening /admin is returned to Home, and staff APIs return HTTP 403. AF2 - Expired session: System redirects the staff member to the shared sign-in page. AF3 - No data available: System displays an empty-state message within the shared dashboard. AF4 - Filters return no results: System displays a no-results message and allows filter reset. AF5 - Analytics service error: System displays an error message without internal stack traces and allows the administrator to retry.
Functional Requirements	FR-ADMIN-01, FR-ADMIN-02, FR-ADMIN-03, FR-ADMIN-17, FR-ADMIN-18, FR-ADMIN-19, FR-ADMIN-20, FR-ADMIN-21, FR-ADMIN-22, FR-ADMIN-23, FR-ADMIN-24

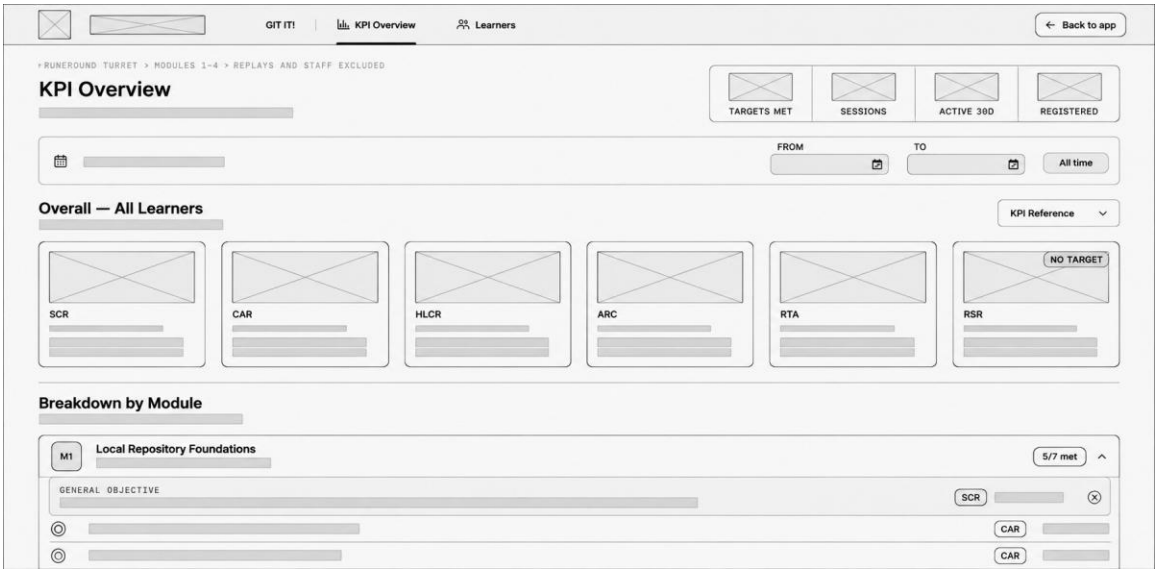
Activity Diagram



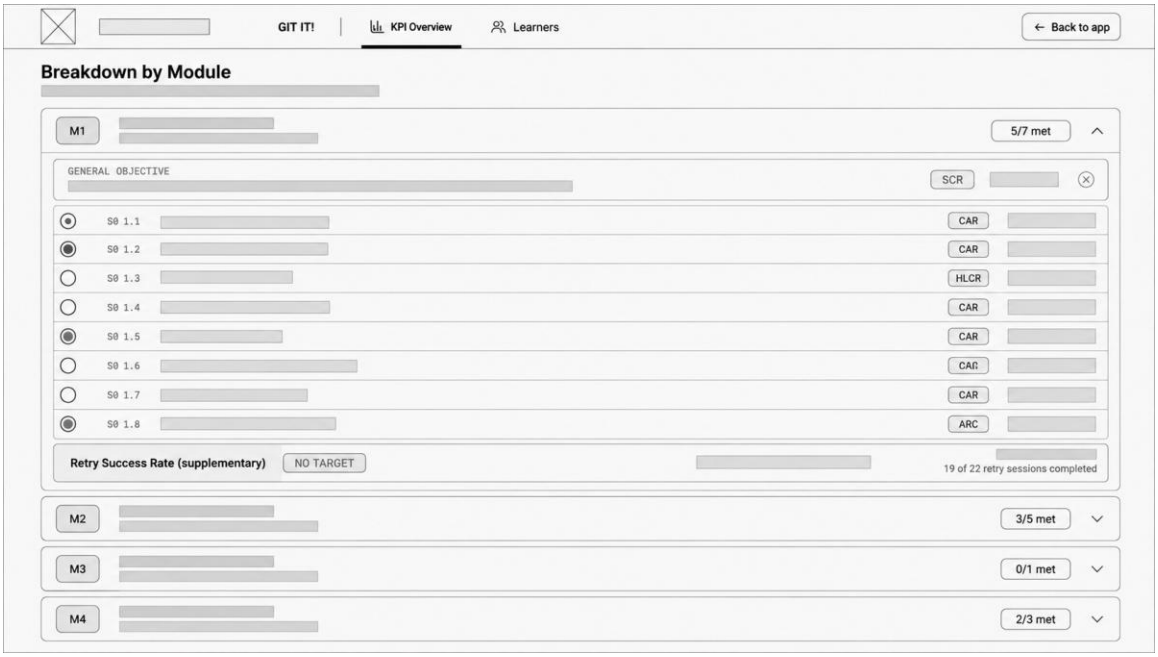
Wireframe

The following views show the shared dashboard and its distinct review states. KPI summaries, filters, and module or scenario details belong to the same administrative workspace.

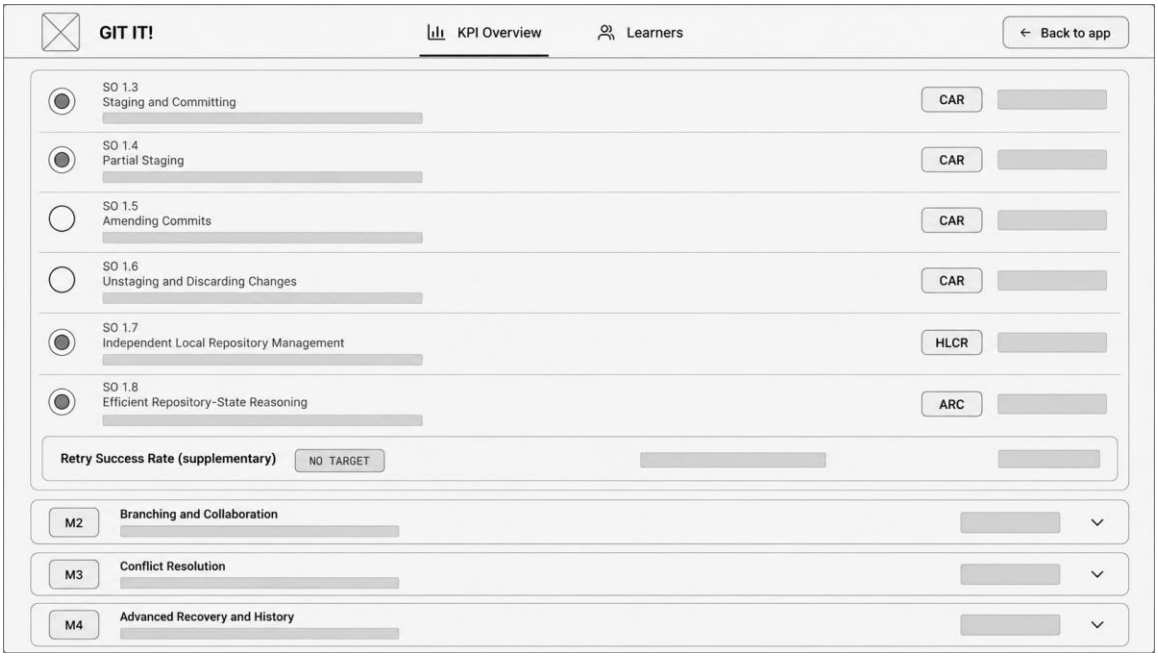
Shared dashboard with KPI overview and date filters



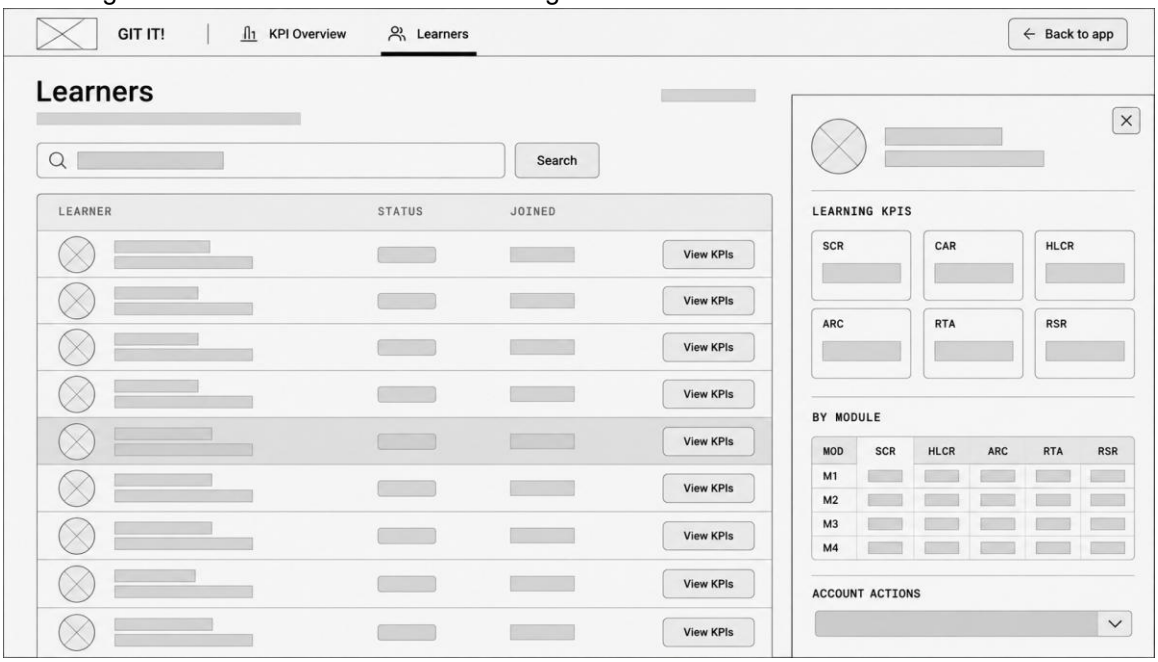
Expanded module and scenario performance details



Additional module and learning objective detail



Learner management reached from the shared navigation



AI-Assisted Learning Support

AI.1 Query AI Support

Use Case Diagram

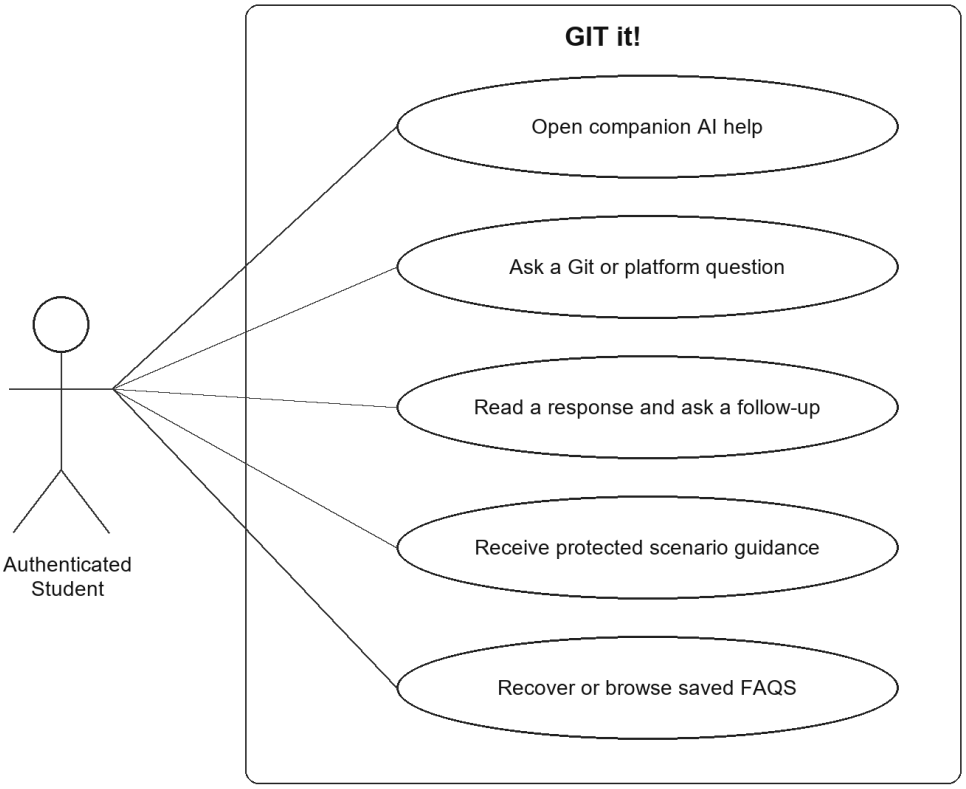


Figure AI-SRS-1. UC-AI-01 Query AI Support. The student uses general help or a signed command guide. Protected responses and recovery belong to the same use case and are detailed in its alternative flows.

Use Case Description

Field	Description
Use Case ID	UC-AI-01
Use Case Name	Query AI Support
Actor(s)	Authenticated Student
Preconditions	The student is signed in. General help is opened from Home, the story map, or the shop. For command-guide help, a guide context is available for a run owned by the student, with a valid signed guide token. An active practice session does not prohibit command-guide help.
Main Flow	1. The student opens the equipped companion's AI chat from a general-help page or the workspace command guide. 2. The system shows the chat, suggested topics, and any conversation retained for that context. The student enters a Git, version-control, platform, or command-guide question. 3. The client sends the question and bounded conversation history with the appropriate context to the authenticated tutor service. For guide help, the server verifies run ownership and the signed guide context. 4. The service checks learner messages for scenario content. A recognized scenario on Home follows Alternative Flow A1; uncertain scenario content and all command-guide help use protected conceptual teaching.

	5. The service obtains a reusable validated reply or requests a provider response. Protected replies are checked for scenario-answer disclosure before display. 6. The system displays the Git explanation or protected guidance. The student may ask a follow-up question, select a suggested topic, browse FAQS, or start a new chat.
Postconditions	The student receives a general explanation, protected conceptual guidance, or an explicit recovery state. Successful exchanges remain in the browser's bounded conversation context. Chat does not execute Git commands or change repository state, run completion, mastery, or rewards.
Alternative Flows	A1 Recognized scenario on Home: The system declines to solve the task without requesting a generated reply and directs the student toward concepts or command-guide help. Declined scenario turns are excluded from later provider conversation context. A2 Protected content: For uncertain matches and command-guide help, the tutor explains concepts and syntax without supplying task-specific answers. If a draft exposes protected answer content, the system regenerates it once; repeated disclosure produces fixed conceptual guidance. A3 Provider or connection failure: The system retains the question and previous successful exchanges, explains offline, timeout, rate-limit, or unavailable status, and offers retry or FAQS where appropriate. Rate-limited retries wait for the cooldown; failed exchanges are not added to later history. A4 Invalid context or input: The system rejects invalid requests or guide context. The student reopens the guide, shortens the question or starts a new chat for history limits, or signs in again when authentication expires.
Functional Requirements	FR-AI-01: The system shall provide authenticated, companion-based AI support on Home, the story map, and the shop, and within command-guide context for learner-owned runs. It shall support Git and version-control questions, verified platform explanations, suggested topics, and conversational follow-ups. Questions shall contain 1 to 4,000 characters; supplied history shall contain at most 20 turns, 6,000 characters per turn, and 32,000 characters in total. FR-AI-02: The system shall recognize scenario-related learner content, decline recognized scenario tasks on Home, and apply protected conceptual teaching to uncertain matches and command-guide help. It shall verify run ownership and signed guide context, instruct providers against scenario-specific solutions, and check protected replies against server-side answer data before display. Detected leakage shall trigger at most one regeneration followed by conceptual fallback if leakage persists. FR-AI-03: The system shall handle offline state, rate limits, timeout, provider unavailability, invalid guide context, and authentication or validation errors through explicit messages and appropriate recovery actions. It shall preserve the question and prior successful conversation on failure, enforce retry cooldowns and bounded request timing, and allow retry or FAQS when appropriate. Chat shall not execute commands or modify assessed progress.

Activity Diagram

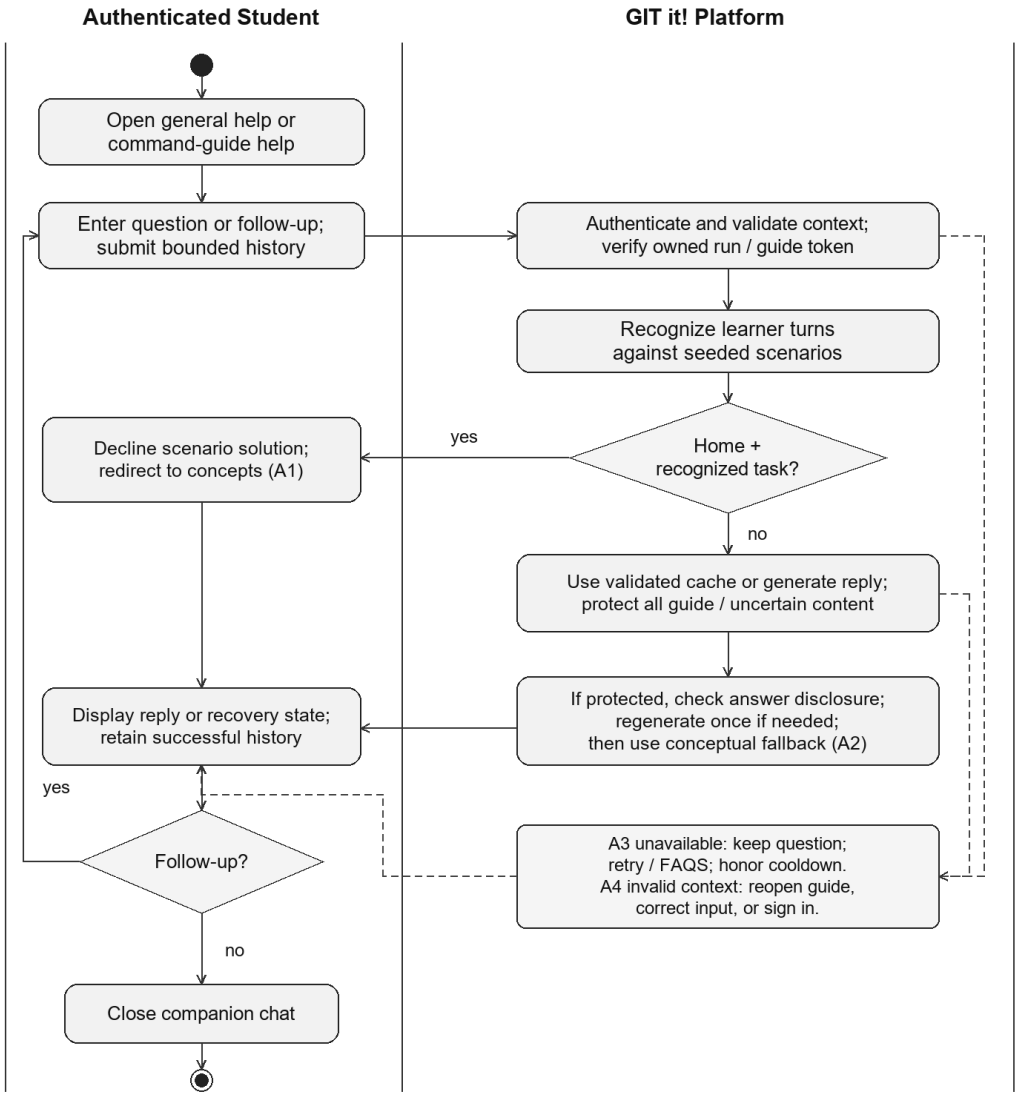


Figure AI-SRS-2. UC-AI-01 activity flow. A1 through A4 correspond to the use-case alternative flows. Dashed paths indicate failure recovery; recognized Home tasks are declined without requesting a generated reply. The student can continue with follow-up questions or close chat.

Wireframe

General companion help on Home

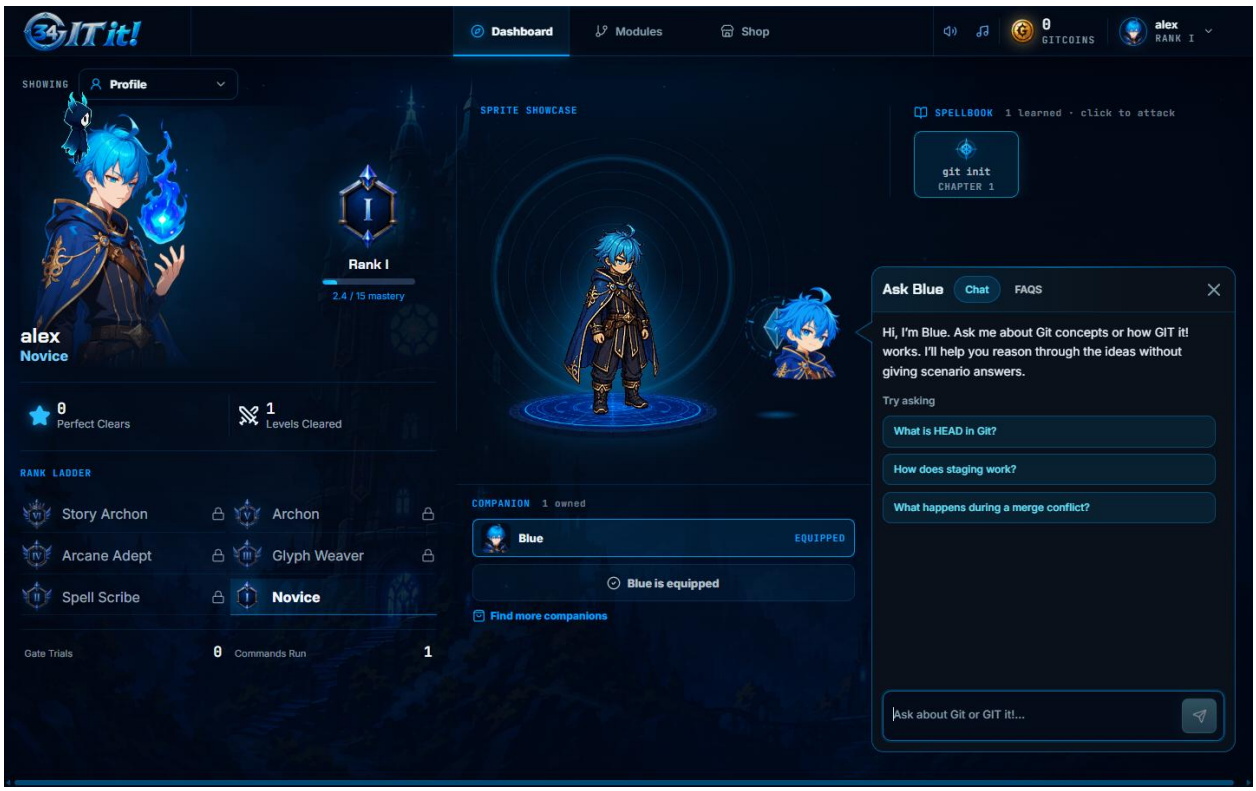


Figure AI-SRS-3. General companion help on Home. The same general-help context is used on the story map and shop. The student can select a suggested question, type a question or follow-up, switch to FAQS, start a new chat, or close the movable window.

Command guide help

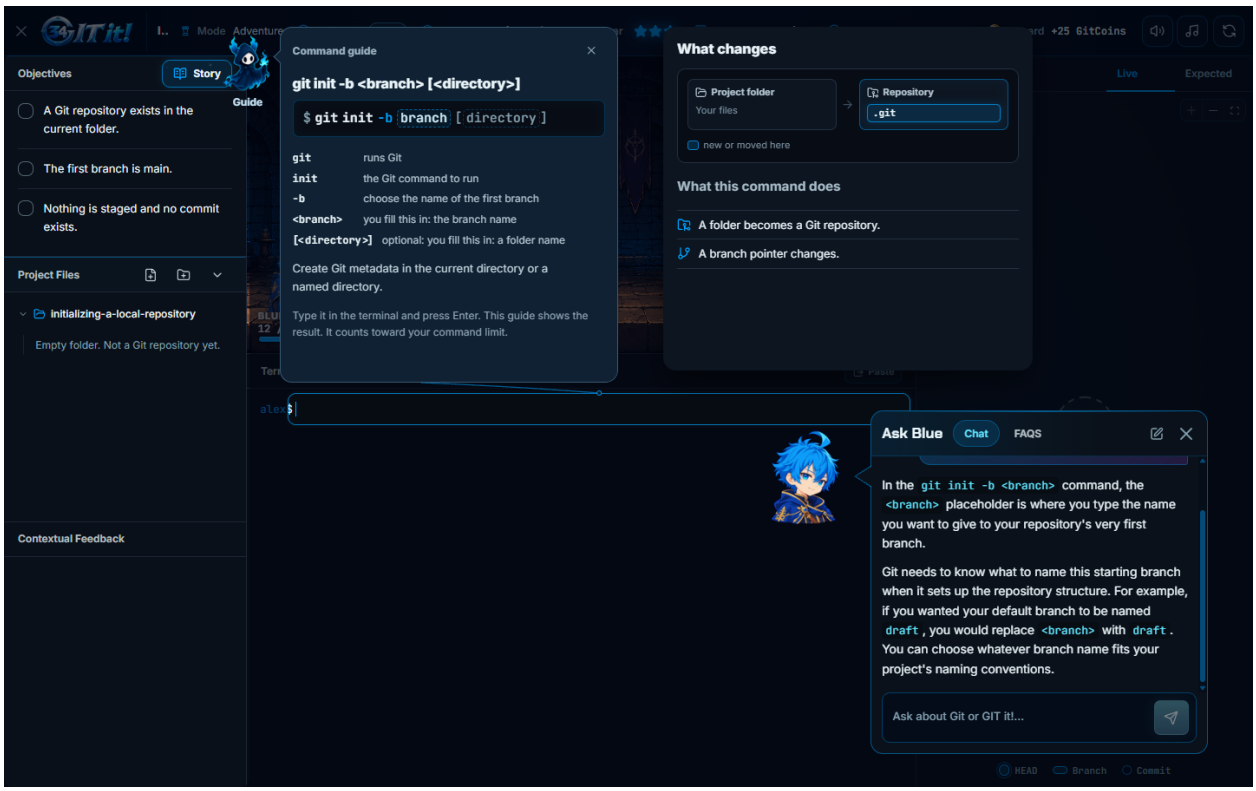


Figure AI-SRS-4. Help alongside an available command guide. Syntax uses placeholders; the tutor explains concepts without choosing the task values or solving the scenario. The guide and chat are separate windows, and chat does not execute the simulated terminal.

AI recovery

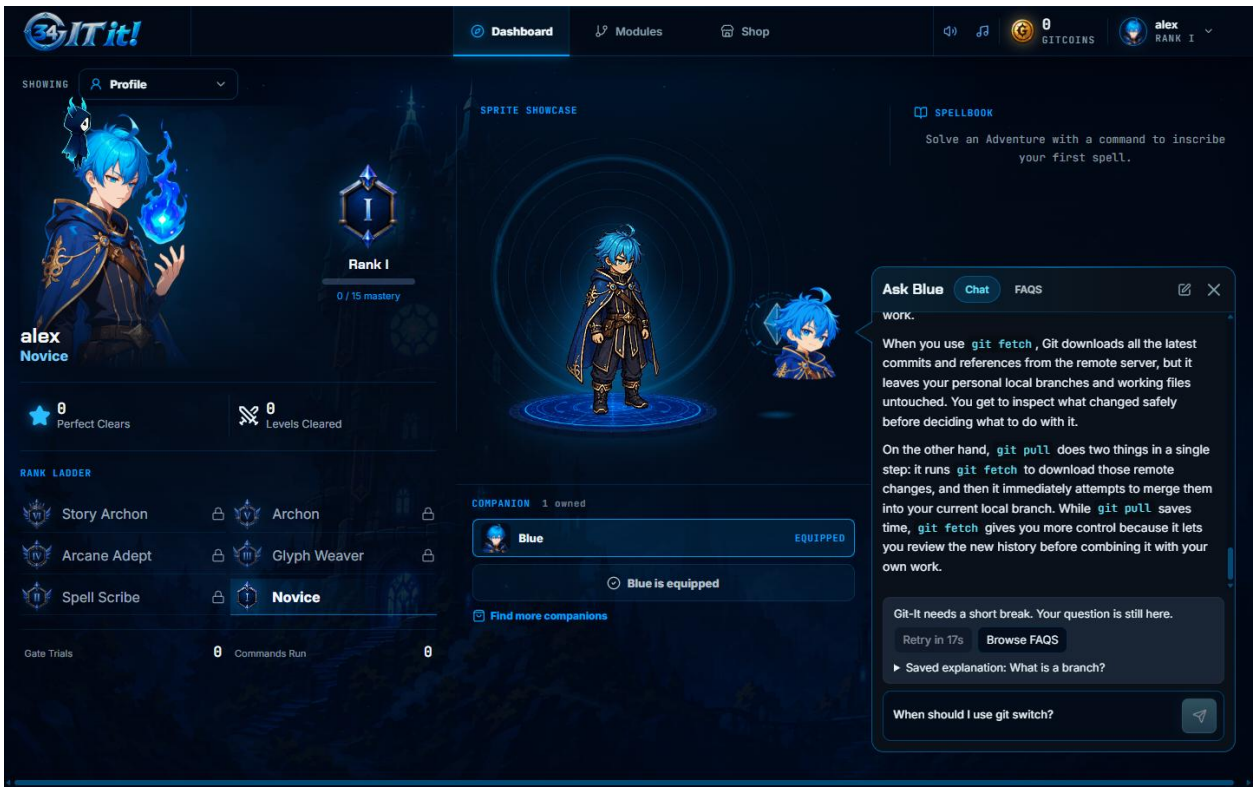


Figure AI-SRS-5. Rate-limit recovery example. The question and earlier successful exchanges remain available. Retry is disabled during cooldown; Browse FAQS opens saved explanations. Offline, timeout, and provider-unavailable states use corresponding messages. Invalid context directs the student to reopen the guide or sign in as applicable.

Saved FAQS

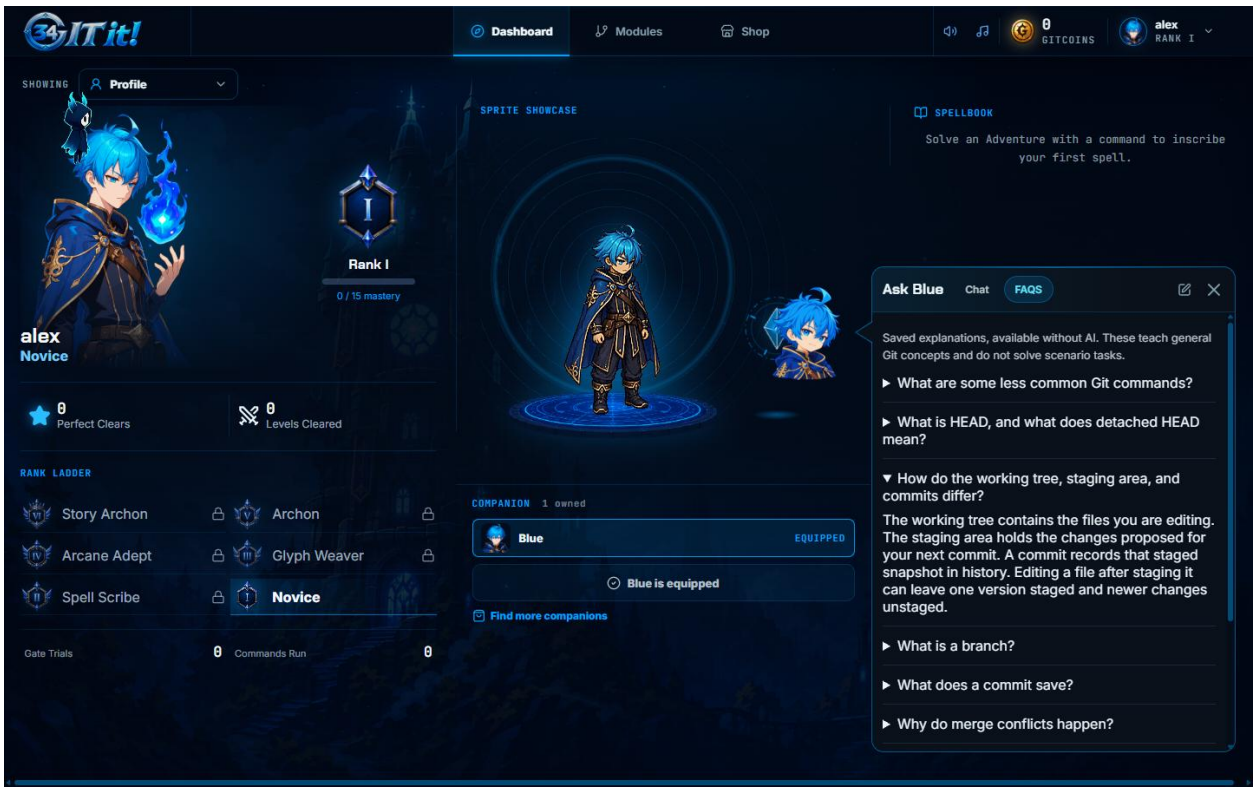


Figure AI-SRS-6. FAQS provides expandable saved Git explanations without a provider request and remains readable offline. Switching back to Chat restores the composer and retained conversation for the current context.

3.3. Non-functional requirements

Performance targets are measured on the production deployment from a client in the Philippines while the API is awake, unless a requirement states otherwise. The targets fit the free hosting tier (one API instance with 0.1 CPU); version 1.4 revised NFR-PERF-01, NFR-PERF-04, NFR-PERF-06 and NFR-PERF-07 after performance testing (STD, INC-10). The ID changes for requirements moved in version 1.3 are listed after the tables.

Performance

Req ID	Description	Target
NFR-PERF-01	Command confirmation time: the server verifies and records a submitted command and returns its result. Measured from the browser under the NFR-PERF-04 workload. The terminal has already shown the simulated result (NFR-PERF-02), so the learner can keep typing while confirmation is pending.	≤ 1.5 s at the 95th percentile with up to 20 learners; ≤ 4 s with 40 learners
NFR-PERF-02	Immediate feedback: the terminal output and repository graph update in the browser from the local simulation, before the server confirms the command.	≤ 200 ms after the command is submitted
NFR-PERF-03	Home load time: Home becomes usable (profile and progress shown) on a 10 Mbps connection.	≤ 3 s
NFR-PERF-04	Concurrent learners: 40 learners join over about 10 minutes (one every 15 seconds, as a class arrives), then play at once for 10 minutes, each submitting a command about every 10 seconds (about 4 commands per second overall) with page loads in between. NFR-PERF-01 holds and the API keeps pace with the offered load. Sign-in hashes the password, so the free tier cannot sign in 40 learners within one minute. Supporting 100 learners requires a larger server (see DEPLOYMENT.md).	40 concurrent learners joining one every 15 s; throughput ≥ 4 requests/s
NFR-PERF-05	Database query time: during the NFR-PERF-04 workload, the ten queries with the highest total time in Supabase's query performance report.	Average ≤ 100 ms each

NFR-PERF-06	Level start time: starting or resuming a tier until the workspace is ready. The workspace opens only after all of the encounter's battle art has loaded, so battles never flicker. When that art is not yet cached (a learner's first visit to a level, or a monster not met before), the browser first downloads up to about 4 MB.	≤ 2 s at the 95th percentile when the art is cached; ≤ 8 s when it must be downloaded, on a 10 Mbps connection
NFR-PERF-07	Error rate: share of requests that fail with HTTP 5xx, are rejected by the hosting platform (HTTP 429) or time out during the NFR-PERF-04 workload.	< 1%
NFR-PERF-08	Cold start: the free hosting tier puts the API to sleep after 15 minutes without traffic. The uptime monitor (NFR-REL-01) normally prevents this; if the API does sleep, the first request completes while the app shows "Restoring session" instead of an error.	≤ 120 s
NFR-PERF-09	AI tutor response time: the tutor returns a reply, a decline or its recovery message. The server stops working on a question after 30 s and the browser stops waiting after 40 s.	≤ 30 s

Performance target revisions (version 1.4)

Req ID	Version 1.3 target	Version 1.4 target	Reason for the change	Evidence
NFR-PERF-01	≤ 1 s at the 95th percentile with 40 learners	≤ 1.5 s at the 95th percentile with up to 20 learners; ≤ 4 s with 40	The free instance (0.1 CPU) could not confirm within 1 s under load, even after the database moved to the API's region and the server threads were raised. The terminal already shows the simulated result at once (NFR-PERF-02), so confirmation time does not hold up the learner. The new targets are the Run 5 measurements (1.02 s and 2.90 s) with a margin.	STD Runs 4 to 6; INC-10

Req ID	Version 1.3 target	Version 1.4 target	Reason for the change	Evidence
NFR-PERF-04	40 learners playing at once, all signing in within one minute; ≥ 4 requests/s	40 learners joining one every 15 s, then 10 minutes at 40; ≥ 4 requests/s	Each sign-in hashes the password, which takes seconds on 0.1 CPU. 40 sign-ins within one minute overloaded the instance and the hosting platform rejected requests. A class arriving over about 10 minutes is within capacity.	STD Runs 4 to 6 (Run A and Run B); INC-10
NFR-PERF-06	≤ 2 s at the 95th percentile	≤ 2 s at the 95th percentile when the art is cached; ≤ 8 s when it must be downloaded, at 10 Mbps	The loading screen deliberately waits for all of the encounter's battle art so that battles never flicker. A first visit downloads up to about 4 MB, about 3 s at 10 Mbps on its own, after the battle images had already been made smaller.	STD Runs 5 and 6; INC-10
NFR-PERF-07	< 1% of requests fail with HTTP 5xx or time out	< 1%, also counting HTTP 429 rejections by the hosting platform	Made stricter. In Run 4 the hosting platform rejected 4.5% to 10.7% of requests with HTTP 429; learners see these as failures, but version 1.3 did not count them.	STD Run 4; INC-10

The project runs on free hosting tiers by team decision, so these targets were set to the capacity measured on that tier. A larger server would allow the version 1.3 targets (DEPLOYMENT.md). The measurements are in the STD, version 3.2.

Security

Req ID	Description
NFR-SEC-01	All passwords shall be stored as secure one-way hashes using an industry-standard password hashing algorithm. Plaintext passwords shall never be persisted, logged, or returned to clients.

NFR-SEC-02	All client-server communication shall be transmitted over HTTPS (TLS 1.2 or higher). Plain HTTP connections shall be automatically redirected to HTTPS. This requirement applies to all endpoints system-wide and is specified in full in Section 3.1.3 (Communications Interfaces).
NFR-SEC-03	JWT access tokens shall expire after 15 minutes. Refresh tokens shall expire after 7 days.
NFR-SEC-04	Refresh tokens shall be stored exclusively in httpOnly, Secure, SameSite=Strict cookies — inaccessible to JavaScript running in the browser.
NFR-SEC-05	Refresh tokens shall be single-use: each refresh issues a new token and revokes the old one. On logout, or when the learner signs out other sessions or all sessions, the refresh session shall be revoked in the database, which is the authoritative record, and the revocation cached in Redis until the token would have expired. A revoked token shall be rejected even while Redis is unavailable.
NFR-SEC-06	Browsers shall reach the API through the same origin as the app (Vercel forwards /api to the API server), so the refresh cookie never crosses sites. Cross-origin requests shall be allowed only from configured origins. Sign-in, refresh, logout and other cookie-based authentication requests shall carry the application-client header (X-Git-It-Client: web); requests without it are rejected with HTTP 403.
NFR-SEC-07	All user input shall be validated by the server, which is authoritative; the client checks forms first as a usability aid, and rendered content is escaped by React. This applies to authentication forms, terminal commands, project-file edits, tutor messages and admin inputs. See FR-SEC-01.
NFR-SEC-08	SQL injection shall be prevented through exclusive use of parameterized queries or a trusted ORM. Raw string interpolation into SQL is prohibited.
NFR-SEC-09	Rate limits: sign-in locks for 5 minutes after 5 failed attempts for the same username or email from the same IP address (HTTP 429). Per IP address: registration 60/hour, token refresh 600/hour, password reset requests 20/hour, reset confirmations 60/hour, email-verification attempts 120/hour, code resends 60/hour, GitHub and Google sign-in 120/hour. Per learner: 120 commands or file actions per minute and 12 tutor messages per minute. Per-IP limits leave room for a class sharing one network address.
NFR-SEC-10	Personal data shall comply with the Philippine Data Privacy Act of 2012 (RA 10173). The system stores a username, email address and password hash; an optional GitHub or Google account link; the IP address and browser user agent of each sign-in session; and the learner's learning records. Data is stored in Supabase

	PostgreSQL, encrypted at rest (AES-256) and in transit (TLS). Only these processors receive personal data: Brevo (email address, to send verification and password-reset emails), Google AI Studio and Groq (tutor conversation text, without account identifiers), and GitHub or Google (only when the learner chooses to sign in with them).
NFR-SEC-11	Learner commands shall be interpreted only by the built-in Git simulator in the browser and the transition verifier on the server. No learner input shall reach an operating-system shell, an external process, or a Git binary.

Reliability and Availability

Req ID	Description	Target
NFR-REL-01	Availability during the evaluation period, measured by an external uptime monitor that requests https://git-it-34.vercel.app/api/health/ready/ every 5 minutes. The monitor also keeps the free-tier API awake; this uses about 744 of Render's 750 free instance hours a month, so no other free service may run in the same Render workspace.	≥ 99% of checks succeed
NFR-REL-02	Cache outage: if Redis is unavailable, the application shall keep serving from PostgreSQL. Sign-in, token refresh, gameplay and progress continue; rate limits and the sign-in lockout pause; GitHub and Google sign-in is unavailable; the readiness check reports "degraded" (HTTP 503) until Redis returns. No request fails with an unhandled error.	Graceful degradation; no crash or data loss
NFR-REL-03	Run recovery: an unfinished run is stored on the server, so after a reload, browser crash or new sign-in, reopening the level resumes it with every accepted command. Squire's Drill resumes mid-drill.	Required
NFR-REL-04	Database backups: Supabase's Free plan has no automatic backups, so during the evaluation period the team shall export the full database at least weekly and before each deployment that changes the database. Each export is encrypted (AES-256) and kept off-site by the team, never in the source repository, because it contains student personal data. A restore into a local database is	Weekly, and before each database-changing deployment

	rehearsed once, and each export is recorded in a backup log (date, person, file checksum).	
NFR-REL-05	Structured error responses: all API errors, including unknown /api/ addresses and unexpected server errors, shall return structured JSON with an HTTP status code and a message; no stack traces in production responses.	Required
NFR-REL-06	Data integrity — completion records: all database writes for completion records shall execute within transactions; unique constraints shall prevent duplicate completion entries.	Required
NFR-REL-07	Data integrity — step logs: step log entries shall be written atomically to prevent partial records from persisting.	Required
NFR-REL-14	Measurement records: run, command and checkpoint records shall be structured and queryable so that SCR, CAR, HLCR, ARC and RTA are computed automatically, as defined in Section 2.1. Replay and transfer runs are recorded separately from practice runs, and administrative actions are recorded in a separate admin action log.	Required
NFR-REL-16	Database connections: the API connects through Supabase's connection pooler over TLS and reuses each connection for up to 60 s. A 10 s lock timeout and a 30 s statement timeout stop one stuck request from holding connections.	Required
NFR-REL-17	Health checks: /api/health/live/ reports that the API process is running; /api/health/ready/ also checks PostgreSQL and Redis and returns HTTP 503 when either is unreachable.	Required
NFR-REL-18	Request logging: every API request is logged as one JSON line with a request ID, method, path, status code and duration.	Required

Usability and Accessibility

Req ID	Description	Target

NFR-USE-01	Keyboard accessibility: all primary student actions, including navigation, command submission, lesson completion, and scenario selection, shall be operable via keyboard input without requiring a pointer device. See FR-ACC-01. (Formerly NFR-REL-12.)	Required
NFR-USE-02	Supported screen sizes: every primary page and the level workspace shall be usable without horizontal scrolling from a 390 × 844 phone screen up to desktop. See FR-ACC-02. (Formerly NFR-REL-13, which required 1280 × 720.)	390 px wide and up

Maintainability

Req ID	Description
NFR-MNT-01	Scenario content maintainability: scenario definitions, variant pools, repository-state definitions, target-state guidance data, and support-level visibility settings shall be stored as seeded content records so that scenario text, diagrams, and evaluator targets are not hardcoded into frontend pages; target-state guidance shall be generated from the same structured target-state data used by the evaluator, regenerated by a command and never edited by hand. (Formerly NFR-REL-15.)
NFR-MNT-02	Every change to the main branch shall pass the CI quality gates: the backend and frontend test suites, Ruff, ESLint and Knip, the type-checked production build, and the repository guards for architecture boundaries, the API contract, generated targets, documentation and deployment settings.
NFR-MNT-03	Frontend API types shall be generated from the backend OpenAPI schema; CI fails when the committed contract is out of date or a frontend API wrapper bypasses it.

Research Validation Targets

Req ID	Description	Target
NFR-RES-01	Usability validation target: SUS score administered after the student's first completed scenario, using the instrument in Appendix B. (Formerly NFR-REL-08.)	≥ 70

NFR-RES-02	Technology acceptance validation target: Perceived Usefulness (PU), using the instrument in Appendix C. (Formerly NFR-REL-09.)	≥ 4.0 on a 5-point Likert scale
NFR-RES-03	Technology acceptance validation target: Perceived Ease of Use (PEOU). (Formerly NFR-REL-10.)	≥ 3.5 on a 5-point Likert scale
NFR-RES-04	Technology acceptance validation target: Behavioral Intention to Use (BI). (Formerly NFR-REL-11.)	≥ 4.0 on a 5-point Likert scale

ID changes in version 1.3: NFR-REL-08 to NFR-REL-11 moved to NFR-RES-01 to NFR-RES-04, NFR-REL-12 and NFR-REL-13 moved to NFR-USE-01 and NFR-USE-02, and NFR-REL-15 moved to NFR-MNT-01. NFR-PERF-07 to NFR-PERF-09, NFR-REL-17, NFR-REL-18, NFR-MNT-02 and NFR-MNT-03 are new. Retired IDs are not reused.

Appendix A: Needs Assessment Survey Summary

A.1. Overview

A needs assessment survey titled *Git Literacy & Collaboration Survey* was administered to third-year BSIT students at Cebu Institute of Technology – University (CIT-U) to identify Git-related knowledge gaps, behavioral patterns, and avoidance strategies within the target population. The survey ran for 72 days and collected **44 responses**, with an average completion time of 15 minutes and 50 seconds.

Survey platform: Microsoft Forms
Target population: Third-year BSIT students at CIT-U
Total responses: 44
Survey duration: 72 days

A.2. Respondent Profile

- **91%** of respondents confirmed they are third-year students (40 of 44).
- **95%** are enrolled in Information Technology (42 of 44); 5% are in Computer Science (2 of 44).
- Nearly all respondents had taken or were currently enrolled in both **Information Management 2** (40 respondents) and **Applications Development** (41 respondents), confirming they represent the intended target population.

A.3. Key Findings

A.3.1 Git Usage Context

- **66%** of respondents use Git in a collaborative setting (i.e., a few group projects per semester).
- **23%** use Git often across multiple active repositories at any given time.
- No respondent reported never using Git in a collaborative setting.

A.3.2 Error Response Behavior

When encountering a Git error they did not understand, respondents' most common first actions were:

Response	Count	Percentage
Paste error into AI (ChatGPT, Gemini, etc.) and run whatever it says	40	**91%**
Ask a more confident teammate	26	59%

Google the error and read StackOverflow or blog posts	17	39%
Read the Git documentation	8	18%
Delete the local folder and re-clone the repository	6	14%

Finding: AI dependency without comprehension is the dominant error-response behavior. 91% of students execute AI-suggested commands without understanding them.

A.3.3 Common Mistakes in Team Projects

The most frequently reported team mistakes were:

Mistake	Count
Someone pushed directly to main and broke the working version	28
The wrong branch was merged into main before it was ready	25
Someone committed and pushed files they shouldn't have (e.g., node_modules, .env)	18
A teammate ran git push --force and overwrote several days of commits	12
A branch got so far out of sync that nobody on the team knew how to catch it up	8
A teammate squashed the entire project history into a single commit	9

A.3.4 Conflict and Panic Situations

The most commonly reported situations that caused confusion or panic:

Situation	Count	Percentage

A merge conflict appeared and the team didn't know how to resolve it, so someone just "picked one side"	33	**75%**
The team had a repository where most commits were named things like 'update', 'fix', or 'test'	19	43%
Someone ran a Git command an AI told them to run without understanding it, and it made things worse	19	43%
Nobody on the team could explain what the Git history actually represented	10	23%
git pull was avoided because everyone was scared it would cause conflicts	9	20%

A.3.5 How Problems Were "Resolved"

Resolution method	Count	Percentage
A teammate with more Git experience just "took over" and fixed it without explaining	27	**61%**
The local folder was deleted and the repo was re-cloned to "start fresh"	20	**45%**
Code was shared via Discord, Messenger, or a ZIP file because Git was "too complicated"	9	20%
The project was abandoned on GitHub and a new repository was created from scratch	7	**16%**

Finding: Avoidance and workaround behaviors are pervasive. 45% of students have resorted to deleting and re-cloning, and 16% have abandoned repositories entirely.

A.4. Student-Identified Learning Needs

Student-Identified Learning Needs

When asked what they most wished a Git learning platform built for CIT students would teach them (Q18, 43 responses), the most prominent themes in open-ended responses were:

- **Merge conflicts** — the dominant theme across responses
- **Git reset and undoing mistakes**
- **Git commands and functions**
- **Errors and conflicts during commits and pushes**
- **Git rebase and reflog**

This directly informed the scenario content and module structure of GIT it!, particularly the prioritization of merge conflict resolution, branch recovery, and force-push consequence scenarios.

A.5. Connection to System Requirements

The survey findings directly informed the following system design decisions:

Finding	System Response
91% run AI-suggested commands without understanding	Terminal interface requires students to reason through commands; AI help is limited to protected conceptual guidance that does not supply scenario-specific answers or execute commands
75% resolve merge conflicts by discarding code	Merge conflict scenarios included across difficulty levels with state-based evaluation
45% delete and re-clone to escape repository states	Scenarios simulate broken repository states that must be resolved in-place
Students requested step-by-step guided practice	Module 0 provides guided walkthroughs before scenario sessions begin

Students requested consequence-safe practice	All sessions run on isolated simulated repositories; no real repositories are affected

Appendix B: SUS Questionnaire Instrument

B.1. Overview

The System Usability Scale (SUS) is a validated 10-item Likert-scale instrument developed by John Brooke (1986) for measuring the perceived usability of a system. Each item is rated on a 5-point scale from **Strongly Disagree (1)** to **Strongly Agree (5)**. The final SUS score is computed on a scale of 0–100, where a score of ≥ 70 is considered acceptable usability.

Administered to: Third-year BSIT students at CIT-U
Timing: After the student's first completed scenario session
Scale: 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree)
Scoring target: SUS score ≥ 70 (NFR-REL-08)

B.2. Instrument

#	Statement	1 Strongly Disagree	2 Disagree	3 Neutral	4 Agree	5 Strongly Agree
1	I think that I would like to use this system frequently.					
2	I found the system unnecessarily complex.					
3	I thought the system was easy to use.					
4	I think that I would need the support of a technical person to be able to use this system.					
5	I found the various functions in this system were well integrated.					
6	I thought there was too much inconsistency in this system.					
7	I would imagine that most people would learn to use this system very quickly.					
8	I found the system very cumbersome to use.					
9	I felt very confident using the system.					

10	I needed to learn a lot of things before I could get going with this system.					

Scoring Method

1. For **odd-numbered items** (1, 3, 5, 7, 9): subtract 1 from the response score.
2. For **even-numbered items** (2, 4, 6, 8, 10): subtract the response score from 5.
3. Sum all converted scores and multiply by **2.5** to obtain the final SUS score (0–100).

A SUS score of **≥ 70** indicates acceptable usability. Scores of ≥ 85 are considered excellent.

Appendix C: TAM Questionnaire Instrument

C.1. Overview

The Technology Acceptance Model (TAM) instrument measures students' perceptions of a system across three constructs: **Perceived Usefulness (PU)**, **Perceived Ease of Use (PEOU)**, and **Behavioral Intention to Use (BI)**. The instrument is adapted from Davis (1989) and subsequent TAM validation studies.

Administered to: Third-year BSIT students at CIT-U
Timing: After the student's first completed scenario session
Scale: 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree)

Scoring Targets

Construct	Target	NFR Reference
Perceived Usefulness (PU)	Mean ≥ 4.0	NFR-REL-09
Perceived Ease of Use (PEOU)	Mean ≥ 3.5	NFR-REL-10
Behavioral Intention to Use (BI)	Mean ≥ 4.0	NFR-REL-11

C.2. Instrument

Perceived Usefulness (PU)

#	Statement	1 Strongly Disagree	2 Disagree	3 Neutral	4 Agree	5 Strongly Agree
PU1	Using GIT it! improves my ability to practice Git commands in a realistic setting.					
PU2	Using GIT it! enhances my understanding of Git repository states and version control concepts.					

PU3	Using GIT it! allows me to practice resolving Git errors in a consequence-safe environment.					
PU4	Overall, I find GIT it! useful for developing my Git skills.					

Perceived Ease of Use (PEOU)

#	Statement	1 Strongly Disagree	2 Disagree	3 Neutral	4 Agree	5 Strongly Agree
PEOU1	The terminal interface in GIT it! is easy to interact with.					
PEOU2	The DAG visualization helps me understand the current state of the repository clearly.					
PEOU3	The instructional scaffolding (hints and support tools) provided in GIT it! is easy to understand and use.					
PEOU4	Overall, I find GIT it! easy to use.					

Behavioral Intention to Use (BI)

#	Statement	1 Strongly Disagree	2 Disagree	3 Neutral	4 Agree	5 Strongly Agree
BI1	I intend to use GIT it! again to practice Git skills beyond this evaluation.					
BI2	I would recommend GIT it! to other BSIT students who want to improve their Git skills.					
BI3	I would use GIT it! to prepare for group projects that require Git collaboration.					

Scoring Method

Each construct score is computed as the **mean of its item scores**.

Example: PU score = (PU1 + PU2 + PU3 + PU4) ÷ 4

A construct mean of ≥ 4.0 (PU, BI) or ≥ 3.5 (PEOU) on a 5-point scale indicates the target threshold has been met per the system's non-functional requirements.